Qualcomm Technologies, Inc.

# Qualcomm Hexagon QuRT RTOS

## User Guide

80-VB419-78 Rev. P

June 12, 2025

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:40:42 GMT
hyungchul.won@lge.com

**Confidential - Qualcomm Technologies, Inc. and/or its affiliated companies - May Contain Trade Secrets**

**NO PUBLIC DISCLOSURE PERMITTED:** Please report postings of this document on public servers or websites to: DocCtrlAgent@qualcomm.com.

© Qualcomm Technologies, Inc. and/or its subsidiaries. All rights reserved.

# Revision History

| Revision | Date | Description |
|---|---|---|
| A | Aug 2011 | Initial release. |
| B | Nov 2011 | Update predefined symbols; update TLB API; add VID API. |
| C | Apr 2012 | Synchronize API with current RTOS release (1.02.01); move fastint, TLB, power, and system services to a new user guide supplement document; fix various document bugs. |
| D | Mar 2013 | Engineering updates.<br>Consolidated QuRT Services sections and QuRT API sections into chapters. |
| E | Apr 2013 | Updated Exception Handling (Chapter 18).<br>Added:<br>• Processes (Chapter 4)<br>• Signals (Chapter 8)<br>• System Environment (Chapter 22) |
| F | Dec 2014 | Updated for QuRT version 02.03.36.<br>Added Q7, Q8, and Q9 to references Table 1-1.<br>Added section 2.5 and qurt_rmutex2_t (Section 5.7.1.2).<br>Updated sections 1.9, 3.17, 4.3, 5.8.1.2, 14.2, 14.6, 15.20.1.2, 17.7, 17.9 17.11, 21.2.1.1, 21.3, 21.15.1.1.<br>Added new functions:<br>• qurt_thread_set_cache_partition (Section 3.21)<br>• qurt_process_cmdline_get (Section 4.7)<br>• qurt_signal_wait_cancellable (Section 8.7)<br>• qurt_pipe_attr_set_buffer_partition (Section 14.5)<br>• qurt_pipe_receive_cancellable (Section 14.14)<br>• qurt_pipe_send_cancellable (Section 14.15)<br>• qurt_exception_shutdown_fatal2 (Section 19.7)<br>• qurt_exception_wait_pagefault (Section 19.10)<br>• qurt_lookup_physaddr_64 (Section 21.3)<br>• qurt_mapping_create_64 (Section 21.5)<br>• qurt_mapping_remove_64 (Section 21.7)<br>• qurt_mem_l2cache_line_lock (Section 21.09)<br>• qurt_mem_l2cache_line_unlock (Section 21.10)<br>• qurt_mem_map_static_query_64 (Section 21.12)<br>• qurt_mem_pool_create (Section 21.17)<br>• qurt_mem_region_attr_get_bus_attr (Section 21.20)<br>• qurt_mem_region_attr_get_physaddr_64 (Section 21.26)<br>• qurt_mem_region_attr_set_bus_attr (Section 21.33)<br>• qurt_mem_region_attr_set_physaddr_64 (Section 21.34)<br>• qurt_mem_region_query_64 (Section 21.38)<br>• qurt_l2fetch_disable (Section 21.39) |

Qualcomm
Confidential - May Contain Trade Secrets
2015-12-17 05:10:42 GMT
hyderabad, wahab, ccm

| Revision | Date | Description |
|---|---|---|
| G | Nov 2015 | Updated for QuRT version 02.4.14.<br>Updated qurt_thread_attr_init (Section 3.2).<br>Added new data types (Sections 8.11, 12.5, 14.17, 15.19, 21.45, 22.9, 25.1, and 28.2).<br>Added new functions:<br>• qurt_thread_attr_set_bus_priority (Section 3.6)<br>• qurt_thread_attr_get (Section 3.11)<br>• qurt_thread_get_l2cache_partition (Section 3.24)<br>• qurt_pipe_is_empty (Section 14.16)<br>• qurt_sysclock_get_hw_ticks_32 (Section 16.5)<br>• qurt_sysclock_get_hw_ticks_16 (Section 16.6)<br>• qurt_mem_pool_add_pages (Section 21.19)<br>• qurt_mem_pool_remove_pages (Section 21.20)<br>• qurt_mem_barrier (Section 21.42)<br>• qurt_mem_syncht (Section 21.43)<br>• qurt_mem_configure_cache_partition (Section 21.44)<br>• qurt_sysenv_get_stack_profile_count (Section 22.8) |
| H | Feb 2020 | Updated Section 19.1.<br>Added Appendix B Debugging Errors and Cause Codes. |

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:40:42 GMT
hyungchul.won@lge.com

| Revision | Date | Description |
|---|---|---|
| J | Oct 2020 | Updated for QuRT version 04.00.xx.<br><br>Updated:<br>• Sections 3.2.1.1, 22.4.1.1, and 24.3.1.1<br><br>Added new Sections:<br>• Data Types (Sections 3.27.1.2 and 8.17.2.2).<br>• Macros (Sections 3.32.1, 21.50, 24.4, and 28.5).<br><br>Added new Chapters:<br>• Atomic Operations (Chapter 27)<br>• SRM Drivers (Chapter 29)<br><br>Added new functions:<br>• qurt_thread_attr_set_stack_size2 (Section 3.8)<br>• qurt_thread_attr_set_detachstate (Section 3.23)<br>• qurt_thread_stid_set (Section 3.26)<br>• qurt_sleep (Section 3.28)<br>• qurt_thread_get_tls_base (Section 3.29)<br>• qurt_qurt_busy_wait (Section 3.30)<br>• qurt_process_get_uid (Section 4.8)<br>• qurt_process_attr_set_max_threads (Section 4.9)<br>• qurt_process_attr_set_ceiling_prio (Section 4.10)<br>• qurt_process_get_thread_count (Section 4.13)<br>• qurt_process_get_thread_ids (Section 4.14)<br>• qurt_process_dump_get_mappings (Section 4.16)<br>• qurt_process_attr_get (Section 4.17)<br>• qurt_process_dump_register_cb (Section 4.18)<br>• qurt_process_dump_deregister_cb (Section 4.19)<br>• qurt_mutex_lock_timed (Section 5.6)<br>• qurt_rmutex_lock_timed (Section 6.6)<br>• qurt_rmutex_try_lock_block_once (Section 6.7)<br>• qurt_signal_wait_timed (Section 8.10)<br>• qurt_signal_64_init (Section 8.11)<br>• qurt_signal_64_destroy (Section 8.12)<br>• qurt_signal_64_wait (Section 8.13)<br>• qurt_signal_64_set (Section 8.14)<br>• qurt_signal_64_get (Section 8.15)<br>• qurt_signal_64_clear (Section 8.16)<br>• qurt_anysignal_wait_timed (Section 9.7)<br>• qurt_sem_down_timed (Section 11.9)<br>• qurt_assert_error (Section 19.6)<br>• qurt_lookup_physaddr2 (Section 21.2)<br>• qurt_mem_cache_phys_clean (Section 21.12)<br>• qurt_mem_pool_is_available (Section 21.22)<br>• qurt_sysenv_get_hw_threads (Section 22.8)<br>• qurt_get_hthread_pcycles (Section 23.7)<br>• qurt_get_hthread_commits (Section 23.8)<br>• qurt_profile_enable2 (Section 23.3)<br>• qurt_profile_get (Section 23.4)<br>• qurt_etm_set_pc_range (Section 26.3)<br>• qurt_etm_set_atb (Section 26.4) |

Qualcomm
Confidential - May Contain Trade Secrets
2026-03-05 10:47 GMT
hwangpu@qualcomm.com

| Revision | Date | Description |
|---|---|---|
| J (cont.) | Oct 2020 | Added new functions:<br>• qurt_stm_trace_set_config (Section 26.5)<br>• qurt_cb_data_set_cbarg (Section 28.2)<br>• qurt_cb_data_set_cbfunc (Section 28.3)<br>• qurt_cb_data_init (Section 28.4) |
| K | Feb 2021 | Added Chapter 30 HVX.<br>Updated Table C.1. |
| L | July 2023 | Updated for QuRT 4.3.<br>Updated Tables B.5 and B.6.<br>Added new Chapters:<br>• Memory Mapping (Chapter 22)<br>• SRM Drivers (Chapter 30)<br>Added new functions:<br>• qurt_thread_attr_set_autostack (Section 3.4)<br>• qurt_thread_set_autostack (Section 3.5)<br>• qurt_thread_get_running_ids (Section 3.27)<br>• qurt_thread_get_thread_id (Section 3.28)<br>• qurt_system_set_priority_floor (Section 3.30)<br>• qurt_process_attr_set_binary_path (Section 4.3)<br>• qurt_process_attr_set_dtb_path (Section 4.4)<br>• qurt_process_attr_set_sw_id (Section 4.10)<br>• qurt_process_exit (Section 4.17)<br>• qurt_process_kill (Section 4.18)<br>• qurt_debugger_register_process (Section 4.19)<br>• qurt_debugger_deregister_process (Section 4.20)<br>• qurt_process_exec_callback (Section 4.21)<br>• qurt_process_get_pid (Section 4.22)<br>• qurt_interrupt_register2 (Section 17.9)<br>• qurt_interrupt_set_config2 (Section 17.11)<br>• qurt_interrupt_set_config3 (Section 17.12)<br>• qurt_interrupt_raise2 (Section 17.15)<br>• qurt_exception_wait3 (Section 19.5)<br>• qurt_mem_pool_attach2 (Section 21.19)<br>• qurt_sysenv_get_process_name2 (Section 23.5)<br>• qurt_pmu_get_pmucnt (Section 25.4)<br>• qurt_etm_set_range (Section 27.4)<br>• qurt_etm_set_sync_period (Section 27.6) |

Qualcomm
Confidential - Qualcomm Technologies, Inc. and/or its affiliated companies - May Contain Trade Secrets
2020-02-24 07:49:49 GMT
kumarchin@qc.com

| Revision | Date | Description |
|---|---|---|
| M | May 2024 | Updated for QuRT 4.4.<br>Updated:<br>• Description for qurt_profile_get_thread_pcycles (Section 24.7)<br>• Predefined Symbols (Chapter 31)<br>• Table C.1 Related Documents<br>Deprecated:<br>• qurt_profile_get_idle_pcycles, which is replaced by qurt_profile_get_idle_pcycles2 (Section 24.6)<br>• qurt_profile_get_threadid_pcycles, which is replaced by qurt_profile_get_threadid_pcycles2 (Section 24.11)<br>Added:<br>• qurt_thread_resume_thread (Section 3.21)<br>• qurt_thread_attr_set_group_id (Section 3.27)<br>• QURT_THREAD_RESUME_DEFAULT (Section 3.34.1.19)<br>• qurt_process_vtcm_window_set (Section 4.24)<br>• Constants and macros in Section 4.26 and Section 31.2<br>• Data Types (Section 6.8)<br>• qurt_pimutex_lock_timed (Section 7.6)<br>• qurt_mapping_attr_get (Section 21.4)<br>• qurt_mapping_attr_get_cache_mode (Section 21.5)<br>• qurt_mapping_attr_get_physaddr (Section 21.6)<br>• qurt_mapping_attr_get_perms (Section 21.7)<br>• qurt_mapping_attr_get_size (Section 21.8)<br>• struct qurt_mem_pool_status_t (Section 21.50.2.3)<br>• qurt_mem_pool_status_get (Section 22.9) |
| N | April 2025 | Updated for QuRT 4.5<br>Added:<br>• Device tree<br>• qurt_memalign<br>• qurt_sysenv_get_instance_id<br>• func_qurt_exception_wait_ext<br>• qurt_enable_floating_point_exception<br>• qurt_mapping_attr64_t<br>• qurt_mem_pool_attr_get_addr_64<br>• qurt_mapping_attr_get_64<br>• qurt_physmem_get_addr_width<br>• qurt_physmem_encode_page<br>• qurt_physmem_decode_page<br>• qurt_mem_preempt_virtual_memory<br>• qurt_mem_resume_virtual_memory<br>• qurt_process_vtcm_window_set2<br>• qurt_process_vtcm_window_get2<br>Deprecated:<br>• qurt_cond_wait2 |

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:40:42 GMT
www.qualcomm.com

| Revision | Date | Description |
|---|---|---|
| P | June 2025 | Updated for QuRT API Version 11<br>Added:<br>• Note<br>• OS services<br>• Physical memory manager<br>• printf<br>• QURT_ETIMERADJUST<br>• qurt_mem_region_attr_set_perms<br>• qurt_mem_region_attr_set_owner<br>• qurt_sem_down_cancellable<br>• qurt_thread_set_group_id<br>Updated:<br>• qurt_malloc<br>• qurt_calloc<br>• qurt_realloc<br>• qurt_free<br>• qurt_memalign<br>• qurt_clade2_get<br>• qurt_clade2_set<br>• qurt_cond_destroy |

**Note:** There is no Rev. I, O, Q, S, X, or Z per Mil. standards.

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:40:42 GMT
hyungchul.won@lge.com

# Contents


**1 Introduction** **43**
- 1.1 Purpose . . . 43
- 1.2 Conventions . . . 43
- 1.3 Technical Assistance . . . 43
- 1.4 QuRT Features . . . 44
- 1.5 Processor Versions . . . 45

**2 Using QuRT** **46**
- 2.1 User Programs . . . 46
- 2.2 Build Procedure . . . 47
- 2.3 API . . . 47
- 2.4 Objects . . . 48
- 2.5 QuRT Services . . . 49

**3 Threads** **50**
- 3.1 Overview . . . 50
- 3.2 qurt_thread_attr_get() . . . 55
  - 3.2.1 Function Documentation . . . 55
    - 3.2.1.1 qurt_thread_attr_get . . . 55
- 3.3 qurt_thread_attr_init() . . . 56
  - 3.3.1 Function Documentation . . . 56
    - 3.3.1.1 qurt_thread_attr_init . . . 56
- 3.4 qurt_thread_attr_set_bus_priority() . . . 57
  - 3.4.1 Function Documentation . . . 57
    - 3.4.1.1 qurt_thread_attr_set_bus_priority . . . 57
- 3.5 qurt_thread_attr_set_autostack() . . . 58
  - 3.5.1 Function Documentation . . . 58
    - 3.5.1.1 qurt_thread_attr_set_autostack . . . 58
- 3.6 qurt_thread_set_autostack() . . . 59
  - 3.6.1 Function Documentation . . . 59
    - 3.6.1.1 qurt_thread_set_autostack . . . 59
- 3.7 qurt_thread_attr_set_name() . . . 60
  - 3.7.1 Function Documentation . . . 60
    - 3.7.1.1 qurt_thread_attr_set_name . . . 60
- 3.8 qurt_thread_attr_set_priority() . . . 61
  - 3.8.1 Function Documentation . . . 61
    - 3.8.1.1 qurt_thread_attr_set_priority . . . 61
- 3.9 qurt_thread_attr_set_stack_addr() . . . 62
  - 3.9.1 Function Documentation . . . 62
    - 3.9.1.1 qurt_thread_attr_set_stack_addr . . . 62



Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:40:42 GMT
kwang.hul.woi@lge.com

3.10 qurt_thread_attr_set_stack_size() . . . 63
3.10.1 Function Documentation . . . 63
3.10.1.1 qurt_thread_attr_set_stack_size . . . 63
3.11 qurt_thread_attr_set_stack_size2() . . . 64
3.11.1 Function Documentation . . . 64
3.11.1.1 qurt_thread_attr_set_stack_size2 . . . 64
3.12 qurt_thread_attr_set_tcb_partition() . . . 65
3.12.1 Function Documentation . . . 65
3.12.1.1 qurt_thread_attr_set_tcb_partition . . . 65
3.13 qurt_thread_attr_set_timetest_id() . . . 66
3.13.1 Function Documentation . . . 66
3.13.1.1 qurt_thread_attr_set_timetest_id . . . 66
3.14 qurt_thread_create() . . . 67
3.14.1 Function Documentation . . . 67
3.14.1.1 qurt_thread_create . . . 67
3.15 qurt_thread_stop() . . . 68
3.15.1 Function Documentation . . . 68
3.15.1.1 qurt_thread_stop . . . 68
3.16 qurt_thread_exit() . . . 69
3.16.1 Function Documentation . . . 69
3.16.1.1 qurt_thread_exit . . . 69
3.17 qurt_thread_get_id() . . . 70
3.17.1 Function Documentation . . . 70
3.17.1.1 qurt_thread_get_id . . . 70
3.18 qurt_thread_get_l2cache_partition() . . . 71
3.18.1 Function Documentation . . . 71
3.18.1.1 qurt_thread_get_l2cache_partition . . . 71
3.19 qurt_thread_get_name() . . . 72
3.19.1 Function Documentation . . . 72
3.19.1.1 qurt_thread_get_name . . . 72
3.20 qurt_thread_get_priority() . . . 73
3.20.1 Function Documentation . . . 73
3.20.1.1 qurt_thread_get_priority . . . 73
3.21 qurt_thread_get_timetest_id() . . . 74
3.21.1 Function Documentation . . . 74
3.21.1.1 qurt_thread_get_timetest_id . . . 74
3.22 qurt_thread_join() . . . 75
3.22.1 Function Documentation . . . 75
3.22.1.1 qurt_thread_join . . . 75
3.23 qurt_thread_resume_thread() . . . 76
3.23.1 Function Documentation . . . 76
3.23.1.1 qurt_thread_resume_thread . . . 76
3.24 qurt_thread_set_thread_property() . . . 77
3.24.1 Function Documentation . . . 77
3.24.1.1 qurt_thread_set_thread_property . . . 77
3.25 qurt_thread_get_group_id() . . . 78
3.25.1 Function Documentation . . . 78
3.25.1.1 qurt_thread_get_group_id . . . 78
3.26 qurt_thread_set_group_id() . . . 79
3.26.1 Function Documentation . . . 79

QUALCOMM Confidential - May Contain Trade Secrets 2025-03-27 05:40:42 GMT hwanchul.won@lge.com

3.26.1.1 qurt_thread_set_group_id . . . 79
3.27 qurt_thread_set_cache_partition() . . . 80
3.27.1 Function Documentation . . . 80
3.27.1.1 qurt_thread_set_cache_partition . . . 80
3.28 qurt_thread_set_priority() . . . 81
3.28.1 Function Documentation . . . 81
3.28.1.1 qurt_thread_set_priority . . . 81
3.29 qurt_thread_attr_set_detachstate() . . . 82
3.29.1 Function Documentation . . . 82
3.29.1.1 qurt_thread_attr_set_detachstate . . . 82
3.30 qurt_thread_set_timetest_id() . . . 83
3.30.1 Function Documentation . . . 83
3.30.1.1 qurt_thread_set_timetest_id . . . 83
3.31 qurt_thread_stid_set() . . . 84
3.31.1 Function Documentation . . . 84
3.31.1.1 qurt_thread_stid_set . . . 84
3.32 qurt_thread_attr_set_group_id() . . . 85
3.32.1 Function Documentation . . . 85
3.32.1.1 qurt_thread_attr_set_group_id . . . 85
3.33 qurt_thread_get_proc_id() . . . 86
3.33.1 Function Documentation . . . 86
3.33.1.1 qurt_thread_get_proc_id . . . 86
3.34 qurt_thread_sched_yield() . . . 87
3.34.1 Function Documentation . . . 87
3.34.1.1 qurt_thread_sched_yield . . . 87
3.35 qurt_thread_get_thread_id() . . . 88
3.35.1 Function Documentation . . . 88
3.35.1.1 qurt_thread_get_thread_id . . . 88
3.36 qurt_sleep() . . . 89
3.36.1 Function Documentation . . . 89
3.36.1.1 qurt_sleep . . . 89
3.37 qurt_system_set_priority_floor() . . . 90
3.37.1 Function Documentation . . . 90
3.37.1.1 qurt_system_set_priority_floor . . . 90
3.38 qurt_thread_get_tls_base() . . . 91
3.38.1 Function Documentation . . . 91
3.38.1.1 qurt_thread_get_tls_base . . . 91
3.39 qurt_busywait() . . . 92
3.39.1 Function Documentation . . . 92
3.39.1.1 qurt_busywait . . . 92
3.40 Data Types . . . 93
3.40.1 Define Documentation . . . 93
3.40.1.1 CCCC_PARTITION . . . 93
3.40.1.2 MAIN_PARTITION . . . 93
3.40.1.3 AUX_PARTITION . . . 93
3.40.1.4 MINIMUM_PARTITION . . . 93
3.40.2 Data Structure Documentation . . . 93
3.40.2.1 struct qurt_thread_attr_t . . . 93
3.40.2.2 struct qurt_tls_info . . . 93
3.40.3 Typedef Documentation . . . 93

Qualcomm Confidential - May Contain Trade Secrets 2025-02-27 05:40:42 GMT hongchul.won@lge.com

3.40.3.1 qurt_cache_partition_t . . . 93
3.40.3.2 qurt_thread_t . . . 93
3.40.4 Variable Documentation . . . 94
3.40.4.1 thread_handle . . . 94
3.41 Constants and Macros . . . 95
3.41.1 Define Documentation . . . 95
3.41.1.1 QURT_MAX_HTHREAD_LIMIT . . . 95
3.41.1.2 QURT_THREAD_CFG_BITMASK_ALL . . . 95
3.41.1.3 QURT_THREAD_BUS_PRIO_DISABLED . . . 95
3.41.1.4 QURT_THREAD_BUS_PRIO_ENABLED . . . 95
3.41.1.5 QURT_THREAD_AUTOSTACK_DISABLED . . . 95
3.41.1.6 QURT_THREAD_AUTOSTACK_ENABLED . . . 95
3.41.1.7 QURT_THREAD_ATTR_CREATE_LEGACY . . . 95
3.41.1.8 QURT_THREAD_ATTR_CREATE_JOINABLE . . . 95
3.41.1.9 QURT_THREAD_ATTR_CREATE_DETACHED . . . 95
3.41.1.10 QURT_THREAD_ATTR_TCB_PARTITION_DEFAULT . . . 95
3.41.1.11 QURT_THREAD_ATTR_PRIORITY_DEFAULT . . . 95
3.41.1.12 QURT_THREAD_ATTR_ASID_DEFAULT . . . 96
3.41.1.13 QURT_THREAD_ATTR_AFFINITY_DEFAULT . . . 96
3.41.1.14 QURT_THREAD_ATTR_BUS_PRIO_DEFAULT . . . 96
3.41.1.15 QURT_THREAD_ATTR_AUTOSTACK_DEFAULT . . . 96
3.41.1.16 QURT_THREAD_ATTR_TIMETEST_ID_DEFAULT . . . 96
3.41.1.17 QURT_THREAD_ATTR_STID_DEFAULT . . . 96
3.41.1.18 QURT_THREAD_ATTR_STID_ENABLE . . . 96
3.41.1.19 QURT_PRIORITY_FLOOR_DEFAULT . . . 96
3.41.1.20 QURT_THREAD_RESUME_DEFAULT . . . 96
3.41.1.21 QURT_THREAD_HTID_BITMASK . . . 96
3.41.1.22 QURT_THREAD_COREID_BITMASK . . . 96
3.41.1.23 QURT_THREAD_HTID_SHIFT . . . 96
3.41.1.24 QURT_THREAD_COREID_SHIFT . . . 96

**4 Processes** **97**
4.1 Overview . . . 97
4.2 qurt_process_create() . . . 99
4.2.1 Function Documentation . . . 99
4.2.1.1 qurt_process_create . . . 99
4.3 qurt_process_get_id() . . . 100
4.3.1 Function Documentation . . . 100
4.3.1.1 qurt_process_get_id . . . 100
4.4 qurt_process_attr_init() . . . 101
4.4.1 Function Documentation . . . 101
4.4.1.1 qurt_process_attr_init . . . 101
4.5 qurt_process_attr_set_executable() . . . 102
4.5.1 Function Documentation . . . 102
4.5.1.1 qurt_process_attr_set_executable . . . 102
4.6 qurt_process_attr_set_binary_path . . . 103
4.6.1 Function Documentation . . . 103
4.6.1.1 qurt_process_attr_set_binary_path . . . 103
4.7 qurt_process_attr_set_dtb_path() . . . 104
4.7.1 Function Documentation . . . 104

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:00:42 GMT myungchul.won@lge.com

4.7.1.1 qurt_process_attr_set_dtb_path . . . 104
4.8 qurt_process_attr_set_flags() . . . 105
4.8.1 Function Documentation . . . 105
4.8.1.1 qurt_process_attr_set_flags . . . 105
4.9 qurt_process_cmdline_get() . . . 106
4.9.1 Function Documentation . . . 106
4.9.1.1 qurt_process_cmdline_get . . . 106
4.10 qurt_process_attr_set_max_threads() . . . 107
4.10.1 Function Documentation . . . 107
4.10.1.1 qurt_process_attr_set_max_threads . . . 107
4.11 qurt_process_attr_set_sw_id() . . . 108
4.11.1 Function Documentation . . . 108
4.11.1.1 qurt_process_attr_set_sw_id . . . 108
4.12 qurt_process_attr_set_ceiling_prio() . . . 109
4.12.1 Function Documentation . . . 109
4.12.1.1 qurt_process_attr_set_ceiling_prio . . . 109
4.13 qurt_process_get_thread_count() . . . 110
4.13.1 Function Documentation . . . 110
4.13.1.1 qurt_process_get_thread_count . . . 110
4.14 qurt_process_get_thread_ids() . . . 111
4.14.1 Function Documentation . . . 111
4.14.1.1 qurt_process_get_thread_ids . . . 111
4.15 qurt_process_attr_get() . . . 112
4.15.1 Function Documentation . . . 112
4.15.1.1 qurt_process_attr_get . . . 112
4.16 qurt_process_dump_register_cb() . . . 113
4.16.1 Function Documentation . . . 113
4.16.1.1 qurt_process_dump_register_cb . . . 113
4.17 qurt_process_dump_deregister_cb() . . . 114
4.17.1 Function Documentation . . . 114
4.17.1.1 qurt_process_dump_deregister_cb . . . 114
4.18 qurt_process_exit() . . . 115
4.18.1 Function Documentation . . . 115
4.18.1.1 qurt_process_exit . . . 115
4.19 qurt_process_kill() . . . 116
4.19.1 Function Documentation . . . 116
4.19.1.1 qurt_process_kill . . . 116
4.20 qurt_debugger_register_process() . . . 117
4.20.1 Function Documentation . . . 117
4.20.1.1 qurt_debugger_register_process . . . 117
4.21 qurt_debugger_deregister_process() . . . 118
4.21.1 Function Documentation . . . 118
4.21.1.1 qurt_debugger_deregister_process . . . 118
4.22 qurt_process_exec_callback() . . . 119
4.22.1 Function Documentation . . . 119
4.22.1.1 qurt_process_exec_callback . . . 119
4.23 qurt_process_get_pid() . . . 120
4.23.1 Function Documentation . . . 120
4.23.1.1 qurt_process_get_pid . . . 120
4.24 qurt_process_get_dm_status() . . . 121

Qualcomm Confidential - May Contain Trade Secrets 2025-02-27 05:40:42 GMT hongshui.wondge.com

4.24.1 Function Documentation . . . 121
4.24.1.1 qurt_process_get_dm_status . . . 121
4.25 qurt_process_vtcm_window_set() . . . 122
4.25.1 Function Documentation . . . 122
4.25.1.1 qurt_process_vtcm_window_set . . . 122
4.26 qurt_process_vtcm_window_set2() . . . 123
4.26.1 Function Documentation . . . 123
4.26.1.1 qurt_process_vtcm_window_set2 . . . 123
4.27 qurt_process_vtcm_window_get() . . . 124
4.27.1 Function Documentation . . . 124
4.27.1.1 qurt_process_vtcm_window_get . . . 124
4.28 qurt_process_vtcm_window_get2() . . . 125
4.28.1 Function Documentation . . . 125
4.28.1.1 qurt_process_vtcm_window_get2 . . . 125
4.29 Data Types . . . 126
4.29.1 Define Documentation . . . 126
4.29.1.1 QURT_PROCESS_ATTR_NAME_MAXLEN . . . 126
4.29.1.2 QURT_PROCESS_ATTR_BIN_PATH_MAXLEN . . . 126
4.29.1.3 QURT_PROCESS_DEFAULT_CEILING_PRIO . . . 126
4.29.1.4 QURT_PROCESS_DEFAULT_MAX_THREADS . . . 126
4.29.1.5 QURT_PROCESS_NON_SYSTEM_CRITICAL . . . 126
4.29.1.6 QURT_PROCESS_ISLAND_RESIDENT . . . 126
4.29.1.7 QURT_PROCESS_RESTARTABLE . . . 126
4.29.1.8 QURT_PROCESS_UNTRUSTED . . . 126
4.29.1.9 QURT_DEBUG_NOT_START . . . 126
4.29.1.10 QURT_DEBUG_START . . . 126
4.29.1.11 QURT_PROCESS_SUSPEND_DEFAULT . . . 126
4.29.1.12 QURT_PROCESS_RESUME_DEFAULT . . . 126
4.29.2 Data Structure Documentation . . . 127
4.29.2.1 struct qurt_pd_dump_attr_t . . . 127
4.29.2.2 struct qurt_process_attr_t . . . 127
4.29.3 Enumeration Type Documentation . . . 127
4.29.3.1 qurt_process_type_t . . . 127
4.29.3.2 qurt_process_dump_cb_type_t . . . 127
4.30 Macros . . . 128
4.30.1 Define Documentation . . . 128
4.30.1.1 QURT_VTCM_WINDOW_ENABLE . . . 128
4.30.1.2 QURT_VTCM_WINDOW_DISABLE . . . 128
4.30.1.3 QURT_VTCM_WINDOW_HI_OFFSET_DEFAULT . . . 128
4.30.1.4 QURT_VTCM_WINDOW_LO_OFFSET_DEFAULT . . . 128

**5 Mutexes** **129**
5.1 Overview . . . 129
5.2 qurt_mutex_destroy() . . . 131
5.2.1 Function Documentation . . . 131
5.2.1.1 qurt_mutex_destroy . . . 131
5.3 qurt_mutex_init() . . . 132
5.3.1 Function Documentation . . . 132
5.3.1.1 qurt_mutex_init . . . 132
5.4 qurt_mutex_lock() . . . 133

Qualcomm Confidential - May Contain Trade Secrets 2025-02-27 05:19:42 PST hwungh@qualcomm.com

5.4.1 Function Documentation . . . 133
5.4.1.1 qurt_mutex_lock . . . 133
5.5 qurt_mutex_try_lock() . . . 134
5.5.1 Function Documentation . . . 134
5.5.1.1 qurt_mutex_try_lock . . . 134
5.6 qurt_mutex_unlock() . . . 135
5.6.1 Function Documentation . . . 135
5.6.1.1 qurt_mutex_unlock . . . 135
5.7 qurt_mutex_lock_timed() . . . 136
5.7.1 Function Documentation . . . 136
5.7.1.1 qurt_mutex_lock_timed . . . 136
5.8 Data Types . . . 137
5.8.1 Typedef Documentation . . . 137
5.8.1.1 qurt_mutex_t . . . 137
5.8.1.2 qurt_rmutex2_t . . . 137
5.9 Constants and Macros . . . 138
5.9.1 Define Documentation . . . 138
5.9.1.1 MUTEX_MAGIC . . . 138
5.9.1.2 QURT_MUTEX_INIT . . . 138

**6 Recursive Mutexes 139**
6.1 Overview . . . 139
6.2 qurt_rmutex_destroy() . . . 140
6.2.1 Function Documentation . . . 140
6.2.1.1 qurt_rmutex_destroy . . . 140
6.3 qurt_rmutex_init() . . . 141
6.3.1 Function Documentation . . . 141
6.3.1.1 qurt_rmutex_init . . . 141
6.4 qurt_rmutex_lock() . . . 142
6.4.1 Function Documentation . . . 142
6.4.1.1 qurt_rmutex_lock . . . 142
6.5 qurt_rmutex_try_lock() . . . 143
6.5.1 Function Documentation . . . 143
6.5.1.1 qurt_rmutex_try_lock . . . 143
6.6 qurt_rmutex_unlock() . . . 144
6.6.1 Function Documentation . . . 144
6.6.1.1 qurt_rmutex_unlock . . . 144
6.7 qurt_rmutex_lock_timed() . . . 145
6.7.1 Function Documentation . . . 145
6.7.1.1 qurt_rmutex_lock_timed . . . 145
6.8 qurt_rmutex_try_lock_block_once() . . . 146
6.8.1 Function Documentation . . . 146
6.8.1.1 qurt_rmutex_try_lock_block_once . . . 146

**7 Priority Inheritance Mutexes 147**
7.1 Overview . . . 147
7.2 qurt_pimutex_destroy() . . . 148
7.2.1 Function Documentation . . . 148
7.2.1.1 qurt_pimutex_destroy . . . 148
7.3 qurt_pimutex_init() . . . 149

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hwungkul.won@lge.com

7.3.1 Function Documentation . . . 149
7.3.1.1 qurt_pimutex_init . . . 149
7.4 qurt_pimutex_lock() . . . 150
7.4.1 Function Documentation . . . 150
7.4.1.1 qurt_pimutex_lock . . . 150
7.5 qurt_pimutex_try_lock() . . . 151
7.5.1 Function Documentation . . . 151
7.5.1.1 qurt_pimutex_try_lock . . . 151
7.6 qurt_pimutex_unlock() . . . 152
7.6.1 Function Documentation . . . 152
7.6.1.1 qurt_pimutex_unlock . . . 152
7.7 func_qurt_pimutex_lock_timed() . . . 153
7.7.1 Function Documentation . . . 153
7.7.1.1 qurt_pimutex_lock_timed . . . 153

**8 Signals 154**
8.1 Overview . . . 154
8.2 qurt_signal_clear() . . . 156
8.2.1 Function Documentation . . . 156
8.2.1.1 qurt_signal_clear . . . 156
8.3 qurt_signal_destroy() . . . 157
8.3.1 Function Documentation . . . 157
8.3.1.1 qurt_signal_destroy . . . 157
8.4 qurt_signal_get() . . . 158
8.4.1 Function Documentation . . . 158
8.4.1.1 qurt_signal_get . . . 158
8.5 qurt_signal_init() . . . 159
8.5.1 Function Documentation . . . 159
8.5.1.1 qurt_signal_init . . . 159
8.6 qurt_signal_set() . . . 160
8.6.1 Function Documentation . . . 160
8.6.1.1 qurt_signal_set . . . 160
8.7 qurt_signal_wait() . . . 161
8.7.1 Function Documentation . . . 161
8.7.1.1 qurt_signal_wait . . . 161
8.8 qurt_signal_wait_all() . . . 162
8.8.1 Function Documentation . . . 162
8.8.1.1 qurt_signal_wait_all . . . 162
8.9 qurt_signal_wait_any() . . . 163
8.9.1 Function Documentation . . . 163
8.9.1.1 qurt_signal_wait_any . . . 163
8.10 qurt_signal_wait_cancellable() . . . 164
8.10.1 Function Documentation . . . 164
8.10.1.1 qurt_signal_wait_cancellable . . . 164
8.11 qurt_signal_wait_timed() . . . 165
8.11.1 Function Documentation . . . 165
8.11.1.1 qurt_signal_wait_timed . . . 165
8.12 qurt_signal_64_init() . . . 167
8.12.1 Function Documentation . . . 167
8.12.1.1 qurt_signal_64_init . . . 167

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungsoul.won@lge.com

8.13 qurt_signal_64_destroy() . . . 168
8.13.1 Function Documentation . . . 168
8.13.1.1 qurt_signal_64_destroy . . . 168
8.14 qurt_signal_64_wait() . . . 169
8.14.1 Function Documentation . . . 169
8.14.1.1 qurt_signal_64_wait . . . 169
8.15 qurt_signal_64_set() . . . 170
8.15.1 Function Documentation . . . 170
8.15.1.1 qurt_signal_64_set . . . 170
8.16 qurt_signal_64_get() . . . 171
8.16.1 Function Documentation . . . 171
8.16.1.1 qurt_signal_64_get . . . 171
8.17 qurt_signal_64_clear() . . . 172
8.17.1 Function Documentation . . . 172
8.17.1.1 qurt_signal_64_clear . . . 172
8.18 Data Types . . . 173
8.18.1 Define Documentation . . . 173
8.18.1.1 QURT_SIGNAL_ATTR_WAIT_ANY . . . 173
8.18.1.2 QURT_SIGNAL_ATTR_WAIT_ALL . . . 173
8.18.2 Data Structure Documentation . . . 173
8.18.2.1 union qurt_signal_t . . . 173
8.18.2.2 struct qurt_signal_64_t . . . 173

**9 Any-signals 174**
9.1 Overview . . . 174
9.2 qurt_anysignal_clear() . . . 175
9.2.1 Function Documentation . . . 175
9.2.1.1 qurt_anysignal_clear . . . 175
9.3 qurt_anysignal_destroy() . . . 176
9.3.1 Function Documentation . . . 176
9.3.1.1 qurt_anysignal_destroy . . . 176
9.4 qurt_anysignal_get() . . . 177
9.4.1 Function Documentation . . . 177
9.4.1.1 qurt_anysignal_get . . . 177
9.5 qurt_anysignal_init() . . . 178
9.5.1 Function Documentation . . . 178
9.5.1.1 qurt_anysignal_init . . . 178
9.6 qurt_anysignal_set() . . . 179
9.6.1 Function Documentation . . . 179
9.6.1.1 qurt_anysignal_set . . . 179
9.7 qurt_anysignal_wait() . . . 180
9.7.1 Function Documentation . . . 180
9.7.1.1 qurt_anysignal_wait . . . 180
9.8 qurt_anysignal_wait_timed() . . . 181
9.8.1 Function Documentation . . . 181
9.8.1.1 qurt_anysignal_wait_timed . . . 181
9.9 Data Types . . . 182
9.9.1 Typedef Documentation . . . 182
9.9.1.1 qurt_anysignal_t . . . 182

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hungchul.wong@qc.com

**10 All-signals** 183
10.1 Overview . . . 183
10.2 qurt_allsignal_destroy() . . . 184
10.2.1 Function Documentation . . . 184
10.2.1.1 qurt_allsignal_destroy . . . 184
10.3 qurt_allsignal_get() . . . 185
10.3.1 Function Documentation . . . 185
10.3.1.1 qurt_allsignal_get . . . 185
10.4 qurt_allsignal_init() . . . 186
10.4.1 Function Documentation . . . 186
10.4.1.1 qurt_allsignal_init . . . 186
10.5 qurt_allsignal_set() . . . 187
10.5.1 Function Documentation . . . 187
10.5.1.1 qurt_allsignal_set . . . 187
10.6 qurt_allsignal_wait() . . . 188
10.6.1 Function Documentation . . . 188
10.6.1.1 qurt_allsignal_wait . . . 188
10.7 Data Types . . . 189
10.7.1 Data Structure Documentation . . . 189
10.7.1.1 union qurt_allsignal_t . . . 189

**11 Semaphores** 190
11.1 Overview . . . 190
11.2 qurt_sem_add() . . . 192
11.2.1 Function Documentation . . . 192
11.2.1.1 qurt_sem_add . . . 192
11.3 qurt_sem_destroy() . . . 193
11.3.1 Function Documentation . . . 193
11.3.1.1 qurt_sem_destroy . . . 193
11.4 qurt_sem_down() . . . 194
11.4.1 Function Documentation . . . 194
11.4.1.1 qurt_sem_down . . . 194
11.5 qurt_sem_get_val() . . . 195
11.5.1 Function Documentation . . . 195
11.5.1.1 qurt_sem_get_val . . . 195
11.6 func_qurt_sem_down_cancellable() . . . 196
11.6.1 Function Documentation . . . 196
11.6.1.1 qurt_sem_down_cancellable . . . 196
11.7 qurt_sem_init() . . . 197
11.7.1 Function Documentation . . . 197
11.7.1.1 qurt_sem_init . . . 197
11.8 qurt_sem_init_val() . . . 198
11.8.1 Function Documentation . . . 198
11.8.1.1 qurt_sem_init_val . . . 198
11.9 qurt_sem_try_down() . . . 199
11.9.1 Function Documentation . . . 199
11.9.1.1 qurt_sem_try_down . . . 199
11.10 qurt_sem_up() . . . 200
11.10.1 Function Documentation . . . 200
11.10.1.1 qurt_sem_up . . . 200

Qualcomm Confidential - May Contain Trade Secrets 2026-02-22 05:40:42 GMT hyunsoul.won@lge.com

11.11 qurt_sem_down_timed() . . . 201
11.11.1 Function Documentation . . . 201
11.11.1.1 qurt_sem_down_timed . . . 201
11.12 Data Types . . . 202
11.12.1 Typedef Documentation . . . 202
11.12.1.1 qurt_sem_t . . . 202

**12 Barriers** **203**
12.1 Overview . . . 203
12.2 qurt_barrier_destroy() . . . 204
12.2.1 Function Documentation . . . 204
12.2.1.1 qurt_barrier_destroy . . . 204
12.3 qurt_barrier_init() . . . 205
12.3.1 Function Documentation . . . 205
12.3.1.1 qurt_barrier_init . . . 205
12.4 qurt_barrier_wait() . . . 206
12.4.1 Function Documentation . . . 206
12.4.1.1 qurt_barrier_wait . . . 206
12.5 Data Types . . . 207
12.5.1 Define Documentation . . . 207
12.5.1.1 QURT_BARRIER_SERIAL_THREAD . . . 207
12.5.1.2 QURT_BARRIER_OTHER . . . 207
12.5.2 Data Structure Documentation . . . 207
12.5.2.1 union qurt_barrier_t . . . 207

**13 Condition Variables** **208**
13.1 Overview . . . 208
13.2 qurt_cond_broadcast() . . . 210
13.2.1 Function Documentation . . . 210
13.2.1.1 qurt_cond_broadcast . . . 210
13.3 qurt_cond_destroy() . . . 211
13.3.1 Function Documentation . . . 211
13.3.1.1 qurt_cond_destroy . . . 211
13.4 qurt_cond_init() . . . 212
13.4.1 Function Documentation . . . 212
13.4.1.1 qurt_cond_init . . . 212
13.5 qurt_cond_signal() . . . 213
13.5.1 Function Documentation . . . 213
13.5.1.1 qurt_cond_signal . . . 213
13.6 qurt_cond_wait() . . . 214
13.6.1 Function Documentation . . . 214
13.6.1.1 qurt_cond_wait . . . 214
13.7 qurt_cond_wait2() . . . 215
13.7.1 Function Documentation . . . 215
13.7.1.1 qurt_cond_wait2 . . . 215
13.8 Data Types . . . 216
13.8.1 Data Structure Documentation . . . 216
13.8.1.1 union qurt_cond_t . . . 216

**14 Pipes** **217**

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyunchul.won@lge.com

14.1 Overview . . . 217
14.2 qurt_pipe_attr_init() . . . 219
14.2.1 Function Documentation . . . 219
14.2.1.1 qurt_pipe_attr_init . . . 219
14.3 qurt_pipe_attr_set_buffer() . . . 220
14.3.1 Function Documentation . . . 220
14.3.1.1 qurt_pipe_attr_set_buffer . . . 220
14.4 qurt_pipe_attr_set_buffer_partition() . . . 221
14.4.1 Function Documentation . . . 221
14.4.1.1 qurt_pipe_attr_set_buffer_partition . . . 221
14.5 qurt_pipe_attr_set_elements() . . . 222
14.5.1 Function Documentation . . . 222
14.5.1.1 qurt_pipe_attr_set_elements . . . 222
14.6 qurt_pipe_create() . . . 223
14.6.1 Function Documentation . . . 223
14.6.1.1 qurt_pipe_create . . . 223
14.7 qurt_pipe_delete() . . . 224
14.7.1 Function Documentation . . . 224
14.7.1.1 qurt_pipe_delete . . . 224
14.8 qurt_pipe_destroy() . . . 225
14.8.1 Function Documentation . . . 225
14.8.1.1 qurt_pipe_destroy . . . 225
14.9 qurt_pipe_init() . . . 226
14.9.1 Function Documentation . . . 226
14.9.1.1 qurt_pipe_init . . . 226
14.10 qurt_pipe_is_empty() . . . 227
14.10.1 Function Documentation . . . 227
14.10.1.1 qurt_pipe_is_empty . . . 227
14.11 qurt_pipe_receive() . . . 228
14.11.1 Function Documentation . . . 228
14.11.1.1 qurt_pipe_receive . . . 228
14.12 qurt_pipe_receive_cancellable() . . . 229
14.12.1 Function Documentation . . . 229
14.12.1.1 qurt_pipe_receive_cancellable . . . 229
14.13 qurt_pipe_send() . . . 230
14.13.1 Function Documentation . . . 230
14.13.1.1 qurt_pipe_send . . . 230
14.14 qurt_pipe_send_cancellable() . . . 231
14.14.1 Function Documentation . . . 231
14.14.1.1 qurt_pipe_send_cancellable . . . 231
14.15 qurt_pipe_try_receive() . . . 232
14.15.1 Function Documentation . . . 232
14.15.1.1 qurt_pipe_try_receive . . . 232
14.16 qurt_pipe_try_send() . . . 233
14.16.1 Function Documentation . . . 233
14.16.1.1 qurt_pipe_try_send . . . 233
14.17 Data Types . . . 234
14.17.1 Define Documentation . . . 234
14.17.1.1 QURT_PIPE_MAGIC . . . 234
14.17.1.2 QURT_PIPE_ATTR_MEM_PARTITION_RAM . . . 234

Qualcomm Confidential - May Contain Trade Secrets 2025-02-27 05:40:42 GMT hyungchul.won@lge.com

14.17.1.3 QURT_PIPE_ATTR_MEM_PARTITION_TCM . . . 234
14.17.2 Data Structure Documentation . . . 234
14.17.2.1 struct qurt_pipe_t . . . 234
14.17.2.2 struct qurt_pipe_attr_t . . . 234
14.17.3 Typedef Documentation . . . 234
14.17.3.1 qurt_pipe_data_t . . . 234

**15 Timers** **235**
15.1 Overview . . . 235
15.2 qurt_timer_attr_get_duration() . . . 238
15.2.1 Function Documentation . . . 238
15.2.1.1 qurt_timer_attr_get_duration . . . 238
15.3 qurt_timer_attr_get_group() . . . 239
15.3.1 Function Documentation . . . 239
15.3.1.1 qurt_timer_attr_get_group . . . 239
15.4 qurt_timer_attr_get_remaining() . . . 240
15.4.1 Function Documentation . . . 240
15.4.1.1 qurt_timer_attr_get_remaining . . . 240
15.5 qurt_timer_attr_get_type() . . . 241
15.5.1 Function Documentation . . . 241
15.5.1.1 qurt_timer_attr_get_type . . . 241
15.6 qurt_timer_attr_init() . . . 242
15.6.1 Function Documentation . . . 242
15.6.1.1 qurt_timer_attr_init . . . 242
15.7 qurt_timer_attr_set_duration() . . . 243
15.7.1 Function Documentation . . . 243
15.7.1.1 qurt_timer_attr_set_duration . . . 243
15.8 qurt_timer_attr_set_expiry() . . . 244
15.8.1 Function Documentation . . . 244
15.8.1.1 qurt_timer_attr_set_expiry . . . 244
15.9 qurt_timer_attr_set_group() . . . 245
15.9.1 Function Documentation . . . 245
15.9.1.1 qurt_timer_attr_set_group . . . 245
15.10 qurt_timer_attr_set_type() . . . 246
15.10.1 Function Documentation . . . 246
15.10.1.1 qurt_timer_attr_set_type . . . 246
15.11 qurt_timer_create() . . . 247
15.11.1 Function Documentation . . . 247
15.11.1.1 qurt_timer_create . . . 247
15.12 qurt_timer_delete() . . . 248
15.12.1 Function Documentation . . . 248
15.12.1.1 qurt_timer_delete . . . 248
15.13 qurt_timer_get_attr() . . . 249
15.13.1 Function Documentation . . . 249
15.13.1.1 qurt_timer_get_attr . . . 249
15.14 qurt_timer_group_disable() . . . 250
15.14.1 Function Documentation . . . 250
15.14.1.1 qurt_timer_group_disable . . . 250
15.15 qurt_timer_group_enable() . . . 251
15.15.1 Function Documentation . . . 251

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:10:42 GMT hyunchun.won@lge.com

15.15.1.1 qurt_timer_group_enable . . . 251
15.16 qurt_timer_restart() . . . 252
15.16.1 Function Documentation . . . 252
15.16.1.1 qurt_timer_restart . . . 252
15.17 qurt_timer_sleep() . . . 253
15.17.1 Function Documentation . . . 253
15.17.1.1 qurt_timer_sleep . . . 253
15.18 qurt_timer_stop() . . . 254
15.18.1 Function Documentation . . . 254
15.18.1.1 qurt_timer_stop . . . 254
15.19 Constants and Macros . . . 255
15.19.1 Define Documentation . . . 255
15.19.1.1 QURT_TIMER_DEFAULT_TYPE . . . 255
15.19.1.2 QURT_TIMER_DEFAULT_DURATION . . . 255
15.19.1.3 QURT_TIMER_DEFAULT_EXPIRY . . . 255
15.19.1.4 QURT_TIMER_TIMETICK_FROM_US . . . 255
15.19.1.5 QURT_TIMER_TIMETICK_TO_US . . . 255
15.19.1.6 QURT_TIMER_MIN_DURATION . . . 255
15.19.1.7 QURT_TIMER_MAX_DURATION . . . 255
15.19.1.8 QURT_TIMER_MAX_DURATION_TICKS . . . 255
15.19.1.9 QURT_TIMETICK_ERROR_MARGIN . . . 255
15.19.1.10 QURT_TIMER_MAX_GROUPS . . . 255
15.19.1.11 QURT_TIMER_DEFAULT_GROUP . . . 255
15.20 Data Types . . . 256
15.20.1 Data Structure Documentation . . . 256
15.20.1.1 struct qurt_timer_attr_t . . . 256
15.20.2 Typedef Documentation . . . 256
15.20.2.1 qurt_timer_t . . . 256
15.20.2.2 qurt_timer_duration_t . . . 256
15.20.2.3 qurt_timer_time_t . . . 256
15.20.3 Enumeration Type Documentation . . . 256
15.20.3.1 qurt_timer_type_t . . . 256

**16 System Clock** **257**
16.1 Overview . . . 257
16.2 qurt_sysclock_get_hw_ticks() . . . 258
16.2.1 Function Documentation . . . 258
16.2.1.1 qurt_sysclock_get_hw_ticks . . . 258
16.3 qurt_sysclock_get_hw_ticks_32() . . . 259
16.3.1 Function Documentation . . . 259
16.3.1.1 qurt_sysclock_get_hw_ticks_32 . . . 259
16.4 qurt_sysclock_get_hw_ticks_16() . . . 260
16.4.1 Function Documentation . . . 260
16.4.1.1 qurt_sysclock_get_hw_ticks_16 . . . 260

**17 Interrupts** **261**
17.1 Overview . . . 261
17.2 qurt_interrupt_acknowledge() . . . 263
17.2.1 Function Documentation . . . 263
17.2.1.1 qurt_interrupt_acknowledge . . . 263

QUALCOMM Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT kyungchul.won@lge.com

17.3 qurt_interrupt_clear() . . . 264
17.3.1 Function Documentation . . . 264
17.3.1.1 qurt_interrupt_clear . . . 264
17.4 qurt_interrupt_deregister() . . . 265
17.4.1 Function Documentation . . . 265
17.4.1.1 qurt_interrupt_deregister . . . 265
17.5 qurt_interrupt_enable() . . . 266
17.5.1 Function Documentation . . . 266
17.5.1.1 qurt_interrupt_enable . . . 266
17.6 qurt_interrupt_disable() . . . 267
17.6.1 Function Documentation . . . 267
17.6.1.1 qurt_interrupt_disable . . . 267
17.7 qurt_interrupt_get_config() . . . 268
17.7.1 Function Documentation . . . 268
17.7.1.1 qurt_interrupt_get_config . . . 268
17.8 qurt_interrupt_raise() . . . 269
17.8.1 Function Documentation . . . 269
17.8.1.1 qurt_interrupt_raise . . . 269
17.9 qurt_interrupt_register() . . . 270
17.9.1 Function Documentation . . . 270
17.9.1.1 qurt_interrupt_register . . . 270
17.10 qurt_interrupt_register2() . . . 272
17.10.1 Function Documentation . . . 272
17.10.1.1 qurt_interrupt_register2 . . . 272
17.11 qurt_interrupt_set_config() . . . 274
17.11.1 Function Documentation . . . 274
17.11.1.1 qurt_interrupt_set_config . . . 274
17.12 qurt_interrupt_set_config2() . . . 275
17.12.1 Function Documentation . . . 275
17.12.1.1 qurt_interrupt_set_config2 . . . 275
17.13 qurt_interrupt_set_config3() . . . 276
17.13.1 Function Documentation . . . 276
17.13.1.1 qurt_interrupt_set_config3 . . . 276
17.14 qurt_interrupt_status() . . . 277
17.14.1 Function Documentation . . . 277
17.14.1.1 qurt_interrupt_status . . . 277
17.15 qurt_interrupt_get_status() . . . 278
17.15.1 Function Documentation . . . 278
17.15.1.1 qurt_interrupt_get_status . . . 278
17.16 qurt_interrupt_raise2() . . . 279
17.16.1 Function Documentation . . . 279
17.16.1.1 qurt_interrupt_raise2 . . . 279
17.17 Constants . . . 280
17.17.1 Define Documentation . . . 280
17.17.1.1 SIG_INT_ABORT . . . 280
17.17.1.2 QURT_INT_NON_DELAYED_ACK . . . 280
17.17.1.3 QURT_INT_DELAYED_ACK . . . 280
17.17.1.4 QURT_INT_ACK_DEFAULT . . . 280
17.17.1.5 QURT_INT_DRV_DEFAULT . . . 280
17.17.1.6 QURT_INT_PRIORITY_DEFAULT . . . 280

QUALCOMM Confidential - May Contain Trade Secrets 2026-02-27 05:10:42 GMT hwanchulwon@lge.com

17.17.1.7 QURT_INT_CONFIGID_POLARITY . . . 280
17.17.1.8 QURT_INT_CONFIGID_LOCK . . . 280
17.17.1.9 QURT_INT_LOCK_DEFAULT . . . 280
17.17.1.10 QURT_INT_LOCK_DISABLE . . . 280
17.17.1.11 QURT_INT_LOCK_ENABLE . . . 280
17.18 Interrupt types . . . 281
17.18.1 Define Documentation . . . 281
17.18.1.1 QURT_INT_TRIGGER_TYPE_SET . . . 281
17.18.1.2 QURT_INT_TRIGGER_LEVEL_LOW . . . 281
17.18.1.3 QURT_INT_TRIGGER_LEVEL_HIGH . . . 281
17.18.1.4 QURT_INT_TRIGGER_RISING_EDGE . . . 281
17.18.1.5 QURT_INT_TRIGGER_FALLING_EDGE . . . 281
17.18.1.6 QURT_INT_TRIGGER_DUAL_EDGE . . . 281
17.18.1.7 QURT_INT_TRIGGER_USE_DEFAULT . . . 281

**18 Thread Local Storage 282**
18.1 Overview . . . 282
18.2 qurt_tls_create_key() . . . 283
18.2.1 Function Documentation . . . 283
18.2.1.1 qurt_tls_create_key . . . 283
18.3 qurt_tls_delete_key() . . . 284
18.3.1 Function Documentation . . . 284
18.3.1.1 qurt_tls_delete_key . . . 284
18.4 qurt_tls_get_specific() . . . 285
18.4.1 Function Documentation . . . 285
18.4.1.1 qurt_tls_get_specific . . . 285
18.5 qurt_tls_set_specific() . . . 286
18.5.1 Function Documentation . . . 286
18.5.1.1 qurt_tls_set_specific . . . 286

**19 Exception Handling 287**
19.1 Overview . . . 287
19.2 qurt_exception_enable_fp_exceptions() . . . 290
19.2.1 Function Documentation . . . 290
19.2.1.1 qurt_exception_enable_fp_exceptions . . . 290
19.3 qurt_exception_raise_nonfatal() . . . 291
19.3.1 Function Documentation . . . 291
19.3.1.1 qurt_exception_raise_nonfatal . . . 291
19.4 qurt_exception_raise_fatal() . . . 292
19.4.1 Function Documentation . . . 292
19.4.1.1 qurt_exception_raise_fatal . . . 292
19.5 qurt_enable_floating_point_exception() . . . 293
19.5.1 Function Documentation . . . 293
19.5.1.1 qurt_enable_floating_point_exception . . . 293
19.6 qurt_exception_wait() . . . 294
19.6.1 Function Documentation . . . 294
19.6.1.1 qurt_exception_wait . . . 294
19.7 qurt_exception_wait3() . . . 295
19.7.1 Function Documentation . . . 295
19.7.1.1 qurt_exception_wait3 . . . 295

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyunsuil.yoon@lge.com

19.8 qurt_exception_wait_ext() . . . 296
19.8.1 Function Documentation . . . 296
19.8.1.1 qurt_exception_wait_ext . . . 296
19.9 qurt_assert_error() . . . 297
19.9.1 Function Documentation . . . 297
19.9.1.1 qurt_assert_error . . . 297

**20 Memory Allocation 298**
20.1 Overview . . . 298
20.2 qurt_calloc() . . . 299
20.2.1 Function Documentation . . . 299
20.2.1.1 qurt_calloc . . . 299
20.3 qurt_free() . . . 300
20.3.1 Function Documentation . . . 300
20.3.1.1 qurt_free . . . 300
20.4 qurt_memalign() . . . 301
20.4.1 Function Documentation . . . 301
20.4.1.1 qurt_memalign . . . 301
20.5 qurt_malloc() . . . 302
20.5.1 Function Documentation . . . 302
20.5.1.1 qurt_malloc . . . 302
20.6 qurt_realloc() . . . 303
20.6.1 Function Documentation . . . 303
20.6.1.1 qurt_realloc . . . 303

**21 Memory Management 304**
21.1 Overview . . . 304
21.2 qurt_lookup_physaddr() . . . 309
21.2.1 Function Documentation . . . 309
21.2.1.1 qurt_lookup_physaddr . . . 309
21.3 qurt_lookup_physaddr2() . . . 310
21.3.1 Function Documentation . . . 310
21.3.1.1 qurt_lookup_physaddr2 . . . 310
21.4 qurt_lookup_physaddr_64() . . . 311
21.4.1 Function Documentation . . . 311
21.4.1.1 qurt_lookup_physaddr_64 . . . 311
21.5 qurt_mapping_create() . . . 312
21.5.1 Function Documentation . . . 312
21.5.1.1 qurt_mapping_create . . . 312
21.6 qurt_mapping_create_64() . . . 313
21.6.1 Function Documentation . . . 313
21.6.1.1 qurt_mapping_create_64 . . . 313
21.7 qurt_mapping_remove() . . . 314
21.7.1 Function Documentation . . . 314
21.7.1.1 qurt_mapping_remove . . . 314
21.8 qurt_mapping_remove_64() . . . 315
21.8.1 Function Documentation . . . 315
21.8.1.1 qurt_mapping_remove_64 . . . 315
21.9 qurt_mem_barrier() . . . 316
21.9.1 Function Documentation . . . 316

Qualcomm Confidential - May Contain Trade Secrets 2020-02-27 05:45:42 GMT hyungchul.yoon@lge.com

21.9.1.1 qurt_mem_barrier . . . 316
21.10 qurt_mem_cache_clean() . . . 317
21.10.1 Function Documentation . . . 317
21.10.1.1 qurt_mem_cache_clean . . . 317
21.11 qurt_mem_cache_clean2() . . . 318
21.11.1 Function Documentation . . . 318
21.11.1.1 qurt_mem_cache_clean2 . . . 318
21.12 qurt_mem_cache_phys_clean() . . . 319
21.12.1 Function Documentation . . . 319
21.12.1.1 qurt_mem_cache_phys_clean . . . 319
21.13 qurt_mem_configure_cache_partition() . . . 320
21.13.1 Function Documentation . . . 320
21.13.1.1 qurt_mem_configure_cache_partition . . . 320
21.14 qurt_mem_l2cache_line_lock() . . . 321
21.14.1 Function Documentation . . . 321
21.14.1.1 qurt_mem_l2cache_line_lock . . . 321
21.15 qurt_mem_l2cache_line_unlock() . . . 322
21.15.1 Function Documentation . . . 322
21.15.1.1 qurt_mem_l2cache_line_unlock . . . 322
21.16 qurt_mem_map_static_query() . . . 323
21.16.1 Function Documentation . . . 323
21.16.1.1 qurt_mem_map_static_query . . . 323
21.17 qurt_mem_map_static_query_64() . . . 324
21.17.1 Function Documentation . . . 324
21.17.1.1 qurt_mem_map_static_query_64 . . . 324
21.18 qurt_mem_pool_add_pages() . . . 325
21.18.1 Function Documentation . . . 325
21.18.1.1 qurt_mem_pool_add_pages . . . 325
21.19 qurt_mem_pool_attach() . . . 326
21.19.1 Function Documentation . . . 326
21.19.1.1 qurt_mem_pool_attach . . . 326
21.20 qurt_mem_pool_attach2() . . . 327
21.20.1 Function Documentation . . . 327
21.20.1.1 qurt_mem_pool_attach2 . . . 327
21.21 qurt_mem_pool_attr_get() . . . 328
21.21.1 Function Documentation . . . 328
21.21.1.1 qurt_mem_pool_attr_get . . . 328
21.22 qurt_mem_pool_attr_get_addr() . . . 329
21.22.1 Function Documentation . . . 329
21.22.1.1 qurt_mem_pool_attr_get_addr . . . 329
21.23 qurt_mem_pool_attr_get_addr_64() . . . 330
21.23.1 Function Documentation . . . 330
21.23.1.1 qurt_mem_pool_attr_get_addr_64 . . . 330
21.24 qurt_mem_pool_is_available() . . . 331
21.24.1 Function Documentation . . . 331
21.24.1.1 qurt_mem_pool_is_available . . . 331
21.25 qurt_mem_pool_attr_get_size() . . . 332
21.25.1 Function Documentation . . . 332
21.25.1.1 qurt_mem_pool_attr_get_size . . . 332
21.26 qurt_mem_pool_create() . . . 333

Qualcomm
Confidential - May Contain Trade Secrets
2025-02-27 15:40:42 GMT
hwangbui.won@lge.com

21.26.1 Function Documentation . . . 333
21.26.1.1 qurt_mem_pool_create . . . 333
21.27 qurt_mem_pool_remove_pages() . . . 334
21.27.1 Function Documentation . . . 334
21.27.1.1 qurt_mem_pool_remove_pages . . . 334
21.28 qurt_mem_region_attr_get() . . . 335
21.28.1 Function Documentation . . . 335
21.28.1.1 qurt_mem_region_attr_get . . . 335
21.29 qurt_mem_region_attr_get_bus_attr() . . . 336
21.29.1 Function Documentation . . . 336
21.29.1.1 qurt_mem_region_attr_get_bus_attr . . . 336
21.30 qurt_mem_region_attr_set_owner() . . . 337
21.30.1 Function Documentation . . . 337
21.30.1.1 qurt_mem_region_attr_set_owner . . . 337
21.31 qurt_mem_region_attr_set_perms() . . . 338
21.31.1 Function Documentation . . . 338
21.31.1.1 qurt_mem_region_attr_set_perms . . . 338
21.32 qurt_mem_region_attr_get_cache_mode() . . . 339
21.32.1 Function Documentation . . . 339
21.32.1.1 qurt_mem_region_attr_get_cache_mode . . . 339
21.33 qurt_mem_region_attr_get_mapping() . . . 340
21.33.1 Function Documentation . . . 340
21.33.1.1 qurt_mem_region_attr_get_mapping . . . 340
21.34 qurt_mem_region_attr_get_physaddr() . . . 341
21.34.1 Function Documentation . . . 341
21.34.1.1 qurt_mem_region_attr_get_physaddr . . . 341
21.35 qurt_mem_region_attr_get_size() . . . 342
21.35.1 Function Documentation . . . 342
21.35.1.1 qurt_mem_region_attr_get_size . . . 342
21.36 qurt_mem_region_attr_get_type() . . . 343
21.36.1 Function Documentation . . . 343
21.36.1.1 qurt_mem_region_attr_get_type . . . 343
21.37 qurt_mem_region_attr_get_virtaddr() . . . 344
21.37.1 Function Documentation . . . 344
21.37.1.1 qurt_mem_region_attr_get_virtaddr . . . 344
21.38 qurt_mem_region_attr_get_physaddr_64() . . . 345
21.38.1 Function Documentation . . . 345
21.38.1.1 qurt_mem_region_attr_get_physaddr_64 . . . 345
21.39 qurt_mem_region_attr_init() . . . 346
21.39.1 Function Documentation . . . 346
21.39.1.1 qurt_mem_region_attr_init . . . 346
21.40 qurt_mem_region_attr_set_bus_attr() . . . 347
21.40.1 Function Documentation . . . 347
21.40.1.1 qurt_mem_region_attr_set_bus_attr . . . 347
21.41 qurt_mem_region_attr_set_cache_mode() . . . 348
21.41.1 Function Documentation . . . 348
21.41.1.1 qurt_mem_region_attr_set_cache_mode . . . 348
21.42 qurt_mem_region_attr_set_mapping() . . . 349
21.42.1 Function Documentation . . . 349
21.42.1.1 qurt_mem_region_attr_set_mapping . . . 349

QUALCOMM Confidential - May Contain Trade Secrets 2022-02-17 05:40:12 GMT hungchun.lin@lge.com

21.43qurt_mem_region_attr_set_physaddr() . . . 350
21.43.1 Function Documentation . . . 350
21.43.1.1 qurt_mem_region_attr_set_physaddr . . . 350
21.44qurt_mem_region_attr_set_physaddr_64() . . . 351
21.44.1 Function Documentation . . . 351
21.44.1.1 qurt_mem_region_attr_set_physaddr_64 . . . 351
21.45qurt_mem_region_attr_set_type() . . . 352
21.45.1 Function Documentation . . . 352
21.45.1.1 qurt_mem_region_attr_set_type . . . 352
21.46qurt_mem_region_attr_set_virtaddr() . . . 353
21.46.1 Function Documentation . . . 353
21.46.1.1 qurt_mem_region_attr_set_virtaddr . . . 353
21.47qurt_mem_region_create() . . . 354
21.47.1 Function Documentation . . . 354
21.47.1.1 qurt_mem_region_create . . . 354
21.48qurt_mem_region_delete() . . . 356
21.48.1 Function Documentation . . . 356
21.48.1.1 qurt_mem_region_delete . . . 356
21.49qurt_mem_region_query() . . . 357
21.49.1 Function Documentation . . . 357
21.49.1.1 qurt_mem_region_query . . . 357
21.50qurt_mem_region_query_64() . . . 358
21.50.1 Function Documentation . . . 358
21.50.1.1 qurt_mem_region_query_64 . . . 358
21.51qurt_mem_syncht() . . . 359
21.51.1 Function Documentation . . . 359
21.51.1.1 qurt_mem_syncht . . . 359
21.52qurt_mapping_attr_get() . . . 360
21.52.1 Function Documentation . . . 360
21.52.1.1 qurt_mapping_attr_get . . . 360
21.53qurt_mapping_attr_get_cache_mode() . . . 361
21.53.1 Function Documentation . . . 361
21.53.1.1 qurt_mapping_attr_get_cache_mode . . . 361
21.54qurt_mapping_attr_get_physaddr() . . . 362
21.54.1 Function Documentation . . . 362
21.54.1.1 qurt_mapping_attr_get_physaddr . . . 362
21.55qurt_mapping_attr_get_perms() . . . 363
21.55.1 Function Documentation . . . 363
21.55.1.1 qurt_mapping_attr_get_perms . . . 363
21.56qurt_mapping_attr_get_size() . . . 364
21.56.1 Function Documentation . . . 364
21.56.1.1 qurt_mapping_attr_get_size . . . 364
21.57qurt_mapping_attr_get_64() . . . 365
21.57.1 Function Documentation . . . 365
21.57.1.1 qurt_mapping_attr_get_64 . . . 365
21.58qurt_physmem_get_addr_width() . . . 366
21.58.1 Function Documentation . . . 366
21.58.1.1 qurt_physmem_get_addr_width . . . 366
21.59qurt_physmem_encode_page() . . . 367
21.59.1 Function Documentation . . . 367

Qualcomm Confidential - May Contain Trade Secrets 2025-02-27 07:00:42 GMT jungwon.lawon@lge.com

21.59.1.1 qurt_physmem_encode_page . . . 367
21.60 qurt_physmem_decode_page() . . . 368
21.60.1 Function Documentation . . . 368
21.60.1.1 qurt_physmem_decode_page . . . 368
21.61 qurt_mem_preempt_virtual_memory() . . . 369
21.61.1 Function Documentation . . . 369
21.61.1.1 qurt_mem_preempt_virtual_memory . . . 369
21.62 qurt_mem_resume_virtual_memory() . . . 370
21.62.1 Function Documentation . . . 370
21.62.1.1 qurt_mem_resume_virtual_memory . . . 370
21.63 Memory Management Data Types . . . 371
21.63.1 Define Documentation . . . 371
21.63.1.1 QURT_POOL_REMOVE_ALL_OR_NONE . . . 371
21.63.1.2 QURT_MEM_CACHE_WRITEBACK_NONL2CACHEABLE . . . 371
21.63.1.3 QURT_MEM_CACHE_WRITETHROUGH_NONL2CACHEABLE . . . 371
21.63.1.4 QURT_MEM_CACHE_DEVICE_SFC . . . 371
21.63.1.5 QURT_MEM_CACHE_DEVICE . . . 371
21.63.1.6 QURT_MEM_CACHE_WRITETHROUGH . . . 371
21.63.1.7 QURT_MEM_CACHE_NONE_SHARED . . . 371
21.63.1.8 QURT_MEM_CACHE_WRITEBACK . . . 371
21.63.1.9 QURT_MEM_CACHE_INVALID . . . 371
21.63.1.10 QURT_MEM_CACHE_WRITEBACK_L2CACHEABLE . . . 372
21.63.1.11 QURT_MEM_CACHE_WRITETHROUGH_L2CACHEABLE . . . 372
21.63.1.12 QURT_MEM_CACHE_NONE . . . 372
21.63.2 Data Structure Documentation . . . 372
21.63.2.1 struct qurt_mem_region_attr_t . . . 372
21.63.2.2 struct qurt_mem_pool_attr_t . . . 372
21.63.2.3 struct qurt_mem_pool_status_t . . . 372
21.63.2.4 struct qurt_mem_physpool_attr64_t . . . 372
21.63.3 Typedef Documentation . . . 372
21.63.3.1 qurt_addr_t . . . 372
21.63.3.2 qurt_paddr_t . . . 372
21.63.3.3 qurt_addr_64_t . . . 372
21.63.3.4 qurt_paddr_64_t . . . 373
21.63.3.5 qurt_size_64_t . . . 373
21.63.3.6 qurt_mem_region_t . . . 373
21.63.3.7 qurt_mem_fs_region_t . . . 373
21.63.3.8 qurt_mem_pool_t . . . 373
21.63.3.9 qurt_size_t . . . 373
21.63.3.10 qurt_mem_cache_mode_t . . . 373
21.63.3.11 qurt_perm_t . . . 373
21.63.4 Enumeration Type Documentation . . . 373
21.63.4.1 qurt_mem_mapping_t . . . 373
21.63.4.2 qurt_mem_cache_type_t . . . 374
21.63.4.3 qurt_mem_cache_op_t . . . 374
21.63.4.4 qurt_mem_region_type_t . . . 374
21.63.4.5 qurt_cache_type_t . . . 374
21.63.4.6 qurt_cache_partition_size_t . . . 374
21.63.5 Variable Documentation . . . 375
21.63.5.1 qurt_mem_default_pool . . . 375


Qualcomm Confidential - May Contain Trade Secrets 2022-03-27 05:40:07 GMT hwangbui.won@lge.com

21.64 Macros . . . 376
21.64.1 Define Documentation . . . 376
21.64.1.1 QURT_SYSTEM_ALLOC_VIRTUAL . . . 376

**22 Memory Mapping 377**
22.1 qurt_mem_mmap() . . . 378
22.1.1 Function Documentation . . . 378
22.1.1.1 qurt_mem_mmap . . . 378
22.2 qurt_mem_mmap2() . . . 379
22.2.1 Function Documentation . . . 379
22.2.1.1 qurt_mem_mmap2 . . . 379
22.3 qurt_mem_mmap_by_name() . . . 380
22.3.1 Function Documentation . . . 380
22.3.1.1 qurt_mem_mmap_by_name . . . 380
22.4 qurt_mem_mprotect2() . . . 381
22.4.1 Function Documentation . . . 381
22.4.1.1 qurt_mem_mprotect2 . . . 381
22.5 qurt_mem_mprotect() . . . 382
22.5.1 Function Documentation . . . 382
22.5.1.1 qurt_mem_mprotect . . . 382
22.6 qurt_mem_munmap() . . . 383
22.6.1 Function Documentation . . . 383
22.6.1.1 qurt_mem_munmap . . . 383
22.7 qurt_mem_munmap2() . . . 384
22.7.1 Function Documentation . . . 384
22.7.1.1 qurt_mem_munmap2 . . . 384
22.8 qurt_mem_munmap3() . . . 385
22.8.1 Function Documentation . . . 385
22.8.1.1 qurt_mem_munmap3 . . . 385
22.9 qurt_mem_pool_status_get() . . . 386
22.9.1 Function Documentation . . . 386
22.9.1.1 qurt_mem_pool_status_get . . . 386
22.10 Memory Mapping Macros . . . 387
22.10.1 Define Documentation . . . 387
22.10.1.1 QURT_MAP_NAMED_MEMSECTION . . . 387
22.10.1.2 QURT_MAP_FIXED . . . 387
22.10.1.3 QURT_MAP_RENAME . . . 387
22.10.1.4 QURT_MAP_NORESERVE . . . 387
22.10.1.5 QURT_MAP_INHERIT . . . 387
22.10.1.6 QURT_MAP_NONPROCESS_VPOOL . . . 387
22.10.1.7 QURT_MAP_HASSEMAPHORE . . . 387
22.10.1.8 QURT_MAP_TRYFIXED . . . 387
22.10.1.9 QURT_MAP_WIRED . . . 387
22.10.1.10 QURT_MAP_FILE . . . 387
22.10.1.11 QURT_MAP_ANON . . . 387
22.10.1.12 QURT_MAP_VA_ONLY . . . 387
22.10.1.13 QURT_MAP_FAR_ADDR . . . 388
22.10.1.14 QURT_MAP_FAILED . . . 388
22.10.1.15 QURT_PROT_CACHE_MODE . . . 388
22.10.1.16 QURT_PROT_BUS_ATTR . . . 388

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyunchul.woo@lge.com

22.10.1.17 QURT_PROT_USER_MODE . . . 388
22.10.1.18 QURT_MAP_PHYSADDR . . . 388
22.10.1.19 QURT_MAP_TYPE . . . 388
22.10.1.20 QURT_MAP_REGION . . . 388
22.11 Memory Mapping Types . . . 389
22.11.1 Data Structure Documentation . . . 389
22.11.1.1 struct qurt_mapping_attr_t . . . 389
22.11.1.2 struct qurt_mapping_attr64_t . . . 389

**23 System Environment** **390**
23.1 Overview . . . 390
23.2 qurt_sysenv_get_app_heap() . . . 391
23.2.1 Function Documentation . . . 391
23.2.1.1 qurt_sysenv_get_app_heap . . . 391
23.3 qurt_sysenv_get_arch_version() . . . 392
23.3.1 Function Documentation . . . 392
23.3.1.1 qurt_sysenv_get_arch_version . . . 392
23.4 qurt_sysenv_get_max_hw_threads() . . . 393
23.4.1 Function Documentation . . . 393
23.4.1.1 qurt_sysenv_get_max_hw_threads . . . 393
23.5 qurt_sysenv_get_max_pi_prio() . . . 394
23.5.1 Function Documentation . . . 394
23.5.1.1 qurt_sysenv_get_max_pi_prio . . . 394
23.6 qurt_sysenv_get_process_name2() . . . 395
23.6.1 Function Documentation . . . 395
23.6.1.1 qurt_sysenv_get_process_name2 . . . 395
23.7 qurt_sysenv_get_process_name() . . . 396
23.7.1 Function Documentation . . . 396
23.7.1.1 qurt_sysenv_get_process_name . . . 396
23.8 qurt_sysenv_get_stack_profile_count() . . . 397
23.8.1 Function Documentation . . . 397
23.8.1.1 qurt_sysenv_get_stack_profile_count . . . 397
23.9 qurt_sysenv_get_instance_id() . . . 398
23.9.1 Function Documentation . . . 398
23.9.1.1 qurt_sysenv_get_instance_id . . . 398
23.10 qurt_sysenv_get_hw_threads() . . . 399
23.10.1 Function Documentation . . . 399
23.10.1.1 qurt_sysenv_get_hw_threads . . . 399
23.11 Data Types . . . 400
23.11.1 Data Structure Documentation . . . 400
23.11.1.1 struct qurt_sysenv_swap_pools_t . . . 400
23.11.1.2 struct qurt_sysenv_app_heap_t . . . 400
23.11.1.3 struct qurt_arch_version_t . . . 400
23.11.1.4 struct qurt_sysenv_max_hthreads_t . . . 400
23.11.1.5 struct qurt_sysenv_hthreads_t . . . 400
23.11.1.6 struct qurt_sysenv_max_pi_prio_t . . . 400
23.11.1.7 struct qurt_sysenv_procname_t . . . 400
23.11.1.8 struct qurt_sysenv_stack_profile_count_t . . . 400
23.11.1.9 struct qurt_sysevent_error_t . . . 400
23.11.1.10 struct qurt_sysevent_error_1_t . . . 401

Qualcomm Confidential - May Contain Trade Secrets 2026-03-27 05:10:52 GMT yunhyeoul.woo@lge.com

23.11.1.1 struct qurt_sysevent_pagefault_t . . . 401

**24 Profiling 402**
24.1 Overview . . . 402
24.2 qurt_get_core_pcycles() . . . 405
24.2.1 Function Documentation . . . 405
24.2.1.1 qurt_get_core_pcycles . . . 405
24.3 qurt_profile_enable() . . . 406
24.3.1 Function Documentation . . . 406
24.3.1.1 qurt_profile_enable . . . 406
24.4 qurt_profile_enable2() . . . 407
24.4.1 Function Documentation . . . 407
24.4.1.1 qurt_profile_enable2 . . . 407
24.5 qurt_profile_get() . . . 408
24.5.1 Function Documentation . . . 408
24.5.1.1 qurt_profile_get . . . 408
24.6 qurt_profile_get_idle_pcycles() . . . 409
24.6.1 Function Documentation . . . 409
24.6.1.1 qurt_profile_get_idle_pcycles . . . 409
24.7 qurt_profile_get_thread_pcycles() . . . 410
24.7.1 Function Documentation . . . 410
24.7.1.1 qurt_profile_get_thread_pcycles . . . 410
24.8 qurt_get_hthread_pcycles() . . . 411
24.8.1 Function Documentation . . . 411
24.8.1.1 qurt_get_hthread_pcycles . . . 411
24.9 qurt_get_hthread_commits() . . . 412
24.9.1 Function Documentation . . . 412
24.9.1.1 qurt_get_hthread_commits . . . 412
24.10 qurt_profile_get_threadid_pcycles() . . . 413
24.10.1 Function Documentation . . . 413
24.10.1.1 qurt_profile_get_threadid_pcycles . . . 413
24.11 qurt_profile_reset_idle_pcycles() . . . 414
24.11.1 Function Documentation . . . 414
24.11.1.1 qurt_profile_reset_idle_pcycles . . . 414
24.12 qurt_profile_reset_threadid_pcycles() . . . 415
24.12.1 Function Documentation . . . 415
24.12.1.1 qurt_profile_reset_threadid_pcycles . . . 415
24.13 Data Types . . . 416
24.13.1 Data Structure Documentation . . . 416
24.13.1.1 union qurt_profile_result_t . . . 416
24.13.1.2 struct qurt_profile_result_t.thread_ready_time . . . 416
24.14 Constants and Macros . . . 417
24.14.1 Define Documentation . . . 417
24.14.1.1 QURT_PROFILE_DISABLE . . . 417
24.14.1.2 QURT_PROFILE_ENABLE . . . 417
24.14.1.3 QURT_PROFILE_PARAM_THREAD_READY_TIME . . . 417
24.14.1.4 QURT_PROFILE_REG_GROUP_CYCLE_NT . . . 417
24.14.1.5 QURT_PROFILE_REG_GROUP_COMMIT_NT . . . 417

**25 Device tree 418**

Qualcomm
Confidential - May Contain Trade Secrets
2025-05-27 05:40:42 GMT
huichul.wong@qc.com

25.1 Overview . . . 418
25.1.1 QuRT Kernel Locating DT image . . . 419
25.1.2 Read the Registers . . . 420
25.1.3 QDI and Device Tree . . . 420
25.1.4 Device Tree Overlay . . . 422
25.1.5 Supported API . . . 422
25.1.6 FAQ's . . . 422
25.2 qurt_devtree_init() . . . 424
25.2.1 Function Documentation . . . 424
25.2.1.1 qurt_devtree_init . . . 424
25.3 qurt_dt_mapping_create() . . . 425
25.3.1 Function Documentation . . . 425
25.3.1.1 qurt_dt_mapping_create . . . 425
25.4 qurt_dt_isr_register() . . . 426
25.4.1 Function Documentation . . . 426
25.4.1.1 qurt_dt_isr_register . . . 426
25.5 qurt_dt_blob_id_get() . . . 428
25.5.1 Function Documentation . . . 428
25.5.1.1 qurt_dt_blob_id_get . . . 428
25.6 Constants and Macros . . . 429
25.6.1 Define Documentation . . . 429
25.6.1.1 DEFAULT_BLOB_ID . . . 429
25.6.1.2 QURT_DT_MAPPING_FAILED . . . 429
25.6.1.3 QURT_DT_BLOB_TYPE_ROOT . . . 429
25.6.1.4 QURT_DT_BLOB_TYPE_LOCAL . . . 429

**26 Performance Monitor 430**
26.1 Overview . . . 430
26.2 qurt_pmu_enable() . . . 431
26.2.1 Function Documentation . . . 431
26.2.1.1 qurt_pmu_enable . . . 431
26.3 qurt_pmu_get() . . . 432
26.3.1 Function Documentation . . . 432
26.3.1.1 qurt_pmu_get . . . 432
26.4 qurt_pmu_set() . . . 433
26.4.1 Function Documentation . . . 433
26.4.1.1 qurt_pmu_set . . . 433
26.5 qurt_pmu_get_pmucnt() . . . 434
26.5.1 Function Documentation . . . 434
26.5.1.1 qurt_pmu_get_pmucnt . . . 434
26.6 Constants and Macros . . . 435
26.6.1 Define Documentation . . . 435
26.6.1.1 QURT_PMUCNT0 . . . 435
26.6.1.2 QURT_PMUCNT1 . . . 435
26.6.1.3 QURT_PMUCNT2 . . . 435
26.6.1.4 QURT_PMUCNT3 . . . 435
26.6.1.5 QURT_PMUCFG . . . 435
26.6.1.6 QURT_PMUEVTCFG . . . 435
26.6.1.7 QURT_PMUCNT4 . . . 435
26.6.1.8 QURT_PMUCNT5 . . . 435

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hungchul.won@lge.com

26.6.1.9 QURT_PMUCNT6 . . . 435
26.6.1.10 QURT_PMUCNT7 . . . 435
26.6.1.11 QURT_PMUEVTCFG1 . . . 435
26.6.1.12 QURT_PMUSTID0 . . . 435
26.6.1.13 QURT_PMUSTID1 . . . 435
26.6.1.14 QURT_PMUCNTSTID0 . . . 435
26.6.1.15 QURT_PMUCNTSTID1 . . . 435
26.6.1.16 QURT_PMUCNTSTID2 . . . 435
26.6.1.17 QURT_PMUCNTSTID3 . . . 435
26.6.1.18 QURT_PMUCNTSTID4 . . . 435
26.6.1.19 QURT_PMUCNTSTID5 . . . 435
26.6.1.20 QURT_PMUCNTSTID6 . . . 435
26.6.1.21 QURT_PMUCNTSTID7 . . . 435

**27 Error Results** **436**
27.0.1 Define Documentation . . . 436
27.0.1.1 QURT_EOK . . . 436
27.0.1.2 QURT_EVAL . . . 436
27.0.1.3 QURT_EMEM . . . 436
27.0.1.4 QURT_EINVALID . . . 436
27.0.1.5 QURT_EFAILED . . . 436
27.0.1.6 QURT_ENOTALLOWED . . . 436
27.0.1.7 QURT_ENOREGISTERED . . . 436
27.0.1.8 QURT_ETLSAVAIL . . . 436
27.0.1.9 QURT_ETLSENTRY . . . 436
27.0.1.10 QURT_EINT . . . 437
27.0.1.11 QURT_ESIG . . . 437
27.0.1.12 QURT_ENOTHREAD . . . 437
27.0.1.13 QURT_EALIGN . . . 437
27.0.1.14 QURT_EDEREGISTERED . . . 437
27.0.1.15 QURT_EEXISTS . . . 437
27.0.1.16 QURT_ENAMETOOLONG . . . 437
27.0.1.17 QURT_EPRIVILEGE . . . 437
27.0.1.18 QURT_ECANCEL . . . 437
27.0.1.19 QURT_EISLANDUSEREXIT . . . 437
27.0.1.20 QURT_ENOISLANDENTRY . . . 437
27.0.1.21 QURT_EISLANDINVALIDINT . . . 437
27.0.1.22 QURT_ETIMEDOUT . . . 437
27.0.1.23 QURT_EALREADY . . . 437
27.0.1.24 QURT_ERETRY . . . 437
27.0.1.25 QURT_EDISABLED . . . 438
27.0.1.26 QURT_EDUPLICATE . . . 438
27.0.1.27 QURT_EBADR . . . 438
27.0.1.28 QURT_ETLB . . . 438
27.0.1.29 QURT_ENOTSUPPORTED . . . 438
27.0.1.30 QURT_ENORESOURCE . . . 438
27.0.1.31 QURT_EDTINIT . . . 438
27.0.1.32 QURT_EBUFLOCK . . . 438
27.0.1.33 QURT_ELOCKED . . . 438
27.0.1.34 QURT_ECLOSED . . . 438

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-25 15:49:42 GMT
rajkumar@lge.com

27.0.1.35 QURT_EMSGSIZE . . . 438
27.0.1.36 QURT_ENOTCONFIGURED . . . 438
27.0.1.37 QURT_EBANDWIDTHLIMIT . . . 438
27.0.1.38 QURT_ECFIVIOLATION . . . 438
27.0.1.39 QURT_EDESTROY . . . 438
27.0.1.40 QURT_EHMXNOTAVAIL . . . 439
27.0.1.41 QURT_ETIMERADJUST . . . 439
27.0.1.42 QURT_EFATAL . . . 439
27.0.1.43 QURT_EXCEPT_PRECISE . . . 439
27.0.1.44 QURT_EXCEPT_NMI . . . 439
27.0.1.45 QURT_EXCEPT_TLBMISS . . . 439
27.0.1.46 QURT_EXCEPT_RSVD_VECTOR . . . 439
27.0.1.47 QURT_EXCEPT_ASSERT . . . 439
27.0.1.48 QURT_EXCEPT_BADTRAP . . . 439
27.0.1.49 QURT_EXCEPT_UNDEF_TRAP1 . . . 439
27.0.1.50 QURT_EXCEPT_EXIT . . . 439
27.0.1.51 QURT_EXCEPT_TLBMISS_X . . . 439
27.0.1.52 QURT_EXCEPT_STOPPED . . . 439
27.0.1.53 QURT_EXCEPT_FATAL_EXIT . . . 439
27.0.1.54 QURT_EXCEPT_INVALID_INT . . . 440
27.0.1.55 QURT_EXCEPT_FLOATING_POINT . . . 440
27.0.1.56 QURT_EXCEPT_DBG_SINGLE_STEP . . . 440
27.0.1.57 QURT_EXCEPT_TLBMISS_RW_ISLAND . . . 440
27.0.1.58 QURT_EXCEPT_TLBMISS_X_ISLAND . . . 440
27.0.1.59 QURT_EXCEPT_SYNTHETIC_FAULT . . . 440
27.0.1.60 QURT_EXCEPT_INVALID_ISLAND_TRAP . . . 440
27.0.1.61 QURT_EXCEPT_UNDEF_TRAP0 . . . 440
27.0.1.62 QURT_EXCEPT_PRECISE_DMA_ERROR . . . 440
27.0.1.63 QURT_EXCEPT_KERNEL_STACK_CORRUPT . . . 440
27.0.1.64 QURT_ECODE_UPPER_LIBC . . . 440
27.0.1.65 QURT_ECODE_UPPER_QURT . . . 440
27.0.1.66 QURT_ECODE_UPPER_ERR_SERVICES . . . 440
27.0.1.67 QURT_SYNTH_ERR . . . 440
27.0.1.68 QURT_SYNTH_INVALID_OP . . . 440
27.0.1.69 QURT_SYNTH_DATA_ALIGNMENT_FAULT . . . 440
27.0.1.70 QURT_SYNTH_FUTEX_INUSE . . . 441
27.0.1.71 QURT_SYNTH_FUTEX_BOGUS . . . 441
27.0.1.72 QURT_SYNTH_FUTEX_ISLAND . . . 441
27.0.1.73 QURT_SYNTH_FUTEX_DESTROYED . . . 441
27.0.1.74 QURT_SYNTH_PRIVILEGE_ERR . . . 441
27.0.1.75 QURT_ABORT_FUTEX_WAKE_MULTIPLE . . . 441
27.0.1.76 QURT_ABORT_WAIT_WAKEUP_SINGLE_MODE . . . 441
27.0.1.77 QURT_ABORT_TCXO_SHUTDOWN_NOEXIT . . . 441
27.0.1.78 QURT_ABORT_FUTEX_ALLOC_QUEUE_FAIL . . . 441
27.0.1.79 QURT_ABORT_INVALID_CALL_QURTK_WARM_INIT . . . 441
27.0.1.80 QURT_ABORT_THREAD_SCHEDULE_SANITY . . . 441
27.0.1.81 QURT_ABORT_REMAP . . . 441
27.0.1.82 QURT_ABORT_NOMAP . . . 441
27.0.1.83 QURT_ABORT_INVALID_MEM_MAPPING_TYPE . . . 441
27.0.1.84 QURT_ABORT_NOPOOL . . . 441

Qualcomm Confidential - May Contain Trade Secrets
Please Contact Qualcomm Legal
www.qualcomm.com

27.0.1.85 QURT_ABORT_LIFO_REMOVE_NON_EXIST_ITEM . . . . . 441
27.0.1.86 QURT_ABORT_ASSERT . . . . . 441
27.0.1.87 QURT_ABORT_FATAL . . . . . 442
27.0.1.88 QURT_ABORT_FUTEX_RESUME_INVALID_QUEUE . . . . . 442
27.0.1.89 QURT_ABORT_FUTEX_WAIT_INVALID_QUEUE . . . . . 442
27.0.1.90 QURT_ABORT_FUTEX_RESUME_INVALID_FUTEX . . . . . 442
27.0.1.91 QURT_ABORT_NO_ERHNDLR . . . . . 442
27.0.1.92 QURT_ABORT_ERR_REAPER . . . . . 442
27.0.1.93 QURT_ABORT_FREEZE_UNKNOWN_CAUSE . . . . . 442
27.0.1.94 QURT_ABORT_FUTEX_WAIT_WRITE_FAILURE . . . . . 442
27.0.1.95 QURT_ABORT_ERR_ISLAND_EXP_HANDLER . . . . . 442
27.0.1.96 QURT_ABORT_L2_TAG_DATA_CHECK_FAIL . . . . . 442
27.0.1.97 QURT_ABORT_ERR_SECURE_PROCESS . . . . . 442
27.0.1.98 QURT_ABORT_ERR_EXP_HANDLER . . . . . 442
27.0.1.99 QURT_ABORT_ERR_NO_PCB . . . . . 442
27.0.1.100 QURT_ABORT_NO_PHYS_ADDR . . . . . 442
27.0.1.101 QURT_ABORT_OUT_OF_FASTINT_CONTEXTS . . . . . 443
27.0.1.102 QURT_ABORT_CLADE_ERR . . . . . 443
27.0.1.103 QURT_ABORT_ETM_ERR . . . . . 443
27.0.1.104 QURT_ABORT_ECC_DED_ASSERT . . . . . 443
27.0.1.105 QURT_ABORT_VTLB_ERR . . . . . 443
27.0.1.106 QURT_ABORT_TLB_ENCODE_DECODE_FAILURE . . . . . 443
27.0.1.107 QURT_ABORT_VTLB_WALKOBJS_BOUND_FAILURE . . . . . 443
27.0.1.108 QURT_ABORT_PHY_MEMORY_OWNERSHIP_FAILURE . . . . . 443
27.0.1.109 QURT_ABORT_JTLB_SIZE_CHECK_FAIL . . . . . 443
27.0.1.110 QURT_ABORT_AUTOSTACK_ASSERT . . . . . 443
27.0.1.111 QURT_TLB_MISS_X_FETCH_PC_PAGE . . . . . 443
27.0.1.112 QURT_TLB_MISS_X_2ND_PAGE . . . . . 443
27.0.1.113 QURT_TLB_MISS_X_ICINVA . . . . . 443
27.0.1.114 QURT_TLB_MISS_RW_MEM_READ . . . . . 443
27.0.1.115 QURT_TLB_MISS_RW_MEM_WRITE . . . . . 443
27.0.1.116 QURT_FLOATING_POINT_EXEC_ERR . . . . . 443
27.0.1.117 QURT_AUTOSTACKV2_CANARY_NOT_MATCH . . . . . 443
27.0.1.118 QURT_CFI_VIOLATION . . . . . 444
27.0.1.119 QURT_FP_EXCEPTION_ALL . . . . . 444
27.0.1.120 QURT_FP_EXCEPTION_INEXACT . . . . . 444
27.0.1.121 QURT_FP_EXCEPTION_UNDERFLOW . . . . . 444
27.0.1.122 QURT_FP_EXCEPTION_OVERFLOW . . . . . 444
27.0.1.123 QURT_FP_EXCEPTION_DIVIDE0 . . . . . 444
27.0.1.124 QURT_FP_EXCEPTION_INVALID . . . . . 444

**28 Function Tracing** **445**
28.1 qurt_trace_changed() . . . . . 446
28.1.1 Function Documentation . . . . . 446
28.1.1.1 qurt_trace_changed . . . . . 446
28.2 qurt_trace_get_marker() . . . . . 447
28.2.1 Function Documentation . . . . . 447
28.2.1.1 qurt_trace_get_marker . . . . . 447
28.3 qurt_etm_set_pc_range() . . . . . 448
28.3.1 Function Documentation . . . . . 448

28.3.1.1 qurt_etm_set_pc_range . . . 448
28.4 qurt_etm_set_range() . . . 449
28.4.1 Function Documentation . . . 449
28.4.1.1 qurt_etm_set_range . . . 449
28.5 qurt_etm_set_atb() . . . 450
28.5.1 Function Documentation . . . 450
28.5.1.1 qurt_etm_set_atb . . . 450
28.6 qurt_etm_set_sync_period() . . . 451
28.6.1 Function Documentation . . . 451
28.6.1.1 qurt_etm_set_sync_period . . . 451
28.7 qurt_stm_trace_set_config() . . . 452
28.7.1 Function Documentation . . . 452
28.7.1.1 qurt_stm_trace_set_config . . . 452
28.8 Data Types . . . 453
28.8.1 Data Structure Documentation . . . 453
28.8.1.1 struct qurt_stm_trace_info_t . . . 453
28.9 Macros . . . 454
28.9.1 Define Documentation . . . 454
28.9.1.1 QURT_ETM_SOURCE_PC . . . 454
28.9.1.2 QURT_ETM_SOURCE_DATA . . . 454
28.9.1.3 QURT_ETM_ASYNC_PERIOD . . . 454
28.9.1.4 QURT_ETM_ISYNC_PERIOD . . . 454
28.9.1.5 QURT_ETM_GSYNC_PERIOD . . . 454
28.9.1.6 QURT_ETM_SETUP_OK . . . 454
28.9.1.7 QURT_ETM_SETUP_ERR . . . 454
28.9.1.8 QURT_ATB_OFF . . . 454
28.9.1.9 QURT_ATB_ON . . . 454
28.9.1.10 QURT_TRACE . . . 454

**29 Atomic Operations** **455**
29.1 qurt_atomic_set() . . . 457
29.1.1 Function Documentation . . . 457
29.1.1.1 qurt_atomic_set . . . 457
29.2 qurt_atomic_and() . . . 458
29.2.1 Function Documentation . . . 458
29.2.1.1 qurt_atomic_and . . . 458
29.3 qurt_atomic_and_return() . . . 459
29.3.1 Function Documentation . . . 459
29.3.1.1 qurt_atomic_and_return . . . 459
29.4 qurt_atomic_or() . . . 460
29.4.1 Function Documentation . . . 460
29.4.1.1 qurt_atomic_or . . . 460
29.5 qurt_atomic_or_return() . . . 461
29.5.1 Function Documentation . . . 461
29.5.1.1 qurt_atomic_or_return . . . 461
29.6 qurt_atomic_xor() . . . 462
29.6.1 Function Documentation . . . 462
29.6.1.1 qurt_atomic_xor . . . 462
29.7 qurt_atomic_xor_return() . . . 463
29.7.1 Function Documentation . . . 463

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT shunchul.won@lge.com

29.7.1.1 qurt_atomic_xor_return . . . 463
29.8 qurt_atomic_set_bit() . . . 464
29.8.1 Function Documentation . . . 464
29.8.1.1 qurt_atomic_set_bit . . . 464
29.9 qurt_atomic_clear_bit() . . . 465
29.9.1 Function Documentation . . . 465
29.9.1.1 qurt_atomic_clear_bit . . . 465
29.10 qurt_atomic_change_bit() . . . 466
29.10.1 Function Documentation . . . 466
29.10.1.1 qurt_atomic_change_bit . . . 466
29.11 qurt_atomic_add() . . . 467
29.11.1 Function Documentation . . . 467
29.11.1.1 qurt_atomic_add . . . 467
29.12 qurt_atomic_add_return() . . . 468
29.12.1 Function Documentation . . . 468
29.12.1.1 qurt_atomic_add_return . . . 468
29.13 qurt_atomic_add_unless() . . . 469
29.13.1 Function Documentation . . . 469
29.13.1.1 qurt_atomic_add_unless . . . 469
29.14 qurt_atomic_sub() . . . 470
29.14.1 Function Documentation . . . 470
29.14.1.1 qurt_atomic_sub . . . 470
29.15 qurt_atomic_sub_return() . . . 471
29.15.1 Function Documentation . . . 471
29.15.1.1 qurt_atomic_sub_return . . . 471
29.16 qurt_atomic_inc() . . . 472
29.16.1 Function Documentation . . . 472
29.16.1.1 qurt_atomic_inc . . . 472
29.17 qurt_atomic_inc_return() . . . 473
29.17.1 Function Documentation . . . 473
29.17.1.1 qurt_atomic_inc_return . . . 473
29.18 qurt_atomic_dec() . . . 474
29.18.1 Function Documentation . . . 474
29.18.1.1 qurt_atomic_dec . . . 474
29.19 qurt_atomic_dec_return() . . . 475
29.19.1 Function Documentation . . . 475
29.19.1.1 qurt_atomic_dec_return . . . 475
29.20 qurt_atomic_compare_and_set() . . . 476
29.20.1 Function Documentation . . . 476
29.20.1.1 qurt_atomic_compare_and_set . . . 476
29.21 qurt_atomic_barrier() . . . 477
29.21.1 Function Documentation . . . 477
29.21.1.1 qurt_atomic_barrier . . . 477
29.22 qurt_atomic64_set() . . . 478
29.22.1 Function Documentation . . . 478
29.22.1.1 qurt_atomic64_set . . . 478
29.23 qurt_atomic64_and_return() . . . 479
29.23.1 Function Documentation . . . 479
29.23.1.1 qurt_atomic64_and_return . . . 479
29.24 qurt_atomic64_or() . . . 480

Qualcomm Confidential - May Contain Trade Secrets 2025-02-27 15:40:42 GMT hongchui.wong@lge.com

29.24.1 Function Documentation . . . 480
29.24.1.1 qurt_atomic64_or . . . 480
29.25 qurt_atomic64_or_return() . . . 481
29.25.1 Function Documentation . . . 481
29.25.1.1 qurt_atomic64_or_return . . . 481
29.26 qurt_atomic64_xor_return() . . . 482
29.26.1 Function Documentation . . . 482
29.26.1.1 qurt_atomic64_xor_return . . . 482
29.27 qurt_atomic64_set_bit() . . . 483
29.27.1 Function Documentation . . . 483
29.27.1.1 qurt_atomic64_set_bit . . . 483
29.28 qurt_atomic64_clear_bit() . . . 484
29.28.1 Function Documentation . . . 484
29.28.1.1 qurt_atomic64_clear_bit . . . 484
29.29 qurt_atomic64_change_bit() . . . 485
29.29.1 Function Documentation . . . 485
29.29.1.1 qurt_atomic64_change_bit . . . 485
29.30 qurt_atomic64_add() . . . 486
29.30.1 Function Documentation . . . 486
29.30.1.1 qurt_atomic64_add . . . 486
29.31 qurt_atomic64_add_return() . . . 487
29.31.1 Function Documentation . . . 487
29.31.1.1 qurt_atomic64_add_return . . . 487
29.32 qurt_atomic64_sub_return() . . . 488
29.32.1 Function Documentation . . . 488
29.32.1.1 qurt_atomic64_sub_return . . . 488
29.33 qurt_atomic64_inc() . . . 489
29.33.1 Function Documentation . . . 489
29.33.1.1 qurt_atomic64_inc . . . 489
29.34 qurt_atomic64_inc_return() . . . 490
29.34.1 Function Documentation . . . 490
29.34.1.1 qurt_atomic64_inc_return . . . 490
29.35 qurt_atomic64_dec_return() . . . 491
29.35.1 Function Documentation . . . 491
29.35.1.1 qurt_atomic64_dec_return . . . 491
29.36 qurt_atomic64_compare_and_set() . . . 492
29.36.1 Function Documentation . . . 492
29.36.1.1 qurt_atomic64_compare_and_set . . . 492
29.37 qurt_atomic64_barrier() . . . 493
29.37.1 Function Documentation . . . 493
29.37.1.1 qurt_atomic64_barrier . . . 493

**30 QuRT Callbacks 494**
30.1 __hexagon_bsp_init() . . . 495
30.2 qurt_cb_data_set_cbarg() . . . 496
30.2.1 Function Documentation . . . 496
30.2.1.1 qurt_cb_data_set_cbarg . . . 496
30.3 qurt_cb_data_set_cbfunc() . . . 497
30.3.1 Function Documentation . . . 497
30.3.1.1 qurt_cb_data_set_cbfunc . . . 497

Qualcomm Confidential - May Contain Trade Secrets 2025-02-27 05:40:42 GMT hwanghui.wan@lge.com

30.4 qurt_cb_data_init() . . . 498
30.4.1 Function Documentation . . . 498
30.4.1.1 qurt_cb_data_init . . . 498
30.5 Data Types . . . 499
30.5.1 Data Structure Documentation . . . 499
30.5.1.1 struct qurt_cb_data_t . . . 499

**31 HVX** **500**
31.1 qurt_hvx_get_units() . . . 501
31.1.1 Function Documentation . . . 501
31.1.1.1 qurt_hvx_get_units . . . 501
31.2 qurt_hvx_lock() . . . 502
31.2.1 Function Documentation . . . 502
31.2.1.1 qurt_hvx_lock . . . 502
31.3 qurt_hvx_unlock . . . 503
31.3.1 Function Documentation . . . 503
31.3.1.1 qurt_hvx_unlock . . . 503

**32 Predefined Symbols** **504**
32.0.1 Define Documentation . . . 504
32.0.1.1 QURT_UPDATE_PROCESS_STATE . . . 504
32.1 qurt_api_version() . . . 505
32.1.1 Function Documentation . . . 505
32.1.1.1 qurt_api_version . . . 505
32.2 Constants and Macros . . . 506
32.2.1 Define Documentation . . . 506
32.2.1.1 QURT_API_VER_MAJOR . . . 506
32.2.1.2 QURT_API_VER_MINOR . . . 506
32.2.1.3 QURT_API_VER_RELEASE . . . 506
32.2.1.4 QURT_API_VER_PATCH . . . 506
32.2.1.5 QURT_API_VERSION_ENCODE . . . 506
32.2.1.6 QURT_API_VERSION . . . 506

**33 Note** **507**
33.1 Macros . . . 508
33.1.1 Define Documentation . . . 508
33.1.1.1 QURT_NOTE_TYPE_POOL_INFO . . . 508
33.1.1.2 QURT_NOTE_TYPE_SEGMENT_INFO . . . 508
33.1.1.3 QURT_NOTE_TYPE_ARB_INFO . . . 508
33.1.1.4 QURT_NOTE_NAME . . . 508
33.1.1.5 QURT_NOTE_NAME_SIZE . . . 508
33.2 Types . . . 509
33.2.1 Data Structure Documentation . . . 509
33.2.1.1 struct qurt_note_t . . . 509
33.2.1.2 struct qurt_note_type_pool_info_t . . . 509
33.2.1.3 struct qurt_note_type_pool_seg_info_t . . . 509

**34 OS services** **510**
34.1 Macros . . . 511
34.1.1 Define Documentation . . . 511
34.1.1.1 QURT_OS_SERVICE_THREAD . . . 511

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:42:51 GMT yungchul.wong@qc.com

34.1.1.2 QURT_OS_SERVICE_FS_HUB . . . 511
34.1.1.3 QURT_OS_SERVICE_CALLBACK . . . 511
34.1.1.4 QURT_OS_SERVICE_INTERRUPTS . . . 511
34.1.1.5 QURT_OS_SERVICE_PROXY . . . 511
34.1.1.6 QURT_OS_SERVICE_MEMORY . . . 511
34.1.1.7 QURT_OS_SERVICE_MEMPOOL . . . 511
34.1.1.8 QURT_OS_SERVICE_PROCESS . . . 511
34.1.1.9 QURT_OS_SERVICE_MMAP . . . 511

**35 Physical memory manager 512**
35.1 func_qurt_pmem_acquire() . . . 513
35.1.1 Function Documentation . . . 513
35.1.1.1 qurt_pmem_acquire . . . 513
35.2 func_qurt_pmem_release() . . . 514
35.2.1 Function Documentation . . . 514
35.2.1.1 qurt_pmem_release . . . 514
35.3 Physical memory management macros . . . 515
35.3.1 Define Documentation . . . 515
35.3.1.1 QURT_PMEM_SUCCESS . . . 515
35.3.1.2 QURT_PMEM_NO_PRIV . . . 515
35.3.1.3 QURT_PMEM_RETRY . . . 515
35.3.1.4 QURT_PMEM_OVERLAP . . . 515
35.3.1.5 QURT_PMEM_NOT_EXIST . . . 515
35.3.1.6 QURT_PMEM_INIT_FAILURE . . . 515
35.3.1.7 QURT_PMEM_OUTSTANDING_MAPPING . . . 515
35.3.1.8 QURT_PMEM_GENERIC_FAILURE . . . 515
35.3.1.9 QURT_PMEM_ENTRY_FOUND . . . 515
35.3.1.10 QURT_PMEM_REACH_END . . . 515
35.3.1.11 QURT_PMEM_UNCLAIMED . . . 515
35.3.1.12 QURT_PMEM_ALREADY_CLAIMED . . . 515

**36 Printf 516**
36.1 printf() . . . 517
36.1.1 Function Documentation . . . 517
36.1.1.1 qurt_printf . . . 517
36.2 vprintf() . . . 518
36.2.1 Function Documentation . . . 518
36.2.1.1 qurt_vprintf . . . 518

**A Thread-level Profiling 519**
A.1 Overview . . . 519
A.2 Server Behavior . . . 519
A.3 Client Behavior . . . 520
A.4 Profiling the System . . . 521

**B Debugging Errors and Cause Codes 522**
B.1 Debugging Errors and Exceptions . . . 522
B.2 Cause Codes . . . 523
B.3 Debugging a Fatal Error . . . 523
B.4 Debugging a Nonfatal Error . . . 523
B.5 Cause . . . 524

Qualcomm Confidential - Qualcomm Trade Secrets 2026-02-27 05:45:43 hyungchul.won@lge.com

B.6 Cause 2 . . . . . 525

**C References 528**

C.1 Related Documents . . . . . 528

C.2 Acronyms and Terms . . . . . 528

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:40:42 GMT
hyungchul.won@lge.com

# List of Figures

2-1 User program image . . . . . 47
3-1 Thread state transitions . . . . . 51
5-1 Mutex example . . . . . 129
21-1 Memory region mappings . . . . . 306
25.1 Q6 DT Architecture . . . . . 418
25.2 Q6_BOOT_PARAMS . . . . . 419
25.3 QuRT Kernel Locating DT image . . . . . 420
25-4 DT_QDI_Driver_Model . . . . . 420
25.5 Multi compatible strings support . . . . . 421
25.6 DT_Overlay_Example . . . . . 422

# List of Tables

3-1 Thread states . . . . . 50
4-2 Process attribute defaults . . . . . 101

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:40:42 GMT
hyungchul.won@lge.com

# 1 Introduction

## 1.1 Purpose

This document is designed to serve as a reference for C programmers experienced in real-time software development. It provides only basic information on real-time concurrent programming. For more information, see ISBN 0470128720.

The QuRT™ operating system is a real-time operating system (RTOS) for the Qualcomm Hexagon™ processor. It supports multithreading, thread communication and synchronization, interrupt handling, and memory management.

**Note:** This document has been updated to reflect APIs introduced in QuRT 4.5.

## 1.2 Conventions

Function declarations, function names, type declarations, and code samples appear in a different font, for example, `#include`.

Code variables appear in angle brackets, for example, `<number>`.

Commands and command variables appear in a different font, for example, **`copy a:*.* b:`**.

Parameter directions are indicated as follows:

- `[in]` indicates an input parameter.
- `[out]` indicates an output parameter.
- `[in,out]` indicates a parameter used for both input and output.

## 1.3 Technical Assistance

For assistance or clarification on information in this document, open a technical support case at https://support.qualcomm.com.

You will need to register for a Qualcomm ID account and your company must have support enabled to access our Case system.

Other systems and support resources are listed on https://support.qualcomm.com.

If you need further assistance, you can send an email to qualcomm.support@qti.qualcomm.com.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 1.4 QuRT Features

QuRT offers low overhead (both in memory and processing), simple implementation, and eases porting porting standalone user programs to the QuRT environment and modification to accommodate specific target requirements

QuRT consists of:

- The kernel, which provides system operations for a minimal set of operating system facilities. The kernel handles thread creation, scheduling, blocking, and performs basic memory management.
- The library, which provides an application programming interface (API) to the kernel operations and additional library functions to aid in programming.
- The configuration files, which encapsulate target-specific information used to configure QuRT for target platforms.

**Note:** QuRT is a simplified operating system – it does not provide many facilities that are commonly available in other operating systems.

QuRT offers:

- Real-time priority-based preemptive Multithreading:
  - Multiple threads (or flows of execution) can execute at the same time in a user program. QuRT initially assigns the program a single thread of execution, the program can then create additional threads. The Hexagon processor can execute a fixed number of threads simultaneously – additional threads must share the processor. QuRT handles the sharing details.
  - Each thread is assigned a priority level that determines which thread has execution priority.
  - A thread can be preempted – have the processor taken away – when a higher-priority thread is ready to execute.
  - The operating system can perform operations within certain periods of time.
- Processes, which enable programs and threads to execute in separate protected address spaces for improved system security and stability.
- Mutexes, which synchronize threads to ensure mutually exclusive access to shared resources.
- Signals: QuRT objects that synchronize threads on sets of mutex-like signals.
- Semaphores, which synchronize threads to ensure limited access to shared resources.
- Barriers: QuRT object used to synchronize threads to meet at a specific point in a user program.
- Condition variables: QuRT objects used to synchronize threads based on the value of a data item.
- Pipes, which support synchronized data exchange between threads.
- Timers, with which threads can schedule actions to occur at specific times or intervals.
- Interrupt handling, which registers threads to serve as interrupt handlers.
- Thread local storage, which allocates global storage that is private to specific threads.
- Exception handling, which supports exception handling for fatal and non-fatal exceptions.
- Memory management, which enables user programs to dynamically manage their memory space.
- Profiling, which record cycle counts (both running and idle) for specific threads.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@qc.com

- Performance monitor, which supports code performance measurement during user program execution.
- Function tracing, which supports debugging macros for tracing function calls and returns.

## 1.5 Processor Versions

QuRT 4.5 supports Hexagon processor versions v68-v81. For more information, see 80-N2040-8.

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:40:42 GMT
hyungchul.won@lge.com

# 2 Using QuRT

## 2.1 User Programs

A QuRT system contains one or more user programs. Each user program is a complete program which uses the QuRT API (see Section 2.3) to access the QuRT services. When a user program is started it is assigned a single thread – to create additional threads, the program uses the QuRT thread services.

A user program typically consists of one or more C or assembly source files (some of which include the QuRT API header file).

A user program memory image includes the following items:

- Default global heap
- Main thread call stack
- Data and text sections of the program
- Heaps and thread call stacks allocated by the program

The size of the global heap is specified by the user when the user program is built (Section 2.2).

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:40:42 GMT
hyungchul.won@lge.com

Figure 2-1 shows a functional diagram of a user program image.

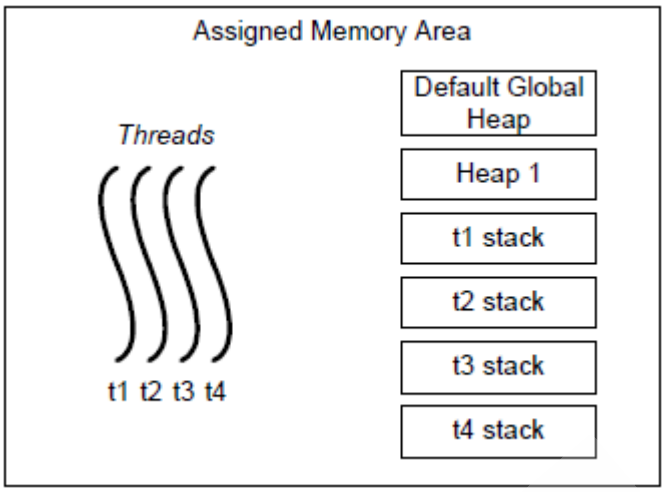


**Figure 2-1 User program image**

QuRT prevents user programs from accessing unauthorized areas of system memory. If a thread attempts to access memory outside its assigned memory area, QuRT generates a memory exception.

## 2.2 Build Procedure

QuRT user programs are written in C/C++ and Hexagon assembly language, and use the QuRT APIs to access the RTOS services.

The build procedure for a QuRT user program is similar to the standard procedure for building a stand-alone C/C++ program.

All QuRT libraries (including the RTOS kernel) are provided as object files – no source code is provided. Multiple versions of the QuRT libraries are provided to support different hardware and software targets. Each library version is optimized for its specific target.

Before building a QuRT system, users must define the system configuration in a user-editable configuration file. This file is then used to generate a configuration object file, which is linked with the QuRT RTOS when it is built.

Building a QuRT system creates a single boot image, which can be executed in two ways:

- Software simulation using the Hexagon simulator
- In-circuit emulation using a hardware test platform (Rumi, ZeBu, SURF)

For more information on the QuRT system build procedure refer to [80-VB419-79].

**Note:** QuRT user programs use the standard C library to perform operations supported by the standard library (in particular, malloc and printf).

## 2.3 API

The QuRT application program interfaces (APIs) are a C header file named qurt.h, which is included into the source code of each QuRT user program. For example:

```
#include ``qurt.h''
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
...
qurt_mutex_lock(&my_mutex); /* QuRT API function */
```

The function, type, and constant names defined in the QuRT API begin with the prefix qurt_ to indicate that they are part of QuRT. Preprocessor definitions in the QuRT API include the prefix QURT_. Functions and data structures in the kernel include the prefix QURTK_.

For more information on QuRT API functions, see Section 2.5.

## 2.4 Objects

A QuRT user program accesses most QuRT services by defining objects and performing operations on them. For example:

```
qurt_mutex_t my_mutex; /* mutex object */
...
qurt_mutex_init(&my_mutex); /* init mutex object */
...
qurt_mutex_lock(&my_mutex); /* lock mutex */
...
qurt_mutex_destroy(&my_mutex); /* destroy mutex object */
```

QuRT objects support two sets of operations for managing objects:

- The init/destroy operations (shown in the example above) are used for objects that are stored wholly in memory allocated by the user program.
- The create/delete operations are used for objects that are stored partly in memory allocated automatically by the RTOS kernel.

Pipe objects support both operation pairs: init/destroy are used when the pipe buffer is user-allocated, while create/delete are used when the pipe buffer is automatically allocated by the kernel as part of initializing a pipe object.

Timer objects support only create/delete for object management. All other QuRT objects support only init/destroy for object management.

In addition to object management, most objects define additional operations which perform services associated with that object (qurt_mutex_lock in the example above).

**Note:** Objects must be destroyed (with the destroy or delete operation) when no longer in use. Failure to do so causes resource leaks in the QuRT kernel.

QuRT objects should be treated as having opaque types. They should be accessed only through QuRT functions.

Qualcomm Confidential - May Contain Trade Secrets 2016-02-27 05:40:42 GMT hyoungchul.woo@lge.com

## 2.5 QuRT Services

The following chapters describe the QuRT APIs.

- Threads
- Processes
- Mutexes
- Recursive Mutexes
- Priority Inheritance Mutexes
- Signals
- Any-signals
- All-signals
- Semaphores
- Barriers
- Condition Variables
- Pipes
- Timers
- System Clock
- Interrupts
- Thread Local Storage
- Exception Handling
- Memory Allocation
- Memory Management
- System Environment
- Profiling
- Performance Monitor
- Error Results
- Function Tracing
- QuRT Callbacks
- Predefined Symbols

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 3 Threads

## 3.1 Overview

Multitasking allows multiple instruction sequences in an application to execute in parallel. Each sequence of instructions in a running application is called a thread.

Threads are represented as shared objects in QuRT. Thread objects support the following operations:

- Create thread – Create new thread and make it executable.
- Resume thread – Awaken the specified thread.
- Exit thread – Stop current thread and destroy it.
- Join thread – Suspend current thread until the specified thread stops.
- Get current thread – Return reference to current thread.

Once started, a thread exists in one of four states. Table 3-1 lists the states.

**Table 3-1 Thread states**

| State | Description |
|---|---|
| Ready | The thread is ready to run, but prevented from running because a higher priority thread is executing. |
| Running | The thread is currently executing. |
| Waiting | The thread is waiting for an event to occur or a shared resource to become available. |
| Stopped | The thread no longer exists, having been destroyed. |

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:49:42 GMT hyunchul.wang@qc.com

The kernel is responsible for switching threads between these states. It uses a scheduler to determine which threads to run – the scheduler always selects the highest-priority ready threads.

A thread is suspended when it changes state from Running to Ready or Waiting, and awakened when it changes from Waiting to Ready. All threads are initialized to Ready. During system startup the scheduler selects the highest-priority threads for execution and changes their thread state to Running.

The action of suspending one thread and resuming another is called a context switch.

Figure 3-1 shows the events that can cause the kernel to perform context switches.

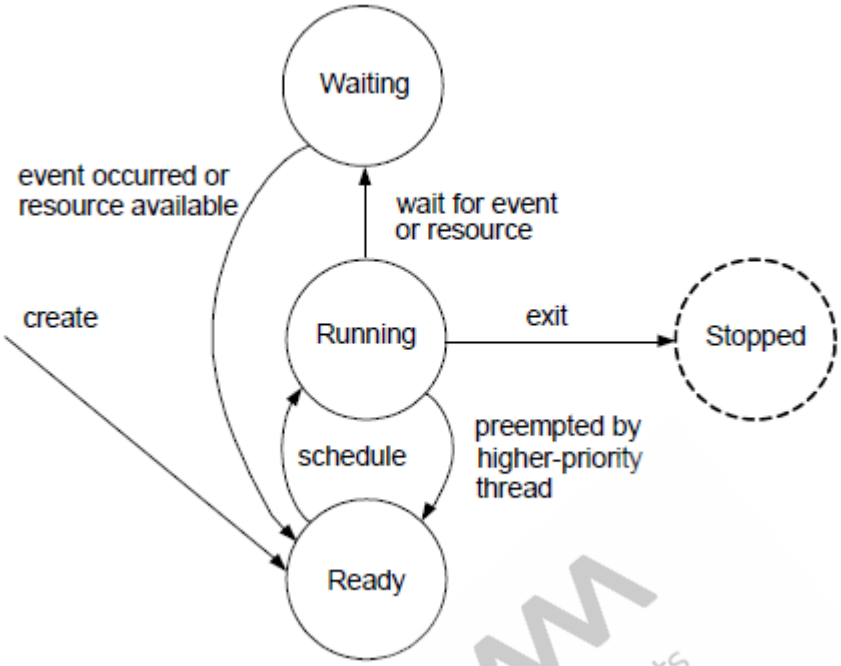


**Figure 3-1 Thread state transitions**

Qualcomm Confidential - May Contain Trade Secrets 2025-02-27 05:40:42 GMT hyungchul.kim@lge.com

QuRT is preemptive – a context switch occurs when a kernel operation suspends the current thread or awakens a higher-priority thread. The following kernel operations can cause a context switch:

- Creating or exiting a thread
- Changing a thread priority
- Waiting on or releasing a mutex or semaphore
- Waiting on or resuming from a signal, barrier, or condition variable
- Reading or writing from a pipe
- Interrupt

**Thread priorities**

The priority of a thread determines how often the thread executes relative to the other threads in the system: if two ready-state threads have different priorities, but only one hardware thread is available, the kernel executes the thread with higher priority until it is suspended.

Threads are assigned priorities when they are first created; however, in some cases an application system may need to adjust the priority of a thread after it has been created. For instance, in order to prevent priority inversion a thread may need to raise its own priority or the priority of another thread.

Priorities are specified as numeric values in a range as large as 0-255, with lower values representing higher priorities. 0 represents the highest possible thread priority.

**Note:** QuRT can be configured to have different priority ranges (Section 2.2).

### Thread attributes

Threads have the following attributes:

- Name – Character string identifier used to uniquely identify the thread.
- Timetest ID – Numeric trace identifier used during hardware debugging.
- Entry point – Function that the thread executes.
- Argument – Pointer passed to the thread function when it is executed.
- Stack address – Base address of the memory area used for the thread call stack.
- Stack size – Size (in bytes) of the memory area used for the thread call stack.
- Priority – Thread execution priority.
- Affinity – Hardware threads used by the thread.
- TCB partition – Memory used for allocating thread control blocks (TCBs).
- Signal – Signal object created by QuRT for each thread.

The thread name and timetest identifier are used to uniquely identify threads during debugging or profiling. Note that these attributes differ from the kernel-generated thread identifiers used to specify threads in the API thread operations.

The thread entry point is the function executed by the thread when it is started. The function is defined in the application, and must accept a single void pointer as a function parameter.

The thread argument is a pointer that is passed to the thread function when the thread is started. It allows a single function to be written so it can be executed by multiple threads.

The thread stack address and size specify the memory area used as a call stack for the thread. The user is responsible for allocating the memory area used for the stack.

The thread priority determines the execution priority of the thread.

The thread affinity specifies which Hexagon processor hardware threads the thread can execute on.

The thread TCB partition specifies the maximum number of threads that have their thread control blocks allocated in TCM/LPM instead of regular memory.

The thread signal is an signal object which is created by QuRT for each thread.

**Note:** Threads are specified by the thread identifier returned by the thread create operation. This identifier is distinct from the thread name or timetest ID.

### Setting thread attributes

Threads have two kinds of attributes:

- Static attributes cannot be changed after a thread is created
- Dynamic attributes can be changed after the thread is created

The only dynamic thread attributes are priority and timetest ID – all the other thread attributes are static.

Static attributes are set both before a thread is created (using the qurt_thread_attr_init and qurt_thread_attr_set functions) and when a thread is created (by directly passing the attributes as arguments to qurt_thread_create).

Dynamic attributes are set after a thread is created using the qurt_thread_set functions.

Qualcomm Confidential - May Contain Trade Secrets 2026-03-27 05:49:42 GMT hyungchunwon@lge.com

**Note:** Two thread attributes – the thread identifier and thread signal – are read-only attributes set by the kernel.
The timetest ID attribute is stored in a Hexagon processor register. For more information refer to [80-N2040-8].

**Functions**

Threads are represented as shared objects in QuRT. Thread objects support the following operations:

- qurt_thread_attr_get()
- qurt_thread_attr_init()
- qurt_thread_attr_set_bus_priority()
- qurt_thread_attr_set_autostack()
- qurt_thread_set_autostack()
- qurt_thread_attr_set_name()
- qurt_thread_attr_set_priority()
- qurt_thread_attr_set_stack_addr()
- qurt_thread_attr_set_stack_size()
- qurt_thread_attr_set_stack_size2()
- qurt_thread_attr_set_tcb_partition()
- qurt_thread_attr_set_timetest_id()
- qurt_thread_create()
- qurt_thread_stop()
- qurt_thread_exit()
- qurt_thread_get_id()
- qurt_thread_get_l2cache_partition()
- qurt_thread_get_name()
- qurt_thread_get_priority()
- qurt_thread_get_timetest_id()
- qurt_thread_join()
- qurt_thread_resume_thread()
- qurt_thread_set_thread_property()
- qurt_thread_get_group_id()
- qurt_thread_set_group_id()
- qurt_thread_set_cache_partition()
- qurt_thread_set_priority()
- qurt_thread_attr_set_detachstate()
- qurt_thread_set_timetest_id()

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

- qurt_thread_stid_set()
- qurt_thread_attr_set_group_id()
- qurt_thread_get_proc_id()
- qurt_thread_sched_yield()
- qurt_thread_get_thread_id()
- qurt_sleep()
- qurt_system_set_priority_floor()
- qurt_thread_get_tls_base()
- qurt_busywait()
- Data Types
- Constants and Macros

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:40:42 GMT
hyungchul.won@lge.com

## 3.2 qurt_thread_attr_get()

### 3.2.1 Function Documentation

#### 3.2.1.1 int qurt_thread_attr_get ( qurt_thread_t *thread_id,* qurt_thread_attr_t ∗ *attr* )

Gets the attributes of the specified thread.

**Associated data types**

qurt_thread_t
qurt_thread_attr_t

**Parameters**

| | | |
|---|---|---|
| in | *thread_id* | Thread identifier. |
| out | *attr* | Pointer to the destination structure for thread attributes. |

**Returns**

QURT_EOK – Success.
QURT_EINVALID – Invalid argument.

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 3.3 qurt_thread_attr_init()

### 3.3.1 Function Documentation

#### 3.3.1.1 static void qurt_thread_attr_init ( qurt_thread_attr_t ∗ *attr* )

Initializes the structure used to set the thread attributes when a thread is created. After an attribute structure is initialized, use the thread attribute operations to explicity set the individual attributes in the structure.

The initialize operation sets the following default attribute values:

- Name – NULL string
- TCB partition – QURT_THREAD_ATTR_TCB_PARTITION_DEFAULT
- Priority – QURT_THREAD_ATTR_PRIORITY_DEFAULT
- Autostack – QURT_THREAD_ATTR_AUTOSTACK_DEFAULT
- Bus priority – QURT_THREAD_ATTR_BUS_PRIO_DEFAULT
- Timetest ID – QURT_THREAD_ATTR_TIMETEST_ID_DEFAULT
- stack_size – 0
- stack_addr – NULL
- detach state – QURT_THREAD_ATTR_CREATE_LEGACY
- STID – QURT_THREAD_ATTR_STID_DEFAULT

**Associated data types**

qurt_thread_attr_t

**Parameters**

| | | |
|---|---|---|
| `in,out` | *attr* | Pointer to the thread attribute structure. |

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 3.4 qurt_thread_attr_set_bus_priority()

### 3.4.1 Function Documentation

#### 3.4.1.1 static void qurt_thread_attr_set_bus_priority ( qurt_thread_attr_t ∗ *attr,* unsigned short *bus_priority* )

Sets the internal bus priority state in the Hexagon core for this software thread attribute. Memory requests generated by the thread with bus priority enabled are given priority over requests generated by the thread with bus priority disabled. The default value of bus priority is disabled.

**NOTE** Sets the internal bus priority for Hexagon processor v60 or greater. The priority is not propagated to the bus fabric.

**Associated data types**

qurt_thread_attr_t

**Parameters**

| | | |
|---|---|---|
| in | *attr* | Pointer to the thread attribute structure. |
| in | *bus_priority* | Enable flag. Values:<br>• QURT_THREAD_BUS_PRIO_DISABLED<br>• QURT_THREAD_BUS_PRIO_ENABLED |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 3.5 qurt_thread_attr_set_autostack()

### 3.5.1 Function Documentation

#### 3.5.1.1 static void qurt_thread_attr_set_autostack ( qurt_thread_attr_t ∗ *attr,* unsigned short *autostack* )

Enables the autostack v2 feature in the thread attributes.

When autostack is enabled by the subsystem and an autostack enabled thread gets a framelimit exception, the kernel allocates more stack for the thread and returns to normal execution.

When autostack is not enabled by the subsystem or not enabled for the thread, the framelimit exception is fatal.

**Associated data types**

qurt_thread_attr_t

**Parameters**

| | | |
|---|---|---|
| in | *attr* | Pointer to the thread attribute structure. |
| in | *autostack* | Autostack enable or disable flag. Values:<br>• QURT_THREAD_AUTOSTACK_DISABLED<br>• QURT_THREAD_AUTOSTACK_ENABLED |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 3.6 qurt_thread_set_autostack()

### 3.6.1 Function Documentation

#### 3.6.1.1 void qurt_thread_set_autostack ( void * )

Sets autostack enable in the TCB. Takes as input a pointer to the user general pointer (UGP).

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 3.7 qurt_thread_attr_set_name()

### 3.7.1 Function Documentation

#### 3.7.1.1 static void qurt_thread_attr_set_name ( qurt_thread_attr_t * *attr,* const char * *name* )

Sets the thread name attribute.

This function specifies the name to use by a thread. Thread names identify a thread during debugging or profiling. Maximum name length is 16 charactes.

**NOTE** Thread names differ from the kernel-generated thread identifiers used to specify threads in the API thread operations.

**Associated data types**

qurt_thread_attr_t

**Parameters**

| | | |
|---|---|---|
| in,out | *attr* | Pointer to the thread attribute structure. |
| in | *name* | Pointer to the character string that contains the thread name. |

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 3.8 qurt_thread_attr_set_priority()

### 3.8.1 Function Documentation

#### 3.8.1.1 static void qurt_thread_attr_set_priority ( qurt_thread_attr_t ∗ *attr,* unsigned short *priority* )

Sets the thread priority to assign to a thread. Thread priorities are specified as numeric values in the range 1 to 254. 1 represents the highest priority. The kernel internally uses priority 0 and 255 are internally for special purposes.

**Associated data types**

qurt_thread_attr_t

**Parameters**

| | | |
|---|---|---|
| in,out | *attr* | Pointer to the thread attribute structure. |
| in | *priority* | Thread priority. |

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 3.9 qurt_thread_attr_set_stack_addr()

## 3.9.1 Function Documentation

### 3.9.1.1 static void qurt_thread_attr_set_stack_addr ( qurt_thread_attr_t ∗ *attr,* void ∗ *stack_addr* )

Sets the thread stack address attribute.

Specifies the base address of the memory area to use for a call stack of a thread.

stack_addr must contain an address value that is 8-byte aligned.

The thread stack address and stack size (Section 3.10.1.1) specify the memory area used as a call stack for the thread.

**NOTE** The user is responsible for allocating the memory area used for the thread stack. The memory area must be large enough to contain the stack that the thread creates.

#### Associated data types

qurt_thread_attr_t

#### Parameters

| | | |
|---|---|---|
| in,out | *attr* | Pointer to the thread attribute structure. |
| in | *stack_addr* | Pointer to the 8-byte aligned address of the thread stack. |

#### Safe API

Yes

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 3.10 qurt_thread_attr_set_stack_size()

### 3.10.1 Function Documentation

#### 3.10.1.1 static void qurt_thread_attr_set_stack_size ( qurt_thread_attr_t ∗ *attr,* unsigned int *stack_size* )

Sets the thread stack size attribute.

Specifies the size of the memory area to use for a call stack of a thread.

The thread stack address (Section 3.9.1.1) and stack size specify the memory area used as a call stack for the thread. The user is responsible for allocating the memory area used for the stack.

**Associated data types**

qurt_thread_attr_t

**Parameters**

| | | |
|---|---|---|
| in,out | *attr* | Pointer to the thread attribute structure. |
| in | *stack_size* | Size (in bytes) of the thread stack. |

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 3.11 qurt_thread_attr_set_stack_size2()

### 3.11.1 Function Documentation

#### 3.11.1.1 static void qurt_thread_attr_set_stack_size2 ( qurt_thread_attr_t ∗ *attr,* unsigned short *user_stack_size,* unsigned short *root_stack_size* )

Sets the thread stack size attribute for island threads that require a higher guest OS stack size than the stack size defined in the configuration XML.

Specifies the size of the memory area to use for a call stack of an island thread in User and Guest mode.

The thread stack address (Section 3.9.1.1) and stack size specify the memory area used as a call stack for the thread. The user is responsible for allocating the memory area used for the stack.

**Associated data types**

qurt_thread_attr_t

**Parameters**

| | | |
|---|---|---|
| in,out | *attr* | Pointer to the thread attribute structure. |
| in | *user_stack_size* | Size (in bytes) of the stack usage in User mode. |
| in | *root_stack_size* | Size (in bytes) of the stack usage in Guest mode. |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 3.12 qurt_thread_attr_set_tcb_partition()

### 3.12.1 Function Documentation

#### 3.12.1.1 static void qurt_thread_attr_set_tcb_partition ( qurt_thread_attr_t ∗ *attr,* unsigned char *tcb_partition* )

Sets the thread TCB partition attribute. Specifies the memory type where a TCB of a thread is allocated. Allocates TCBs in RAM or TCM/LPM.

**Associated data types**

qurt_thread_attr_t

**Parameters**

| | | |
|---|---|---|
| in,out | *attr* | Pointer to the thread attribute structure. |
| in | *tcb_partition* | TCB partition. Values:<br>• 0 – TCB resides in RAM<br>• 1 – TCB resides in TCM/LCM |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 3.13 qurt_thread_attr_set_timetest_id()

### 3.13.1 Function Documentation

#### 3.13.1.1 static void qurt_thread_attr_set_timetest_id ( qurt_thread_attr_t * *attr,* unsigned short *timetest_id* )

Sets the thread timetest attribute.

Specifies the timetest identifier to use by a thread.

Timetest identifiers are used to identify a thread during debugging or profiling.

**NOTE** Timetest identifiers differ from the kernel-generated thread identifiers used to specify threads in the API thread operations.

**Associated data types**

qurt_thread_attr_t

**Parameters**

| | | |
|---|---|---|
| in,out | *attr* | Pointer to the thread attribute structure. |
| in | *timetest_id* | Timetest identifier value. |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 3.14 qurt_thread_create()

### 3.14.1 Function Documentation

#### 3.14.1.1 int qurt_thread_create ( qurt_thread_t ∗ *thread_id,* qurt_thread_attr_t ∗ *attr,* void(∗)(void ∗fn_arg) *entrypoint,* void ∗ *args* )

Creates a thread with the specified attributes, and makes it executable.

**Associated data types**

qurt_thread_t
qurt_thread_attr_t

**Parameters**

| | | |
|---|---|---|
| out | *thread_id* | Returns a pointer to the thread identifier if the thread was successfully created. |
| in | *attr* | Pointer to the initialized thread attribute structure that specifies the attributes of the created thread. |
| in | *entrypoint* | C function pointer, which specifies the main function of a thread. |
| in | *args* | Pointer to a thread-specific argument structure |

**Returns**

QURT_EOK – Thread created.
QURT_EFAILED – Thread not created.

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 3.15 qurt_thread_stop()

### 3.15.1 Function Documentation

#### 3.15.1.1 void qurt_thread_stop ( void )

Stops the current thread, frees the kernel TCB, and yields to the next highest ready thread.

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 3.16 qurt_thread_exit()

## 3.16.1 Function Documentation

### 3.16.1.1 void qurt_thread_exit ( int *status* )

Stops the current thread, awakens threads joined to it, then destroys the stopped thread.

Threads that are suspended on the current thread (by performing a thread join Section 3.22.1.1) are awakened and passed a user-defined status value that indicates the status of the stopped thread.

**NOTE** Exit must be called in the context of the thread to stop.

**Parameters**

| | | |
|---|---|---|
| in | *status* | User-defined thread exit status value. |

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 3.17 qurt_thread_get_id()

## 3.17.1 Function Documentation

### 3.17.1.1 qurt_thread_t qurt_thread_get_id ( void )

Gets the identifier of the current thread.

Returns the thread identifier for the current thread.

**Returns**

Thread identifier – Identifier of the current thread.

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 3.18 qurt_thread_get_l2cache_partition()

### 3.18.1 Function Documentation

#### 3.18.1.1 qurt_cache_partition_t qurt_thread_get_l2cache_partition ( void )

Returns the current value of the L2 cache partition assigned to the caller thread.

**Returns**

Value of the qurt_cache_partition_t data type.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 3.19 qurt_thread_get_name()

## 3.19.1 Function Documentation

### 3.19.1.1 void qurt_thread_get_name ( char * *name,* unsigned char *max_len* )

Gets the thread name of current thread.

Returns the thread name of the current thread. Thread names are assigned to threads as thread attributes, see qurt_thread_attr_set_name(). Thread names identify a thread during debugging or profiling.

**Parameters**

| | | |
|---|---|---|
| `out` | *name* | Pointer to a character string, which specifies the address where the returned thread name is stored. |
| `in` | *max_len* | Maximum length of the character string that can be returned. |

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 3.20 qurt_thread_get_priority()

### 3.20.1 Function Documentation

#### 3.20.1.1 int qurt_thread_get_priority ( qurt_thread_t *threadid* )

Gets the priority of the specified thread.

Returns the thread priority of the specified thread.

Thread priorities are specified as numeric values in a range as large as 1 through 254, with lower values representing higher priorities. 1 represents the highest possible thread priority.

Priority 0 and 255 are internally used by the kernel for special purposes.

**NOTE** QuRT can be configured to have different priority ranges.

**Associated data types**

qurt_thread_t

**Parameters**

| | | |
|---|---|---|
| in | *threadid* | Thread identifier. |

**Returns**

-1 – Invalid thread identifier.
1 through 254 – Thread priority value.

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 3.21 qurt_thread_get_timetest_id()

### 3.21.1 Function Documentation

#### 3.21.1.1 unsigned short qurt_thread_get_timetest_id ( void )

Gets the timetest identifier of the current thread.

Returns the timetest identifier of the current thread.

Timetest identifiers are used to identify a thread during debugging or profiling.

**NOTE** Timetest identifiers differ from the kernel-generated thread identifiers used to specify threads in the API thread operations.

**Returns**

Integer – Timetest identifier.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 3.22 qurt_thread_join()

### 3.22.1 Function Documentation

#### 3.22.1.1 int qurt_thread_join ( unsigned int *tid,* int ∗ *status* )

Waits for a specified thread to finish; the specified thread is another thread within the same process. The caller thread is suspended until the specified thread exits. When the unspecified thread exits, the caller thread is awakened.

**NOTE** If the specified thread has already exited, this function returns immediately with the result value QURT_ENOTHREAD.

Two threads cannot call qurt_thread_join to wait for the same thread to finish. If this occurs, QuRT generates an exception (see Section 19.1).

**Parameters**

| | | |
|---|---|---|
| in | *tid* | Thread identifier. |
| out | *status* | Destination variable for thread exit status. Returns an application-defined value that indicates the termination status of the specified thread. |

**Returns**

QURT_ENOTHREAD – Thread has already exited.
QURT_EOK – Thread successfully joined with valid status value.

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 3.23 qurt_thread_resume_thread()

## 3.23.1 Function Documentation

### 3.23.1.1 int qurt_thread_resume_thread ( unsigned int *thread_id,* unsigned int *option* )

Resumes a QuRT thread with its thread identifier. The target thread can be in a signed user process or an unsigned user process. The caller thread can be a thread from the same user process of the target thread, or from its parent process. After the target thread resumes, the kernel scheduler can schedule the thread to run based on the thread priority.

The optional option argument has the QURT_THREAD_RESUME_DEFAULT value to resume the target thread.

By default, this is an asynchronous function. The function returns after the kernel moves the target thread from a suspended state to runnable state. The thread is scheduled to run based on its thread priority.

When the target thread is set as non-resumable, this function call does not resume the target thread.

When the target thread has already resumed, this function returns success.

When the target thread is in a secure user process or CPZ process, this function call returns an error and does not resume the operation.

When the target thread runs in the guest OS or root process via a QDI call, this function call clears the mark of suspend-pending on the target thread, and the target thread is not suspended when it exits the guest OS.

**Parameters**

| | | |
|---|---|---|
| in | *thread_id* | Thread identifier. |
| in | *option* | Optional argument QURT_THREAD_RESUME_DEFAULT resumes the target thread. |

**Returns**

QURT_EOK – Success
QURT_EINVALID – Failure due to invalid thread_id input
QURT_ENOTALLOWED – Failure due to operation is not allowed, for example, in a secure or CPZ process.
QURT_EHMXNOTAVAIL – Failure because when HMX thread resumes, HMX is not available for the HMX thread resume.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:49 GMT hyunseul.won@lge.com

## 3.24 qurt_thread_set_thread_property()

### 3.24.1 Function Documentation

#### 3.24.1.1 int qurt_thread_set_thread_property ( unsigned int *thread_id,* unsigned int *property_id,* unsigned int *value* )

Set a QuRT thread property with its thread identifier. The target thread can be in a signed user process or an unsigned user process. The caller thread can be from the same user process of the target thread or from its parent process.

If the target thread is in a secure user process or CPZ process, this function call returns an error without a change to the property of the target thread.

**Parameters**

| | | |
|---|---|---|
| in | *thread_id* | Thread identifier. |
| in | *property_id* | Thread property identifier.<br>#QURT_THREAD_PROPERTY_SUSPENDABLE – thread is suspendable. Default is TRUE.<br>#QURT_THREAD_PROPERTY_RESUMEABLE – thread is resumable. Default is TRUE. |
| in | *value* | Property value:<br>1 – TRUE<br>0 – FALSE |

**Returns**

QURT_EOK – Success
QURT_EINVALID – Failure due to invalid thread_id input
QURT_ENOTALLOWED – Failure because the operation is not allowed, for example, in a secure process/CPZ process.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 3.25 qurt_thread_get_group_id()

### 3.25.1 Function Documentation

#### 3.25.1.1 int qurt_thread_get_group_id ( qurt_thread_t *thread_id,* unsigned int * *group_id* )

Gets the group ID of the thread specified by thread_id.

**Parameters**

| | | |
|---|---|---|
| in | *thread_id* | Thread identifier. |
| out | *group_id* | Pointer to the variable of the group identifier. |

**Returns**

QURT_EOK – Success.
QURT_EINVALID – Thread ID is invalid, or the process has no groups enabled.
QURT_ENOTALLOWED – Operation is not allowed.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 3.26 qurt_thread_set_group_id()

### 3.26.1 Function Documentation

#### 3.26.1.1 int qurt_thread_set_group_id ( qurt_thread_t *thread_id,* unsigned int *group_id* )

Set thread group identifier of the thread specified by thread_id. The caller shall be from a thread in group0 in the same user process, from a thread in the root process, or from a user thread that is running in root process via QDI call. The thread group indicated by the group ID shall be enabled already. The target thread shall not have HVX/HLX/HMX attached. The target thread shall be either in waiting state, e.g., waiting for a signal/mutex/semaphore, or running in root process via QDI call where the target thread is the caller thread itself. The target thread shall not be in the suspended state (caused by memory preemption or thread suspend function). If thread priority of target thread is higher than the ceiling priority of the new group indicated by the group ID, the thread priority of target thread is change to the ceiling priority of the new group.

**Parameters**

| | | |
|---|---|---|
| in | *thread_id* | Thread identifier. |
| out | *group_id* | group identifier. |

**Returns**

QURT_EOK – Success.
QURT_EINVALID – Caller is invalid, or thread ID is invalid, or the user process has no groups enabled.
QURT_ENOTALLOWED – Operation is not allowed, e.g., wrong thread running conditions.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 3.27 qurt_thread_set_cache_partition()

## 3.27.1 Function Documentation

### 3.27.1.1 void qurt_thread_set_cache_partition ( qurt_cache_partition_t *l1_icache,* qurt_cache_partition_t *l1_dcache,* qurt_cache_partition_t *l2_cache* )

Sets the cache partition for the current thread. This function uses the qurt_cache_partition_t type to select the cache partition of the current thread for the L1 Icache, L1 Dcache, and L2 cache.

**Associated data types**

qurt_cache_partition_t

**Parameters**

| | | |
|---|---|---|
| in | *l1_icache* | L1 I cache partition. |
| in | *l1_dcache* | L1 D cache partition. |
| in | *l2_cache* | L2 cache partition. |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 3.28 qurt_thread_set_priority()

## 3.28.1 Function Documentation

### 3.28.1.1 int qurt_thread_set_priority ( qurt_thread_t *threadid,* unsigned short *newprio* )

Sets the priority of the specified thread.

Thread priorities are specified as numeric values in a range as large as 1 through 254, with lower values that represent higher priorities. 1 represents the highest possible thread priority. Priority 0 and 255 are internally used by the kernel for special purposes.

**NOTE** QuRT can be configured to have different priority ranges. For more information, see Section 2.2.

#### Associated data types

qurt_thread_t

#### Parameters

| | | |
|---|---|---|
| in | *threadid* | Thread identifier. |
| in | *newprio* | New thread priority value. |

#### Returns

0 – Priority successfully set.
-1 – Invalid thread identifier.

#### Safe API

Yes

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 3.29 qurt_thread_attr_set_detachstate()

## 3.29.1 Function Documentation

### 3.29.1.1 static void qurt_thread_attr_set_detachstate ( qurt_thread_attr_t ∗ *attr,* unsigned short *detachstate* )

Sets the thread detach state with which the thread is created. The following values specify the thread detach state:

- QURT_THREAD_ATTR_CREATE_JOINABLE – Joinable
- QURT_THREAD_ATTR_CREATE_DETACHED – Detached

When a detached thread is created (QURT_THREAD_ATTR_CREATE_DETACHED), its thread ID and other resources are reclaimed as soon as the thread exits.

When a joinable thread is created (QURT_THREAD_ATTR_CREATE_JOINABLE), it is assumed that some thread waits to use a qurt_thread_join() call to join on it. By default, detached state is QURT_THREAD_ATTR_CREATE_LEGACY. When detached state is QURT_THREAD_ATTR_CREATE_LEGACY, another thread can join before the thread exits but it does not wait on the other thread to join.

**NOTE** For a joinable thread (QURT_THREAD_ATTR_CREATE_JOINABLE), it is critical that some thread joins on it after it terminates, otherwise the resources of that thread are not reclaimed and can cause memory leaks.

#### Associated data types

qurt_thread_attr_t

#### Parameters

| | | |
|---|---|---|
| in,out | *attr* | Pointer to the thread attribute structure. |
| in | *detachstate* | Thread detach state. |

#### Safe API

No

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 09:40:42 GMT hyungchul.won@nsr.com

# 3.30 qurt_thread_set_timetest_id()

## 3.30.1 Function Documentation

### 3.30.1.1 void qurt_thread_set_timetest_id ( unsigned short *tid* )

Sets the timetest identifier of the current thread. Timetest identifiers are used to identify a thread during debugging or profiling.

**NOTE** Timetest identifiers differ from the kernel-generated thread identifiers used to specify threads in the API thread operations.

**Parameters**

| | | |
|---|---|---|
| in | *tid* | Timetest identifier. |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 3.31 qurt_thread_stid_set()

## 3.31.1 Function Documentation

### 3.31.1.1 int qurt_thread_stid_set ( char *stid* )

Sets the STID for a specified thread.

#### Associated data types

qurt_thread_t

#### Parameters

| | | |
|---|---|---|
| in | *stid* | Thread identifier. |

#### Returns

QURT_EOK – STID set created.
QURT_EFAILED – STID not set.

#### Safe API

No

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 3.32 qurt_thread_attr_set_group_id()

### 3.32.1 Function Documentation

#### 3.32.1.1 static void qurt_thread_attr_set_group_id ( qurt_thread_attr_t ∗ *attr,* unsigned int *group_id* )

Sets the group ID in the thread attributes.

The primordial or first thread has group ID zero. When a new thread is created without an assigned group_id, it inherits the group ID from its parent thread.

**NOTE** 1. Group ID can only be set before thread creation. Group ID cannot be changed after the thread is created. 2. If a non-activated group_id is passed, thread creation fails. 3. Only a thread with Group ID zero can set the Group ID for its child threads. 4. If a thread with a nonzero group ID set the group ID for its child threads, QuRT ignores this parameter and the child threads inherit the group ID of the parent thread. When the passed group ID is not activated, thread creation fails. 5. Input group_id will be bitmasked and trimmed to unsigned char before assignement.

**Associated data types**

qurt_thread_attr_t

**Parameters**

| | | |
|---|---|---|
| in | *attr* | Pointer to the thread attribute structure. |
| in | *group_id* | Group identifier; valid range is 0 through 63. |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 3.33 qurt_thread_get_proc_id()

## 3.33.1 Function Documentation

### 3.33.1.1 unsigned int qurt_thread_get_proc_id ( void )

Gets the processor identifier for a current thread.

The processor identifier include the hardware thread identifier, and core identifier.

The processor information returned is not guaranteed to be current. The caller must account for the possibility that the information returned may have changed by the time the function call completes, due to the dynamic nature of thread scheduling and preemption.

**Parameters**

| | |
|---|---|
| *None* | |

**Note**

caller shall use the following bitmask to read processor information QURT_THREAD_HTID_BITMASK – hardware threads Identifier. QURT_THREAD_COREID_BITMASK – core Identifier.

**Returns**

processor information for the current thread. If hardware thread information is unavailable, 0xFF will be returned as the hardware thread identifier. If core information is unavailable, 0xFF will be returned as the core identifier

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 3.34 qurt_thread_sched_yield()

### 3.34.1 Function Documentation

#### 3.34.1.1 int qurt_thread_sched_yield ( void )

Move the current software thread to the end of kernel scheduler's ready queue of the same priority as the current software thread. If there are no other software threads in the kernel scheduler's ready queue with the same priority, this function returns immediately. Otherwise, the current software thread is offloaded from the hardware thread and placed at the end of the ready queue, while the first task in the ready queue of the same priority is loaded onto the current hardware thread to run.

**Returns**

QURT_EOK – Success.
QURT_ENOTALLOWED – Operation is not allowed, e.g., wrong thread running conditions.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 3.35 qurt_thread_get_thread_id()

## 3.35.1 Function Documentation

### 3.35.1.1 int qurt_thread_get_thread_id ( qurt_thread_t ∗ *thread_id,* char ∗ *name* )

Gets the thread identifier of the thread with the name that matches in the same process of the caller.

#### Associated data types

qurt_thread_t

#### Parameters

| | | |
|---|---|---|
| out | *thread_id* | Pointer to the thread identifier. |
| in | *name* | Pointer to the name of the thread. |

#### Returns

QURT_EINVALID – There is no thread with a name that matches in the process of the caller.
QURT_EOK – Success.

#### Safe API

No

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 3.36 qurt_sleep()

### 3.36.1 Function Documentation

#### 3.36.1.1 void qurt_sleep ( unsigned long long int *duration* )

Suspends the current thread for the specified amount of time.

**NOTE** Because QuRT timers are deferrable, this call is guaranteed to block at least for the specified amount of time. If power collapse is enabled, the maximum amount of time this call can block depends on the earliest wakeup from power collapse past the specified duration.

**Parameters**

| | | |
|---|---|---|
| in | *duration* | Duration (in microseconds) for which the thread is suspended. |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 3.37 qurt_system_set_priority_floor()

### 3.37.1 Function Documentation

#### 3.37.1.1 int qurt_system_set_priority_floor ( unsigned int *priority_floor* )

Sets a priority floor to move threads with thread priority lower than the floor out of the running state. Running threads with thread priority lower than the priority floor are moved into the kernel ready queue, and they are not scheduled to run when the thread priority is lower than the floor. Later, the caller should reset the priority floor back to the default value of QURT_PRIORITY_FLOOR_DEFAULT. Threads in the kernel ready queue are scheduled to run when the thread priority is higher than the floor.

The priority floor is set and associated to the user process of the caller. When the caller enters QuRTOS and sets a new floor, the new floor is associated to its original user process rather than the QuRTOS process. The floor associated to the user process is reset when the user process exits or is killed, but not at the time when the user thread of the caller exits.

The priority floor cannot be set to a priority higher than the thread priority of the caller, or lower than the default QURT_PRIORITY_FLOOR_DEFAULT system floor.

This function is not supported in Island mode.

After the system floor is set above QURT_PRIORITY_FLOOR_DEFAULT, power collapse is skipped and the sleep task is not scheduled to run.

**Parameters**

| | | |
|---|---|---|
| in | *priority_floor* | Priority floor. |

**Returns**

QURT_EOK – Success
QURT_ENOTALLOWED – Floor setting is not allowed

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:46:42 GMT hyungchul.won@lge.com

## 3.38 qurt_thread_get_tls_base()

### 3.38.1 Function Documentation

#### 3.38.1.1 void∗ qurt_thread_get_tls_base ( qurt_tls_info ∗ *info* )

Gets the base address of thread local storage (TLS) of a dynamically loaded module for the current thread.

**Associated data types**

qurt_tls_info

**Parameters**

| | | |
|---|---|---|
| in | *info* | Pointer to the TLS information for a module. |

**Returns**

Pointer to the TLS object for the dynamically loaded module.
NULL – TLS information is invalid.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 3.39 qurt_busywait()

## 3.39.1 Function Documentation

### 3.39.1.1 void qurt_busywait ( unsigned int *pause_time_us* )

Pauses the execution of a thread for a specified time.

Use for small microsecond delays.

**NOTE** The function does not return to the caller until the time duration has expired.

**Parameters**

| | | |
|---|---|---|
| in | *pause_time_us* | Time to pause in microseconds. |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 3.40 Data Types

This section describes data types for thread services.

Threads in QuRT are identified by values of type qurt_thread_t.

Thread priorities in QuRT are identified by values of type unsigned short.

Thread attributes in QuRT are stored in structures of type qurt_thread_attr_t.

### 3.40.1 Define Documentation

#### 3.40.1.1 #define CCCC_PARTITION 0U

Use the CCCC page attribute bits to determine the main or auxiliary partition.

#### 3.40.1.2 #define MAIN_PARTITION 1U

Use the main partition.

#### 3.40.1.3 #define AUX_PARTITION 2U

Use the auxiliary partition.

#### 3.40.1.4 #define MINIMUM_PARTITION 3U

Use the minimum. Allocates the least amount of cache (no-allocate policy possible) for this thread.

### 3.40.2 Data Structure Documentation

#### 3.40.2.1 struct qurt_thread_attr_t

Thread attribute structure. Always use the `qurt_thread_attr_set_*()` APIs to set thread attributes.

#### 3.40.2.2 struct qurt_tls_info

Dynamic TLS attributes.

**Data fields**

| Type | Parameter | Description |
|---|---|---|
| unsigned int | module_id | Module ID of the loaded dynamic linked library. |
| unsigned int | tls_start | Start address of the TLS data. |
| unsigned int | tls_data_end | End address of the TLS RW data. |
| unsigned int | tls_end | End address of the TLS data. |

### 3.40.3 Typedef Documentation

#### 3.40.3.1 typedef unsigned int qurt_cache_partition_t

QuRT cache partition type.

#### 3.40.3.2 typedef unsigned int qurt_thread_t

Thread ID type.

Qualcomm Confidential - May Contain Trade Secrets 2026-05-07 05:40:42 GMT hyungkyu.kim@lge.com

### 3.40.4 Variable Documentation

#### 3.40.4.1 int thread_handle

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:40:42 GMT
hyungchul.won@lge.com

## 3.41 Constants and Macros

This section describes constants for thread services, and macros for thread configuration and QuRT thread attributes.

Bitmask configuration is for selecting DSP hardware threads. To select all the hardware threads, use QURT_THREAD_CFG_BITMASK_ALL.

### 3.41.1 Define Documentation

#### 3.41.1.1 #define QURT_MAX_HTHREAD_LIMIT 8U

Limit on the maximum number of hardware threads supported by QuRT for any

#### 3.41.1.2 #define QURT_THREAD_CFG_BITMASK_ALL 0x000000ffU

Select all the hardware threads.

#### 3.41.1.3 #define QURT_THREAD_BUS_PRIO_DISABLED 0

Thread internal bus priority disabled.

#### 3.41.1.4 #define QURT_THREAD_BUS_PRIO_ENABLED 1

Thread internal bus priority enabled.

#### 3.41.1.5 #define QURT_THREAD_AUTOSTACK_DISABLED 0

Thread has autostack v2 feature disabled.

#### 3.41.1.6 #define QURT_THREAD_AUTOSTACK_ENABLED 1

Thread has autostack v2 feature enabled.

#### 3.41.1.7 #define QURT_THREAD_ATTR_CREATE_LEGACY 0U

Create a legacy QuRT thread by default. If a thread has this as a detach state, the thread can be joined on until it exits.

#### 3.41.1.8 #define QURT_THREAD_ATTR_CREATE_JOINABLE 1U

Create a joinable thread.

#### 3.41.1.9 #define QURT_THREAD_ATTR_CREATE_DETACHED 2U

Create a detached thread.

#### 3.41.1.10 #define QURT_THREAD_ATTR_TCB_PARTITION_DEFAULT QURT_THREAD_ATTR_TCB_PARTITION_RAM

Backward compatibility.

#### 3.41.1.11 #define QURT_THREAD_ATTR_PRIORITY_DEFAULT 254

Priority.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 03:40:42 GMT kyungchul.won@nxp.com

#### 3.41.1.12 #define QURT_THREAD_ATTR_ASID_DEFAULT 0

ASID.

#### 3.41.1.13 #define QURT_THREAD_ATTR_AFFINITY_DEFAULT (-1)

Affinity.

#### 3.41.1.14 #define QURT_THREAD_ATTR_BUS_PRIO_DEFAULT 255

Bus priority.

#### 3.41.1.15 #define QURT_THREAD_ATTR_AUTOSTACK_DEFAULT 0

Default autostack v2 disabled thread.

#### 3.41.1.16 #define QURT_THREAD_ATTR_TIMETEST_ID_DEFAULT (-2)

Timetest ID.

#### 3.41.1.17 #define QURT_THREAD_ATTR_STID_DEFAULT QURT_STID_DEFAULT

STID.

#### 3.41.1.18 #define QURT_THREAD_ATTR_STID_ENABLE 1

Indicate to allocate STID during thread creation.

#### 3.41.1.19 #define QURT_PRIORITY_FLOOR_DEFAULT 255U

Default floor.

#### 3.41.1.20 #define QURT_THREAD_RESUME_DEFAULT 0x0

Resume default.

#### 3.41.1.21 #define QURT_THREAD_HTID_BITMASK 0x000000ffU

Select hardware threads Identifier.

#### 3.41.1.22 #define QURT_THREAD_COREID_BITMASK 0x0000ff00U

Select cores Identifier.

#### 3.41.1.23 #define QURT_THREAD_HTID_SHIFT 0x0U

Shift for hardware threads Identifier.

#### 3.41.1.24 #define QURT_THREAD_COREID_SHIFT 0x8U

Shift for cores Identifier.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:19:42 GMT hyungchul.won@lg.com

# 4 Processes

## 4.1 Overview

A process is a grouping of an executable program, an address space, and one or more threads. Each thread in a process shares the process memory area.

A process cannot access the memory in another process, except through the use of an OS-defined mechanism for resource sharing. QuRT uses the QDI framework to share resources across processes - for more information refer to [80-VB419-84].

Processes are represented as shared objects in QuRT. Process objects support the following operations:

- Create process – Creates a new process with the specified attributes.
- Get current process – Returns a reference to the current process.

When a process is created it automatically starts running the code in the specified executable file. A newly-created process is assigned an identifier value which uniquely identifies the process.

The get current process operation returns the process identifier for the current process.

**Process attributes**
Processes have the following attributes:

- Name - Character string identifier used to uniquely identify a process.
- Flags - Bit array used to specify properties of a process.

The process name specifies a process object which is already loaded in memory as part of the QuRT system.

The processor flags specify properties of a newly-created process. The properties are represented as defined symbols which map into bits 0-31 of the 32-bit flag value. Multiple properties are specified by OR'ing together the individual property symbols.

**Functions**
Process services are accessed with the following QuRT functions:

- qurt_process_create()
- qurt_process_get_id()
- qurt_process_attr_init()
- qurt_process_attr_set_executable()
- qurt_process_attr_set_binary_path
- qurt_process_attr_set_dtb_path()
- qurt_process_attr_set_flags()

Qualcomm Confidential - May Contain Trade Secrets
2026-02-27 05:40:42 GMT
hyungsul.wan@qce.com

- qurt_process_cmdline_get()
- qurt_process_attr_set_max_threads()
- qurt_process_attr_set_sw_id()
- qurt_process_attr_set_ceiling_prio()
- qurt_process_get_thread_count()
- qurt_process_get_thread_ids()
- qurt_process_attr_get()
- qurt_process_dump_register_cb()
- qurt_process_dump_deregister_cb()
- qurt_process_exit()
- qurt_process_kill()
- qurt_debugger_register_process()
- qurt_debugger_deregister_process()
- qurt_process_exec_callback()
- qurt_process_get_pid()
- qurt_process_get_dm_status()
- qurt_process_vtcm_window_set()
- qurt_process_vtcm_window_set2()
- qurt_process_vtcm_window_get()
- qurt_process_vtcm_window_get2()
- Data Types
- Macros

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 4.2 qurt_process_create()

### 4.2.1 Function Documentation

#### 4.2.1.1 int qurt_process_create ( qurt_process_attr_t * *attr* )

Creates a process with the specified attributes, and starts the process.

The process executes the code in the specified ELF.

**Associated data types**

qurt_process_attr_t

**Parameters**

| | | |
|---|---|---|
| in | *attr* | Process attribute structure, which specifies the attributes of the created process. |

**Returns**

Postive return value – Process ID
Negative return value – Error:
QURT_EPRIVILEGE – Caller does not have privileges for this operation
QURT_EMEM – Not enough memory to perform the operation
QURT_EFAILED – Operation failed
QURT_ENOTALLOWED – Operation not allowed
QURT_ENOREGISTERED – Not registered
QURT_ENORESOURCE – Resource exhaustion
QURT_EINVALID – Invalid argument value
QURT_EFATAL – attr is NULL

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 4.3 qurt_process_get_id()

### 4.3.1 Function Documentation

#### 4.3.1.1 int qurt_process_get_id ( void )

Returns the process identifier for the current thread.

**Safe API**

No

**Returns**

Process identifier for the current thread.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 4.4 qurt_process_attr_init()

### 4.4.1 Function Documentation

#### 4.4.1.1 static void qurt_process_attr_init ( qurt_process_attr_t * *attr* )

Initializes the structure that sets the process attributes when a thread is created.

After an attribute structure is initialized, use the process attribute operations to explicitly set the individual attributes in the structure.

Table 4-2 lists the default attribute values set by the initialize operation.

**Table 4-2 Process attribute defaults**

| Attribute | Default value |
|---|---|
| Name | Null string |
| Flags | 0 |

**Associated data types**

qurt_process_attr_t

**Parameters**

| | | |
|---|---|---|
| `out` | *attr* | Pointer to the structure to initialize. |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 4.5 qurt_process_attr_set_executable()

### 4.5.1 Function Documentation

#### 4.5.1.1 void qurt_process_attr_set_executable ( qurt_process_attr_t ∗ *attr,* const char ∗ *name* )

Sets the process name in the specified process attribute structure.

Process names identify process objects that are already loaded in memory as part of the QuRT system.

**NOTE** Process objects are incorporated into the QuRT system at build time.

**NOTE** Maximum length of name string is limited to QURT_PROCESS_ATTR_NAME_MAXLEN - 1.

**Associated data types**

qurt_process_attr_t

**Parameters**

| | | |
|---|---|---|
| in | *attr* | Pointer to the process attribute structure. |
| in | *name* | Pointer to the process name. |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 4.6 qurt_process_attr_set_binary_path

### 4.6.1 Function Documentation

#### 4.6.1.1 void qurt_process_attr_set_binary_path ( qurt_process_attr_t * *attr,* char * *path* )

Sets the binary path for the process that loads in the specified process attribute structure.

Path specifies the binary to load for this process.

**NOTE** The maximum length of the path string is limited to QURT_PROCESS_ATTR_BIN_PATH_MAXLEN - 1.

**Associated data types**

qurt_process_attr_t

**Parameters**

| | | |
|---|---|---|
| in | *attr* | Pointer to the process attribute structure. |
| in | *path* | Pointer to the binary path. |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 4.7 qurt_process_attr_set_dtb_path()

### 4.7.1 Function Documentation

#### 4.7.1.1 void qurt_process_attr_set_dtb_path ( qurt_process_attr_t ∗ *attr,* char ∗ *path* )

Sets the DTB binary path for the process that loads in the specified process attribute structure.

The path parameter specifies the DTB binary to load for this process.

**NOTE** The maximum length of the path string is limited to QURT_PROCESS_ATTR_BIN_PATH_MAXLEN - 1.

**Associated data types**

qurt_process_attr_t

**Parameters**

| | | |
|---|---|---|
| in | *attr* | Pointer to the process attribute structure. |
| in | *path* | Pointer to the binary path. |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 4.8 qurt_process_attr_set_flags()

### 4.8.1 Function Documentation

#### 4.8.1.1 static void qurt_process_attr_set_flags ( qurt_process_attr_t * *attr,* int *flags* )

Sets the process properties in the specified process attribute structure. Process properties are represented as defined symbols that map into bits 0 through 31 of the 32-bit flag value. To specify multiple properties, OR together the individual property symbols.

**Associated data types**

qurt_process_attr_t

**Parameters**

| | | |
|---|---|---|
| in | *attr* | Pointer to the process attribute structure. |
| in | *flags* | Values:<br>QURT_PROCESS_NON_SYSTEM_CRITICAL Process is considered non system-critical. Error services use this attribute to decide whether to kill the user PD or the entire subsystem.<br>QURT_PROCESS_ISLAND_RESIDENT Process is marked as an island resident.<br>QURT_PROCESS_RESTARTABLE Process is marked as restartable.<br>QURT_PROCESS_UNTRUSTED Process is marked as an unsigned process. |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 4.9 qurt_process_cmdline_get()

## 4.9.1 Function Documentation

### 4.9.1.1 void qurt_process_cmdline_get ( char * *buf,* unsigned *buf_siz* )

Gets the command line string associated with the current process. This function retrieves the Hexagon simulator command line arguments as long as the call is made in the process of the QuRT installation, and with the requirement that the program runs in a simulation environment.

If the function modifies the provided buffer, it zero-terminates the string. It is possible that the function does not modify the provided buffer, so the caller must set buf[0] to a NULL byte before making the call. A truncated command line is returned when the command line is longer than the provided buffer.

**Parameters**

| | | |
|---|---|---|
| in | *buf* | Pointer to a character buffer that must be filled in. |
| in | *buf_siz* | Size (in bytes) of the buffer pointed to by the buf argument. |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 4.10 qurt_process_attr_set_max_threads()

### 4.10.1 Function Documentation

#### 4.10.1.1 static void qurt_process_attr_set_max_threads ( qurt_process_attr_t * *attr,* unsigned *max_threads* )

Sets the maximum number of threads allowed in the specified process attribute structure.

**Associated data types**

qurt_process_attr_t

**Parameters**

| | | |
|---|---|---|
| in | *attr* | Pointer to the process attribute structure. |
| in | *max_threads* | Maximum number of threads allowed for this process. |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 4.11 qurt_process_attr_set_sw_id()

## 4.11.1 Function Documentation

### 4.11.1.1 static void qurt_process_attr_set_sw_id ( qurt_process_attr_t * *attr,* unsigned int *sw_id* )

Sets the software ID of the process to load in the specified process attribute structure.

#### Associated data types

qurt_process_attr_t

#### Parameters

| | | |
|---|---|---|
| in | *attr* | Pointer to the process attribute structure. |
| in | *sw_id* | Software ID of the process, used in authentication. |

#### Safe API

No

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 4.12 qurt_process_attr_set_ceiling_prio()

### 4.12.1 Function Documentation

#### 4.12.1.1 static void qurt_process_attr_set_ceiling_prio ( qurt_process_attr_t ∗ *attr,* unsigned short *prio* )

Sets the highest thread priority allowed in the specified process attribute structure. Refer to qurt_thread.h for priority ranges.

**Associated data types**

qurt_process_attr_t

**Parameters**

| | | |
|---|---|---|
| in | *attr* | Pointer to the process attribute structure. |
| in | *prio* | Priority. |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 4.13 qurt_process_get_thread_count()

## 4.13.1 Function Documentation

### 4.13.1.1 int qurt_process_get_thread_count ( unsigned int *pid* )

Gets the number of threads present in the process indicated by the PID.

**Parameters**

| | | |
|---|---|---|
| in | *pid* | PID of the process for which the information is required. |

**Returns**

Number of threads in the process indicated by PID, if a positive value is obtained.
Negative error code if failed include:
QURT_EFATAL - Invalid PID
QURT_ENOTALLOWED - Current process does not have access to target process indicated by PID

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 4.14 qurt_process_get_thread_ids()

### 4.14.1 Function Documentation

#### 4.14.1.1 int qurt_process_get_thread_ids ( unsigned int *pid,* unsigned int ∗ *ptr,* unsigned *thread_num* )

Gets the thread IDs for a process indicated by PID.

**Parameters**

| | | |
|---|---|---|
| in | *pid* | PID of the process for which the information is required. |
| in | *ptr* | Pointer to a user passed buffer that must be filled in with thread IDs. |
| in | *thread_num* | Number of thread IDs requested. |

**Returns**

QURT_EOK - Success
QURT_EFATAL - Failed, ptr is NULL

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 4.15 qurt_process_attr_get()

## 4.15.1 Function Documentation

### 4.15.1.1 int qurt_process_attr_get ( unsigned int *pid,* qurt_process_attr_t ∗ *attr* )

Gets the attributes of the process with which it was created.

#### Associated data types

qurt_process_attr_t

#### Parameters

| | | |
|---|---|---|
| in | *pid* | PID of the process for which the information is required. |
| in,out | *attr* | Pointer to the user allocated attribute structure. |

#### Returns

QURT_EOK - Success
QURT_EINVALID - Invalid PID
QURT_EFATAL - attr is NULL

#### Safe API

No

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 4.16 qurt_process_dump_register_cb()

### 4.16.1 Function Documentation

#### 4.16.1.1 int qurt_process_dump_register_cb ( qurt_cb_data_t ∗ *cb_data,* qurt_-process_dump_cb_type_t *type,* unsigned short *priority* )

Registers the process domain dump callback.

**Associated data types**

qurt_cb_data_t
qurt_process_dump_cb_type_t

**Parameters**

| | | |
|---|---|---|
| in | *cb_data* | Pointer to the callback information. |
| in | *type* | Callback type; these callbacks are called in the context of the user process domain.<br>QURT_PROCESS_DUMP_CB_PRESTM – Before threads of the exiting process are frozen.<br>QURT_PROCESS_DUMP_CB_ERROR – After threads are frozen and captured.<br>QURT_PROCESS_DUMP_CB_ROOT – After threads are frozen and captured, and CB_ERROR type of callbacks are called. |
| in | *priority* | Priority. |

**Returns**

QURT_EOK – Success
QURT_EFATAL – Failure; cb_data is NULL
QURT_EINVALID – Failure; invalid cb_type
QURT_EFAILED – Failure; invalid cb_data

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 03:40:42 GMT hyungchul.kim@lge.com

## 4.17 qurt_process_dump_deregister_cb()

### 4.17.1 Function Documentation

#### 4.17.1.1 int qurt_process_dump_deregister_cb ( qurt_cb_data_t * *cb_data,* qurt_process_dump_cb_type_t *type* )

Deregisters the process domain dump callback.

**Associated data types**

qurt_cb_data_t
qurt_process_dump_cb_type_t

**Parameters**

| | | |
|---|---|---|
| in | *cb_data* | Pointer to the callback information to deregister. |
| in | *type* | Callback type. |

**Returns**

QURT_EOK – Success.
QURT_EFATAL – Failure; cb_data is NULL
QURT_EINVALID – Failure; invalid cb_type
QURT_EFAILED – Failure; invalid cb_data

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 4.18 qurt_process_exit()

### 4.18.1 Function Documentation

#### 4.18.1.1 int qurt_process_exit ( int *exitcode* )

Exits the current user process with an exit code.

**Parameters**

| | | |
|---|---|---|
| in | *exitcode* | Exit code. |

**Returns**

QURT_EFATAL – No client is found with the specified PID value.
QURT_EINVALID – Invalid client.
QURT_ENOTALLOWED – User does not have permission to perform this operation.
QURT_EOK – Success.

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 4.19 qurt_process_kill()

## 4.19.1 Function Documentation

### 4.19.1.1 int qurt_process_kill ( int *pid,* int *exitcode* )

Kills the process represented by the PID with the exit code.

**Parameters**

| | | |
|---|---|---|
| in | *pid* | PID of the process to kill. |
| in | *exitcode* | Exit code. |

**Returns**

QURT_EFATAL – No client is found with the specified PID value.
QURT_EINVALID – Invalid client.
QURT_ENOTALLOWED – User does not have permission to perform this operation.
QURT_EOK – Success.

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 4.20 qurt_debugger_register_process()

### 4.20.1 Function Documentation

#### 4.20.1.1 int qurt_debugger_register_process ( int *pid,* unsigned int *adr* )

Registers the process indicated by the PID with the debug monitor.

**Parameters**

| | | |
|---|---|---|
| in | *pid* | PID of the process. |
| in | *adr* | Address. |

**Returns**

QURT_EOK – Success.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 4.21 qurt_debugger_deregister_process()

### 4.21.1 Function Documentation

#### 4.21.1.1 int qurt_debugger_deregister_process ( int *pid* )

Deregister the process indicated by the PID with the debug monitor.

**Parameters**

| | | |
|---|---|---|
| in | *pid* | PID of the process. |

**Returns**

QURT_EOK – Success

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 4.22 qurt_process_exec_callback()

### 4.22.1 Function Documentation

#### 4.22.1.1 int qurt_process_exec_callback ( int *client_handle,* unsigned *callback_fn,* unsigned *stack_base,* unsigned *stack_size* )

Executes callbacks in the user process as indicated by the client_handle argument.

**Parameters**

| | | |
|---|---|---|
| in | *client_handle* | Client handle obtained from the current invocation function. See Section 3.4.1 in the Hexagon QuRT QDI User Guide (80-VB419-84). |
| in | *callback_fn* | Callback function to execute. |
| in | *stack_base* | Stack address to use. |
| in | *stack_size* | Stack size. |

**Returns**

QURT_EOK – Success.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 4.23 qurt_process_get_pid()

### 4.23.1 Function Documentation

#### 4.23.1.1 int qurt_process_get_pid ( int *client_handle,* int ∗ *pid* )

Gets the process ID of the process that the client_handle argument represents.

**NOTE** This API is not supported for unsigned PDs. For unsigned PDs, use qurt_process_get_id().

**Parameters**

| | | |
|---|---|---|
| in | *client_handle* | Client handle obtained from the current invocation function. See Section 3.4.1 in the Hexagon QuRT QDI User Guide (80-VB419-84). |
| in | *pid* | Pointer to the address to store the PID. |

**Returns**

QURT_EOK – Success.
QURT_EFATAL – PID pointer passed as NULL.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 4.24 qurt_process_get_dm_status()

### 4.24.1 Function Documentation

#### 4.24.1.1 int qurt_process_get_dm_status ( unsigned int *pid,* unsigned int ∗ *status* )

Gets the debugging session status on the process represented by the PID argument.

**Parameters**

| | | |
|---|---|---|
| in | *pid* | Process ID. |
| in,out | *status* | Address to store the status:<br>QURT_DEBUG_NOT_START<br>QURT_DEBUG_START |

**Returns**

QURT_EOK - Success.
QURT_EINVALID - Error.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 4.25 qurt_process_vtcm_window_set()

## 4.25.1 Function Documentation

### 4.25.1.1 int qurt_process_vtcm_window_set ( int *pid,* unsigned int *enable,* unsigned int *high_offset,* unsigned int *low_offset* )

Sets a VTCM access window for a process and its thread groups. The caller thread must be in the SRM process.

This synchronous function ensures that all running threads of the process have the requested window in effect. The requested view for all non-running thread takes effect when they are scheduled.

**Parameters**

| | | |
|---|---|---|
| in | *pid* | Process identifier. |
| in | *enable* | Values:<br>QURT_VTCM_WINDOW_ENABLE – Enforces the VTCM access window defined by the high and low offset.<br>QURT_VTCM_WINDOW_DISABLE – High and low offset is ignored and VTCM access is fully disabled for the process. |
| in | *high_offset* | Specifies the high window offset in 4K increments from the base address of the VTCM.<br>QURT_VTCM_WINDOW_HI_OFFSET_DEFAULT – Restore the high offset to the reset value. |
| in | *low_offset* | Specifies the low window offset in 4K increments from the base address of the VTCM.<br>QURT_VTCM_WINDOW_LO_OFFSET_DEFAULT – Restore thelow offset to the reset value. |

**NOTE** When high_offset is set to QURT_VTCM_WINDOW_HI_OFFSET_DEFAULT and low offset is set as QURT_VTCM_WINDOW_LO_OFFSET_DEFAULT, the full VTCM range is accessible. Access to VTCM is controlled via MMU mapping for the process.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 15:29:42 GMT hyungchul.wo@qc.com

**Returns**

QURT_EOK – Success.
QURT_EVAL – Failure due to invalid inputs.
QURT_EPRIVILEGE – Failure because the caller does not have enough privilege for this operation.
QURT_ENOTSUPPORTED – Failure because the operation is not supported due to hardware limitation.

**Safe API**

No

**Dependencies**

None.

## 4.26 qurt_process_vtcm_window_set2()

### 4.26.1 Function Documentation

#### 4.26.1.1 int qurt_process_vtcm_window_set2 ( unsigned int *pid,* unsigned int *group_id,* unsigned int *enable,* unsigned int *high_offset,* unsigned int *low_offset* )

The caller thread must be in the SRM process.

This synchronous function ensures that all running threads of the process have the requested window in effect. The requested view for all non-running thread takes effect when they are scheduled.

**Parameters**

| | | |
|---|---|---|
| in | *pid* | Process identifier.‘ |
| in | *group_id* | group identifier |
| in | *enable* | Values:<br>QURT_VTCM_WINDOW_ENABLE – Enforces the VTCM access window defined by the high and low offset.<br>QURT_VTCM_WINDOW_DISABLE – High and low offset is ignored and VTCM access is fully disabled for the process. |
| in | *high_offset* | Specifies the high window offset in 4K increments from the base address of the VTCM.<br>QURT_VTCM_WINDOW_HI_OFFSET_DEFAULT – Restore the high offset to the reset value. |
| in | *low_offset* | Specifies the low window offset in 4K increments from the base address of the VTCM.<br>QURT_VTCM_WINDOW_LO_OFFSET_DEFAULT – Restore thelow offset to the reset value. |

**NOTE** When high_offset is set to QURT_VTCM_WINDOW_HI_OFFSET_DEFAULT and low offset is set as QURT_VTCM_WINDOW_LO_OFFSET_DEFAULT, the full VTCM range is accessible. Access to VTCM is controlled via MMU mapping for the process.

**NOTE** User can pass default group_id (QURT_THREAD_DEFAULT_GROUP_ID) to set a process-level vtcm window. When a non-default group_id is specified, the vtcm window is set only or a thread group within a process.

**Returns**

QURT_EOK – Success.
QURT_EVAL – Failure due to invalid inputs.
QURT_ENOTSUPPORTED – Failure because the operation is not supported due to hardware limitation.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2025-03-27 10:42 GMT hyungchulwon@naver.com

## 4.27 qurt_process_vtcm_window_get()

### 4.27.1 Function Documentation

#### 4.27.1.1 int qurt_process_vtcm_window_get ( int *pid*, unsigned int ∗ *enable*, unsigned int ∗ *high_offset*, unsigned int ∗ *low_offset* )

Get the VTCM window for a process. The caller thread must be in the SRM process.

**Parameters**

| | | |
|---|---|---|
| in | *pid* | Process identifier. |
| out | *enable* | Address to store the enable status when set. |
| out | *high_offset* | Address to return the high window offset in 4K increments from the base address of the VTCM. |
| out | *low_offset* | Address to return the low window offset in 4K increments from the base address of the VTCM. |

**NOTE** The user must first check the returned value of enable before it checks high and low offset.

**Returns**

QURT_EOK – Success.
QURT_EVAL – Failure because of invalid inputs.
QURT_EPRIVILEGE – Failure because caller does not have privileges for this operation.
QURT_ENOTSUPPORTED – Failure because the operation is not supported due to hardware limitation.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 4.28 qurt_process_vtcm_window_get2()

### 4.28.1 Function Documentation

#### 4.28.1.1 int qurt_process_vtcm_window_get2 ( unsigned int *pid,* unsigned int *group_id,* unsigned int ∗ *enable,* unsigned int ∗ *high_offset,* unsigned int ∗ *low_offset* )

Get the VTCM window for a process or or for a thread group within a process. The caller thread must be in the SRM process.

**Parameters**

| | | |
|---|---|---|
| in | *pid* | Process identifier. |
| in | *group_id* | group identifier |
| out | *enable* | Address to store the enable status when set. |
| out | *high_offset* | Address to return the high window offset in 4K increments from the base address of the VTCM. |
| out | *low_offset* | Address to return the low window offset in 4K increments from the base address of the VTCM. |

**NOTE** To get a process-level vtcm window when set, user can pass default group_id (QURT_THREAD_DEFAULT_GROUP_ID) When a non-default group_id is specified, the vtcm window is returned for a thread group when set.

**NOTE** The user must first check the returned value of enable before it checks high and low offset.

**Returns**

QURT_EOK – Success.
QURT_EVAL – Failure because of invalid inputs.
QURT_ENOTSUPPORTED – Failure because the operation is not supported due to hardware limitation.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:49:42 GMT hyungchul.won@kde.com

## 4.29 Data Types

This section describes data types for processes.

### 4.29.1 Define Documentation

#### 4.29.1.1 #define QURT_PROCESS_ATTR_NAME_MAXLEN QURT_MAX_NAME_LEN

Maximum length of the process name.

#### 4.29.1.2 #define QURT_PROCESS_ATTR_BIN_PATH_MAXLEN 128

Maximum length of the path of binary/ELF for this process.

#### 4.29.1.3 #define QURT_PROCESS_DEFAULT_CEILING_PRIO 0

Default ceiling priority of the threads in the new process.

#### 4.29.1.4 #define QURT_PROCESS_DEFAULT_MAX_THREADS -1

Default number of threads in the new process. -1 indicates that the limit is set to the maximum supported by the system.

#### 4.29.1.5 #define QURT_PROCESS_NON_SYSTEM_CRITICAL (1u << 1)

Starts the new process as non system-critical.

#### 4.29.1.6 #define QURT_PROCESS_ISLAND_RESIDENT (1u << 2)

Process is island resident.

#### 4.29.1.7 #define QURT_PROCESS_RESTARTABLE (1u << 3)

Indicates that the process is restartable

#### 4.29.1.8 #define QURT_PROCESS_UNTRUSTED (1u << 7)

Starts the new process as unsigned process.

#### 4.29.1.9 #define QURT_DEBUG_NOT_START 0

Debug is not started.

#### 4.29.1.10 #define QURT_DEBUG_START 1

Debug has started.

#### 4.29.1.11 #define QURT_PROCESS_SUSPEND_DEFAULT 0

Process suspend options

#### 4.29.1.12 #define QURT_PROCESS_RESUME_DEFAULT 0

Process resume options

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:49:42 GMT hyungchul.yon@lge.com

## 4.29.2 Data Structure Documentation

### 4.29.2.1 struct qurt_pd_dump_attr_t

QuRT process dump attributes.

### 4.29.2.2 struct qurt_process_attr_t

internal_only QuRT process attributes.

## 4.29.3 Enumeration Type Documentation

### 4.29.3.1 enum qurt_process_type_t

**Enumerator:**

***QURT_PROCESS_TYPE_RESERVED*** Process type is reserved.
***QURT_PROCESS_TYPE_KERNEL*** Kernel process.
***QURT_PROCESS_TYPE_SRM*** SRM process.
***QURT_PROCESS_TYPE_SECURE*** Secure process.
***QURT_PROCESS_TYPE_ROOT*** Root process.
***QURT_PROCESS_TYPE_USER*** User process.

### 4.29.3.2 enum qurt_process_dump_cb_type_t

QuRT process callback types.

**Enumerator:**

***QURT_PROCESS_DUMP_CB_ROOT*** Register the callback that executes in the root process context.
***QURT_PROCESS_DUMP_CB_ERROR*** Register the user process callback that is called after threads in the process are frozen.
***QURT_PROCESS_DUMP_CB_PRESTM*** Register the user process callback that is called before threads in the process are frozen.
***QURT_PROCESS_DUMP_CB_MAX*** Reserved for error checking.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:49:42 GMT hyungchul.won@lge.com

## 4.30 Macros

### 4.30.1 Define Documentation

#### 4.30.1.1 #define QURT_VTCM_WINDOW_ENABLE 1U

VTCM window enable.

#### 4.30.1.2 #define QURT_VTCM_WINDOW_DISABLE 0U

VTCM window disable.

#### 4.30.1.3 #define QURT_VTCM_WINDOW_HI_OFFSET_DEFAULT 0xFFFU

VTCM window high offset default.

#### 4.30.1.4 #define QURT_VTCM_WINDOW_LO_OFFSET_DEFAULT 0U

VTCM window low offset default.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 5 Mutexes

## 5.1 Overview

Threads use mutexes to synchronize their execution to ensure mutually exclusive access to shared resources. Mutexes are shared objects which support the following operations:

- Init mutex – Initialize mutex.
- Destroy mutex – Destroy the specified mutex.
- Lock mutex – Request access to a shared resource.
- Unlock mutex – Release access to a shared resource.
- Try lock mutex –Request access to a shared resource (does not suspend).

If a thread performs a lock operation on a mutex that is not being used, the thread gains access to the shared resource that is protected by the mutex, and continues executing.

If a thread performs a lock operation on a mutex that is already being used by another thread, the thread is suspended on the mutex. When the mutex becomes available (because the other thread has unlocked it), the suspended thread is awakened and gains access to the shared resource.

More than one thread can be suspended on a mutex. When the mutex is unlocked, only the highest-priority thread waiting on the mutex is awakened. If the awakened thread has higher priority than the current thread, a context switch may occur.

Figure 5-1 shows an example of using mutexes.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

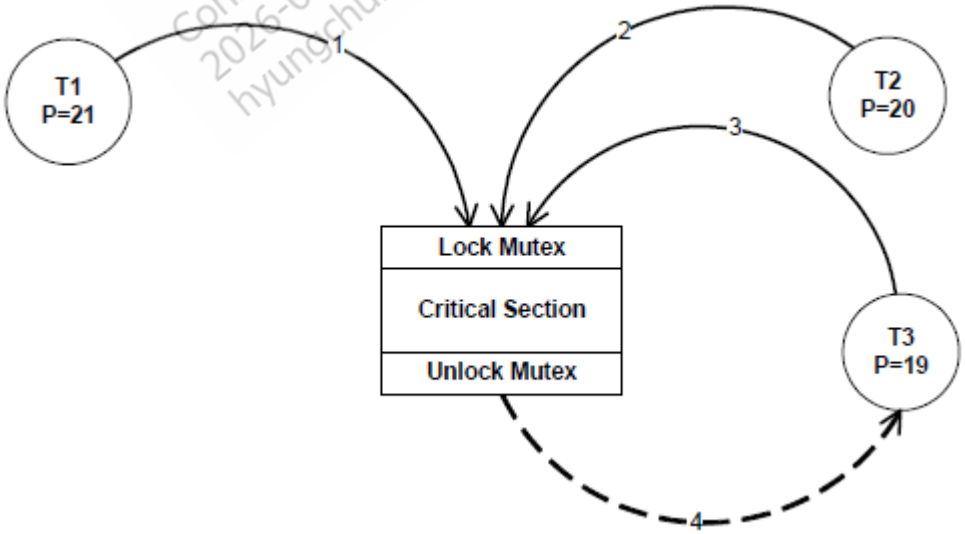


**Figure 5-1 Mutex example**

In figure 5-1 the following sequence of events occurs:

1. Thread T1 successfully locks the mutex and enters the critical section of code that is protected by the mutex.
2. Thread T2 attempts to lock the mutex but is blocked by T1. T2 is suspended on the mutex.
3. Thread T3 also tries to lock the mutex and it too is suspended. Because the thread priority of T3 is higher than the priority of T2, T3 is inserted into the mutex wait queue ahead of T2.
4. T1 exits the critical section and unlocks the mutex. T3 is selected from the mutex wait queue and awakened (because it is the highest-priority thread waiting on the mutex). Because T3 has higher priority than T1 (19 versus 21 respectively), T1 is suspended and T3 resumes execution, locking the mutex and entering the critical section.

The try lock operation enables a thread to try locking a mutex without the risk of getting suspended if the mutex is already locked:

- If the mutex is unlocked, try lock is identical to the regular lock operation.
- If the mutex is locked, try lock returns with a value indicating the locked state.

**Functions**

Mutexes are shared objects which support the following operations:

- qurt_mutex_destroy()
- qurt_mutex_init()
- qurt_mutex_lock()
- qurt_mutex_try_lock()
- qurt_mutex_unlock()
- qurt_mutex_lock_timed()
- Data Types
- Constants and Macros

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 5.2 qurt_mutex_destroy()

### 5.2.1 Function Documentation

#### 5.2.1.1 void qurt_mutex_destroy ( qurt_mutex_t * *lock* )

Destroys the specified mutex.

**NOTE** Mutexes must be destroyed when they are no longer in use. Failure to do this causes resource leaks in the QuRT kernel.

Mutexes must not be destroyed while they are still in use. If this occurs, the behavior of QuRT is undefined.

**Associated data types**

qurt_mutex_t

**Parameters**

| | | |
|---|---|---|
| in | *lock* | Pointer to the mutex object to destroy. |

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 5.3 qurt_mutex_init()

### 5.3.1 Function Documentation

#### 5.3.1.1 void qurt_mutex_init ( qurt_mutex_t * *lock* )

Initializes a mutex object. The mutex is initially unlocked.

**NOTE** Each mutex-based object has one or more kernel resources associated with it; to prevent resource leaks, call qurt_mutex_destroy() when this object is not used anymore

**Associated data types**

qurt_mutex_t

**Parameters**

| | | |
|---|---|---|
| out | *lock* | Pointer to the mutex object. Returns the initialized object. |

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 5.4 qurt_mutex_lock()

### 5.4.1 Function Documentation

#### 5.4.1.1 void qurt_mutex_lock ( qurt_mutex_t * *lock* )

Locks the specified mutex. If a thread performs a lock operation on a mutex that is not in use, the thread gains access to the shared resource that is protected by the mutex, and continues to execute.

If a thread performs a lock operation on a mutex that is already in use by another thread, the thread is suspended. When the mutex becomes available again (because the other thread has unlocked it), the thread is awakened and given access to the shared resource.

**NOTE** A thread is suspended indefinitely if it locks a mutex that it has already locked. To avoid this, use recursive mutexes (Section 6).

**Associated data types**

qurt_mutex_t

**Parameters**

| | | |
|---|---|---|
| in | *lock* | Pointer to the mutex object. Specifies the mutex to lock. |

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 5.5 qurt_mutex_try_lock()

### 5.5.1 Function Documentation

#### 5.5.1.1 int qurt_mutex_try_lock ( qurt_mutex_t ∗ *lock* )

Attempts to lock the specified mutex. If a thread performs a try_lock operation on a mutex that is not in use, the thread gains access to the shared resource that is protected by the mutex, and continues to execute.

**NOTE** If a thread performs a try_lock operation on a mutex that it has already locked or is in use by another thread, qurt_mutex_try_lock immediately returns with a nonzero result value.

**Associated data types**

qurt_mutex_t

**Parameters**

| | | |
|---|---|---|
| in | *lock* | Pointer to the mutex object. Specifies the mutex to lock. |

**Returns**

0 – Success.
Nonzero – Failure.

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 5.6 qurt_mutex_unlock()

### 5.6.1 Function Documentation

#### 5.6.1.1 void qurt_mutex_unlock ( qurt_mutex_t * *lock* )

Unlocks the specified mutex.

More than one thread can be suspended on a mutex. When the mutex is unlocked, only the highest-priority thread that waits on the mutex is awakened. If the awakened thread has higher priority than the current thread, a context switch occurs.

**NOTE** The behavior of QuRT is undefined if a thread unlocks a mutex that it did not first lock.

**Associated data types**

qurt_mutex_t

**Parameters**

| | | |
|---|---|---|
| in | *lock* | Pointer to the mutex object. Specifies the mutex to unlock. |

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 5.7 qurt_mutex_lock_timed()

### 5.7.1 Function Documentation

#### 5.7.1.1 int qurt_mutex_lock_timed ( qurt_mutex_t * *lock*, unsigned long long int *duration* )

Locks the specified mutex. When a thread performs a lock operation on a mutex that is not in use, the thread gains access to the shared resource that is protected by the mutex, and continues to execute.

When a thread performs a lock operation on a mutex that is already in use by another thread, the thread is suspended. When the mutex becomes available again (because the other thread has unlocked it), the thread is awakened and given access to the shared resource. If the duration of suspension exceeds the timeout duration, wait is terminated and no access to mutex is granted.

**Associated data types**

qurt_mutex_t

**Parameters**

| | | |
|---|---|---|
| in | *lock* | Pointer to the mutex object; specifies the mutex to lock. |
| in | *duration* | Interval (in microseconds) that the duration value must be between QURT_TIMER_MIN_DURATION and QURT_TIMER_MAX_DURATION. |

**Returns**

QURT_EOK – Success
QURT_ETIMEDOUT – Timeout

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 5.8 Data Types

This section describes data types for mutex services.

Mutexes are represented in QuRT as objects of type qurt_mutex_t.

### 5.8.1 Typedef Documentation

#### 5.8.1.1 typedef union qurt_mutex_aligned8 qurt_mutex_t

QuRT mutex type.

Both non-recursive mutex lock and unlock, and recursive mutex lock and unlock can be applied to this type.

#### 5.8.1.2 typedef struct qurt_rmutex2 qurt_rmutex2_t

QuRT rmutex2 type.

Mutex type used with rmutex2 APIs.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 5.9 Constants and Macros

This section describes constants and macros for mutex services.

### 5.9.1 Define Documentation

#### 5.9.1.1 #define MUTEX_MAGIC 0xfe

#### 5.9.1.2 #define QURT_MUTEX_INIT {{MUTEX_MAGIC, 0, QURTK_FUTEX_FREE_MAGIC,0}}

Suitable as an initializer for a variable of type qurt_mutex_t.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 6 Recursive Mutexes

## 6.1 Overview

QuRT supports a variant of mutexes known as recursive mutexes. Recursive mutexes are shared objects which support the following operations:

- Init recursive mutex – Initialize the recursive mutex.
- Destroy recursive mutex – Destroy the specified recursive mutex.
- Lock recursive mutex – Request access to a shared resource.
- Unlock recursive mutex – Release access to a shared resource.
- Try lock recursive mutex – Request access to a shared resource (without suspend).

Recursive mutexes are functionally equivalent to regular mutexes (Section 5.1), except that they enable a thread to perform nested locking on a mutex:

- If a thread performs a lock operation on a recursive mutex that is already being used by another thread, the thread is suspended.
- If a thread performs a lock on a recursive mutex that is already being used by itself, the operation is treated as a nested lock and the thread continues executing as if the mutex were unlocked. However, the recursive mutex does not become available again until the thread performs a balanced number of unlocks on it.

The regular and recursive mutex operations are identical except for the change within the function names from mutex to rmutex.

**Note:** With recursive mutexes, the try lock operation handles a nested lock as if the mutex were unlocked.

**Functions**
Recursive mutexes are shared objects that support the following operations:

- qurt_rmutex_destroy()
- qurt_rmutex_init()
- qurt_rmutex_lock()
- qurt_rmutex_try_lock()
- qurt_rmutex_unlock()
- qurt_rmutex_lock_timed()
- qurt_rmutex_try_lock_block_once()

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.wo@lge.com

## 6.2 qurt_rmutex_destroy()

### 6.2.1 Function Documentation

#### 6.2.1.1 void qurt_rmutex_destroy ( qurt_mutex_t * *lock* )

Destroys the specified recursive mutex.

**NOTE** Recursive mutexes must not be destroyed while they are still in use. If this occurs, the behavior of QuRT is undefined.

**Associated data types**

qurt_mutex_t

**Parameters**

| | | |
|---|---|---|
| in | *lock* | Pointer to the recursive mutex object to destroy. |

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 6.3 qurt_rmutex_init()

### 6.3.1 Function Documentation

#### 6.3.1.1 void qurt_rmutex_init ( qurt_mutex_t ∗ *lock* )

Initializes a recursive mutex object. The recursive mutex is initialized in unlocked state.

**Associated data types**

qurt_mutex_t

**Parameters**

| | | |
|---|---|---|
| out | *lock* | Pointer to the recursive mutex object. |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 6.4 qurt_rmutex_lock()

### 6.4.1 Function Documentation

#### 6.4.1.1 void qurt_rmutex_lock ( qurt_mutex_t * *lock* )

Locks the specified recursive mutex.

If a thread performs a lock operation on a mutex that is not in use, the thread gains access to the shared resource that the mutex protects, and continues to execute.

If a thread performs a lock operation on a mutex that is already use by another thread, the thread is suspended. When the mutex becomes available again (because the other thread has unlocked it), the thread is awakened and given access to the shared resource.

**NOTE** A thread is not suspended if it locks a recursive mutex that it has already locked. However, the mutex does not become available to other threads until the thread performs a balanced number of unlocks on the mutex.

**Associated data types**

qurt_mutex_t

**Parameters**

| | | |
|---|---|---|
| in | *lock* | Pointer to the recursive mutex object to lock. |

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 6.5 qurt_rmutex_try_lock()

### 6.5.1 Function Documentation

#### 6.5.1.1 int qurt_rmutex_try_lock ( qurt_mutex_t ∗ *lock* )

Attempts to lock the specified recursive mutex.

If a thread performs a try_lock operation on a recursive mutex that is not in use, the thread gains access to the shared resource that is protected by the mutex, and continues to execute.

If a thread performs a try_lock operation on a recursive mutex that another thread has already locked, qurt_rmutex_try_lock immediately returns with a nonzero result value.

**Associated data types**

qurt_mutex_t

**Parameters**

| | | |
|---|---|---|
| in | *lock* | Pointer to the recursive mutex object to lock. |

**Returns**

0 – Success.
Nonzero – Failure.

**Safe API**

Yes

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 6.6 qurt_rmutex_unlock()

### 6.6.1 Function Documentation

#### 6.6.1.1 void qurt_rmutex_unlock ( qurt_mutex_t * *lock* )

Unlocks the specified recursive mutex.

More than one thread can be suspended on a mutex. When the mutex is unlocked, the thread that waits on the mutex awakens. If the awakened thread has higher priority than the current thread, a context switch occurs.

**NOTE** When a thread unlocks a recursive mutex, the mutex is not available until the balanced number of locks and unlocks has been performed on the mutex.

**Associated data types**

qurt_mutex_t

**Parameters**

| | | |
|---|---|---|
| in | *lock* | Pointer to the recursive mutex object to unlock. |

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 6.7 qurt_rmutex_lock_timed()

### 6.7.1 Function Documentation

#### 6.7.1.1 int qurt_rmutex_lock_timed ( qurt_mutex_t ∗ *lock,* unsigned long long int *duration* )

Locks the specified recursive mutex. The wait must be terminated when the specified timeout expires.

If a thread performs a lock operation on a mutex that is not in use, the thread gains access to the shared resource that the mutex protects, and continues to execute.

If a thread performs a lock operation on a mutex that is already in use by another thread, the thread is suspended. When the mutex becomes available again (because the other thread has unlocked it), the thread is awakened and given access to the shared resource.

**NOTE** A thread is not suspended if it locks a recursive mutex that it has already locked by itself. However, the mutex does not become available to other threads until the thread performs a balanced number of unlocks on the mutex. If timeout expires, this wait must be terminated and no access to the mutex is granted.

**Associated data types**

qurt_mutex_t

**Parameters**

| | | |
|---|---|---|
| in | *lock* | Pointer to the recursive mutex object to lock. |
| in | *duration* | Interval (in microseconds) duration value must be between QURT_TIMER_MIN_DURATION and QURT_TIMER_MAX_DURATION |

**Returns**

QURT_EOK – Success
QURT_ETIMEDOUT – Timeout

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 6.8 qurt_rmutex_try_lock_block_once()

### 6.8.1 Function Documentation

#### 6.8.1.1 int qurt_rmutex_try_lock_block_once ( qurt_mutex_t ∗ *lock* )

Attempts to lock a mutex object recursively. If the mutex is available, it locks the mutex. If the mutex is held by the current thread, it increases the internal counter and returns 0. If not, it returns a nonzero value. If the mutex is already locked by another thread, the caller thread is suspended. When the mutex becomes available again (because the other thread has unlocked it), the caller thread is awakened and tries to lock the mutex; and if it fails, this function returns failure with a nonzero value. If it succeeds, this function returns success with zero.

**Associated data types**

qurt_mutex_t

**Parameters**

| | | |
|---|---|---|
| in | *lock* | Pointer to the qurt_mutex_t object. |

**Returns**

0 – Success.
Nonzero – Failure.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 7 Priority Inheritance Mutexes

## 7.1 Overview

QuRT supports a variant of recursive mutexes known as priority inheritance mutexes. Priority inheritance mutexes are shared objects which support the following operations:

- Init priority inheritance mutex – Initialize the priority inheritance mutex.
- Destroy priority inheritance mutex – Destroy the specified priority inheritance mutex.
- Lock priority inheritance mutex – Request access to a shared resource.
- Unlock priority inheritance mutex – Release access to a shared resource.
- Try lock priority inheritance mutex – Request access to a shared resource (without suspend).

Priority inheritance mutexes are functionally equivalent to recursive mutexes (Section 6), except that they enable a thread to perform priority inheritance after locking a mutex:

- If a thread has locked a priority inheritance mutex, and another thread with higher priority (Section 3.1) becomes suspended on the mutex, then the thread with the lock acquires the higher priority of the suspended thread.
- If multiple threads are suspended on a priority inheritance mutex, then the thread with the lock acquires the priority of the highest-priority suspended thread (if it is higher than the thread's original priority.

The change in priority is only temporary – when a thread unlocks a priority inheritance mutex, its thread priority is restored to its original value.

The regular and priority inheritance mutex operations are identical except for the change within the function names from mutex to pimutex.

**Note:** With priority inheritance mutexes, the try lock operation handles a nested lock as if the mutex were unlocked.

**Functions**

Priority inheritance mutexes are shared objects that support the following operations:

- qurt_pimutex_destroy()
- qurt_pimutex_init()
- qurt_pimutex_lock()
- qurt_pimutex_try_lock()
- qurt_pimutex_unlock()
- func_qurt_pimutex_lock_timed()

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungshul.woo@lge.com

## 7.2 qurt_pimutex_destroy()

### 7.2.1 Function Documentation

#### 7.2.1.1 void qurt_pimutex_destroy ( qurt_mutex_t * *lock* )

Destroys the specified priority inheritance mutex.

**NOTE** Priority inheritance mutexes must be destroyed when they are no longer in use. Failure to do this causes resource leaks in the QuRT kernel.

Priority inheritance mutexes must not be destroyed while they are still in use. If this occurs, the behavior of QuRT is undefined.

**Associated data types**

qurt_mutex_t

**Parameters**

| | | |
|---|---|---|
| in | *lock* | Pointer to the priority inheritance mutex object to destroy. |

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 7.3 qurt_pimutex_init()

### 7.3.1 Function Documentation

#### 7.3.1.1 void qurt_pimutex_init ( qurt_mutex_t * *lock* )

Initializes a priority inheritance mutex object. The priority inheritance mutex is initially unlocked.

This function works the same as qurt_mutex_init().

**NOTE** Each pimutex-based object has one or more kernel resources associated with it; to prevent resource leaks, call qurt_pimutex_destroy() when this object is not used anymore

**Associated data types**

qurt_mutex_t

**Parameters**

| | | |
|---|---|---|
| out | *lock* | Pointer to the priority inheritance mutex object. |

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 7.4 qurt_pimutex_lock()

### 7.4.1 Function Documentation

#### 7.4.1.1 void qurt_pimutex_lock ( qurt_mutex_t ∗ *lock* )

Requests access to a shared resources. If a thread performs a lock operation on a mutex that has not been locked by other threads, the thread gains access to the shared resource that the mutex protects, and continues to execute.

If a thread performs a lock operation on a mutex that is already locked by another thread, the thread is suspended. When the mutex becomes available again (because the other thread has unlocked it), the thread is awakened and given access to the shared resource.

If a thread is waiting on a priority inheritance mutex, and the priority of the waiting thread is higher than the priority of the thread that has locked the mutex, priority of the thread that has locked the mutex is raised to the same priority of the waiting thread.

**NOTE** A thread is not moved into waiting state if it locks a priority inheritance mutex that it has already locked . However, the mutex does not become available to other threads until the thread performs a balanced number of unlocks on the mutex.

When there is no contention, PI mutex lock has the similar performance as regular Qurt recursive mutex. When there is contention, i.e., the PI mutex has been locked by another thread, kernel moves the caller thread into waiting state, and also raise the priority of the holder thread if its priority is lower than the waiting thread, i.e., in this case, there will be longer processing time in kernel for thread priority adjustment.

**Associated data types**

qurt_mutex_t

**Parameters**

| | | |
|---|---|---|
| in | *lock* | Pointer to the priority inheritance mutex object to lock. |

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 7.5 qurt_pimutex_try_lock()

### 7.5.1 Function Documentation

#### 7.5.1.1 int qurt_pimutex_try_lock ( qurt_mutex_t ∗ *lock* )

Requests access to a shared resource (without suspend). Attempts to lock the specified priority inheritance mutex.

If a thread performs a try_lock operation on a priority inheritance mutex that is not in use, the thread gains access to the shared resource that is protected by the mutex, and continues to execute. If a thread performs a try_lock operation on a priority inheritance mutex that is already in use by another thread, qurt_pimutex_try_lock immediately returns with a nonzero result value.

**Associated data types**

qurt_mutex_t

**Parameters**

| | | |
|---|---|---|
| in | *lock* | Pointer to the priority inheritance mutex object to lock. |

**Returns**

0 – Success.
Nonzero – Failure.

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 7.6 qurt_pimutex_unlock()

### 7.6.1 Function Documentation

#### 7.6.1.1 void qurt_pimutex_unlock ( qurt_mutex_t ∗ *lock* )

Releases access to a shared resource; unlocks the specified priority inheritance mutex.

More than one thread can be waiting on a priority inheritance mutex. When the mutex is unlocked, only the highest-priority thread that waits on the mutex is awakened. If the awakened thread has higher priority than the current thread, a context switch occurs.

When a thread unlocks a priority inheritance mutex, its thread priority is restored to its original value from any higher priority value that it acquired from another thread waiting on the mutex.

**NOTE** When there is no thread waiting for the PI mutex, PI mutex unlock has the similar performance as regular Qurt recursive mutex; When there are threads waiting for the PI mutex, in PI mutex unlock, kernel waits up one waiting thread, and also adjust the priority of the thread if its priority has been changed, i.e., in this case, there will be longer processing time in kernel for thread priority adjustment.

**Associated data types**

qurt_mutex_t

**Parameters**

| | | |
|---|---|---|
| in | *lock* | Pointer to the priority inheritance mutex object to unlock. |

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 7.7 func_qurt_pimutex_lock_timed()

### 7.7.1 Function Documentation

#### 7.7.1.1 int qurt_pimutex_lock_timed ( qurt_mutex_t * *lock,* unsigned long long int *duration* )

Locks a priority inheritance mutex with timeout.

A thread can lock a priority inheritance mutex for multiple times. The mutex is not available to other threads until the thread performs the same number of mutex unlock operations.

If a thread performs a lock operation on a mutex that is already locked by another thread, the thread is moved to waiting state. When the mutex becomes available again (because the other thread has unlocked the mutex), the thread is awakened and tries to lock the mutex.

If a thread waits on a priority inheritance mutex, and the priority of the waiting thread is higher than the priority of the thread that has locked the mutex, the priority of the thread that has locked the mutex is raised to the same priority of the waiting thread.

If the wait duration exceeds the timeout duration, the wait is terminated, and the function returns QURT_ETIMEDOUT as a failure of the mutex lock.

**Associated data types**

qurt_mutex_t

**Parameters**

| | | |
|---|---|---|
| in | *lock* | Pointer to the mutex object to lock. |
| in | *duration* | Duration (in microseconds) to wait. The duration value must be between QURT_TIMER_MIN_DURATION and QURT_TIMER_MAX_DURATION. |

**Returns**

QURT_EOK – Success
QURT_ETIMEDOUT – Timeout
QURT_EINVALID – Duration is out of range

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:42:01 UTC hyungchul.won@lge.com

# 8 Signals

## 8.1 Overview

Threads use signals to synchronize their execution based on the occurrence of one or more internal events. Signals are stored in shared objects which support the following operations:

- Init signal - Initialize a signal object.
- Destroy signal - Destroy the specified signal object.
- Wait on any signal - Suspend the current thread until one of the specified signals is set.
- Wait on all signals - Suspend the current thread until all of the specified signals are set.
- Set signal - Set signals in the specified signal object.
- Get signal - Return the current value of the specified signal object.
- Clear signal - Clear signals in the specified signal object.

If a thread is waiting on a signal object for any of the specified set of signals to be set, and one or more of those signals is set in the signal object, then the thread is awakened.

If a thread is waiting on a signal object for all of the specified set of signals to be set, and all of those signals are set in the signal object, then the thread is awakened.

A signal object contains 32 signals, which are represented as bits 0-31 in a 32-bit value. The bit value 1 indicates that a signal is set, and 0 indicates that it is cleared.

**Note:** At most one thread can wait on a signal object at any given time.

The qurt_signal_wait() function waits for any or all signals, depending on its wait type argument.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.yoon@qce.com

### Functions

Signal services are accessed with the following QuRT functions:

- qurt_signal_clear()
- qurt_signal_destroy()
- qurt_signal_get()
- qurt_signal_init()
- qurt_signal_set()
- qurt_signal_wait()
- qurt_signal_wait_all()
- qurt_signal_wait_any()
- qurt_signal_wait_cancellable()
- qurt_signal_wait_timed()
- qurt_signal_64_init()
- qurt_signal_64_destroy()
- qurt_signal_64_wait()
- qurt_signal_64_set()
- qurt_signal_64_get()
- qurt_signal_64_clear()
- Data Types

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 8.2 qurt_signal_clear()

### 8.2.1 Function Documentation

#### 8.2.1.1 void qurt_signal_clear ( qurt_signal_t ∗ *signal,* unsigned int *mask* )

Clears signals in the specified signal object.

Signals are represented as bits 0 through 31 in the 32-bit mask value. A mask bit value of 1 indicates that a signal must be cleared, and 0 indicates not to clear the signal.

**NOTE** Signals must be explicitly cleared by a thread when awakened – the wait operations do not automatically clear them.

**Associated data types**

qurt_signal_t

**Parameters**

| | | |
|---|---|---|
| in | *signal* | Pointer to the signal object to modify. |
| in | *mask* | Mask value that identifies the individual signals to clear in the signal object. |

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 8.3 qurt_signal_destroy()

### 8.3.1 Function Documentation

#### 8.3.1.1 void qurt_signal_destroy ( qurt_signal_t * *signal* )

Destroys the specified signal object.

**NOTE** Signal objects must be destroyed when they are no longer in use. Failure to do this causes resource leaks in the QuRT kernel.

Signal objects must not be destroyed while they are still in use. If this occurs, the behavior of QuRT is undefined.

**Associated data types**

qurt_signal_t

**Parameters**

| | | |
|---|---|---|
| in | *signal* | Pointer to the signal object to destroy. |

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 8.4 qurt_signal_get()

### 8.4.1 Function Documentation

#### 8.4.1.1 unsigned int qurt_signal_get ( qurt_signal_t * *signal* )

Gets a signal from a signal object.

Returns the current signal values of the specified signal object.

**Associated data types**

qurt_signal_t

**Parameters**

| | | |
|---|---|---|
| in | *signal* | Pointer to the signal object to access. |

**Returns**

A 32-bit word with current signals.

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 8.5 qurt_signal_init()

### 8.5.1 Function Documentation

#### 8.5.1.1 void qurt_signal_init ( qurt_signal_t * *signal* )

Initializes a signal object. Signal returns the initialized object. The signal object is initially cleared.

**NOTE** Each signal-based object has one or more kernel resources associated with it; to prevent resource leaks, call qurt_signal_destroy() when this object is not used anymore

**Associated data types**

qurt_signal_t

**Parameters**

| | | |
|---|---|---|
| in | *signal* | Pointer to the initialized object. |

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 8.6 qurt_signal_set()

### 8.6.1 Function Documentation

#### 8.6.1.1 void qurt_signal_set ( qurt_signal_t * *signal,* unsigned int *mask* )

Sets signals in the specified signal object.

Signals are represented as bits 0 through 31 in the 32-bit mask value. A mask bit value of 1 indicates to set the signal, and 0 indicates not to set it.

**Associated data types**

qurt_signal_t

**Parameters**

| | | |
|---|---|---|
| in | *signal* | Pointer to the signal object to modify. |
| in | *mask* | Mask value that identifies the individual signals to set in the signal object. |

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 8.7 qurt_signal_wait()

### 8.7.1 Function Documentation

#### 8.7.1.1 unsigned int qurt_signal_wait ( qurt_signal_t * *signal,* unsigned int *mask,* unsigned int *attribute* )

Suspends the current thread until the specified signals are set.

Signals are represented as bits 0 through 31 in the 32-bit mask value. A mask bit value of 1 indicates to wait on a signal, and 0 indicates not to wait on the signal.

If a thread waits on a signal object for any of the specified set of signals to be set, and one or more of those signals is set in the signal object, the thread is awakened.

If a thread waits on a signal object for all of the specified set of signals to be set, and all of those signals are set in the signal object, the thread is awakened.

The specified set of signals can be cleared when the signal is set.

**NOTE** At most, one thread can wait on a signal object at any given time.

**Associated data types**

qurt_signal_t

**Parameters**

| | | |
|---|---|---|
| in | *signal* | Pointer to the signal object to wait on. |
| in | *mask* | Mask value that identifies the individual signals in the signal object to wait on. |
| in | *attribute* | Indicates whether the thread waits to set any of the signals, or to set all of them.<br>**NOTE** The wait-any and wait-all types are mutually exclusive.<br>Values:<br>• QURT_SIGNAL_ATTR_WAIT_ANY<br>• QURT_SIGNAL_ATTR_WAIT_ALL |

**Returns**

32-bit word with current signals.

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential May Contain Trade Secrets 2026-02-27 05:40:46 GMT hyungchul.kim@lge.com

## 8.8 qurt_signal_wait_all()

### 8.8.1 Function Documentation

#### 8.8.1.1 static unsigned int qurt_signal_wait_all ( qurt_signal_t * *signal,* unsigned int *mask* )

Suspends the current thread until all of the specified signals are set.

Signals are represented as bits 0 through 31 in the 32-bit mask value. A mask bit value of 1 indicates to wait on a signal, and 0 indicates not to wait on it.

If a thread waits on a signal object for all of the specified set of signals to be set, and all of those signals are set in the signal object, the thread is awakened.

**NOTE** At most, one thread can wait on a signal object at any given time.

**Associated data types**

qurt_signal_t

**Parameters**

| | | |
|---|---|---|
| in | *signal* | Pointer to the signal object to wait on. |
| in | *mask* | Mask value that identifies the individual signals in the signal object to wait on. |

**Returns**

A 32-bit word with current signals.

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 8.9 qurt_signal_wait_any()

### 8.9.1 Function Documentation

#### 8.9.1.1 static unsigned int qurt_signal_wait_any ( qurt_signal_t ∗ *signal,* unsigned int *mask* )

Suspends the current thread until any of the specified signals are set.

Signals are represented as bits 0 through 31 in the 32-bit mask value. A mask bit value of 1 indicates to wait on a signal, and 0 indicates not to wait on the thread.

If a thread waits on a signal object for any of the specified set of signals to be set, and one or more of those signals is set in the signal object, the thread is awakened.

**NOTE** At most, one thread can wait on a signal object at any given time.

**Associated data types**

qurt_signal_t

**Parameters**

| | | |
|---|---|---|
| in | *signal* | Pointer to the signal object to wait on. |
| in | *mask* | Mask value that identifies the individual signals in the signal object to wait on. |

**Returns**

32-bit word with current signals.

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 8.10 qurt_signal_wait_cancellable()

### 8.10.1 Function Documentation

#### 8.10.1.1 int qurt_signal_wait_cancellable ( qurt_signal_t ∗ *signal,* unsigned int *mask,* unsigned int *attribute,* unsigned int ∗ *return_mask* )

Suspends the current thread until either the specified signals are set or the wait operation is cancelled. The operation is cancelled if the user process of the calling thread is killed, or if the calling thread must finish its current QDI invocation and return to user space.

Signals are represented as bits 0 through 31 in the 32-bit mask value. A mask bit value of 1 indicates that a signal must be waited on, and 0 indicates not to wait on it.

If a thread waits on a signal object for any of the specified set of signals to be set, and one or more of those signals is set in the signal object, the thread is awakened.

If a thread waits on a signal object for all of the specified set of signals to be set, and all of those signals are set in the signal object, the thread is awakened.

**NOTE** At most, one thread can wait on a signal object at any given time.

When the operation is cancelled, the caller must assume that the signal is never set.

**Associated data types**

qurt_signal_t

**Parameters**

| | | |
|---|---|---|
| in | *signal* | Pointer to the signal object to wait on. |
| in | *mask* | Mask value that identifies the individual signals in the signal object to wait on. |
| in | *attribute* | Indicates whether the thread must wait until any of the signals are set, or until all of them are set. Values:<br>• QURT_SIGNAL_ATTR_WAIT_ANY<br>• QURT_SIGNAL_ATTR_WAIT_ALL |
| out | *return_mask* | Pointer to the 32-bit mask value that was originally passed to the function. |

**Returns**

QURT_EOK – Wait completed.
QURT_ECANCEL – Wait cancelled.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential May Contain Trade Secrets 2026-02-27 05:10:41 GMT hyungchul.kim@lge.com

# 8.11 qurt_signal_wait_timed()

## 8.11.1 Function Documentation

### 8.11.1.1 int qurt_signal_wait_timed ( qurt_signal_t * *signal_ptr,* unsigned int *mask,* unsigned int *attribute,* unsigned int * *signals,* unsigned long long int *duration* )

Suspends the current thread until the specified signals are set or until timeout.

Signals are represented as bits 0 through 31 in the 32-bit mask value. A mask bit value of 1 indicates waiting on a signal, and 0 indicates not waiting.

If a thread waits on a signal object for any of the specified set of signals to be set, and one or more of those signals is set in the signal object, the thread is awakened.

If a thread waits on a signal object for all of the specified set of signals to be set, and all of those signals are set in the signal object, the thread is awakened.

The specified set of signals can be cleared after the signal is set.

**NOTE** At most, one thread can wait on a signal object at any given time.

#### Associated data types

qurt_signal_t

#### Parameters

| | | |
|---|---|---|
| in | *signal_ptr* | Pointer to the signal object to wait on. |
| in | *mask* | Mask value that identifies the individual signals in the signal object to wait on. |
| in | *attribute* | Indicates whether the thread must wait until any of the signals are set, or until all of them are set.<br>**NOTE** The wait-any and wait-all types are mutually exclusive.<br>Values:<br>• QURT_SIGNAL_ATTR_WAIT_ANY<br>• QURT_SIGNAL_ATTR_WAIT_ALL |
| out | *signals* | Bitmask of signals that are set. |
| in | *duration* | Duration (microseconds) to wait. Must be in the range [QURT_TIMER_MIN_DURATION ... QURT_TIMER_MAX_DURATION] |

#### Returns

QURT_EOK – Success; one or more signals were set
QURT_ETIMEDOUT – Timed-out
QURT_EINVALID – Duration out of range

#### Safe API

Yes

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.kim@sec.com

### Dependencies

Timed-waiting support in the kernel.

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:40:42 GMT
hyungchul.won@lge.com

## 8.12 qurt_signal_64_init()

### 8.12.1 Function Documentation

#### 8.12.1.1 void qurt_signal_64_init ( qurt_signal_64_t * *signal_ptr* )

Initializes a 64-bit signal object.

The signal argument returns the initialized object. The signal object is initially cleared.

**NOTE** Each signal-based object has one or more kernel resources associated with it; to prevent resource leaks, call qurt_signal_destroy() when this object is not used anymore.

**Associated data types**

qurt_signal_64_t

**Parameters**

| | | |
|---|---|---|
| in | *signal_ptr* | Pointer to the initialized object. |

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 8.13 qurt_signal_64_destroy()

### 8.13.1 Function Documentation

#### 8.13.1.1 void qurt_signal_64_destroy ( qurt_signal_64_t ∗ *signal_ptr* )

Destroys the specified signal object.

**NOTE** 64-bit signal objects must be destroyed when they are no longer in use. Failure to do this causes resource leaks in the QuRT kernel.

Signal objects must not be destroyed while they are still in use. If this occurs, the behavior of QuRT is undefined.

**Associated data types**

qurt_signal_64_t

**Parameters**

| | | |
|---|---|---|
| in | *signal_ptr* | Pointer to the signal object to destroy. |

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 8.14 qurt_signal_64_wait()

## 8.14.1 Function Documentation

### 8.14.1.1 unsigned long long qurt_signal_64_wait ( qurt_signal_64_t * *signal_ptr,* unsigned long long *mask,* unsigned int *attribute* )

Suspends the current thread until all of the specified signals are set.

Signals are represented as bits 0 through 63 in the 64-bit mask value. A mask bit value of 1 indicates that a signal must be waited on, and 0 indicates not wait on it.

If a thread waits on a signal object for all of the specified set of signals to be set, and all of those signals are set in the signal object, the thread is awakened.

**NOTE** At most, one thread can wait on a signal object at any given time.

#### Associated data types

qurt_signal_64_t

#### Parameters

| | | |
|---|---|---|
| in | *signal_ptr* | Pointer to the signal object to wait on. |
| in | *mask* | Mask value, which identifies the individual signals in the signal object to wait on. |
| in | *attribute* | Indicates whether the thread must wait until any of the signals are set, or until all of them are set.<br>**NOTE** The wait-any and wait-all types are mutually exclusive.<br>Values:<br>• QURT_SIGNAL_ATTR_WAIT_ANY<br>• QURT_SIGNAL_ATTR_WAIT_ALL |

#### Returns

A 32-bit word with current signals.

#### Safe API

Yes

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 8.15 qurt_signal_64_set()

## 8.15.1 Function Documentation

### 8.15.1.1 void qurt_signal_64_set ( qurt_signal_64_t ∗ *signal_ptr,* unsigned long long *mask* )

Sets signals in the specified signal object.

Signals are represented as bits 0 through 63 in the 64-bit mask value. A mask bit value of 1 indicates that a signal must be set, and 0 indicates not to set it.

#### Associated data types

qurt_signal_64_t

#### Parameters

| | | |
|---|---|---|
| in | *signal_ptr* | Pointer to the signal object to modify. |
| in | *mask* | Mask value that identifies the individual signals to set in the signal object. |

#### Safe API

Yes

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 8.16 qurt_signal_64_get()

## 8.16.1 Function Documentation

### 8.16.1.1 unsigned long long qurt_signal_64_get ( qurt_signal_64_t * *signal_ptr* )

Gets a signal from a signal object.

Returns the current signal values of the specified signal object.

#### Associated data types

qurt_signal_64_t

#### Parameters

| | | |
|---|---|---|
| in | *signal_ptr* | Pointer to the signal object to access. |

#### Returns

A 64-bit double word with current signals.

#### Safe API

Yes

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 8.17 qurt_signal_64_clear()

## 8.17.1 Function Documentation

### 8.17.1.1 void qurt_signal_64_clear ( qurt_signal_64_t ∗ *signal_ptr,* unsigned long long *mask* )

Clears signals in the specified signal object.

Signals are represented as bits 0 through 63 in the 64-bit mask value. A mask bit value of 1 indicates that a signal must be cleared, and 0 indicates not to clear it.

**NOTE** Signals must be explicitly cleared by a thread when awakened – the wait operations do not automatically clear them.

#### Associated data types

qurt_signal_64_t

#### Parameters

| | | |
|---|---|---|
| in | *signal_ptr* | Pointer to the signal object to modify. |
| in | *mask* | Mask value that identifies the individual signals to clear in the signal object. |

#### Safe API

Yes

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 8.18 Data Types

Any-signals are represented in QuRT as objects of type qurt_signal_t.

### 8.18.1 Define Documentation

#### 8.18.1.1 #define QURT_SIGNAL_ATTR_WAIT_ANY 0x00000000U

Wait any.

#### 8.18.1.2 #define QURT_SIGNAL_ATTR_WAIT_ALL 0x00000001U

Wait all.

### 8.18.2 Data Structure Documentation

#### 8.18.2.1 union qurt_signal_t

QuRT signal type.

#### 8.18.2.2 struct qurt_signal_64_t

QuRT 64-bit signal type.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 9 Any-signals

## 9.1 Overview

Threads use any-signals to synchronize their execution based on the occurrence of any one of a number of internal events. Any-signals are stored in shared objects which support the following operations:

- Init any-signal – Initialize the any-signal object.
- Destroy any-signal – Destroy the specified any-signal object.
- Wait any-signal – Suspend the current thread until one of the specified signals is set.
- Set any-signal – Set signals in the specified any-signal object.
- Get any-signal – Return the current value of the specified any-signal object.
- Clear any-signal – Clear signals in the specified any-signal object.

If a signal is set in an any-signal object, and a thread is waiting on the any-signal object for that signal, then the thread is awakened. If the awakened thread has higher priority than the current thread, a context switch may occur.

Threads are responsible for explicitly clearing any set signals in an any-signal object before waiting on them again. If a thread waits on a signal that has already been set, the thread continues executing.

An any-signal object contains 32 signals, which are represented as bits 0-31 in a 32-bit value. The bit value 1 indicates that a signal is set, and 0 indicates that it is cleared.

**Note:** At most one thread can wait on an any-signal object at any given time.

**Functions**

Any-signals are stored in shared objects that support the following operations:

- qurt_anysignal_clear()
- qurt_anysignal_destroy()
- qurt_anysignal_get()
- qurt_anysignal_init()
- qurt_anysignal_set()
- qurt_anysignal_wait()
- qurt_anysignal_wait_timed()
- Data Types

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 03:49:42 GMT hyunchul.won@qc.com

## 9.2 qurt_anysignal_clear()

### 9.2.1 Function Documentation

#### 9.2.1.1 unsigned int qurt_anysignal_clear ( qurt_anysignal_t * *signal,* unsigned int *mask* )

Clears signals in the specified any-signal object.

Signals are represented as bits 0 through 31 in the 32-bit mask value. A mask bit value of 1 indicates that a signal must be cleared, and 0 indicates not to clear the signal.

**Associated data types**

qurt_anysignal_t

**Parameters**

| | | |
|---|---|---|
| in | *signal* | Pointer to the any-signal object, which specifies the any-signal object to modify. |
| in | *mask* | Signal mask value that identifies the individual signals to clear in the any-signal object. |

**Returns**

Bitmask – Old signal values (before clear).

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 9.3 qurt_anysignal_destroy()

### 9.3.1 Function Documentation

#### 9.3.1.1 static void qurt_anysignal_destroy ( qurt_anysignal_t * *signal* )

Destroys the specified any-signal object.

**NOTE** Any-signal objects must be destroyed when they are no longer in use. Failure to do this causes resource leaks in the QuRT kernel.

Any-signal objects must not be destroyed while they are still in use. If this occurs, the behavior of QuRT is undefined.

**Associated data types**

qurt_anysignal_t

**Parameters**

| | | |
|---|---|---|
| in | *signal* | Pointer to the any-signal object to destroy. |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 9.4 qurt_anysignal_get()

### 9.4.1 Function Documentation

#### 9.4.1.1 static unsigned int qurt_anysignal_get ( qurt_anysignal_t * *signal* )

Gets signal values from the any-signal object.

Returns the current signal values of the specified any-signal object.

**Associated data types**

qurt_anysignal_t

**Parameters**

| | | |
|---|---|---|
| in | *signal* | Pointer to the any-signal object to access. |

**Returns**

A bitmask with the current signal values of the specified any-signal object.

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 9.5 qurt_anysignal_init()

### 9.5.1 Function Documentation

#### 9.5.1.1 static void qurt_anysignal_init ( qurt_anysignal_t * *signal* )

Initializes an any-signal object.

The any-signal object is initially cleared.

**Associated data types**

qurt_anysignal_t

**Parameters**

| | | |
|---|---|---|
| `out` | *signal* | Pointer to the initialized any-signal object. |

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 9.6 qurt_anysignal_set()

### 9.6.1 Function Documentation

#### 9.6.1.1 unsigned int qurt_anysignal_set ( qurt_anysignal_t * *signal,* unsigned int *mask* )

Sets signals in the specified any-signal object.

Signals are represented as bits 0 through 31 in the 32-bit mask value. A mask bit value of 1 indicates that a signal must be set, and 0 indicates not to set the sigmal.

**Associated data types**

qurt_anysignal_t

**Parameters**

| | | |
|---|---|---|
| in | *signal* | Pointer to the any-signal object to modify. |
| in | *mask* | Signal mask value that identifies the individual signals to set in the any-signal object. |

**Returns**

Bitmask of old signal values (before set).

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 9.7 qurt_anysignal_wait()

### 9.7.1 Function Documentation

#### 9.7.1.1 static unsigned int qurt_anysignal_wait ( qurt_anysignal_t * *signal,* unsigned int *mask* )

Wait on the any-signal object.

Suspends the current thread until any one of the specified signals is set.

Signals are represented as bits 0 through 31 in the 32-bit mask value. A mask bit value of 1 indicates that a signal must be waited on, and 0 indicates not to wait on the signal. If a signal is set in an any-signal object, and a thread waits on the any-signal object for that signal, the thread is awakened. If the awakened thread has higher priority than the current thread, a context switch can occur.

**NOTE** At most, one thread can wait on an any-signal object at any given time.

**Associated data types**

qurt_anysignal_t

**Parameters**

| | | |
|---|---|---|
| in | *signal* | Pointer to the any-signal object to wait on. |
| in | *mask* | Signal mask value, which specifies the individual signals in the any-signal object to wait on. |

**Returns**

Bitmask of current signal values.

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 9.8 qurt_anysignal_wait_timed()

### 9.8.1 Function Documentation

#### 9.8.1.1 int qurt_anysignal_wait_timed ( qurt_anysignal_t ∗ *signal_obj,* unsigned int *mask,* unsigned int ∗ *signals,* unsigned long long int *duration* )

Waits on the any-signal object.

Suspends the current thread until any of the specified signals is set or timeout expires.

Signals are represented as bits 0 through 31 in the 32-bit mask value. A mask bit value of 1 indicates that a signal must be waited on, and 0 indicates not to wait on the signal. If a signal is set in an any-signal object, and a thread waits on the any-signal object for that signal, the thread is awakened. If the awakened thread has higher priority than the current thread, a context switch can occur.

**NOTE** At most, one thread can wait on an any-signal object at any given time.

**Associated data types**

qurt_anysignal_t

**Parameters**

| | | |
|---|---|---|
| in | *signal_obj* | Pointer to the any-signal object to wait on. |
| in | *mask* | Signal mask value, which specifies the individual signals in the any-signal object to wait on. |
| out | *signals* | Bitmask of current signal values. |
| in | *duration* | Interval (in microseconds) duration value must be between QURT_TIMER_MIN_DURATION and QURT_TIMER_MAX_DURATION. |

**Returns**

QURT_EOK – Success
QURT_ETIMEDOUT – timeout QURT_EINVALID – Duration out of range

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-03-27 05:40:42 GMT hyungchul.won@lge.com

## 9.9 Data Types

This section describes data types for any-signal services.

- Any-signals are represented in QuRT as objects of type qurt_anysignal_t.

### 9.9.1 Typedef Documentation

#### 9.9.1.1 typedef qurt_signal_t qurt_anysignal_t

qurt_signal_t supersedes qurt_anysignal_t. This type definition was added for backwards compatibility.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 10 All-signals

## 10.1 Overview

Threads use all-signals to synchronize their execution based on the occurrence of one or more internal events. All-signals are stored in shared objects which support the following operations:

- Init all-signal – Initialize the all-signal object.
- Destroy all-signal – Destroy the specified all-signal object.
- Wait all-signal – Suspend the current thread until all of the specified signals are set.
- Set all-signal – Set signals in the specified all-signal object.
- Get all-signal – Return the current value of the specified all-signal object.

If one or more signals is set in an all-signal object, and a thread is waiting on the all-signal object for that particular set of signals to be set, then the thread is awakened. If the awakened thread has higher priority than the current thread, a context switch may occur.

Unlike any-signals, all-signals do not need to explicitly clear any set signals in an all-signal object before waiting on them again – clearing is done automatically by the wait operation.

An all-signal object contains 32 signals, which are represented as bits 0-31 in a 32-bit value. The bit value 0 indicates that a signal is set, and 1 indicates that it is cleared (which is the opposite definition of any-signals).

**Note:** At most one thread can wait on an all-signal object at any given time.

Because signal clearing is done by the wait operation, no clear operation is defined for all-signals.

**Functions**

All-signal services are accessed with the following QuRT functions.

- qurt_allsignal_destroy()
- qurt_allsignal_get()
- qurt_allsignal_init()
- qurt_allsignal_set()
- qurt_allsignal_wait()
- Data Types

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.woo@lge.com

## 10.2 qurt_allsignal_destroy()

### 10.2.1 Function Documentation

#### 10.2.1.1 void qurt_allsignal_destroy ( qurt_allsignal_t * *signal* )

Destroys the specified all-signal object.

**NOTE** All-signal objects must be destroyed when they are no longer in use. Failure to do this causes resource leaks in the QuRT kernel.

All-signal objects must not be destroyed while they are still in use. If this occurs, the behavior of QuRT is undefined.

**Associated data types**

qurt_allsignal_t

**Parameters**

| | | |
|---|---|---|
| in | *signal* | Pointer to the all-signal object to destroy. |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 10.3 qurt_allsignal_get()

### 10.3.1 Function Documentation

#### 10.3.1.1 static unsigned int qurt_allsignal_get ( qurt_allsignal_t * *signal* )

Gets signal values from the all-signal object.

Returns the current signal values of the specified all-signal object.

**Associated data types**

qurt_allsignal_t

**Parameters**

| | | |
|---|---|---|
| in | *signal* | Pointer to the all-signal object to access. |

**Returns**

Bitmask with current signal values.

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 10.4 qurt_allsignal_init()

### 10.4.1 Function Documentation

#### 10.4.1.1 void qurt_allsignal_init ( qurt_allsignal_t * *signal* )

Initializes an all-signal object.

The all-signal object is initially cleared.

**Associated data types**

qurt_allsignal_t

**Parameters**

| | | |
|---|---|---|
| `out` | *signal* | Pointer to the all-signal object to initialize. |

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 10.5 qurt_allsignal_set()

### 10.5.1 Function Documentation

#### 10.5.1.1 void qurt_allsignal_set ( qurt_allsignal_t ∗ *signal,* unsigned int *mask* )

Set signals in the specified all-signal object.

Signals are represented as bits 0 through 31 in the 32-bit mask value. A mask bit value of 1 indicates that a signal must be set, and 0 indicates not to set the signal.

**Associated data types**

qurt_allsignal_t

**Parameters**

| | | |
|---|---|---|
| in | *signal* | Pointer to the all-signal object to modify. |
| in | *mask* | Signal mask value that identifies the individual signals to set in the all-signal object. |

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 10.6 qurt_allsignal_wait()

### 10.6.1 Function Documentation

#### 10.6.1.1 void qurt_allsignal_wait ( qurt_allsignal_t * *signal,* unsigned int *mask* )

Waits on the all-signal object.

Suspends the current thread until all of the specified signals are set. Signals are represented as bits 0 through 31 in the 32-bit mask value. A mask bit value of 1 indicates that a signal must be waited on, and 0 that it is not to be waited on.

If a signal is set in an all-signal object, and a thread waits on the all-signal object for that signal, the thread is awakened. If the awakened thread has higher priority than the current thread, a context switch can occur.

Unlike any-signals, all-signals do not need to explicitly clear any set signals in an all-signal object before waiting on them again – clearing is done automatically by the wait operation.

**NOTE** At most, one thread can wait on an all-signal object at any given time. Because the wait operation clears signals, no clear operation is defined for all-signals.

**Associated data types**

qurt_allsignal_t

**Parameters**

| | | |
|---|---|---|
| in | *signal* | Pointer to the all-signal object to wait on. |
| in | *mask* | Signal mask value that identifies the individual signals in the all-signal object to wait on. |

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 10.7 Data Types

This section describes data types for all-signal services.

- All-signals are represented in QuRT as objects of type qurt_allsignal_t.

### 10.7.1 Data Structure Documentation

#### 10.7.1.1 union qurt_allsignal_t

qurt_signal_t supersedes qurt_allsignal_t. This type definition was added for backwards compatibility.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 11 Semaphores

## 11.1 Overview

Threads use semaphores to synchronize their access to shared resources. Semaphores are shared objects which support the following operations:

- Init semaphore – Initialize the semaphore.
- Init semaphore with value – Initialize the semaphore with the specified value.
- Destroy semaphore – Destroy the specified semaphore.
- Down semaphore – Request access to a shared resource.
- Up semaphore – Release access to a shared resource.
- Add semaphore – Release access (by specified value).
- Try down semaphore – Request access to a shared resource (without suspend).
- Get semaphore value – Return the count value of the specified semaphore.

When a semaphore is initialized it is assigned an integer count value. This value indicates the number of threads that can simultaneously access a shared resource through the semaphore. The default value is 1.

**Down**
When a thread performs a down operation on a semaphore, the result depends on the semaphore count value:

- If the count value is nonzero it is decremented, and the thread gains access to the shared resource and continues executing.
- If the count value is zero it is not decremented, and the thread is suspended on the semaphore. When the count value becomes nonzero (because another thread released the semaphore) it is decremented, and the suspended thread is awakened and gains access to the shared resource.

**Up**
When a thread performs an up operation on a semaphore, the semaphore count value is incremented. The result depends on the number of threads waiting on the semaphore:

- If no threads are waiting the current thread releases access to the shared resource and continues executing.
- If one or more threads are waiting and the semaphore count value is nonzero, then the kernel awakens the highest-priority waiting thread and decrements the semaphore count value. If the awakened thread has higher priority than the current thread, a context switch may occur.

The add operation is similar to up, but can increment the semaphore count value by an amount greater than 1. As a result, add has the potential to awaken multiple waiting threads in a single operation.

Qualcomm Confidential May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

The try down operation enables a thread to try accessing a shared resource without the risk of getting suspended if its semaphore has a count value of zero:

- If the count is nonzero try down is identical to the regular down operation.
- If the count is zero try down returns with a value indicating the zero-count state.

**Functions**

Semaphores are shared objects that support the following operations:

- qurt_sem_add()
- qurt_sem_destroy()
- qurt_sem_down()
- qurt_sem_get_val()
- func_qurt_sem_down_cancellable()
- qurt_sem_init()
- qurt_sem_init_val()
- qurt_sem_try_down()
- qurt_sem_up()
- qurt_sem_down_timed()
- Data Types

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 11.2 qurt_sem_add()

### 11.2.1 Function Documentation

#### 11.2.1.1 int qurt_sem_add ( qurt_sem_t * *sem,* unsigned int *amt* )

Releases access to a shared resource (the specified amount increments the semaphore count value).

When a thread performs an add operation on a semaphore, the specified value increments the semaphore count. The result depends on the number of threads that wait on the semaphore:

- When no threads wait, the current thread releases access to the shared resource and continues to execute.
- When one or more threads wait and the semaphore count value is nonzero, the kernel repeatedly awakens the highest-priority waiting thread and decrements the semaphore count value until either no waiting threads remain or the semaphore count value is zero. If any of the awakened threads has higher priority than the current thread, a context switch can occur.

**Associated data types**

qurt_sem_t

**Parameters**

| | | |
|---|---|---|
| in | *sem* | Pointer to the semaphore object to access. |
| in | *amt* | Amount to increment the semaphore count value. |

**Returns**

Unused integer value.

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 11.3 qurt_sem_destroy()

### 11.3.1 Function Documentation

#### 11.3.1.1 void qurt_sem_destroy ( qurt_sem_t * *sem* )

Destroys the specified semaphore.

**NOTE** Semaphores must be destroyed when they are no longer in use. Failure to do this causes resource leaks in the QuRT kernel.

Semaphores must not be destroyed while they are still in use. If this occurs, the behavior of QuRT is undefined.

**Associated data types**

qurt_sem_t

**Parameters**

| | | |
|---|---|---|
| in | *sem* | Pointer to the semaphore object to destroy. |

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 11.4 qurt_sem_down()

### 11.4.1 Function Documentation

#### 11.4.1.1 int qurt_sem_down ( qurt_sem_t * *sem* )

Requests access to a shared resource. When a thread performs a down operation on a semaphore, the result depends on the semaphore count value:

- When the count value is nonzero, it is decremented, and the thread gains access to the shared resource and continues to execute.
- When the count value is zero, it is not decremented, and the thread is suspended on the semaphore. When the count value becomes nonzero (because another thread released the semaphore) it is decremented, and the suspended thread is awakened and gains access to the shared resource.

**Associated data types**

qurt_sem_t

**Parameters**

| | | |
|---|---|---|
| in | *sem* | Pointer to the semaphore object to access. |

**Returns**

Unused integer value.

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 11.5 qurt_sem_get_val()

### 11.5.1 Function Documentation

#### 11.5.1.1 static unsigned short qurt_sem_get_val ( qurt_sem_t * *sem* )

Gets the semaphore count value.

Returns the current count value of the specified semaphore.

**Associated data types**

qurt_sem_t

**Parameters**

| | | |
|---|---|---|
| in | *sem* | Pointer to the semaphore object to access. |

**Returns**

Integer semaphore count value.

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 11.6 func_qurt_sem_down_cancellable()

### 11.6.1 Function Documentation

#### 11.6.1.1 int qurt_sem_down_cancellable ( qurt_sem_t * *sem* )

Requests access to a shared resource. When a thread performs a down operation on a semaphore, the result depends on the semaphore count value:

- When the count value is nonzero, it is decremented, and the thread gains access to the shared resource and continues to execute.
- When the count value is zero, it is not decremented, and the thread is suspended on the semaphore. When the count value becomes nonzero (because another thread released the semaphore) it is decremented, and the suspended thread is awakened and gains access to the shared resource. The operation is cancelled if the user process of the calling thread is killed, or if the calling thread must finish its current QDI invocation and return to user space.

**Associated data types**

qurt_sem_t

**Parameters**

| | | |
|---|---|---|
| in | *sem* | Pointer to the semaphore object to access. |

**Returns**

Unused integer value.

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 11.7 qurt_sem_init()

### 11.7.1 Function Documentation

#### 11.7.1.1 void qurt_sem_init ( qurt_sem_t * *sem* )

Initializes a semaphore object. The default initial value of the semaphore count value is 1.

**Parameters**

| | | |
|---|---|---|
| `out` | *sem* | Pointer to the initialized semaphore object. |

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 11.8 qurt_sem_init_val()

### 11.8.1 Function Documentation

#### 11.8.1.1 void qurt_sem_init_val ( qurt_sem_t * *sem,* unsigned short *val* )

Initializes a semaphore object with the specified value.

**Associated data types**

qurt_sem_t

**Parameters**

| | | |
|---|---|---|
| out | *sem* | Pointer to the initialized semaphore object. |
| in | *val* | Initial value of the semaphore count value. |

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 11.9 qurt_sem_try_down()

## 11.9.1 Function Documentation

### 11.9.1.1 int qurt_sem_try_down ( qurt_sem_t * *sem* )

Requests access to a shared resource (without suspend). When a thread performs a try down operation on a semaphore, the result depends on the semaphore count value:

- The count value is decremented when it is nonzero. The down operation returns 0 as the function result, and the thread gains access to the shared resource and is free to continue to execute.
- The count value is not decremented when it is zero. The down operation returns -1 as the function result, and the thread does not gain access to the shared resource and should not continue to execute.

**Associated data types**

qurt_sem_t

**Parameters**

| | | |
|---|---|---|
| in | *sem* | Pointer to the semaphore object to access. |

**Returns**

0 – Success.
-1 – Failure.

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 11.10 qurt_sem_up()

### 11.10.1 Function Documentation

#### 11.10.1.1 static int qurt_sem_up ( qurt_sem_t * *sem* )

Releases access to a shared resource. When a thread performs an up operation on a semaphore, the semaphore count value increments. The result depends on the number of threads that wait on the semaphore:

- When no threads wait, the current thread releases access to the shared resource and continues to execute.
- When one or more threads wait and the semaphore count value is nonzero, the kernel awakens the highest-priority waiting thread and decrements the semaphore count value. If the awakened thread has higher priority than the current thread, a context switch can occur.

**Associated data types**

qurt_sem_t

**Parameters**

| | | |
|---|---|---|
| in | *sem* | Pointer to the semaphore object to access. |

**Returns**

Unused integer value.

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 11.11 qurt_sem_down_timed()

### 11.11.1 Function Documentation

#### 11.11.1.1 int qurt_sem_down_timed ( qurt_sem_t * *sem,* unsigned long long int *duration* )

When a thread performs a down operation on a semaphore, the result depends on the semaphore count value:

- When the count value is nonzero, it is decremented, and the thread gains access to the shared resource and continues to execute.
- When the count value is zero, it is not decremented, and the thread is suspended on the semaphore. When the count value becomes nonzero (because another thread released the semaphore) it is decremented, and the suspended thread is awakened and gains access to the shared resource. Terminate the wait when the specified timeout expires. If timeout expires, terminate this wait and grant no access to the shared resource.

**Associated data types**

qurt_sem_t

**Parameters**

| | | |
|---|---|---|
| in | *sem* | Pointer to the semaphore object to access. |
| in | *duration* | Interval (in microseconds) duration value must be between QURT_TIMER_MIN_DURATION and QURT_TIMER_MAX_DURATION |

**Returns**

QURT_EOK – Success
QURT_ETIMEDOUT – Timeout

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 11.12 Data Types

This section describes data types for semaphore services. Semaphores are represented in QuRT as objects of type qurt_sem_t.

### 11.12.1 Typedef Documentation

#### 11.12.1.1 typedef union qurt_sem qurt_sem_t

QuRT semaphore type.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 12 Barriers

## 12.1 Overview

Threads use barriers to synchronize their execution at a specific point in a program. Barriers are shared objects which support the following operations:

- Init barrier – Initialize a barrier.
- Destroy barrier – Destroy the specified barrier.
- Wait barrier – Suspend the current thread on the specified barrier.

When a barrier is initialized it is assigned a user-specified integer value. This value indicates the number of threads to synchronize on the barrier.

When a thread waits on a barrier, it is suspended on the barrier:

- If the total number of threads waiting on the barrier is less than the barrier's assigned value, no other action occurs.
- If the total number of threads waiting on the barrier equals the barrier's assigned value, all threads currently waiting on the barrier are awakened, allowing them to execute past the barrier.

After its waiting threads are awakened, a barrier is automatically reset and can be used again in the program without the need for re-initialization.

**Functions**

Barriers are shared objects that support the following operations:

- qurt_barrier_destroy()
- qurt_barrier_init()
- qurt_barrier_wait()
- Data Types

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.woo@lge.com

## 12.2 qurt_barrier_destroy()

### 12.2.1 Function Documentation

#### 12.2.1.1 int qurt_barrier_destroy ( qurt_barrier_t * *barrier* )

Destroys the specified barrier.

**NOTE** Barriers must be destroyed when they are no longer in use. Failure to do this causes resource leaks in the QuRT kernel.

Barriers must not be destroyed while they are still in use. If this occurs, the behavior of QuRT is undefined.

**Associated data types**

qurt_barrier_t

**Parameters**

| | | |
|---|---|---|
| in | *barrier* | Pointer to the barrier object to destroy. |

**Returns**

Unused integer value.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 12.3 qurt_barrier_init()

### 12.3.1 Function Documentation

#### 12.3.1.1 int qurt_barrier_init ( qurt_barrier_t ∗ *barrier,* unsigned int *threads_total* )

Initializes a barrier object.

**Associated data types**

qurt_barrier_t

**Parameters**

| | | |
|---|---|---|
| out | *barrier* | Pointer to the barrier object to initialize. |
| in | *threads_total* | Total number of threads to synchronize on the barrier. |

**Returns**

Unused integer value.

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 12.4 qurt_barrier_wait()

### 12.4.1 Function Documentation

#### 12.4.1.1 int qurt_barrier_wait ( qurt_barrier_t * *barrier* )

Waits on the barrier.

Suspends the current thread on the specified barrier.

The function return value indicates whether the thread was the last one to synchronize on the barrier. When a thread waits on a barrier, it is suspended on the barrier:

- If the total number of threads that wait on the barrier is less than the assigned value of the barrier, no other action occurs.
- If the total number of threads that wait on the barrier equals the assigned value of the barrier, all threads that currently wait on the barrier are awakened, allowing the threads to execute past the barrier.

**NOTE** After its waiting threads are awakened, a barrier is automatically reset and can be used again in the program without the need for re-initialization.

**Associated data types**

qurt_barrier_t

**Parameters**

| | | |
|---|---|---|
| in | *barrier* | Pointer to the barrier object to wait on. |

**Returns**

QURT_BARRIER_OTHER – Current thread awakened from barrier.
QURT_BARRIER_SERIAL_THREAD – Current thread is last caller of barrier.

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@qc.com

## 12.5 Data Types

This section describes data types for barrier services. Barriers are represented in QuRT as objects of type qurt_barrier_t.

### 12.5.1 Define Documentation

#### 12.5.1.1 #define QURT_BARRIER_SERIAL_THREAD 1

Serial thread.

#### 12.5.1.2 #define QURT_BARRIER_OTHER 0

Other.

### 12.5.2 Data Structure Documentation

#### 12.5.2.1 union qurt_barrier_t

QuRT barrier type.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 13 Condition Variables

## 13.1 Overview

Threads use condition variables to synchronize their execution based on the value in a shared data item. Condition variables are useful in cases where a thread would normally have to continuously poll a data item until it contained a specific value – using a condition variable the thread can efficiently accomplish the same task without the need for polling.

Condition variables are shared objects which support the following operations:

- Init condition – Initialize the condition variable.
- Destroy condition – Destroy the specified condition variable.
- Wait condition – Suspend current thread until the specified condition is true.
- Signal condition – Signal a waiting thread that the specified condition is true.
- Broadcast condition – Signal multiple waiting threads that the specified condition is true.

A condition variable is always used in conjunction with an associated mutex (Section 5.1) to ensure that the shared data item is checked and updated without thread contention.

When a thread wishes to wait for a specific condition on a shared data item, it must first lock the mutex that controls access to the data item. If the condition is not satisfied, the thread then performs the wait condition operation on the condition variable (which both suspends the thread and unlocks the mutex).

When a thread wishes to signal that a condition is true on a shared data item, it must first lock the mutex that controls access to the data item, then perform the signal condition operation, and finally explicitly unlock the mutex.

The signal condition operation is used to awaken a single waiting thread. If multiple threads are waiting on a condition variable, they can all be awakened by using the broadcast condition operation.

**Note:** Failure to properly lock and unlock mutexes with condition variables may cause the threads to never be suspended (or suspended but never awakened).

Because QuRT allows threads to be awakened by spurious conditions, threads should always verify the target condition on being awakened.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT rvdagchur.vdc@lga.com

**Functions**

Condition variables are shared objects that support the following operations:

- qurt_cond_broadcast()
- qurt_cond_destroy()
- qurt_cond_init()
- qurt_cond_signal()
- qurt_cond_wait()
- qurt_cond_wait2()
- Data Types

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 13.2 qurt_cond_broadcast()

### 13.2.1 Function Documentation

#### 13.2.1.1 void qurt_cond_broadcast ( qurt_cond_t * *cond* )

Signals multiple waiting threads that the specified condition is true.

When a thread wishes to broadcast that a condition is true on a shared data item, it must perform the following procedure:

1. Lock the mutex that controls access to the data item.
2. Perform the broadcast condition operation.
3. Unlock the mutex.

**NOTE** Failure to properly lock and unlock the mutex of a condition variable can cause the threads to never be suspended (or suspended but never awakened).

Use condition variables only with regular mutexes – attempting to use recursive mutexes or priority inheritance mutexes results in undefined behavior.

**Associated data types**

qurt_cond_t

**Parameters**

| | | |
|---|---|---|
| in | *cond* | Pointer to the condition variable object to signal. |

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 13.3 qurt_cond_destroy()

### 13.3.1 Function Documentation

#### 13.3.1.1 void qurt_cond_destroy ( qurt_cond_t * *cond* )

Destroys the specified condition variable.

**NOTE** Conditions must be destroyed when they are no longer in use. Failure to do this causes resource leaks in the QuRT kernel.

Conditions must not be destroyed while they are still in use. If this occurs, the behavior of QuRT is undefined.

**Associated data types**

qurt_cond_t

**Parameters**

| | | |
|---|---|---|
| in | *cond* | Pointer to the condition variable object to destroy. |

**Safe API**

Yes

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 13.4 qurt_cond_init()

### 13.4.1 Function Documentation

#### 13.4.1.1 void qurt_cond_init ( qurt_cond_t * *cond* )

Initializes a conditional variable object.

**Associated data types**

qurt_cond_t

**Parameters**

| | | |
|---|---|---|
| out | *cond* | Pointer to the initialized condition variable object. |

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 13.5 qurt_cond_signal()

### 13.5.1 Function Documentation

#### 13.5.1.1 void qurt_cond_signal ( qurt_cond_t * *cond* )

Signals a waiting thread that the specified condition is true.

For a thread to signal that a condition is true on a shared data item, the thread must perform the following procedure:

1. Lock the mutex that controls access to the data item.
2. Perform the signal condition operation.
3. Unlock the mutex.

**NOTE** Failure to properly lock and unlock a mutex of a condition variable can cause the threads to never be suspended (or suspended but never awakened).

Use condition variables only with regular mutexes – attempts to use recursive mutexes or priority inheritance mutexes result in undefined behavior.

**Associated data types**

qurt_cond_t

**Parameters**

| | | |
|---|---|---|
| in | *cond* | Pointer to the condition variable object to signal. |

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 13.6 qurt_cond_wait()

### 13.6.1 Function Documentation

#### 13.6.1.1 void qurt_cond_wait ( qurt_cond_t ∗ *cond,* qurt_mutex_t ∗ *mutex* )

Suspends the current thread until the specified condition is true. When a thread wishes to wait for a specific condition on a shared data item, it must perform the following procedure:

1. Lock the mutex that controls access to the data item.
2. If the condition is not satisfied, perform the wait condition operation on the condition variable (suspends the thread and unlocks the mutex).

**NOTE** Failure to properly lock and unlock the mutex of a condition variable can cause the threads to never be suspended (or suspended but never awakened).

Use condition variables only with regular mutexes – attempting to use recursive mutexes or priority inheritance mutexes results in undefined behavior.

**Associated data types**

qurt_cond_t
qurt_mutex_t

**Parameters**

| | | |
|---|---|---|
| in | *cond* | Pointer to the condition variable object to wait on. |
| in | *mutex* | Pointer to the mutex associated with condition variable to wait on. |

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 13.7 qurt_cond_wait2()

### 13.7.1 Function Documentation

#### 13.7.1.1 void qurt_cond_wait2 ( qurt_cond_t ∗ *cond,* qurt_rmutex2_t ∗ *mutex* )

**NOTE** Failure to properly lock and unlock the mutex of a condition variable can cause the threads to never be suspended (or suspended but never awakened).

Use condition variables only with regular mutexes – attempting to use recursive mutexes or priority inheritance mutexes results in undefined behavior.

This is the same API as qurt_cond_wait(). Use this version when using mutexes of type qurt_rmutex2_t.

**Associated data types**

qurt_cond_t
qurt_rmutex2_t

**Parameters**

| | | |
|---|---|---|
| in | *cond* | Pointer to the condition variable object to wait on. |
| in | *mutex* | Pointer to the mutex associated with the condition variable to wait on. |

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 13.8 Data Types

This section describes data types for condition variable services. Condition variables are represented in QuRT as objects of type qurt_cond_t.

### 13.8.1 Data Structure Documentation

#### 13.8.1.1 union qurt_cond_t

QuRT condition variable type.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 14 Pipes

## 14.1 Overview

Threads use pipes to perform synchronized exchange of data streams. Pipes are shared objects which support the following operations:

- Init pipe – Initialize the pipe.
- Create pipe – Initialize the pipe and allocate the pipe buffer.
- Destroy pipe – Destroy the pipe.
- Delete pipe – Destroy the pipe and deallocate the pipe buffer.
- Send pipe – Write data to the pipe.
- Receive pipe – Read data from the pipe.
- Try send pipe – Write data to the pipe (without suspend).
- Try receive pipe – Read data from the pipe (without suspend).

When a pipe object is initialized it uses a user-allocated FIFO buffer to store one or more elements of pipe data. The pipe buffer address and length are specified as parameters.

When a pipe object is created the pipe buffer is allocated as part of the create operation. In this case only the pipe buffer length is specified as a parameter.

If a thread reads from an empty pipe, it is suspended on the pipe. When another thread writes to the pipe, the suspended thread is awakened and can then read data from the pipe.

If a thread writes to a full pipe, it is suspended on the pipe. When another thread reads from the pipe, the suspended thread is awakened and can then write data to the pipe.

The try operations enable a thread to try reading or writing from a pipe without the risk of getting suspended if the pipe is empty (on a read) or full (on a write). If the operation cannot be performed, it returns with a value indicating the state of the pipe.

Pipe data items are defined as 64-bit values. Pipe reads and writes are limited to transferring a single 64-bit data item per operation. Data items larger than 64 bits can be transferred by reading and writing pointers to the data (rather than the data itself), or by transferring the data in consecutive 64-bit chunks.

**Note:** Multiple threads can read from or write to a single pipe.

**Pipe attributes**
Pipes have the following attributes:

- Buffer – Pipe buffer address.
- Elements – Pipe buffer length.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyeong-hun.wo@lge.com

The pipe buffer address specifies the byte address of the start of the pipe data buffer.

The pipe buffer length specifies the length of the pipe data buffer. The length is expressed in terms of the number of 64-bit data elements that can be stored in the buffer.

**Setting pipe attributes**

The pipe attributes are set before a pipe is created using the using the qurt_pipe_attr_init and qurt_pipe_attr_set functions.

**Note:** The pipe attribute structure stores the pipe buffer address and buffer length. The buffer address attribute is ignored by the pipe create operation – for create operations only the buffer length needs to be set.

**Functions**

Pipes are shared objects that support the following operations:

- qurt_pipe_attr_init()
- qurt_pipe_attr_set_buffer()
- qurt_pipe_attr_set_buffer_partition()
- qurt_pipe_attr_set_elements()
- qurt_pipe_create()
- qurt_pipe_delete()
- qurt_pipe_destroy()
- qurt_pipe_init()
- qurt_pipe_is_empty()
- qurt_pipe_receive()
- qurt_pipe_receive_cancellable()
- qurt_pipe_send()
- qurt_pipe_send_cancellable()
- qurt_pipe_try_receive()
- qurt_pipe_try_send()
- Data Types

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 14.2 qurt_pipe_attr_init()

### 14.2.1 Function Documentation

#### 14.2.1.1 static void qurt_pipe_attr_init ( qurt_pipe_attr_t * *attr* )

Initializes the structure that sets the pipe attributes when a pipe is created.

After an attribute structure is initialized, the individual attributes in the structure are explicitly set using the pipe attribute operations.

The attribute structure is assigned the following default values:

- buffer: 0
- elements: 0
- mem_partition: QURT_PIPE_ATTR_MEM_PARTITION_RAM

**Associated data types**

qurt_pipe_attr_t

**Parameters**

| | | |
|---|---|---|
| in,out | *attr* | Pointer to the pipe attribute structure. |

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 14.3 qurt_pipe_attr_set_buffer()

### 14.3.1 Function Documentation

#### 14.3.1.1 static void qurt_pipe_attr_set_buffer ( qurt_pipe_attr_t ∗ *attr,* qurt_pipe_-data_t ∗ *buffer* )

Sets the pipe buffer address attribute.

Specifies the base address of the memory area to use for the data buffer of a pipe.

The base address and size (Section 14.5.1.1) specify the memory area used as a pipe data buffer. The user is responsible for allocation of the memory area used for the buffer.

**Associated data types**

qurt_pipe_attr_t
qurt_pipe_data_t

**Parameters**

| | | |
|---|---|---|
| in,out | *attr* | Pointer to the pipe attribute structure. |
| in | *buffer* | Pointer to the buffer base address. |

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 14.4 qurt_pipe_attr_set_buffer_partition()

### 14.4.1 Function Documentation

#### 14.4.1.1 static void qurt_pipe_attr_set_buffer_partition ( qurt_pipe_attr_t * *attr,* unsigned char *mem_partition* )

Specifies the memory type where the buffer of a pipe is allocated. Allocate pipes in RAM or TCM/LPM.

**NOTE** If a pipe is specified as allocated in TCM/LPM, it must be created with the qurt_pipe_init() operation. The qurt_pipe_create() operation results in an error.

**Associated data types**

qurt_pipe_attr_t

**Parameters**

| | | |
|---|---|---|
| in,out | *attr* | Pointer to the pipe attribute structure. |
| in | *mem_partition* | Pipe memory partition. Values:<br>• QURT_PIPE_ATTR_MEM_PARTITION_RAM – Pipe resides in RAM<br>• QURT_PIPE_ATTR_MEM_PARTITION_TCM – Pipe resides in TCM/LCM |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 14.5 qurt_pipe_attr_set_elements()

### 14.5.1 Function Documentation

#### 14.5.1.1 static void qurt_pipe_attr_set_elements ( qurt_pipe_attr_t ∗ *attr,* unsigned int *elements* )

Specifies the length of the memory area to use for the data buffer of a pipe.

The length is expressed in terms of the number of 64-bit data elements that can be stored in the buffer.

The base address (Section 14.3.1.1) and size specify the memory area used as a pipe data buffer. The user is responsible for allocation of the memory area used for the buffer.

**Associated data types**

qurt_pipe_attr_t

**Parameters**

| | | |
|---|---|---|
| in,out | *attr* | Pointer to the pipe attribute structure. |
| in | *elements* | Pipe length (64-bit elements). |

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 14.6 qurt_pipe_create()

### 14.6.1 Function Documentation

#### 14.6.1.1 int qurt_pipe_create ( qurt_pipe_t ** *pipe,* qurt_pipe_attr_t * *attr* )

Creates a pipe.

Allocates a pipe object and its associated data buffer, and initializes the pipe object.

**NOTE** The buffer address and size stored in the attribute structure specify how the pipe data buffer is allocated.

If a pipe is specified as allocated in TCM/LPM, it must be created with the qurt_pipe_init() operation. The qurt_pipe_create() operation results in an error.

**Associated data types**

qurt_pipe_t
qurt_pipe_attr_t

**Parameters**

| | | |
|---|---|---|
| out | *pipe* | Pointer to the created pipe object. |
| in | *attr* | Pointer to the attribute structure used to create the pipe. |

**Returns**

QURT_EOK – Pipe created.
QURT_EFAILED – Pipe not created.
QURT_ENOTALLOWED – Pipe cannot be created in TCM/LPM.

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 14.7 qurt_pipe_delete()

### 14.7.1 Function Documentation

#### 14.7.1.1 void qurt_pipe_delete ( qurt_pipe_t * *pipe* )

Deletes the pipe.

Destroys the specified pipe (Section 14.8.1.1) and deallocates the pipe object and its associated data buffer.

**NOTE** Delete pipes only when they were created using qurt_pipe_create() and not qurt_pipe_init(). Otherwise the behavior of QuRT is undefined.

Pipes must be deleted when they are no longer in use. Failure to do this causes resource leaks in the QuRT kernel.

Pipes must not be deleted while they are in use. If this occurs, the behavior of QuRT is undefined.

**Associated data types**

qurt_pipe_t

**Parameters**

| | | |
|---|---|---|
| in | *pipe* | Pointer to the pipe object to destroy. |

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 14.8 qurt_pipe_destroy()

### 14.8.1 Function Documentation

#### 14.8.1.1 void qurt_pipe_destroy ( qurt_pipe_t * *pipe* )

Destroys the specified pipe.

**NOTE** Pipes must be destroyed when they are no longer in use. Failure to do this causes resource leaks in the QuRT kernel. Pipes must not be destroyed while they are still in use. If this occurs, the behavior of QuRT is undefined.

**Associated data types**

qurt_pipe_t

**Parameters**

| | | |
|---|---|---|
| in | *pipe* | Pointer to the pipe object to destroy. |

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 14.9 qurt_pipe_init()

### 14.9.1 Function Documentation

#### 14.9.1.1 int qurt_pipe_init ( qurt_pipe_t ∗ *pipe,* qurt_pipe_attr_t ∗ *attr* )

Uses an existing data buffer to initialize a pipe object .

**NOTE** The buffer address and size stored in the attribute structure must specify a data buffer that the user has already allocated.

**Associated data types**

qurt_pipe_t
qurt_pipe_attr_t

**Parameters**

| | | |
|---|---|---|
| out | *pipe* | Pointer to the pipe object to initialize. |
| in | *attr* | Pointer to the pipe attribute structure used to initialize the pipe. |

**Returns**

QURT_EOK – Success.
QURT_EFAILED – Failure.

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 14.10 qurt_pipe_is_empty()

### 14.10.1 Function Documentation

#### 14.10.1.1 int qurt_pipe_is_empty ( qurt_pipe_t * *pipe* )

Returns a value that indicates whether the specified pipe contains any data.

**Associated data types**

qurt_pipe_t

**Parameters**

| | | |
|---|---|---|
| in | *pipe* | Pointer to the pipe object from which to read. |

**Returns**

1 – Pipe contains no data.
0 – Pipe contains data.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 14.11 qurt_pipe_receive()

## 14.11.1 Function Documentation

### 14.11.1.1 qurt_pipe_data_t qurt_pipe_receive ( qurt_pipe_t * *pipe* )

Reads a data item from the specified pipe.

If a thread reads from an empty pipe, it is suspended on the pipe. When another thread writes to the pipe, the suspended thread is awakened and can then read data from the pipe. Pipe data items are defined as 64-bit values. Pipe reads are limited to transfers of a single 64-bit data item per operation.

**NOTE** To transfer data items larger than 64 bits, read and write pointers to the data or transfer the data in consecutive 64-bit chunks.

#### Associated data types

qurt_pipe_t

#### Parameters

| | | |
|---|---|---|
| in | *pipe* | Pointer to the pipe object from which to read. |

#### Returns

Integer that contains the 64-bit data item from pipe.

#### Safe API

Yes

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 14.12 qurt_pipe_receive_cancellable()

### 14.12.1 Function Documentation

#### 14.12.1.1 int qurt_pipe_receive_cancellable ( qurt_pipe_t ∗ *pipe,* qurt_pipe_data_t ∗ *presult* )

Reads a data item from the specified pipe (with suspend), cancellable.

If a thread reads from an empty pipe, it is suspended on the pipe. When another thread writes to the pipe, the suspended thread is awakened and can then read data from the pipe. The operation is cancelled if the user process of the calling thread is killed or if the calling thread must finish its current QDI invocation and return to user space. The root PD thread can use this API to wait on pipe to receive, and resumes with QURT_EDESTROY if the pipe is destroyed. Pipe data items are defined as 64-bit values. Pipe reads are limited to a transfer of a single 64-bit data item per operation.

**NOTE** To transfer data items larger than 64 bits, read and write pointers to the data or transfer the data in consecutive 64-bit chunks.

**Associated data types**

qurt_pipe_t
qurt_pipe_data_t

**Parameters**

| | | |
|---|---|---|
| in | *pipe* | Pointer to the pipe object from which to read. |
| in | *presult* | Pointer to the integer that contains the 64-bit data item from pipe. |

**Returns**

QURT_EOK – Receive complete.
QURT_ECANCEL – Receive canceled.
QURT_EDESTROY – Receive destroyed.
QURT_ENOTALLOWED – Pipe is not initialized

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.kim@lge.com

## 14.13 qurt_pipe_send()

### 14.13.1 Function Documentation

#### 14.13.1.1 void qurt_pipe_send ( qurt_pipe_t * *pipe,* qurt_pipe_data_t *data* )

Writes a data item to the specified pipe.

If a thread writes to a full pipe, it is suspended on the pipe. When another thread reads from the pipe, the suspended thread is awakened and can then write data to the pipe.

Pipe data items are defined as 64-bit values. Pipe writes are limited to transfers of a single 64-bit data item per operation.

**NOTE** To transfer data items larger than 64 bits, read and write pointers to the data or transfer the data in consecutive 64-bit chunks.

**Associated data types**

qurt_pipe_t
qurt_pipe_data_t

**Parameters**

| | | |
|---|---|---|
| in | *pipe* | Pointer to the pipe object to write to. |
| in | *data* | Data item to write. |

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 14.14 qurt_pipe_send_cancellable()

### 14.14.1 Function Documentation

#### 14.14.1.1 int qurt_pipe_send_cancellable ( qurt_pipe_t * *pipe,* qurt_pipe_data_t *data* )

Writes a data item to the specified pipe (with suspend), cancellable.

If a thread writes to a full pipe, it is suspended on the pipe. When another thread reads from the pipe, the suspended thread is awakened and can then write data to the pipe. The operation is canceled if the user process of the calling thread is killed, or if the calling thread must finish its current QDI invocation and return to user space. The root PD thread can use this API to wait on a pipe to receive and resumes with QURT_EDESTROY when the pipe is destroyed.

Pipe data items are defined as 64-bit values. Pipe writes are limited to a transfer of a single 64-bit data item per operation.

**NOTE** To transfer data items larger than 64 bits, read and write pointers to the data or transfer the data in consecutive 64-bit chunks.

**Associated data types**

qurt_pipe_t
qurt_pipe_data_t

**Parameters**

| | | |
|---|---|---|
| in | *pipe* | Pointer to the pipe object from which to read. |
| in | *data* | Data item to write. |

**Returns**

QURT_EOK – Send complete.
QURT_ECANCEL – Send canceled.
QURT_EDESTROY – Send destroyed.
QURT_ENOTALLOWED – Pipe is not initialized

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-06-27 05:40:42 GMT hyungbum.won@lge.com

## 14.15 qurt_pipe_try_receive()

### 14.15.1 Function Documentation

#### 14.15.1.1 qurt_pipe_data_t qurt_pipe_try_receive ( qurt_pipe_t ∗ *pipe,* int ∗ *success* )

Reads a data item from the specified pipe (without suspending the thread if the pipe is empty).

If a thread reads from an empty pipe, the operation returns immediately with success set to -1. Otherwise, success is always set to 0 to indicate a successful read operation.

Pipe data items are defined as 64-bit values. Pipe reads are limited to a transfer of a single 64-bit data item per operation.

**NOTE** To transfer data items larger than 64 bits, read and write pointers to the data or transfer the data in consecutive 64-bit chunks.

**Associated data types**

qurt_pipe_t

**Parameters**

| | | |
|---|---|---|
| in | *pipe* | Pointer to the pipe object from which to read. |
| out | *success* | Pointer to the operation status result. |

**Returns**

Integer that contains a 64-bit data item from pipe.

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 14.16 qurt_pipe_try_send()

### 14.16.1 Function Documentation

#### 14.16.1.1 int qurt_pipe_try_send ( qurt_pipe_t ∗ *pipe,* qurt_pipe_data_t *data* )

Writes a data item to the specified pipe (without suspending the thread if the pipe is full).

If a thread writes to a full pipe, the operation returns immediately with success set to -1. Otherwise, success is always set to 0 to indicate a successful write operation.

Pipe data items are defined as 64-bit values. Pipe writes are limited to a transfer of a single 64-bit data item per operation.

**NOTE** To transfer data items larger than 64 bits, read and write pointers to the data or transfer the data in consecutive 64-bit chunks.

**Associated data types**

qurt_pipe_t
qurt_pipe_data_t

**Parameters**

| | | |
|---|---|---|
| in | *pipe* | Pointer to the pipe object to write to. |
| in | *data* | Data item to write. |

**Returns**

0 – Success.
-1 – Failure (pipe full).

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 14.17 Data Types

This section describes data types for pipe services.

- Pipes are represented in QuRT as objects of type qurt_pipe_t.
- Pipe data values are represented as objects of type qurt_pipe_data_t.
- Pipe attributes in QuRT are stored in structures of type qurt_pipe_attr_t.

### 14.17.1 Define Documentation

#### 14.17.1.1 #define QURT_PIPE_MAGIC 0xF1FEF1FEU

Magic.

#### 14.17.1.2 #define QURT_PIPE_ATTR_MEM_PARTITION_RAM 0

RAM.

#### 14.17.1.3 #define QURT_PIPE_ATTR_MEM_PARTITION_TCM 1

TCM.

### 14.17.2 Data Structure Documentation

#### 14.17.2.1 struct qurt_pipe_t

QuRT pipe type.

#### 14.17.2.2 struct qurt_pipe_attr_t

QuRT pipe attributes type.

### 14.17.3 Typedef Documentation

#### 14.17.3.1 typedef unsigned long long int qurt_pipe_data_t

QuRT pipe data values type.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 15 Timers

## 15.1 Overview

Threads use timers to perform actions that need to occur at specific intervals. A timer waits for the specified period of time and then generates a timer event.

Timer objects are assigned to specific threads. They support the following operations:

- Create timer – Create the timer with specified attributes and start it.
- Delete timer – Delete the specified timer.
- Stop timer – Stop running the timer.
- Restart timer – Restart the stopped timer.
- Sleep timer – Suspend the current thread for the specified duration.
- Timer group enable – Enable all timers assigned to the specified timer group.
- Timer group disable – Disable all timers assigned to the specified timer group.
- Get timer attributes – Get attributes assigned to a timer when it was created.

When a timer object is created, it is both started and associated with the specified signal object and signal mask. Whenever the timer expires, the signal specified in the signal mask is set in the signal object. A timer event handler must be implemented by the application to wait on that signal in order to handle the timer event.

A running timer can be stopped by calling the timer stop operation. A stopped (or expired) timer can be restarted with a specified duration by calling the timer restart operation.

A thread can suspend itself (Section 3.1) for a specific amount of time by calling the timer sleep operation. The sleep duration specifies the interval (in microseconds) between when the thread is suspended and when it is re-awakened.

Timers can be assigned to groups which make it possible to enable or disable one or more timers with a single operation. A timer state is saved across disabling and subsequent reenabling.

The static attributes of a running timer can be accessed with the get timer attributes operation.

**Note:** Timers can run for up to 36 hours, and have a worst-case error margin of 60 microseconds.

**Attributes**
Timers have the following attributes:

- Duration – Interval between timer events.
- Type – Timer functional behavior (one-shot or periodic).
- Group – Timer group that timer is assigned to.

Qualcomm Confidential - May Contain Trade Secrets 2020-05-27 05:47:45 GMT hyungchul.yun@lge.com

- Remaining – Time remaining before next timer event (read-only).
- Expiry – Absolute time when timer expires (one-shot only).

The timer duration specifies the interval (in microseconds) between the creation of the timer object and the generation of the corresponding timer event.

The timer type specifies the functional behavior of the timer:

- A one-shot timer (QURT_TIMER_ONESHOT) waits for the specified timer duration and then generates a single timer event. After this the timer is nonfunctional.
- A periodic timer (QUR_TIMER_PERIODIC) repeatedly waits for the specified timer duration and then generates a timer event. The result is a series of timer events with interval equal to the timer duration.

The timer group specifies the group that the timer is assigned to. Timer groups are used to enable or disable one or more timers with a single operation.

The timer remaining returns the time remaining (in microseconds) before the generation of the next timer event on the timer. Note that this attribute is read-only.

The timer expiry specifies the absolute time (in microseconds) when the timer expires. Absolute time is defined as the time elapsed since the previous hardware reset of the Hexagon processor. This attribute applies only to one-shot timers.

**Setting timer attributes**

The timer attributes are set before a timer is created using the qurt_timer_attr_init and qurt_timer_attr_set functions.

The timer type must be set on all timers. Depending on the type, either the timer duration or expiry is set – expiry applies only to one-shot timers. The timer group is optional.

**Getting timer attributes**

Timer attributes can be retrieved from a created timer using the qurt_timer_get_attr and qurt_timer_attr_get functions.

Of the various attributes retrieved from a timer, the timer remaining is the only dynamic attribute – it returns the time remaining before the next event occurs on the timer. All the other returned attributes are static, and remain unchanged from when they were set.

**Functions**

Timer objects are assigned to specific threads. They support the following operations:

- qurt_timer_attr_get_duration()
- qurt_timer_attr_get_group()
- qurt_timer_attr_get_remaining()
- qurt_timer_attr_get_type()
- qurt_timer_attr_init()
- qurt_timer_attr_set_duration()
- qurt_timer_attr_set_expiry()
- qurt_timer_attr_set_group()
- qurt_timer_attr_set_type()

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:49:42 GMT hyung-hwan.yun@lge.com

- qurt_timer_create()
- qurt_timer_delete()
- qurt_timer_get_attr()
- qurt_timer_group_disable()
- qurt_timer_group_enable()
- qurt_timer_restart()
- qurt_timer_sleep()
- qurt_timer_stop()
- Constants and Macros
- Data Types

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:40:42 GMT
hyungchul.won@lge.com

## 15.2 qurt_timer_attr_get_duration()

### 15.2.1 Function Documentation

#### 15.2.1.1 void qurt_timer_attr_get_duration ( qurt_timer_attr_t ∗ *attr,* qurt_timer_-duration_t ∗ *duration* )

Gets the timer duration from the specified timer attribute structure. The value returned is the duration that was originally set for the timer.

**NOTE** This function does not return the remaining time of an active timer; use qurt_timer_attr_get_remaining() to get the remaining time.

**Associated data types**

qurt_timer_attr_t
qurt_timer_duration_t

**Parameters**

| | | |
|---|---|---|
| in | *attr* | Pointer to the timer attributes object |
| out | *duration* | Pointer to the destination variable for timer duration. |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 15.3 qurt_timer_attr_get_group()

### 15.3.1 Function Documentation

#### 15.3.1.1 void qurt_timer_attr_get_group ( qurt_timer_attr_t ∗ *attr,* unsigned int ∗ *group* )

Gets the timer group identifier from the specified timer attribute structure.

**Associated data types**

qurt_timer_attr_t

**Parameters**

| | | |
|---|---|---|
| in | *attr* | Pointer to the timer attribute structure. |
| out | *group* | Pointer to the destination variable for the timer group identifier. |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 15.4 qurt_timer_attr_get_remaining()

### 15.4.1 Function Documentation

#### 15.4.1.1 void qurt_timer_attr_get_remaining ( qurt_timer_attr_t ∗ *attr,* qurt_timer_-duration_t ∗ *remaining* )

Gets the timer remaining duration from the specified timer attribute structure.

The timer remaining duration indicates (in microseconds) how much time remains before the generation of the next timer event on the corresponding timer. In most cases, this function assumes that the timer attribute structure was obtained via a call to qurt_timer_get_attr().

**NOTE** This attribute is read-only and thus has no set operation defined for it.

**Associated data types**

qurt_timer_attr_t
qurt_timer_duration_t

**Parameters**

| | | |
|---|---|---|
| in | *attr* | Pointer to the timer attribute object. |
| out | *remaining* | Pointer to the destination variable for the remaining time. |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 15.5 qurt_timer_attr_get_type()

### 15.5.1 Function Documentation

#### 15.5.1.1 void qurt_timer_attr_get_type ( qurt_timer_attr_t * *attr,* qurt_timer_type_t * *type* )

Gets the timer type from the specified timer attribute structure.

**Associated data types**

qurt_timer_attr_t
qurt_timer_type_t

**Parameters**

| | | |
|---|---|---|
| in | *attr* | Pointer to the timer attribute structure. |
| out | *type* | Pointer to the destination variable for the timer type. |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 15.6 qurt_timer_attr_init()

### 15.6.1 Function Documentation

#### 15.6.1.1 void qurt_timer_attr_init ( qurt_timer_attr_t ∗ *attr* )

Initializes the specified timer attribute structure with default attribute values:

- Timer duration: QURT_TIMER_DEFAULT_DURATION (Section 15)
- Timer type: QURT_TIMER_ONESHOT
- Timer group: QURT_TIMER_DEFAULT_GROUP

**Associated data types**

qurt_timer_attr_t

**Parameters**

| | | |
|---|---|---|
| in,out | *attr* | Pointer to the destination structure for the timer attributes. |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 15.7 qurt_timer_attr_set_duration()

### 15.7.1 Function Documentation

#### 15.7.1.1 void qurt_timer_attr_set_duration ( qurt_timer_attr_t ∗ *attr,* qurt_timer_-duration_t *duration* )

Sets the timer duration in the specified timer attribute structure.

The timer duration specifies the interval (in microseconds) between the creation of the timer object and the generation of the corresponding timer event.

The timer duration value must be between QURT_TIMER_MIN_DURATION and QURT_TIMER_MAX_DURATION (Section 15). Otherwise, the set operation is ignored.

**Associated data types**

qurt_timer_attr_t
qurt_timer_duration_t

**Parameters**

| | | |
|---|---|---|
| in,out | *attr* | Pointer to the timer attribute structure. |
| in | *duration* | Timer duration (in microseconds). Valid range is QURT_TIMER_MIN_DURATION to QURT_TIMER_MAX_DURATION. |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 15.8 qurt_timer_attr_set_expiry()

### 15.8.1 Function Documentation

#### 15.8.1.1 void qurt_timer_attr_set_expiry ( qurt_timer_attr_t * *attr,* qurt_timer_time_t *time* )

Sets the absolute expiry time in the specified timer attribute structure.

The timer expiry specifies the absolute time (in microseconds) of the generation of the corresponding timer event.

Timer expiries are relative to when the system first began to execute.

**Associated data types**

qurt_timer_attr_t
qurt_timer_time_t

**Parameters**

| | | |
|---|---|---|
| in,out | *attr* | Pointer to the timer attribute structure. |
| in | *time* | Timer expiry. |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 15.9 qurt_timer_attr_set_group()

### 15.9.1 Function Documentation

#### 15.9.1.1 void qurt_timer_attr_set_group ( qurt_timer_attr_t ∗ *attr,* unsigned int *group* )

Sets the timer group identifier in the specified timer attribute structure.

The timer group identifier specifies the group that the timer belongs to. Timer groups are used to enable or disable one or more timers in a single operation.

The timer group identifier value must be between 0 and (QURT_TIMER_MAX_GROUPS - 1). See Section 15.

**Associated data types**

qurt_timer_attr_t

**Parameters**

| | | |
|---|---|---|
| in,out | *attr* | Pointer to the timer attribute object. |
| in | *group* | Timer group identifier; Valid range is 0 to (QURT_TIMER_MAX_GROUPS - 1). |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 15.10 qurt_timer_attr_set_type()

## 15.10.1 Function Documentation

### 15.10.1.1 void qurt_timer_attr_set_type ( qurt_timer_attr_t * *attr,* qurt_timer_type_t *type* )

Sets the timer type in the specified timer attribute structure.

The timer type specifies the functional behavior of the timer:

- A one-shot timer (QURT_TIMER_ONESHOT) waits for the specified timer duration and then generates a single timer event. After this the timer is nonfunctional.
- A periodic timer (QURT_TIMER_PERIODIC) repeatedly waits for the specified timer duration and then generates a timer event. The result is a series of timer events with interval equal to the timer duration.

#### Associated data types

qurt_timer_attr_t
qurt_timer_type_t

#### Parameters

| | | |
|---|---|---|
| in,out | *attr* | Pointer to the timer attribute structure. |
| in | *type* | Timer type. Values are:<br>• QURT_TIMER_ONESHOT – One-shot timer.<br>• QURT_TIMER_PERIODIC – Periodic timer. |

#### Safe API

No

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 15.11 qurt_timer_create()

## 15.11.1 Function Documentation

### 15.11.1.1 int qurt_timer_create ( qurt_timer_t ∗ *timer,* const qurt_timer_attr_t ∗ *attr,* const qurt_anysignal_t ∗ *signal_ptr,* unsigned int *mask* )

Creates a timer.

Allocates and initializes a timer object, and starts the timer.

**NOTE** A timer event handler must be defined to wait on the specified signal to handle the timer event.

#### Associated data types

qurt_timer_t
qurt_timer_attr_t
qurt_anysignal_t

#### Parameters

| | | |
|---|---|---|
| out | *timer* | Pointer to the created timer object. |
| in | *attr* | Pointer to the timer attribute structure. |
| in | *signal_ptr* | Pointer to the signal object set when timer expires. |
| in | *mask* | Signal mask, which specifies the signal to set in the signal object when the time expires. |

#### Returns

QURT_EOK – Success.
QURT_EMEM – Not enough memory to create the timer.
QURT_EINVALID – One of the arguments in the attr field is invalid.
Other error code – Operation failed.

#### Safe API

No

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:49:48 GMT hyungchul.won@lge.com

## 15.12 qurt_timer_delete()

### 15.12.1 Function Documentation

#### 15.12.1.1 int qurt_timer_delete ( qurt_timer_t *timer* )

Deletes the timer.

Destroys the specified timer and deallocates the timer object.

**Associated data types**

qurt_timer_t

**Parameters**

| | | |
|---|---|---|
| in | *timer* | Timer object. |

**Returns**

QURT_EOK – Success.
QURT_EVAL – Argument passed is not a valid timer.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 15.13 qurt_timer_get_attr()

### 15.13.1 Function Documentation

#### 15.13.1.1 int qurt_timer_get_attr ( qurt_timer_t *timer,* qurt_timer_attr_t ∗ *attr* )

Gets the timer attributes of the specified timer when it was created.

**Associated data types**

qurt_timer_t
qurt_timer_attr_t

**Parameters**

| | | |
|---|---|---|
| in | *timer* | Timer object. |
| out | *attr* | Pointer to the destination structure for timer attributes. |

**Returns**

QURT_EOK – Success.
QURT_EVAL – Argument passed is not a valid timer.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 15.14 qurt_timer_group_disable()

### 15.14.1 Function Documentation

#### 15.14.1.1 int qurt_timer_group_disable ( unsigned int *group* )

Disables all timers that are assigned to the specified timer group. If a specified timer is already disabled, ignore it. If a specified timer is expired, do not process it. If the specified timer group is empty, do nothing.

**NOTE** When a timer is disabled, its remaining time does not change, thus it cannot generate a timer event.

**Parameters**

| | | |
|---|---|---|
| in | *group* | Timer group identifier. |

**Returns**

QURT_EOK – Success.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 15.15 qurt_timer_group_enable()

### 15.15.1 Function Documentation

#### 15.15.1.1 int qurt_timer_group_enable ( unsigned int *group* )

Enables all timers that are assigned to the specified timer group. If a specified timer is already enabled, ignore it. If a specified timer is expired, process it. If the specified timer group is empty, do nothing.

**Parameters**

| | | |
|---|---|---|
| in | *group* | Timer group identifier. |

**Returns**

QURT_EOK – Success.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 15.16 qurt_timer_restart()

### 15.16.1 Function Documentation

#### 15.16.1.1 int qurt_timer_restart ( qurt_timer_t *timer,* qurt_timer_duration_t *time_duration* )

Restarts a stopped timer with the specified duration. The timer must be a one-shot timer. Timers stop after they have expired or after they are explicitly stopped with qurt_timer_stop(). A restarted timer expires after the specified duration, the starting time is when the function is called.

**NOTE** Timers stop after they have expired or after they are explicitly stopped with the timer stop operation, see Section 15.18.1.1.

**Associated data types**

qurt_timer_t
qurt_timer_duration_t

**Parameters**

| | | |
|---|---|---|
| in | *timer* | Timer object. |
| in | *time_duration* | Timer duration (in microseconds) before the restarted timer expires again. The valid range is QURT_TIMER_MIN_DURATION to QURT_TIMER_MAX_DURATION. |

**Returns**

QURT_EOK – Success.
QURT_EINVALID – Invalid timer ID or duration value.
QURT_ENOTALLOWED – Timer is not a one-shot timer.
QURT_EMEM – Out-of-memory error.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:52 GMT hyungchul.won@lge.com

# 15.17 qurt_timer_sleep()

## 15.17.1 Function Documentation

### 15.17.1.1 int qurt_timer_sleep ( qurt_timer_duration_t *duration* )

Suspends the current thread for the specified amount of time. The sleep duration value must be between QURT_TIMER_MIN_DURATION and QURT_TIMER_MAX_DURATION (Section 15).

#### Associated data types

qurt_timer_duration_t

#### Parameters

| | | |
|---|---|---|
| in | *duration* | Interval (in microseconds) between when the thread is suspended and when it is re-awakened. |

#### Returns

QURT_EOK – Success.
QURT_EMEM – Not enough memory to perform the operation.

#### Safe API

No

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 15.18 qurt_timer_stop()

## 15.18.1 Function Documentation

### 15.18.1.1 int qurt_timer_stop ( qurt_timer_t *timer* )

Stops a running timer. The timer must be a one-shot timer.

**NOTE** Restart stopped timers with the timer restart operation, see Section 15.16.1.1.

#### Associated data types

qurt_timer_t

#### Parameters

| | | |
|---|---|---|
| in | *timer* | Timer object. |

#### Returns

QURT_EOK – Success.
QURT_EINVALID – Invalid timer ID or duration value.
QURT_ENOTALLOWED – Timer is not a one shot timer.
QURT_EMEM – Out of memory error.

#### Safe API

No

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 15.19 Constants and Macros

This section describes constants and macros for timer services.

### 15.19.1 Define Documentation

#### 15.19.1.1 #define QURT_TIMER_DEFAULT_TYPE QURT_TIMER_ONESHOT

Default values. One shot.

#### 15.19.1.2 #define QURT_TIMER_DEFAULT_DURATION 1000uL

Default duration.

#### 15.19.1.3 #define QURT_TIMER_DEFAULT_EXPIRY 0uL

Default expiration.

#### 15.19.1.4 #define QURT_TIMER_TIMETICK_FROM_US( *us* ) QURT_SYSCLOCK_TIMETICK_FROM_US(us)

Conversion from microseconds to timer ticks.

#### 15.19.1.5 #define QURT_TIMER_TIMETICK_TO_US( *ticks* ) qurt_timer_timetick_to_us(ticks)

Conversion from timer ticks to microseconds at the nominal frequency.

#### 15.19.1.6 #define QURT_TIMER_MIN_DURATION 100uL

Minimum microseconds value is 100 microseconds (sleep timer).

#### 15.19.1.7 #define QURT_TIMER_MAX_DURATION QURT_SYSCLOCK_MAX_DURATION

Maximum microseconds value for Qtimer is 1,042,499 hours.

#### 15.19.1.8 #define QURT_TIMER_MAX_DURATION_TICKS QURT_SYSCLOCK_MAX_DURATION_TICKS

Timer clock for Qtimer is 19.2 MHz.

#### 15.19.1.9 #define QURT_TIMETICK_ERROR_MARGIN QURT_SYSCLOCK_ERROR_MARGIN

Sleep timer error margin for Qtimer is 1,000 ticks ∼52 us.

#### 15.19.1.10 #define QURT_TIMER_MAX_GROUPS 5U

Maximum groups.

#### 15.19.1.11 #define QURT_TIMER_DEFAULT_GROUP 0U

Default groups.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 15.20 Data Types

This section describes data types for timer services.

- Timers are represented in QuRT as objects of type qurt_timer_t.
- Timer attributes are stored in structures of type qurt_timer_attr_t.
- Timer durations are specified as values of type qurt_timer_duration_t.
- Timer times are specified as values of type qurt_timer_time_t.
- Timer types are specified as values of type qurt_timer_type_t.

### 15.20.1 Data Structure Documentation

#### 15.20.1.1 struct qurt_timer_attr_t

QuRT timer attribute type.

### 15.20.2 Typedef Documentation

#### 15.20.2.1 typedef unsigned int qurt_timer_t

QuRT timer type.

#### 15.20.2.2 typedef unsigned long long qurt_timer_duration_t

QuRT timer duration type.

#### 15.20.2.3 typedef unsigned long long qurt_timer_time_t

QuRT timer time type.

### 15.20.3 Enumeration Type Documentation

#### 15.20.3.1 enum qurt_timer_type_t

QuRT timer types.

**Enumerator:**

***QURT_TIMER_ONESHOT*** One shot.
***QURT_TIMER_PERIODIC*** Periodic.

Qualcomm Confidential - May Contain Trade Secrets 2026-03-27 05:40:42 GMT hyungchul.won@lge.com

# 16 System Clock

## 16.1 Overview

Threads use the QuRT system clock to create alarms and timers, access the current system time, or determine when the next timer event will occur on any active timer.

The system clock time indicates how long (in terms of system ticks) the QuRT application system has been executing. A system tick is defined as one cycle of the Hexagon processor's 19.2 MHz QTIMER clock.

The system clock supports the following operations:

- Register client – Register the client for the system clock event.
- Create alarm – Create the system clock alarm with the specified time and start it.
- Create timer – Create the system clock timer with the specified duration and start it.
- Get hardware ticks – Get the hardware tick count.
- Get timer expiry – Return the time remaining before the next event on any timer.

Unlike regular timers (Section 15.1), system clock alarms and timers are global resources which can notify multiple client threads that a clock event has occurred. When a client thread registers for a system clock event, it specifies a signal object and signal mask.

System clock alarms expire at a specified time, while system clock timers expire after a specified duration. In both cases, when the event occurs, for each registered client thread the signal specified in the registered signal mask is set in the registered signal object.

The get hardware ticks operation returns the current value of a 32-bit hardware counter. The value wraps around to zero when it exceeds the maximum value.

**Note:** This operation must be used with care because of the wrap-around behavior.

The get timer expiry operation returns the number of system clock ticks remaining before the next timer event occurs on any user-defined timer or alarm in the QuRT application system (including those described in Section 15.1).

**Functions**

System clock services are accessed with the following QuRT functions.

- qurt_sysclock_get_hw_ticks()
- qurt_sysclock_get_hw_ticks_32()
- qurt_sysclock_get_hw_ticks_16()

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:44:42 GMT hyungshul.won@lge.com

## 16.2 qurt_sysclock_get_hw_ticks()

### 16.2.1 Function Documentation

#### 16.2.1.1 unsigned long long qurt_sysclock_get_hw_ticks ( void )

Gets the hardware tick count.

Returns the current value of a 64-bit hardware counter. The value wraps around to zero when it exceeds the maximum value.

**NOTE** This operation must be used with care because of the wrap-around behavior.

**Returns**

Integer – Current value of 64-bit hardware counter.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 16.3 qurt_sysclock_get_hw_ticks_32()

### 16.3.1 Function Documentation

#### 16.3.1.1 static unsigned long qurt_sysclock_get_hw_ticks_32 ( void )

Gets the hardware tick count in 32 bits.

Returns the current value of a 32-bit hardware counter. The value wraps around to zero when it exceeds the maximum value.

**NOTE** This operation is implemented as an inline C function, and should be called from a C/C++ program. The returned 32 bits are the lower 32 bits of the Qtimer counter.

**Returns**

Integer – Current value of the 32-bit timer counter.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 16.4 qurt_sysclock_get_hw_ticks_16()

### 16.4.1 Function Documentation

#### 16.4.1.1 static unsigned short qurt_sysclock_get_hw_ticks_16 ( void )

Gets the hardware tick count in 16 bits.

Returns the current value of a 16-bit timer counter. The value wraps around to zero when it exceeds the maximum value.

**NOTE** This operation is implemented as an inline C function, and should be called from a C/C++ program. The returned 16 bits are based on the value of the lower 32 bits in Qtimer counter, right shifted by 16 bits.

**Returns**

Integer – Current value of the 16-bit timer counter, calculated from the lower 32 bits in the Qtimer counter, right shifted by 16 bits.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 17 Interrupts

## 17.1 Overview

Threads use interrupts to respond to external events. Interrupts are processor resources which support the following operations:

- Register interrupt – Enable the signal object to receive a specific interrupt.
- Deregister interrupt – Disable the signal object from receiving a specific interrupt.
- Acknowledge interrupt – Re-enable the interrupt after it has been processed.
- Get registered interrupts – Return the bitmask indicating registered interrupts.
- Interrupt status – Return the pending status of the specified interrupt.
- Clear interrupt – Clear the pending status of the specified interrupt.
- Raise interrupt –Trigger an interrupt from software.
- Get interrupt configuration – Return the L2VIC interrupt type and polarity.
- Set interrupt configuration – Set the L2VIC interrupt type and polarity.

When an interrupt is registered, it is both enabled and associated with the specified signal object and signal mask. When an interrupt occurs, the signal specified in the signal mask is set in the signal object. An interrupt service thread (IST) conventionally waits on that signal in order to handle the interrupt.

Interrupts are automatically disabled after they occur. To re-enable an interrupt, an IST performs the acknowledge interrupt operation after it has finished processing the interrupt and just before suspending itself (i.e., by waiting on the interrupt signal). When an interrupt is deregistered it is disabled and no longer associated with any signal.

Up to 31 separate interrupts can be registered to a single signal object, as determined by the number of individual signals the object can store. (Note that signal 31 is reserved by QuRT.) Thus a single IST can handle several different interrupts.

For more information on interrupts, refer to [80-VB419-92].

**Note:** Only one signal object can be registered to a specific interrupt. Registering multiple signal objects on an interrupt raises an exception (Section 19.1).

Threads that serve as ISTs must not call the exit thread operation.

Interrupts do not support init and destroy operations because no objects (Section 2.4) are created for them.

**Pending interrupts**

A pending interrupt can be explicitly cleared with the clear interrupt operation.

**Note:** This operation is intended for system-level use, and must be used with care.

Qualcomm Confidential - May Contain Trade Secrets 2020-02-27 05:10:42 GMT hyungchul.kim@lge.com

### L2VIC interrupts

In the V4 and V5 Hexagon processors, all interrupts are based on the L2VIC interrupt controller. All interrupts must be specified using the L2VIC interrupt numbers.

L2VIC interrupts can be configured dynamically to have different types (edge-triggered or level-triggered) or polarities (active-low or active-high).

**Note:** L2VIC interrupts must be deregistered before they can be reconfigured.

### Functions

Interrupt services are accessed with the following QuRT functions.

- qurt_interrupt_acknowledge()
- qurt_interrupt_clear()
- qurt_interrupt_deregister()
- qurt_interrupt_enable()
- qurt_interrupt_disable()
- qurt_interrupt_get_config()
- qurt_interrupt_raise()
- qurt_interrupt_register()
- qurt_interrupt_register2()
- qurt_interrupt_set_config()
- qurt_interrupt_set_config2()
- qurt_interrupt_set_config3()
- qurt_interrupt_status()
- qurt_interrupt_get_status()
- qurt_interrupt_raise2()
- Constants
- Interrupt types

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 17.2 qurt_interrupt_acknowledge()

### 17.2.1 Function Documentation

#### 17.2.1.1 int qurt_interrupt_acknowledge ( int *int_num* )

Acknowledges an interrupt after it has been processed.

Re-enables an interrupt and clears its pending status after an interrupt is processed by an IST.

Interrupts are automatically disabled after they occur. To re-enable an interrupt, an IST performs the acknowledge operation after it finishes processing the interrupt and just before suspending itself (such as by waiting on the interrupt signal).

**NOTE** To prevent losing or reprocessing subsequent occurrences of the interrupt, an IST must clear the interrupt signal (Section 9.2.1.1) before acknowledging the interrupt.

**Parameters**

| | | |
|---|---|---|
| in | *int_num* | Interrupt that is being re-enabled. |

**Returns**

QURT_EOK – Interrupt acknowledge was successful.
QURT_EDEREGISTERED – Interrupt is already de-registered.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 17.3 qurt_interrupt_clear()

### 17.3.1 Function Documentation

#### 17.3.1.1 unsigned int qurt_interrupt_clear ( int *int_num* )

Clears the pending status of the specified interrupt.

**NOTE** This operation is intended for system-level use, and must be used with care.

**Parameters**

| | | |
|---|---|---|
| in | *int_num* | Interrupt that is being re-enabled. |

**Returns**

QURT_EOK – Success.
QURT_EINT – Invalid interrupt number.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 17.4 qurt_interrupt_deregister()

### 17.4.1 Function Documentation

#### 17.4.1.1 unsigned int qurt_interrupt_deregister ( int *int_num* )

Disables the specified interrupt and disassociates it from a QuRT signal object. If the specified interrupt was never registered (Section 17.9.1.1), the deregister operation returns the status value QURT_EINT.

**NOTE** If an interrupt is deregistered while an IST waits to receive it, the IST might wait indefinitely for the interrupt to occur. To avoid this problem, the QuRT kernel sends the signal SIG_INT_ABORT to awaken an IST after determining that it has no interrupts registered.

**Parameters**

| | | |
|---|---|---|
| in | *int_num* | L2VIC to deregister; valid range is 0 to 1023. |

**Returns**

QURT_EOK – Success.
QURT_EINT – Invalid interrupt number (not registered).

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 17.5 qurt_interrupt_enable()

### 17.5.1 Function Documentation

#### 17.5.1.1 unsigned int qurt_interrupt_enable ( int *int_num* )

Enables an interrupt with its interrupt number.

The interrupt must be registered prior to calling this function.

**Parameters**

| | | |
|---|---|---|
| in | *int_num* | Interrupt number. |

**Returns**

QURT_EOK – Interrupt successfully enabled.
QURT_EINT – Invalid interrupt number.
QURT_ENOTALLOWED – Interrupt is locked.
QURT_EVAL – Interrupt is not registered.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 17.6 qurt_interrupt_disable()

### 17.6.1 Function Documentation

#### 17.6.1.1 unsigned int qurt_interrupt_disable ( int *int_num* )

Disables an interrupt with its interrupt number.

The interrupt must be registered prior to calling this function. After qurt_interrupt_disable() returns, the Hexagon subsystem can no longer send the corresponding interrupt to the Hexagon core, until qurt_interrupt_enable() is called for the same interrupt.

Avoid calls to qurt_interrupt_disable() and qurt_interrupt_enable() frequently within a short period of time.

- A pending interrupt can already be in the Hexagon core when qurt_interrupt_disable() is called. Therefore, some time later, the pending interrupt is received on a Hexagon hardware thread.
- After the Hexagon subsystem sends an interrupt to the Hexagon core, the Hexagon hardware automatically disables the interrupt until kernel software re-enables the interrupt at the interrupt acknowledgement stage. If qurt_interrupt_enable() is called from a certain thread at an ealier time, the interrupt is re-enabled earlier and can trigger sending a new interrupt to the Hexagon core while kernel software is still processing the previous interrupt.

**Parameters**

| | | |
|---|---|---|
| in | *int_num* | Interrupt number. |

**Returns**

QURT_EOK – Interrupt successfully disabled.
QURT_EINT – Invalid interrupt number.
QURT_ENOTALLOWED – Interrupt is locked.
QURT_EVAL – Interrupt is not registered.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 17.7 qurt_interrupt_get_config()

### 17.7.1 Function Documentation

#### 17.7.1.1 unsigned int qurt_interrupt_get_config ( unsigned int *int_num,* unsigned int ∗ *int_type,* unsigned int ∗ *int_polarity* )

Gets the L2VIC interrupt configuration.

This function returns the type and polarity of the specified L2VIC interrupt.

**Parameters**

| | | |
|---|---|---|
| in | *int_num* | L2VIC interrupt that is being re-enabled. |
| out | *int_type* | Pointer to an interrupt type.<br>0 – Level-triggered interrupt<br>1 – Eedge-triggered interrupt |
| out | *int_polarity* | Pointer to interrupt polarity.<br>0 – Active-high interrupt<br>1 – Active-low interrupt. |

**Returns**

QURT_EOK – Configuration successfully returned.
QURT_EINT – Invalid interrupt number.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 17.8 qurt_interrupt_raise()

### 17.8.1 Function Documentation

#### 17.8.1.1 int qurt_interrupt_raise ( unsigned int *interrupt_num* )

Raises the interrupt.

This function triggers a level-triggered L2VIC interrupt, and accepts interrupt numbers in the range of 0 to 1023.

**Parameters**

| | | |
|---|---|---|
| in | *interrupt_num* | Interrupt number. |

**Returns**

QURT_EOK – Success
-1 – Failure; the interrupt is not supported.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 17.9 qurt_interrupt_register()

### 17.9.1 Function Documentation

#### 17.9.1.1 unsigned int qurt_interrupt_register ( int *int_num,* qurt_anysignal_t * *int_signal,* int *signal_mask* )

Registers the interrupt.

Enables the specified interrupt and associates it with the specified QuRT signal object and signal mask.

Signals are represented as bits 0 through 31 in the 32-bit mask value. A mask bit value of 1 indicates that a signal must be waited on, and 0 indicates not to wait. When the interrupt occurs, the signal specified in the signal mask is set in the signal object. An IST conventionally waits on that signal to handle the interrupt. The thread that registers the interrupt is set as the IST.

Up to 31 separate interrupts can be registered to a single signal object, as determined by the number of individual signals THAT the object can store. QuRT reserves signal 31. Thus a single IST can handle several different interrupts.

QuRT reserves some interrupts for internal use – the remainder are available for use by applications, and thus are valid interrupt numbers. If the specified interrupt number is outside the valid range, the register operation returns the status value QURT_EINT.

Only one thread can be registered at a time to a specific interrupt. Attempts to register an already-registered interrupt return the status value QURT_EVAL. Only one signal bit in a signal object can be registered at a time to a specific interrupt. Attempts to register multiple signal bits to an interrupt return the status value QURT_ESIG.

When the signal registers an interrupt, QuRT can only set its signal bits when receiving the interrupt. The QuRT signal API from another software thread cannot set the signal even for unused signal bits.

**NOTE** The valid range for an interrupt number can differ on target execution environments other than the simulator. For more information, see the appropriate hardware document.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:46:42 GMT hyungchul.won@lge.com

**Associated data types**

qurt_anysignal_t

**Parameters**

| | | |
|---|---|---|
| in | *int_num* | L2VIC interrupt to deregister; valid range is 0 to 1023. |
| in | *int_signal* | Any-signal object to wait on (Section 9). |
| in | *signal_mask* | Signal mask value that indicates the signal to receive the interrupt. |

**Returns**

QURT_EOK – Interrupt successfully registered.
QURT_EINT – Invalid interrupt number.
QURT_ESIG – Invalid signal bitmask (cannot set more than one signal at a time).
QURT_EVAL – Interrupt already registered.

### Safe API

No

### Dependencies

None.

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:40:42 GMT
hyungchul.won@lge.com

## 17.10 qurt_interrupt_register2()

### 17.10.1 Function Documentation

#### 17.10.1.1 unsigned int qurt_interrupt_register2 ( int *int_num,* qurt_anysignal_t * *int_signal,* int *signal_mask,* unsigned int *flags* )

Registers the interrupt.

Enables the specified interrupt, associates it with the specified QuRT signal object and signal mask, and sets interrupt flags.

Signals are represented as bits 0 through 31 in the 32-bit mask value. A mask bit value of 1 indicates that a signal must be waited on, and 0 indicates not to wait.

When the interrupt occurs, the signal specified in the signal mask is set in the signal object. An IST conventionally waits on that signal to handle the interrupt. The thread that registers the interrupt is set as the IST.

Up to 31 separate interrupts can be registered to a single signal object, as determined by the number of individual signals that the object can store. QuRT reserves signal 31. Thus a single IST can handle several different interrupts.

QuRT reserves some interrupts for internal use – the remainder are available for use by applications, and thus are valid interrupt numbers. If the specified interrupt number is outside the valid range, the register operation returns the QURT_EINT status value.

Only one thread can be registered at a time to a specific interrupt. Attempts to register an already-registered interrupt return the QURT_EVAL status value.

Only one signal bit in a signal object can be registered at a time to a specific interrupt. Attempts to register multiple signal bits to an interrupt return the QURT_ESIG status value .

When the signal registers an interrupt, QuRT can only set its signal bits when receiving the interrupt. The QuRT signal API from another software thread cannot set the signal, even for unused signal bits.

**NOTE** The valid range for an interrupt number can differ on target execution environments other than the simulator. For more information, see the appropriate hardware document.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:32 GMT hyungchul.ryoo@lge.com

**Associated data types**

qurt_anysignal_t

**Parameters**

| | | |
|---|---|---|
| in | *int_num* | L2VIC interrupt to deregister; valid range is 0 to 1023. |
| in | *int_signal* | Any-signal object to wait on (Section 9). |
| in | *signal_mask* | Signal mask value that indicates the signal to receive the interrupt. |
| in | *flags* | Interrupt property, supported properties are interrupt lock enable or disable:<br>• QURT_INT_LOCK_ENABLE<br>• QURT_INT_LOCK_DISABLE |

### Returns

QURT_EOK – Interrupt successfully registered.
QURT_EINT – Invalid interrupt number.
QURT_ESIG – Invalid signal bitmask (cannot set more than one signal at a time).
QURT_EVAL – Interrupt already registered.

### Safe API

No

### Dependencies

None.

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:40:42 GMT
hyungchul.won@lge.com

## 17.11 qurt_interrupt_set_config()

### 17.11.1 Function Documentation

#### 17.11.1.1 unsigned int qurt_interrupt_set_config ( unsigned int *int_num,* unsigned int *int_type,* unsigned int *int_polarity* )

Sets the type and polarity of the specified L2VIC interrupt.

**NOTE** Deregister L2VIC interrupts before reconfiguring them.

**Parameters**

| | | |
|---|---|---|
| in | *int_num* | L2VIC interrupt that is being re-enabled. |
| in | *int_type* | Interrupt type.<br>0 – Level-triggered interrupt<br>1 – Edge-triggered interrupt |
| in | *int_polarity* | Interrupt polarity.<br>0 – Active-high interrupt<br>1 – Active-low interrupt |

**Returns**

QURT_EOK – Success.
QURT_ENOTALLOWED – Not allowed; the interrupt is being registered.
QURT_EINT – Invalid interrupt number.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 17.12 qurt_interrupt_set_config2()

### 17.12.1 Function Documentation

#### 17.12.1.1 unsigned int qurt_interrupt_set_config2 ( unsigned int *int_num,* unsigned int *int_type* )

Sets the type and polarity of the specified L2VIC interrupt.

**NOTE** L2VIC interrupts must be deregistered before they can be reconfigured.

**Parameters**

| | | |
|---|---|---|
| in | *int_num* | L2VIC interrupt that is being re-enabled. |
| in | *int_type* | Notified to the hardware configuration callback function and used to modify the L2VIC type. Values:<br>• QURT_INT_TRIGGER_USE_DEFAULT<br>• QURT_INT_TRIGGER_LEVEL_HIGH<br>• QURT_INT_TRIGGER_LEVEL_LOW<br>• QURT_INT_TRIGGER_RISING_EDGE<br>• QURT_INT_TRIGGER_FALLING_EDGE<br>• QURT_INT_TRIGGER_DUAL_EDGE |

**Returns**

QURT_EOK – Success.
QURT_ENOTALLOWED – Not allowed; the interrupt is being registered.
QURT_EINT – Invalid interrupt number.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 17.13 qurt_interrupt_set_config3()

### 17.13.1 Function Documentation

#### 17.13.1.1 unsigned int qurt_interrupt_set_config3 ( unsigned int *int_num,* unsigned int *config_id,* unsigned int *config_val* )

Sets the specified configuration value for the specified property of the specified L2VIC interrupt.

**NOTE** L2VIC interrupts must be deregistered before they can be reconfigured for polarity.

**Parameters**

| | | |
|---|---|---|
| in | *int_num* | L2VIC interrupt to re-enable. |
| in | *config_id* | Property to configure:<br>• QURT_INT_CONFIGID_POLARITY<br>• QURT_INT_CONFIGID_LOCK |
| in | *config_val* | Dependent on the second config_id argument, specifies the value to set.<br>QURT_INT_CONFIGID_POLARITY values:<br>• QURT_INT_TRIGGER_USE_DEFAULT<br>• QURT_INT_TRIGGER_LEVEL_HIGH<br>• QURT_INT_TRIGGER_LEVEL_LOW<br>• QURT_INT_TRIGGER_RISING_EDGE<br>• QURT_INT_TRIGGER_FALLING_EDGE<br>• QURT_INT_TRIGGER_DUAL_EDGE<br>QURT_INT_CONFIGID_LOCK values:<br>• QURT_INT_LOCK_ENABLE<br>• QURT_INT_LOCK_DISABLE |

**Returns**

QURT_EOK – Success.
QURT_ENOTALLOWED – Not allowed; the interrupt is being registered or is locked for enable/disable.
QURT_EINT – Invalid interrupt number.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyunchul.yon@lge.com

## 17.14 qurt_interrupt_status()

### 17.14.1 Function Documentation

#### 17.14.1.1 unsigned int qurt_interrupt_status ( int *int_num*, int ∗ *status* )

Returns a value that indicates the pending status of the specified interrupt.

**Parameters**

| | | |
|---|---|---|
| in | *int_num* | Interrupt number that is being checked. |
| out | *status* | Interrupt status:<br>1 – Interrupt is pending<br>0 – Interrupt is not pending. |

**Returns**

QURT_EOK – Success.
QURT_EINT – Failure; invalid interrupt number.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 17.15 qurt_interrupt_get_status()

### 17.15.1 Function Documentation

#### 17.15.1.1 unsigned int qurt_interrupt_get_status ( int *int_num*, int *status_type*, int * *status* )

Gets the status of the specified interrupt in L2VIC.

**Parameters**

| | | |
|---|---|---|
| in | *int_num* | Interrupt number that is being checked. |
| in | *status_type* | 0 – Interrupt pending status<br>1 – Interrupt enable status |
| out | *status* | 0 – OFF<br>1 – ON |

**Returns**

QURT_EOK – Success.
QURT_EINT – Failure; invalid interrupt number.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 17.16 qurt_interrupt_raise2()

### 17.16.1 Function Documentation

#### 17.16.1.1 unsigned long long qurt_interrupt_raise2 ( unsigned int *interrupt_num* )

Raises the interrupt and returns the current pcycle value.

**Parameters**

| | | |
|---|---|---|
| in | *interrupt_num* | Interrupt number. |

**Returns**

0xFFFFFFFFFFFFFFFF – Failure; the interrupt is not supported.
Other value – pcycle count at the time the interrupt is raised.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 17.17 Constants

### 17.17.1 Define Documentation

#### 17.17.1.1 #define SIG_INT_ABORT 0x80000000

#### 17.17.1.2 #define QURT_INT_NON_DELAYED_ACK 0

#### 17.17.1.3 #define QURT_INT_DELAYED_ACK 1

#### 17.17.1.4 #define QURT_INT_ACK_DEFAULT QURT_INT_NON_DELAYED_ACK

#### 17.17.1.5 #define QURT_INT_DRV_DEFAULT 0

#### 17.17.1.6 #define QURT_INT_PRIORITY_DEFAULT 0xFF

#### 17.17.1.7 #define QURT_INT_CONFIGID_POLARITY 0x1U

QuRT interrupt property.

#### 17.17.1.8 #define QURT_INT_CONFIGID_LOCK 0x2U

#### 17.17.1.9 #define QURT_INT_LOCK_DEFAULT 0x0

QuRT interrupt lock. Default.

#### 17.17.1.10 #define QURT_INT_LOCK_DISABLE 0x0

Interrupt can be enabled, disabled, or de-registered.

#### 17.17.1.11 #define QURT_INT_LOCK_ENABLE 0x1

Interrupt is locked and cannot be enabled, disabled, or deregistered.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 17.18 Interrupt types

### 17.18.1 Define Documentation

#### 17.18.1.1 #define QURT_INT_TRIGGER_TYPE_SET( *pol, edge* ) ((((pol) & 0x01U) << 2) | ((edge) & 0x03U))

Trigger type bit fields for a PDC interrupt:

```
Polarity  Edge  Output
0         00    Level sensitive active low
0         01    Rising edge sensitive
0         10    Falling edge sensitive
0         11    Dual edge sensitive
1         00    Level sensitive active high
1         01    Falling edge sensitive
1         10    Rising edge sensitive
1         11    Dual edge sensitive
```

#### 17.18.1.2 #define QURT_INT_TRIGGER_LEVEL_LOW QURT_INT_TRIGGER_TYPE_SET(0U, 0x00U)

#### 17.18.1.3 #define QURT_INT_TRIGGER_LEVEL_HIGH QURT_INT_TRIGGER_TYPE_SET(1U, 0x00U)

#### 17.18.1.4 #define QURT_INT_TRIGGER_RISING_EDGE QURT_INT_TRIGGER_TYPE_SET(1U, 0x02U)

#### 17.18.1.5 #define QURT_INT_TRIGGER_FALLING_EDGE QURT_INT_TRIGGER_TYPE_SET(0U, 0x02U)

#### 17.18.1.6 #define QURT_INT_TRIGGER_DUAL_EDGE QURT_INT_TRIGGER_TYPE_SET(0U, 0x03U)

#### 17.18.1.7 #define QURT_INT_TRIGGER_USE_DEFAULT 0xffU

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 18 Thread Local Storage

## 18.1 Overview

Threads use thread local storage to allocate global storage which is private to specific threads.

Data items stored in thread local storage can be accessed by any function in a thread (but not by any function outside the thread). As with global storage, the stored data items persist for as long as the thread exists. Destructor functions can be defined that process the stored data items when a thread terminates.

Thread local storage supports the following operations:

- Init TLS – Initialize thread local storage.
- Create key – Create a unique key for accessing a data item in thread local storage.
- Set specific – Store a data item to thread local storage along with the specified key.
- Get specific – Load a data item that is stored in thread local storage with the specified key.
- Delete key – Delete the specified key.

The create key operation returns a value (known as a key) which uniquely specifies a data item within the thread local storage. The key is used to access the data item in subsequent get and set operations. A destructor function for the item can be optionally specified as part of the create key operation.

The set and get specific operations both use key values to specify data items for loading or storing in thread local storage. The data items themselves are always pointers to user data.

The delete key operation deletes the specified key from thread local storage.

**Note:** Deleting a key does not run any destructor function that is associated with it.

Memory used for thread local storage is automatically allocated by the kernel. QuRT's thread local storage service is POSIX-compatible.

Thread local storage keys in QuRT are identified by values of type int.

**Functions**

Thread local storage services are accessed with the following QuRT functions.

- qurt_tls_create_key()
- qurt_tls_delete_key()
- qurt_tls_get_specific()
- qurt_tls_set_specific()

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:49:42 GMT sayanagchul.wan@qc.com

## 18.2 qurt_tls_create_key()

### 18.2.1 Function Documentation

#### 18.2.1.1 int qurt_tls_create_key ( int ∗ *key,* void(∗)(void ∗) *destructor* )

Creates a key to access a thread local storage data item.

Subsequent get and set operations use the key value.

**NOTE** The destructor function performs any clean-up operations required by a thread local storage item when its containing thread is deleted (Section 3.16.1.1). If TLS data is not set for the specified key, the destructor function will not be invoked.

**Parameters**

| | | |
|---|---|---|
| `out` | *key* | Pointer to the newly created thread local storage key value. |
| `in` | *destructor* | Pointer to the key-specific destructor function. Pass NULL to specify that no destructor function is defined for the key. |

**Returns**

QURT_EOK – Key successfully created.
QURT_ETLSAVAIL – No free TLS key available.

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 18.3 qurt_tls_delete_key()

### 18.3.1 Function Documentation

#### 18.3.1.1 int qurt_tls_delete_key ( int *key* )

Deletes the specified key from thread local storage.

**NOTE** Explicitly deleting a key does not execute any destructor function that is associated with the key (Section 18.2.1.1).

**Parameters**

| | | |
|---|---|---|
| in | *key* | Thread local storage key value to delete. |

**Returns**

QURT_EOK – Key successfully deleted.
QURT_ETLSENTRY – Key already free.

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 18.4 qurt_tls_get_specific()

### 18.4.1 Function Documentation

#### 18.4.1.1 void* qurt_tls_get_specific ( int *key* )

Loads the data item from thread local storage.

Returns the data item that is stored in thread local storage with the specified key. The data item is always a pointer to user data.

**Parameters**

| | | |
|---|---|---|
| in | *key* | Thread local storage key value. |

**Returns**

Pointer – Data item indexed by key in thread local storage.
0 (NULL) – Key out of range.

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 18.5 qurt_tls_set_specific()

### 18.5.1 Function Documentation

#### 18.5.1.1 int qurt_tls_set_specific ( int *key*, const void ∗ *value* )

Stores a data item to thread local storage along with the specified key.

**Parameters**

| | | |
|---|---|---|
| in | *key* | Thread local storage key value. |
| in | *value* | Pointer to user data value to store. |

**Returns**

QURT_EOK – Data item successfully stored.
QURT_EINVALID – Invalid key.
QURT_EFAILED – Invoked from a non-thread context.

**Safe API**

Yes

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 19 Exception Handling

## 19.1 Overview

QuRT supports exception handling for software errors and processor-detected hardware exceptions. Exceptions are treated as either fatal or nonfatal, and handled accordingly.

QuRT handles the following types of exceptions:

- Program exceptions (fatal or nonfatal)
- Kernel exceptions
- Imprecise exceptions

**Program exceptions**
Program exceptions are raised by QuRT program code – they include cases such as page faults, misaligned load/store operations, and other Hexagon processor exceptions. They can also be raised explicitly using the QuRT API.

Program exceptions are handled by a thread (Section 3.1) which is registered as the program exception handler.

Program exception handling supports the following operations:

- Wait for exception - Register the current thread as the program exception handler and wait for a program exception.
- Raise nonfatal exception - Raise a nonfatal program exception.
- Raise fatal exception - Raise a fatal system exception.
- Shutdown fatal - Shut down the system due to a fatal exception.
- Register fatal notification - Register a fatal notification handler.
- Enable FP exceptions - Enable floating point exceptions.

Nonfatal program exceptions cause QuRT to take the following actions:

- Save the context of the relevant hardware thread in the task control block (TCB).
- Schedule the registered program exception handler thread (if any), with the error information assigned to the parameters of the wait for exception operation.

Depending on how it is implemented, an program exception handler may handle a nonfatal exception either by reloading the QuRT program (if it has the ability to do so) or by terminating the execution of the QuRT program system.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyunachel.wo@lge.com

**Note:** If no program exception handler is registered, or if the registered handler itself calls raise nonfatal exception, then QuRT raises a kernel exception.

If a thread runs in Supervisor mode, any errors are treated as kernel exceptions.

If multiple program exceptions occurs, all exceptions are forwarded to the program exception handler in the order that the exceptions occur. In order to handle multiple exceptions, the exception handler must make repeated calls to qurt_exception_wait to process the error information from the queued exceptions.

**Fatal program exceptions**

Fatal program exceptions terminate the execution of the QuRT program system without invoking the program exception handler. They are intended for use where the program will handle all the system shutdown operations.

Fatal exceptions are raised by calling the raise fatal exception operation, which masks the Hexagon processor interrupts and stops all the other hardware threads in the Hexagon processor. This operation returns so the program can then perform the necessary program-level shutdown operations (data logging, etc.).

Once the program is ready to shut down the system, it calls the fatal shutdown operation, which performs the following actions:

1. If the raise fatal exception operation was not already called, mask the processor interrupts and stop all the other hardware threads.
2. Save the contexts of all hardware threads.
3. Save the contents of TCM.
4. Save all TLB entries.
5. Flush the caches and update cache flush status.
6. Call the the registered fatal notification handler.
7. Execute an infinite loop in the current hardware thread.

**Kernel exceptions**

Kernel exceptions are raised by the QuRT kernel – they include Supervisor mode exceptions along with page faults and other Hexagon processor exceptions.

Kernel exceptions cause QuRT to terminate the execution of the program system and shut down the system processor, while saving the processor state to assist in post-mortem investigations of the problem that caused the exception.

A kernel exception causes QuRT to perform the following actions:

1. Save the context of the current hardware thread to the kernel error data structure.
2. Save the contexts of all other active hardware threads to their respective TCBs.
3. Stop the other hardware threads.
4. Wait until the other hardware threads stop.
5. Flush the Hexagon processor cache.
6. Mask the Hexagon processor interrupts.
7. Call the registered fatal notification handler.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyongchulwon@lge.com

8. Execute an infinite loop in the current hardware thread.

**Note:** Kernel exceptions do not invoke the program exception handler.

**Imprecise exceptions**

Imprecise exceptions are serious and unrecoverable error conditions which can be raised in either the QuRT kernel or the program code – they include cases such as stores to bad addresses, hardware parity errors, or other imprecise slave error conditions, and also non- maskable interrupt (NMI) exceptions raised from outside the Hexagon processor.

QuRT does not forward imprecise exceptions to the program exception handler. Instead the kernel terminates the execution of the current hardware thread while saving the processor state.

When an imprecise exception occurs, QuRT performs the same procedure used for a kernel exception, except that the thread contexts for all hardware threads are stored in the kernel error data structure.

**Note:** The imprecise exception handler overwrites Hexagon processor register R23. This does not occur with program or kernel exceptions.

**Floating point exceptions**

User programs can selectively enable specific floating point events (inexact, underflow, overflow, divide by zero, and invalid) to generate QuRT program exceptions.

**Functions**

Exception handling services are accessed with the following QuRT functions.

- qurt_exception_enable_fp_exceptions()
- qurt_exception_raise_nonfatal()
- qurt_exception_raise_fatal()
- qurt_enable_floating_point_exception()
- qurt_exception_wait()
- qurt_exception_wait3()
- qurt_exception_wait_ext()
- qurt_assert_error()

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 19.2 qurt_exception_enable_fp_exceptions()

### 19.2.1 Function Documentation

#### 19.2.1.1 static unsigned int qurt_exception_enable_fp_exceptions ( unsigned int *mask* )

Enables the specified floating point exceptions as QuRT program exceptions.

To enable the exceptions, set the corresponding bits in the Hexagon control user status register (USR).

The mask argument specifies a mask value that identifies the individual floating point exceptions to set. The exceptions are represented as defined symbols that map into bits 0 through 31 of the 32-bit flag value. Multiple floating point exceptions are specified by OR'ing together the individual exception symbols.

**NOTE** This function must be called before performing any floating point operations.

**Parameters**

| | | |
|---|---|---|
| in | *mask* | Floating point exception types. Values:<br>• QURT_FP_EXCEPTION_ALL<br>• QURT_FP_EXCEPTION_INEXACT<br>• QURT_FP_EXCEPTION_UNDERFLOW<br>• QURT_FP_EXCEPTION_OVERFLOW<br>• QURT_FP_EXCEPTION_DIVIDE0<br>• QURT_FP_EXCEPTION_INVALID |

**Returns**

Updated contents of the USR.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 19.3 qurt_exception_raise_nonfatal()

### 19.3.1 Function Documentation

#### 19.3.1.1 int qurt_exception_raise_nonfatal ( int *error* )

Raises a nonfatal program exception in the QuRT program system.

For more information on program exceptions, see Section 19.

This operation never returns – the program exception handler is assumed to perform all exception handling before terminating or reloading the QuRT program system.

**NOTE** The C library function abort() calls this operation to indicate software errors.

**Parameters**

| | | |
|---|---|---|
| in | *error* | QuRT error result code (Section 27). |

**Returns**

Integer – Unused.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 19.4 qurt_exception_raise_fatal()

### 19.4.1 Function Documentation

#### 19.4.1.1 void qurt_exception_raise_fatal ( void )

Raises a fatal program exception in the QuRT system.

Fatal program exceptions terminate the execution of the QuRT system without invoking the program exception handler.

For more information on fatal program exceptions, see Section 19.

This operation always returns, so the calling program can perform the necessary shutdown operations (data logging, and so on).

**NOTE** Context switches do not work after this operation has been called.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 19.5 qurt_enable_floating_point_exception()

### 19.5.1 Function Documentation

#### 19.5.1.1 unsigned int qurt_enable_floating_point_exception ( unsigned int *mask* )

Enables the specified floating point exceptions as QuRT program exceptions.

To enable the exceptions, set the corresponding bits in the Hexagon control user status register (USR).

The mask argument specifies a mask value that identifies the individual floating point exceptions to set. The exceptions are represented as defined symbols that map into bits 0 through 31 of the 32-bit flag value. Multiple floating point exceptions are specified by OR'ing together the individual exception symbols.

**NOTE** This function must be called before performing any floating point operations.

**Parameters**

| | | |
|---|---|---|
| in | *mask* | Floating point exception types. Values:<br>• QURT_FP_EXCEPTION_ALL<br>• QURT_FP_EXCEPTION_INEXACT<br>• QURT_FP_EXCEPTION_UNDERFLOW<br>• QURT_FP_EXCEPTION_OVERFLOW<br>• QURT_FP_EXCEPTION_DIVIDE0<br>• QURT_FP_EXCEPTION_INVALID |

**Returns**

Updated contents of the USR.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 19.6 qurt_exception_wait()

### 19.6.1 Function Documentation

#### 19.6.1.1 unsigned int qurt_exception_wait ( unsigned int ∗ *ip*, unsigned int ∗ *sp*, unsigned int ∗ *badva*, unsigned int ∗ *cause* )

Registers the program exception handler. This function assigns the current thread as the QuRT program exception handler and suspends the thread until a program exception occurs.

When a program exception occurs, the thread is awakened with error information assigned to the parameters of this operation.

**NOTE** If no program exception handler is registered, or if the registered handler calls exit, QuRT raises a kernel exception. If a thread runs in Supervisor mode, any errors are treated as kernel exceptions.

**Parameters**

| | | |
|---|---|---|
| `out` | *ip* | Pointer to the instruction memory address where the exception occurred. |
| `out` | *sp* | Stack pointer. |
| `out` | *badva* | Pointer to the virtual data address where the exception occurred. |
| `out` | *cause* | Pointer to the QuRT error result code. |

**Returns**

Registry status:
Thread identifier – Handler successfully registered.
QURT_EFATAL – Registration failed.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 19.7 qurt_exception_wait3()

### 19.7.1 Function Documentation

#### 19.7.1.1 unsigned int qurt_exception_wait3 ( void * *sys_err,* unsigned int *sys_err_size* )

Registers the current thread as the QuRT program exception handler, and suspends the thread until a program exception occurs. When a program exception occurs, the thread is awakened with error information assigned to the specified error event record. If a program exception is raised when no handler is registered (or when a handler is registered, but it calls exit), the exception is treated as fatal.

**NOTE** If a thread runs in Monitor mode, all exceptions are treated as kernel exceptions.

This function differs from qurt_exception_wait() by returning the error information in a data structure rather than as individual variables. It also returns additional information (for example, SSR, FP, and LR).

**Parameters**

| | | |
|---|---|---|
| `out` | *sys_err* | Pointer to the qurt_sysevent_error_1_t type structure. |
| `in` | *sys_err_size* | Size of the qurt_sysevent_error_1_t structure. |

**Returns**

Registry status:

- QURT_EFATAL – Failure.
- Thread ID – Success.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 19.8 qurt_exception_wait_ext()

### 19.8.1 Function Documentation

#### 19.8.1.1 unsigned int qurt_exception_wait_ext ( qurt_sysevent_error_t * *sys_err* )

Registers the current thread as the QuRT program exception handler, and suspends the thread until a program exception occurs. When a program exception occurs, the thread is awakened with error information assigned to the specified error event record. If a program exception is raised when no handler is registered (or when a handler is registered, but it calls exit), the exception is treated as fatal.

**NOTE** If a thread runs in Monitor mode, all exceptions are treated as kernel exceptions.

This function differs from qurt_exception_wait() by returning the error information in a data structure rather than as individual variables. It also returns additional information (for example, SSR, FP, and LR).

**Parameters**

| | | |
|---|---|---|
| `out` | *sys_err* | Pointer to the qurt_sysevent_error_t type structure. |

Registry status:

- QURT_EFATAL – Failure.
- Thread ID – Success.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 19.9 qurt_assert_error()

### 19.9.1 Function Documentation

#### 19.9.1.1 void qurt_assert_error ( const char ∗ *filename,* int *lineno* )

Writes diagnostic information to the debug buffer, and raises an error to the QuRT kernel.

**Associated data types**

None.

**Parameters**

| | | |
|---|---|---|
| in | *filename* | Pointer to the file name string. |
| in | *lineno* | Line number. |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 20 Memory Allocation

## 20.1 Overview

QuRT applications are assigned a default global heap which is accessed by the standard C functions malloc and free (Section 2.1).

Threads use memory allocation to create additional heap-based storage allocators within applications.

Memory allocation supports the following operations:

- Allocate memory – Allocate memory area in heap.
- Allocate array – Allocate an array in heap.
- Reallocate memory – Reallocate existing memory area in heap.
- Deallocate memory – Free allocated memory from heap.

The allocate memory operations are functionally similar to the corresponding operations malloc and calloc that are defined in the C library.

The reallocate memory operation is functionally similar to realloc. It accepts a pointer to an existing memory area on the heap, and resizes the memory area to the specified size while preserving the original contents of the memory area.

The deallocate memory operation is functionally similar to free.

**Note:** Memory allocation cannot allocate memory outside the thread assigned memory area (Section 2.1). This can be done using the QuRT memory management services (Section 21.1).

**Functions**

Memory allocation services are accessed with the following QuRT functions.

- qurt_calloc()
- qurt_free()
- qurt_memalign()
- qurt_malloc()
- qurt_realloc()

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:41:42 GMT hyung-hul.weh@lge.com

## 20.2 qurt_calloc()

### 20.2.1 Function Documentation

#### 20.2.1.1 void* qurt_calloc ( unsigned int *elsize,* unsigned int *num* )

Wrapper around `calloc()` from `stdlib.h`

**Parameters**

| | | |
|---|---|---|
| `in` | *elsize* | Size (in bytes) of each array element. |
| `in` | *num* | Number of array elements. |

**Return values**

| | |
|---|---|
| *non-zero* | Pointer to the allocated memory. |
| *0* | Memory allocation failed. |

**Safe API**

No

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 20.3 qurt_free()

### 20.3.1 Function Documentation

#### 20.3.1.1 void qurt_free ( void * *ptr* )

Wrapper around `free()` from `stdlib.h`

**Parameters**

| | | |
|---|---|---|
| in | *ptr* | Pointer to the address of the memory to deallocate. |

**Safe API**

No

**Dependencies**

The memory item that the ptr value specifies must have been previously allocated using one of the qurt_calloc(), qurt_malloc(), or qurt_realloc() memory allocation functions. Otherwise the behavior of QuRT is undefined.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 20.4 qurt_memalign()

### 20.4.1 Function Documentation

#### 20.4.1.1 void* qurt_memalign ( unsigned int *alignment,* unsigned int *size* )

Wrapper around `memalign()` from `stdlib.h`

**Parameters**

| | | |
|---|---|---|
| `in` | *alignment* | Alignment (in Bytes) requested |
| `in` | *size* | Size of allocation requested |

**Safe API**

No

**Return values**

| | |
|---|---|
| *non-zero* | Pointer to the allocated memory. |
| *0* | Memory allocation failed. |

**Safe API**

No

**Dependencies**

The requested alignment should be a power of 2.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 20.5 qurt_malloc()

### 20.5.1 Function Documentation

#### 20.5.1.1 void* qurt_malloc ( unsigned int *size* )

Wrapper around `malloc()` from `stdlib.h`

**Parameters**

| | | |
|---|---|---|
| `in` | *size* | Size (in bytes) of the memory area. |

**Return values**

| | |
|---|---|
| *non-zero* | Pointer to the allocated memory. |
| *0* | Memory allocation failed. |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 20.6 qurt_realloc()

### 20.6.1 Function Documentation

#### 20.6.1.1 void* qurt_realloc ( void * *ptr,* int *newsize* )

Wrapper around `realloc()` from `stdlib.h`

**Parameters**

| | | |
|---|---|---|
| `in` | *ptr* | Pointer to the address of the memory area. |
| `in` | *newsize* | Size (in bytes) of the reallocated memory area. |

**Return values**

| | |
|---|---|
| *non-zero* | Pointer to reallocated memory area. |
| *0* | Reallocation failed. |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 21 Memory Management

## 21.1 Overview

Threads use memory management to dynamically allocate application memory, share memory with other applications, and manage virtual memory. Memory management is performed with memory pools and regions.

Threads use memory management to perform the following tasks:

- Dynamically allocate memory outside the assigned application memory
- Manage virtual memory

Memory management is performed with memory pools, regions, and mappings, objects which support the following operations:

- Attach pool – Attach a memory pool to a predefined pool.
- Create region – Create a memory region with specified attributes.
- Delete region – Delete the specified region.
- Create mapping – Create a new memory map entry in the page table.
- Remove mapping – Delete the specified memory map entry from the page table.
- Query region – Determine if a region exists for a specified address.
- Query map – Determine if a memory page is statically mapped.
- Lookup physical address – Translate a virtual address to a physical address.
- Cache clean – Perform a cache control operation.

To dynamically allocate memory outside its assigned memory area (Section 2.1), a thread first initializes a memory pool by attaching it to a predefined pool. It then creates one or more memory regions with the pool specified as one of the region attributes. The thread can then access the memory in the newly-allocated memory regions.

**Note:** An application cannot share its original assigned memory with another application – it can only share dynamically-allocated memory regions.

**Memory pools**
Memory pools are used to assign memory regions to different types of physical (not virtual) memory. For example, the Hexagon processor can access SMI, TCM, and EBI memory; therefore, to allocate regions in each of these memories a separate memory pool must be defined for each memory unit (e.g., an SMI pool, TCM pool, etc.). Requests to create new memory regions always specify a memory pool object as a region attribute.

Memory pools are predefined in the system configuration file (Section 2.2), and are specified by their

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyunchol.wo@lge.com

assigned pool name in memory pool attach operations. All memory pools are accessible by all applications in the QuRT application system.

Memory pools must be predefined as part of the system configuration – for more information refer to [80-VB419-79].

QuRT predefines the memory pool object qurt_mem_default_pool which is preattached to the default memory pool in the system configuration file. It is defined to allocate memory regions in SMI memory.

**Memory regions**

Memory regions are used to define memory areas with a fixed set of attributes that specify an area's virtual memory mapping and cache type. A core set of regions is predefined in the system configuration file (Section 2.2), with additional regions created or deleted at runtime to support dynamic memory management.

Memory regions have the following attributes:

- Size – Memory region size (in bytes).
- Pool – Memory pool that the region belongs to.
- Mapping – Memory mapping.
- Physical address – Memory region physical address.
- Virtual address – Memory region virtual address.
- Cache mode – Cache type.
- Type – Memory region type (local/shared).

The pool specifies the memory pool that the region belongs to. Each region must have a corresponding pool.

The mapping indicates how the memory region is mapped in virtual memory (as shown in 21-1):

- Virtual regions can have their virtual address space mapped to noncontiguous areas of physical memory. This makes the most efficient use of virtual memory, and works for most memory use cases.
- Contiguous regions require that the virtual address space be mapped to a contiguous area of physical memory. This is necessary when the memory region is accessed by external devices which bypass Hexagon virtual memory addressing.

The physical address indicates the physical base address of the memory region. It is set only when using physical-contiguous-mapped memory regions.

The virtual address is a read-only attribute which returns the base address of the memory region.

The Cache mode indicates whether the memory region uses the instruction or data cache.

The type indicates whether the memory region is local to an application or shared between applications.

For more information on memory, refer to [80-VB419-92].

**Note:** The memory region size and pool attributes are set directly as parameters in the memory region create operation.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyengcunwo@ige.com

Figure 21-1 shows an example of memory region mappings.

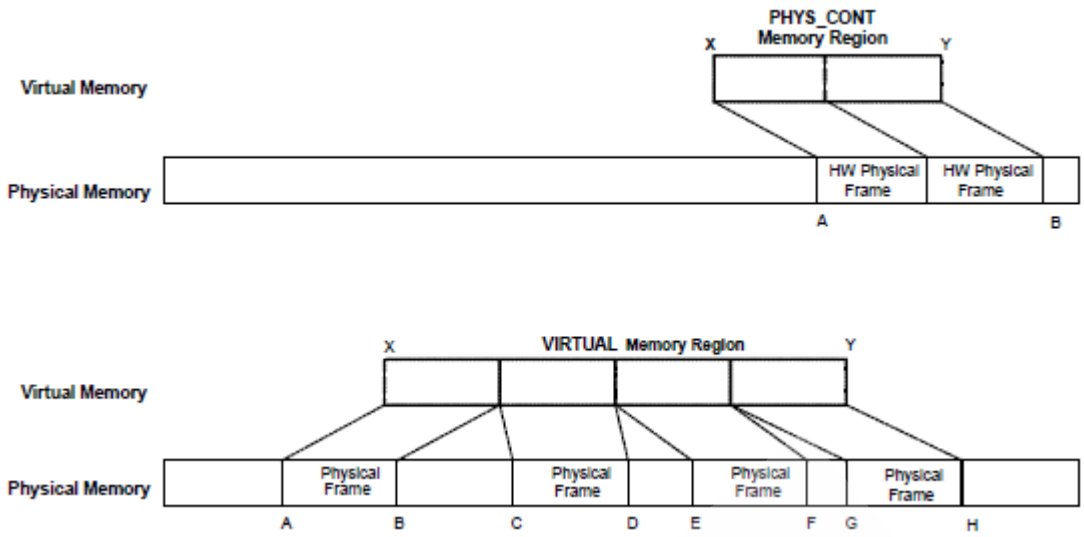


**Figure 21-1 Memory region mappings**

### Setting region attributes

Memory region attributes are set both before a region is created (using the qurt_mem_region_attr_init and qurt_mem_region_attr_set functions) and when a region is created (by directly passing the attributes as arguments to qurt_mem_region_create).

The memory region size and memory region pool are set when a region is created – all the other memory region attributes are set before the create operation.

### Getting region attributes

Memory region attributes can be retrieved from a created memory region using the qurt_mem_region_get_attr and qurt_mem_region_attr_get functions.

The only attribute that cannot be retrieved from a memory region is the memory pool.

### Memory maps

Memory maps specify the mapping between virtual memory and physical memory in the Hexagon processor.

The create mapping and remove mapping operations are used to directly manipulate the memory maps.

The memory map static query operation indicates whether a memory page is statically mapped. If the specified page is statically mapped, the operation returns the page's virtual address. If the page is not statically mapped (or if it does not exist), the operation returns -1 as the virtual address value.

The lookup physical address operation performs virtual to physical address translation. It returns the physical memory address of the specified virtual address.

**Note:** Memory maps operate directly on the page table – therefore, changing the map may affect any memory region defined for the affected memory area.

### Memory cache

The cache clean operation performs the specified operation on the Hexagon memory cache. The operation can specify the data or instruction cache, and supports the following flush options: flush, invalidate, or flush and invalidate.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyung-shul.won@qce.com

**Functions**

Memory management services are accessed with the following QuRT functions.

- qurt_lookup_physaddr()
- qurt_lookup_physaddr2()
- qurt_lookup_physaddr_64()
- qurt_mapping_create()
- qurt_mapping_create_64()
- qurt_mapping_remove()
- qurt_mapping_remove_64()
- qurt_mem_barrier()
- qurt_mem_cache_clean()
- qurt_mem_cache_clean2()
- qurt_mem_cache_phys_clean()
- qurt_mem_configure_cache_partition()
- qurt_mem_l2cache_line_lock()
- qurt_mem_l2cache_line_unlock()
- qurt_mem_map_static_query()
- qurt_mem_map_static_query_64()
- qurt_mem_pool_add_pages()
- qurt_mem_pool_attach()
- qurt_mem_pool_attach2()
- qurt_mem_pool_attr_get()
- qurt_mem_pool_attr_get_addr()
- qurt_mem_pool_attr_get_addr_64()
- qurt_mem_pool_is_available()
- qurt_mem_pool_attr_get_size()
- qurt_mem_pool_create()
- qurt_mem_pool_remove_pages()
- qurt_mem_region_attr_get()
- qurt_mem_region_attr_get_bus_attr()
- qurt_mem_region_attr_set_owner()
- qurt_mem_region_attr_set_perms()
- qurt_mem_region_attr_get_cache_mode()

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

- qurt_mem_region_attr_get_mapping()
- qurt_mem_region_attr_get_physaddr()
- qurt_mem_region_attr_get_size()
- qurt_mem_region_attr_get_type()
- qurt_mem_region_attr_get_virtaddr()
- qurt_mem_region_attr_get_physaddr_64()
- qurt_mem_region_attr_init()
- qurt_mem_region_attr_set_bus_attr()
- qurt_mem_region_attr_set_cache_mode()
- qurt_mem_region_attr_set_mapping()
- qurt_mem_region_attr_set_physaddr()
- qurt_mem_region_attr_set_physaddr_64()
- qurt_mem_region_attr_set_type()
- qurt_mem_region_attr_set_virtaddr()
- qurt_mem_region_create()
- qurt_mem_region_delete()
- qurt_mem_region_query()
- qurt_mem_region_query_64()
- qurt_mem_syncht()
- qurt_mapping_attr_get()
- qurt_mapping_attr_get_cache_mode()
- qurt_mapping_attr_get_physaddr()
- qurt_mapping_attr_get_perms()
- qurt_mapping_attr_get_size()
- qurt_mapping_attr_get_64()
- qurt_physmem_get_addr_width()
- qurt_physmem_encode_page()
- qurt_physmem_decode_page()
- qurt_mem_preempt_virtual_memory()
- qurt_mem_resume_virtual_memory()
- Memory Management Data Types
- Macros

Qualcomm Confidential - May Contain Trade Secrets 2026-02-05:40:42 GMT hyungchul.won@lge.com

## 21.2 qurt_lookup_physaddr()

### 21.2.1 Function Documentation

#### 21.2.1.1 qurt_paddr_t qurt_lookup_physaddr ( qurt_addr_t *vaddr* )

Translates a virtual memory address to the physical memory address to which it maps.

The lookup occurs in the process of the caller. Use qurt_lookup_physaddr2() to lookup the physical address of another process.

**Associated data types**

qurt_addr_t
qurt_paddr_t

**Parameters**

| | | |
|---|---|---|
| in | *vaddr* | Virtual address. |

**Returns**

Nonzero – Physical address to which the virtual address is mapped.
0 – Virtual address not mapped.

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 21.3 qurt_lookup_physaddr2()

### 21.3.1 Function Documentation

#### 21.3.1.1 qurt_paddr_64_t qurt_lookup_physaddr2 ( qurt_addr_t *vaddr,* unsigned int *pid* )

Translates the virtual memory address of the specified process to the 64-bit physical memory address to which it is mapped.

**Associated data types**

qurt_addr_t
qurt_paddr_64_t

**Parameters**

| | | |
|---|---|---|
| in | *vaddr* | Virtual address. |
| in | *pid* | PID. |

**Returns**

Nonzero – 64-bit physical address to which the virtual address is mapped.
0 – Virtual address is not mapped.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 21.4 qurt_lookup_physaddr_64()

### 21.4.1 Function Documentation

#### 21.4.1.1 qurt_paddr_64_t qurt_lookup_physaddr_64 ( qurt_addr_t *vaddr* )

Translates a virtual memory address to the 64-bit physical memory address it is mapped to.

The lookup occurs in the process of the caller. Use qurt_lookup_physaddr2() to lookup the physical address of another process.

**Associated data types**

qurt_paddr_64_t
qurt_addr_t

**Parameters**

| | | |
|---|---|---|
| in | *vaddr* | Virtual address. |

**Returns**

Nonzero – 64-bit physical address to which the virtual address is mapped.
0 – Virtual address has not been mapped.

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 21.5 qurt_mapping_create()

### 21.5.1 Function Documentation

#### 21.5.1.1 int qurt_mapping_create ( qurt_addr_t *vaddr,* qurt_addr_t *paddr,* qurt_size_t *size,* qurt_mem_cache_mode_t *cache_attribs,* qurt_perm_t *perm* )

Creates a memory mapping in the page table. Not supported if called from a user process, always returns QURT_EMEM.

**Associated data types**

qurt_addr_t
qurt_size_t
qurt_mem_cache_mode_t
qurt_perm_t

**Parameters**

| | | |
|---|---|---|
| in | *vaddr* | Virtual address. |
| in | *paddr* | Physical address. |
| in | *size* | Size (4K-aligned) of the mapped memory page. |
| in | *cache_attribs* | Cache mode (writeback, and so on). |
| in | *perm* | Access permissions. |

**Returns**

QURT_EOK – Mapping created.
QURT_EMEM – Failed to create mapping.
QURT_EINVALID – Invalid cache attributes or permissions provided.

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:43 GMT hyungchul.won@lge.com

## 21.6 qurt_mapping_create_64()

### 21.6.1 Function Documentation

#### 21.6.1.1 int qurt_mapping_create_64 ( qurt_addr_t *vaddr,* qurt_paddr_64_t *paddr_64,* qurt_size_t *size,* qurt_mem_cache_mode_t *cache_attribs,* qurt_perm_t *perm* )

Creates a memory mapping in the page table. Not supported if called from a user process, always returns QURT_EMEM.

**Associated data types**

qurt_addr_t
qurt_paddr_64_t
qurt_size_t
qurt_mem_cache_mode_t
qurt_perm_t

**Parameters**

| | | |
|---|---|---|
| in | *vaddr* | Virtual address. |
| in | *paddr_64* | 64-bit physical address. |
| in | *size* | Size (4K-aligned) of the mapped memory page. |
| in | *cache_attribs* | Cache mode (writeback, and so on). |
| in | *perm* | Access permissions. |

**Returns**

QURT_EOK – Success.
QURT_EMEM – Failure.
QURT_EINVALID – Invalid cache attributes or permissions provided.

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@qe.com

## 21.7 qurt_mapping_remove()

### 21.7.1 Function Documentation

#### 21.7.1.1 int qurt_mapping_remove ( qurt_addr_t *vaddr,* qurt_addr_t *paddr,* qurt_size_t *size* )

Deletes the specified memory mapping from the page table.

**Associated data types**

qurt_addr_t
qurt_size_t

**Parameters**

| | | |
|---|---|---|
| in | *vaddr* | Virtual address. |
| in | *paddr* | Physical address. |
| in | *size* | Size of the mapped memory page (4K-aligned). |

**Returns**

QURT_EOK – Mapping created.
QURT_ELOCKED – Buffer is locked, mapping delete failed.

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 21.8 qurt_mapping_remove_64()

### 21.8.1 Function Documentation

#### 21.8.1.1 int qurt_mapping_remove_64 ( qurt_addr_t *vaddr,* qurt_paddr_64_t *paddr_-64,* qurt_size_t *size* )

Deletes the specified memory mapping from the page table.

**Associated data types**

qurt_addr_t
qurt_paddr_64_t
qurt_size_t

**Parameters**

| | | |
|---|---|---|
| in | *vaddr* | Virtual address. |
| in | *paddr_64* | 64-bit physical address. |
| in | *size* | Size of the mapped memory page (4K-aligned). |

**Returns**

QURT_EOK – Success.
QURT_ELOCKED – Buffer is locked. Mapping delete failed.

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 21.9 qurt_mem_barrier()

### 21.9.1 Function Documentation

#### 21.9.1.1 static void qurt_mem_barrier ( void )

Creates a barrier for memory transactions.

This operation ensures that all previous memory transactions are globally observable before any future memory transactions are globally observable.

**NOTE** This operation is implemented as a wrapper for the Hexagon barrier instruction.

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 21.10 qurt_mem_cache_clean()

## 21.10.1 Function Documentation

### 21.10.1.1 int qurt_mem_cache_clean ( qurt_addr_t *addr,* qurt_size_t *size,* qurt_mem_cache_op_t *opcode,* qurt_mem_cache_type_t *type* )

Performs a cache clean operation on the data stored in the specified memory area. Peforms a syncht on all the data cache operations when the Hexagon processor version is v60 or greater.

**NOTE** Perform the flush all operation only on the data cache.

This operation flushes and invalidates the contents of all cache lines from start address to end address (start address + size). The contents of the adjoining buffer can be flushed and invalidated if it falls in any of the cache line.

#### Associated data types

qurt_addr_t
qurt_size_t
qurt_mem_cache_op_t
qurt_mem_cache_type_t

#### Parameters

| | | |
|---|---|---|
| in | *addr* | Address of data to flush. |
| in | *size* | Size (in bytes) of data to flush. |
| in | *opcode* | Type of cache clean operation. Values:<br>• QURT_MEM_CACHE_FLUSH<br>• QURT_MEM_CACHE_INVALIDATE<br>• QURT_MEM_CACHE_FLUSH_INVALIDATE<br>• QURT_MEM_CACHE_FLUSH_ALL<br>**NOTE** QURT_MEM_CACHE_FLUSH_ALL is valid only when the type is QURT_MEM_DCACHE |
| in | *type* | Cache type. Values:<br>• QURT_MEM_ICACHE<br>• QURT_MEM_DCACHE |

#### Returns

QURT_EOK – Cache operation performed successfully.
QURT_EVAL – Invalid cache type.

#### Safe API

Yes

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:42:12 hyungchulwon@qc.com

# 21.11 qurt_mem_cache_clean2()

## 21.11.1 Function Documentation

### 21.11.1.1 int qurt_mem_cache_clean2 ( qurt_addr_t *addr,* qurt_size_t *size,* qurt_mem_cache_op_t *opcode,* qurt_mem_cache_type_t *type* )

Performs a data cache clean operation on the data stored in the specified memory area.

This API only performs the following data cache operations:

- QURT_MEM_CACHE_FLUSH
- QURT_MEM_CACHE_INVALIDATE
- QURT_MEM_CACHE_FLUSH_INVALIDATE – flushes/invalidates the contents of all cache lines from start address to end address (start address + size). The contents of the adjoining buffer can be flushed/invalidated if it falls in any of the cache line.

#### Associated data types

qurt_addr_t
qurt_size_t
qurt_mem_cache_op_t
qurt_mem_cache_type_t

#### Parameters

| | | |
|---|---|---|
| in | *addr* | Address of data to flush. |
| in | *size* | Size (in bytes) of data to flush. |
| in | *opcode* | Type of cache clean operation. Values:<br>QURT_MEM_CACHE_FLUSH<br>QURT_MEM_CACHE_INVALIDATE<br>QURT_MEM_CACHE_FLUSH_INVALIDATE |
| in | *type* | Cache type. Values:<br>QURT_MEM_DCACHE |

#### Returns

QURT_EOK – Cache operation performed successfully.
QURT_EVAL – Invalid cache type.

#### Safe API

Yes

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 hyungchul@qualcomm.com

## 21.12 qurt_mem_cache_phys_clean()

### 21.12.1 Function Documentation

#### 21.12.1.1 int qurt_mem_cache_phys_clean ( unsigned int *mask,* unsigned int *addrmatch,* qurt_mem_cache_op_t *opcode* )

Performs a cache clean operation on the data stored in the specified memory area based on address match and mask. Operate on a cache line when (LINE.PhysicalPageNumber & mask) == addrmatch.

**NOTE** The addrmatch value should be the upper 24-bit physical address to match against.

**Associated data types**

qurt_mem_cache_op_t

**Parameters**

| | | |
|---|---|---|
| in | *mask* | 24-bit address mask. |
| in | *addrmatch* | Physical page number (24 bits) of memory to use as an address match. |
| in | *opcode* | Type of cache clean operation. Values:<br>• QURT_MEM_CACHE_FLUSH<br>• QURT_MEM_CACHE_INVALIDATE |

**Returns**

QURT_EOK – Cache operation performed successfully.
QURT_EVAL – Invalid operation

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 21.13 qurt_mem_configure_cache_partition()

### 21.13.1 Function Documentation

#### 21.13.1.1 int qurt_mem_configure_cache_partition ( qurt_cache_type_t *cache_type,* qurt_cache_partition_size_t *partition_size* )

Configures the Hexagon cache partition at the system level.

A partition size value of SEVEN_EIGHTHS_SIZE is applicable only to the L2 cache. The L1 cache partition is not supported in Hexagon processor version v60 or greater.

**NOTE** Call this operation only with QuRT OS privilege.

**Associated data types**

qurt_cache_type_t
qurt_cache_partition_size_t

**Parameters**

| | | |
|---|---|---|
| in | *cache_type* | Cache type for partition configuration. Values:<br>• HEXAGON_L1_I_CACHE<br>• HEXAGON_L1_D_CACHE<br>• HEXAGON_L2_CACHE |
| in | *partition_size* | Cache partition size. Values:<br>• FULL_SIZE<br>• HALF_SIZE<br>• THREE_QUARTER_SIZE<br>• SEVEN_EIGHTHS_SIZE |

**Returns**

QURT_EOK – Success.
QURT_EVAL – Error.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 21.14 qurt_mem_l2cache_line_lock()

## 21.14.1 Function Documentation

### 21.14.1.1 int qurt_mem_l2cache_line_lock ( qurt_addr_t *addr,* qurt_size_t *size* )

Performs an L2 cache line locking operation. This function locks selective lines in the L2 cache memory.

**NOTE** Perform the line lock operation only on the 32-byte aligned size and address.

#### Associated data types

qurt_addr_t
qurt_size_t

#### Parameters

| | | |
|---|---|---|
| in | *addr* | Address of the L2 cache memory line to lock; the address must be 32-byte aligned. |
| in | *size* | Size (in bytes) of L2 cache memory to line lock; size must be a multiple of 32 bytes. |

#### Returns

QURT_EOK – Success.
QURT_EALIGN – Data alignment or address failure.
QURT_EINVALID – Improper addr and size passed (for example, integer overflow due to addr + size).
QURT_EFAILED – Failed to lock cache line as all the ways are locked for the corresponding set of an address in the range of addr and addr + size or the address range is not L2 cacheable.

#### Safe API

No

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 21.15 qurt_mem_l2cache_line_unlock()

## 21.15.1 Function Documentation

### 21.15.1.1 int qurt_mem_l2cache_line_unlock ( qurt_addr_t *addr,* qurt_size_t *size* )

Performs an L2 cache line unlock operation. This function unlocks selective lines in the L2 cache memory.

**NOTE** Perform the line unlock operation only on a 32-byte aligned size and address.

#### Associated data types

qurt_addr_t
qurt_size_t

#### Parameters

| | | |
|---|---|---|
| in | *addr* | Address of the L2 cache memory line to unlock; the address must be 32-byte aligned. |
| in | *size* | Size (in bytes) of the L2 cache memory line to unlock; size must be a multiple of 32 bytes. |

#### Returns

QURT_EOK – Success.
QURT_EALIGN – Aligning data or address failure.
QURT_EFAILED – Operation failed, cannot find the matching tag.

#### Safe API

No

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 21.16 qurt_mem_map_static_query()

## 21.16.1 Function Documentation

### 21.16.1.1 int qurt_mem_map_static_query ( qurt_addr_t * *vaddr,* qurt_addr_t *paddr,* unsigned int *page_size,* qurt_mem_cache_mode_t *cache_attribs,* qurt_perm_t *perm* )

Determines whether a memory page is statically mapped. Pages are specified by the following attributes: physical address, page size, cache mode, and memory permissions.

- If the specified page is statically mapped, vaddr returns the virtual address of the page.
- If the page is not statically mapped (or if it does not exist as specified), vaddr returns -1 as the virtual address value.
  The system configuration file defines QuRT memory maps.

#### Associated data types

qurt_addr_t
qurt_mem_cache_mode_t
qurt_perm_t

#### Parameters

| | | |
|---|---|---|
| out | *vaddr* | Virtual address that corresponds to paddr. |
| in | *paddr* | Physical address. |
| in | *page_size* | Size of the mapped memory page. |
| in | *cache_attribs* | Cache mode (writeback, and so on). |
| in | *perm* | Access permissions. |

#### Returns

QURT_EOK – Specified page is statically mapped, vaddr returns the virtual address.
QURT_EMEM – Specified page is not statically mapped, vaddr returns -1.
QURT_EVAL – Specified page does not exist.

#### Safe API

No

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.yun@lge.com

## 21.17 qurt_mem_map_static_query_64()

### 21.17.1 Function Documentation

#### 21.17.1.1 int qurt_mem_map_static_query_64 ( qurt_addr_t * *vaddr,* qurt_paddr_64_t *paddr_64,* unsigned int *page_size,* qurt_mem_cache_mode_t *cache_-attribs,* qurt_perm_t *perm* )

Determines if a memory page is statically mapped. The following attributes specify pages: 64-bit physical address, page size, cache mode, and memory permissions.

If the specified page is statically mapped, vaddr returns the virtual address of the page. If the page is not statically mapped (or if it does not exist as specified), vaddr returns -1 as the virtual address value.

QuRT memory maps are defined in the system configuration file.

**Associated data types**

qurt_addr_t
qurt_paddr_64_t
qurt_mem_cache_mode_t
qurt_perm_t

**Parameters**

| | | |
|---|---|---|
| out | *vaddr* | Virtual address that corresponds to paddr. |
| in | *paddr_64* | 64-bit physical address. |
| in | *page_size* | Size of the mapped memory page. |
| in | *cache_attribs* | Cache mode (writeback, and so on). |
| in | *perm* | Access permissions. |

**Returns**

QURT_EOK – Specified page is statically mapped; a virtual address is returned in vaddr.
QURT_EMEM – Specified page is not statically mapped; -1 is returned in vaddr.
QURT_EVAL – Specified page does not exist.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2025-06-27 05:40:42 GMT hyungdum.kim@lge.com

# 21.18 qurt_mem_pool_add_pages()

## 21.18.1 Function Documentation

### 21.18.1.1 int qurt_mem_pool_add_pages ( qurt_mem_pool_t *pool,* unsigned *first_pageno,* unsigned *size_in_pages* )

Adds a physical address range to the specified memory pool object.

**NOTE** Call this operation only with root privileges (guest OS mode).

#### Associated data types

qurt_mem_pool_t

#### Parameters

| | | |
|---|---|---|
| in | *pool* | Memory pool object. |
| in | *first_pageno* | First page number of the physical address range (equivalent to address >> 12) |
| in | *size_in_pages* | Number of pages in the physical address range (equivalent to size >> 12) |

#### Returns

QURT_EOK – Pages successfully added.

#### Safe API

Yes

#### Dependencies

None.

#### Note

When secure PD is spawned through TRUSTED ROOT, API will NOT be supported for secure PD

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 21.19 qurt_mem_pool_attach()

## 21.19.1 Function Documentation

### 21.19.1.1 int qurt_mem_pool_attach ( char ∗ *name,* qurt_mem_pool_t ∗ *pool* )

Initializes a memory pool object to attach to a pool predefined in the system configuration file.

Memory pool objects assign memory regions to physical memory in different Hexagon memory units. They are specified in memory region create operations (Section 21.47.1.1).

**NOTE** QuRT predefines the memory pool object qurt_mem_default_pool (Section 21) for allocation memory regions in SMI memory. The pool attach operation is necessary only when allocating memory regions in nonstandard memory units such as TCM.

#### Associated data types

qurt_mem_pool_t

#### Parameters

| | | |
|---|---|---|
| in | *name* | Pointer to the memory pool name. |
| out | *pool* | Pointer to the memory pool object. |

#### Returns

QURT_EOK – Attach operation successful.

#### Safe API

Yes

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 21.20 qurt_mem_pool_attach2()

## 21.20.1 Function Documentation

### 21.20.1.1 int qurt_mem_pool_attach2 ( int *client_handle,* char ∗ *name,* qurt_mem_-pool_t ∗ *pool* )

Gets the identifier that corresponds to a pool object created specifically for a client, for example, HLOS_PHYSPOOL. The client_handle is used to look up the client specific pool.

Memory pool objects assign memory regions to physical memory in different Hexagon memory units. Memory pool objects are specified during mapping creation operations (qurt_mem_mmap() and qurt_mem_region_create()).

**NOTE** QuRT predefines the memory pool object qurt_mem_default_pool (Section 21) for allocation memory regions in SMI memory. The pool_attach2 operation is necessary only when allocating memory regions in memory units specific to the client.

#### Associated data types

qurt_mem_pool_t

#### Parameters

| | | |
|---|---|---|
| in | *client_handle* | Client identifier used by the OS to lookup the identifier for client specific pool |
| in | *name* | Pointer to the memory pool name. |
| out | *pool* | Pointer to the memory pool object. |

#### Returns

QURT_EOK – Attach operation successful.

#### Safe API

No

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 21.21 qurt_mem_pool_attr_get()

## 21.21.1 Function Documentation

### 21.21.1.1 int qurt_mem_pool_attr_get ( qurt_mem_pool_t *pool,* qurt_mem_pool_attr_t ∗ *attr* )

Gets the memory pool attributes.

Retrieves pool configurations based on the pool handle, and fills in the attribute structure with configuration values.

#### Associated data types

qurt_mem_pool_t
qurt_mem_pool_attr_t

#### Parameters

| | | |
|---|---|---|
| in | *pool* | Pool handle obtained from qurt_mem_pool_attach(). |
| out | *attr* | Pointer to the memory region attribute structure. |

#### Returns

0 – Success.
QURT_EINVALID – Corrupt handle; pool handle is invalid.

#### Safe API

No

#### Note

When secure PD is spawned through TRUSTED ROOT, API will NOT be supported for secure PD

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 21.22 qurt_mem_pool_attr_get_addr()

## 21.22.1 Function Documentation

### 21.22.1.1 static int qurt_mem_pool_attr_get_addr ( qurt_mem_pool_attr_t ∗ *attr,* int *range_id,* qurt_addr_t ∗ *addr* )

Gets the start address of the specified memory pool range.

#### Associated data types

qurt_mem_pool_attr_t
qurt_addr_t

#### Parameters

| | | |
|---|---|---|
| in | *attr* | Pointer to the memory pool attribute structure. |
| in | *range_id* | Memory pool range key. |
| out | *addr* | Pointer to the destination variable for range start address. |

#### Returns

0 – Success.
QURT_EINVALID – Range is invalid.

#### Safe API

No

#### Dependencies

None.

#### Note

When secure PD is spawned through TRUSTED ROOT, API will NOT be supported for secure PD

Qualcomm Confidential - May Contain Trade Secrets 2025-02-27 05:40:42 GMT hyungchul.won@lge.com

# 21.23 qurt_mem_pool_attr_get_addr_64()

## 21.23.1 Function Documentation

### 21.23.1.1 static int qurt_mem_pool_attr_get_addr_64 ( qurt_mem_pool_attr_t ∗ *attr,* int *range_id,* qurt_addr_64_t ∗ *addr* )

Gets the 64 bit start address of the specified memory pool range.

#### Associated data types

qurt_mem_pool_attr_t
qurt_addr_64_t

#### Parameters

| | | |
|---|---|---|
| in | *attr* | Pointer to the memory pool attribute structure. |
| in | *range_id* | Memory pool range key. |
| out | *addr* | Pointer to the destination variable for range start address. |

#### Returns

0 – Success.
QURT_EINVALID – Range is invalid.

#### Safe API

No

#### Dependencies

None.

#### Note

When secure PD is spawned through TRUSTED ROOT, API will NOT be supported for secure PD

Qualcomm Confidential - May Contain Trade Secrets 2025-02-27 05:40:42 GMT hyungchul.won@lge.com

# 21.24 qurt_mem_pool_is_available()

## 21.24.1 Function Documentation

### 21.24.1.1 int qurt_mem_pool_is_available ( qurt_mem_pool_t *pool,* int *page_count,* qurt_mem_mapping_t *mapping_type* )

Checks whether the number of pages that the page_count argument indicates can be allocated from the specified pool.

#### Associated data types

qurt_mem_pool_attr_t
qurt_mem_mapping_t

#### Parameters

| | | |
|---|---|---|
| in | *pool* | Pool handle obtained from qurt_mem_pool_attach(). |
| in | *page_count* | Number of 4K pages. |
| in | *mapping_type* | Variable of type qurt_mem_mapping_t. |

#### Returns

0 – Success.
QURT_EINVALID – Mapping_type is invalid.
QURT_EMEM – Specified pages cannot be allocated from the pool.

#### Safe API

No

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 21.25 qurt_mem_pool_attr_get_size()

## 21.25.1 Function Documentation

### 21.25.1.1 static int qurt_mem_pool_attr_get_size ( qurt_mem_pool_attr_t ∗ *attr,* int *range_id,* qurt_size_t ∗ *size* )

Gets the size of the specified memory pool range.

#### Associated data types

qurt_mem_pool_attr_t
qurt_size_t

#### Parameters

| | | |
|---|---|---|
| in | *attr* | Pointer to the memory pool attribute structure. |
| in | *range_id* | Memory pool range key. |
| out | *size* | Pointer to the destination variable for the range size. |

#### Returns

0 – Success.
QURT_EINVALID – Range is invalid.

#### Safe API

No

#### Dependencies

None.

#### Note

When secure PD is spawned through TRUSTED ROOT, API will NOT be supported for secure PD

Qualcomm Confidential - May Contain Trade Secrets 2025-02-27 05:40:42 GMT hyungchul.won@lge.com

# 21.26 qurt_mem_pool_create()

## 21.26.1 Function Documentation

### 21.26.1.1 int qurt_mem_pool_create ( char ∗ *name,* unsigned *base,* unsigned *size,* qurt_mem_pool_t ∗ *pool* )

Dynamically creates a memory pool object from a physical address range.

The pool is assigned a single memory region with the specified base address and size.

The base address and size values passed to this function must be aligned to 4K byte boundaries, and must be expressed as the actual base address and size values divided by 4K.

For example, the function call:

```
qurt_mem_pool_create ("TCM_PHYSPOOL", 0xd8020, 0x20, &pool)
```

... is equivalent to the following static pool definition in the QuRT system configuration file:

```
<physical_pool name="TCM_PHYSPOOL">
    <region base="0xd8020000" size="0x20000" />
</physical_pool>
```

For more information on the system configuration file, see 80-VB419-79.

**NOTE** Dynamically created pools are not identical to static pools. In particular, qurt_mem_pool_attr_get() is not valid with dynamically created pools.

Dynamic pool creation permanently consumes system resources, and cannot be undone.

#### Associated data types

qurt_mem_pool_t

#### Parameters

| | | |
|---|---|---|
| in | *name* | Pointer to the memory pool name. |
| in | *base* | Base address of the memory region (divided by 4K). |
| in | *size* | Size (in bytes) of the memory region (divided by 4K). |
| out | *pool* | Pointer to the memory pool object. |

#### Returns

QURT_EOK – Success.

#### Safe API

No

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.yun@qualcomm.com

# 21.27 qurt_mem_pool_remove_pages()

## 21.27.1 Function Documentation

### 21.27.1.1 int qurt_mem_pool_remove_pages ( qurt_mem_pool_t *pool,* unsigned *first_pageno,* unsigned *size_in_pages,* unsigned *flags,* void(∗)(void ∗arg_cb) *callback,* void ∗ *args* )

Removes a physical address range from the specified memory pool object.

If any part of the address range is in use, this operation returns an error without a change to the state.

**NOTE** Call this operation only with root privileges (guest-OS mode).

In the future, this operation will support (via the flags parameter) the removal of a physical address range when part of the range is in use.

#### Associated data types

qurt_mem_pool_t

#### Parameters

| | | |
|---|---|---|
| in | *pool* | Memory pool object. |
| in | *first_pageno* | First page number of the physical address range (equivalent to address >> 12). |
| in | *size_in_pages* | Number of pages in the physical address range (equivalent to size >> 12). |
| in | *flags* | Remove options. Values:<br>• 0: Skip holes in the range that are not part of the pool (default)<br>• QURT_POOL_REMOVE_ALL_OR_NONE: Pages are removed only when the specified physical address range is entirely contained (with no holes) in the pool free space. |
| in | *callback* | Callback procedure called when pages are successfully removed. Not called when the operation failed. The callback is not called when 0 is passed as the parameter. |
| in | *args* | Value passed as an argument to the callback procedure. |

#### Returns

QURT_EOK – Pages successfully removed.

#### Safe API

Yes

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:47:52 GMT hyungchul.c@qnet.com

# 21.28 qurt_mem_region_attr_get()

## 21.28.1 Function Documentation

### 21.28.1.1 int qurt_mem_region_attr_get ( qurt_mem_region_t *region,* qurt_mem_region_attr_t ∗ *attr* )

Gets the memory attributes of the specified message region. After a memory region is created, its attributes cannot be changed.

#### Associated data types

qurt_mem_region_t
qurt_mem_region_attr_t

#### Parameters

| | | |
|---|---|---|
| in | *region* | Memory region object. |
| out | *attr* | Pointer to the destination structure for memory region attributes. |

#### Returns

QURT_EOK – Operation successfully performed.
Error code – Failure.

#### Safe API

No

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 21.29 qurt_mem_region_attr_get_bus_attr()

## 21.29.1 Function Documentation

### 21.29.1.1 static void qurt_mem_region_attr_get_bus_attr ( qurt_mem_region_attr_t * *attr,* unsigned * *pbits* )

Gets the (A1, A0) bus attribute bits from the specified memory region attribute structure.

#### Associated data types

qurt_mem_region_attr_t

#### Parameters

| | | |
|---|---|---|
| in | *attr* | Pointer to the memory region attribute structure. |
| out | *pbits* | Pointer to an unsigned integer that is filled in with the (A1, A0) bits from the memory region attribute structure, expressed as a 2-bit binary number. |

#### Safe API

No

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 21.30 qurt_mem_region_attr_set_owner()

## 21.30.1 Function Documentation

### 21.30.1.1 void qurt_mem_region_attr_set_owner ( qurt_mem_region_attr_t ∗ *attr,* int *handle* )

Sets the space id of the process represented by client handle, in the specified memory region attribute structure.

#### Associated data types

qurt_mem_region_attr_t

#### Parameters

| | | |
|---|---|---|
| in,out | *attr* | Pointer to memory region attribute structure. |
| in | *handle* | Client handle of the process. |

#### Safe API

No

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 21.31 qurt_mem_region_attr_set_perms()

## 21.31.1 Function Documentation

### 21.31.1.1 void qurt_mem_region_attr_set_perms ( qurt_mem_region_attr_t ∗ *attr,* unsigned *perms* )

Sets the permission for the memory region to be created, in the specified memory region attribute structure.

#### Associated data types

qurt_mem_region_attr_t

#### Parameters

| | | |
|---|---|---|
| in,out | *attr* | Pointer to memory region attribute structure. |
| in | *perms* | Permission required for the memory region to be created. |

#### Safe API

No

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 21.32 qurt_mem_region_attr_get_cache_mode()

## 21.32.1 Function Documentation

### 21.32.1.1 static void qurt_mem_region_attr_get_cache_mode ( qurt_mem_region_-attr_t ∗ *attr,* qurt_mem_cache_mode_t ∗ *mode* )

Gets the cache operation mode from the specified memory region attribute structure.

#### Associated data types

qurt_mem_region_attr_t
qurt_mem_cache_mode_t

#### Parameters

| | | |
|---|---|---|
| in | *attr* | Pointer to the memory region attribute structure. |
| out | *mode* | Pointer to the destination variable for cache mode. |

#### Safe API

No

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 21.33 qurt_mem_region_attr_get_mapping()

## 21.33.1 Function Documentation

### 21.33.1.1 static void qurt_mem_region_attr_get_mapping ( qurt_mem_region_attr_t * *attr,* qurt_mem_mapping_t * *mapping* )

Gets the memory mapping from the specified memory region attribute structure.

#### Associated data types

qurt_mem_region_attr_t
qurt_mem_mapping_t

#### Parameters

| | | |
|---|---|---|
| in | *attr* | Pointer to the memory region attribute structure. |
| out | *mapping* | Pointer to the destination variable for memory mapping. |

#### Safe API

No

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 21.34 qurt_mem_region_attr_get_physaddr()

## 21.34.1 Function Documentation

### 21.34.1.1 static void qurt_mem_region_attr_get_physaddr ( qurt_mem_region_attr_t * *attr,* unsigned int * *addr* )

Gets the memory region physical address from the specified memory region attribute structure.

#### Associated data types

qurt_mem_region_attr_t

#### Parameters

| | | |
|---|---|---|
| in | *attr* | Pointer to the memory region attribute structure. |
| out | *addr* | Pointer to the destination variable for memory region physical address. |

#### Safe API

No

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 21.35 qurt_mem_region_attr_get_size()

### 21.35.1 Function Documentation

#### 21.35.1.1 static void qurt_mem_region_attr_get_size ( qurt_mem_region_attr_t ∗ *attr,* qurt_size_t ∗ *size* )

Gets the memory region size from the specified memory region attribute structure.

**Associated data types**

qurt_mem_region_attr_t
qurt_size_t

**Parameters**

| | | |
|---|---|---|
| in | *attr* | Pointer to the memory region attribute structure. |
| out | *size* | Pointer to the destination variable for the memory region size. |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 21.36 qurt_mem_region_attr_get_type()

## 21.36.1 Function Documentation

### 21.36.1.1 static void qurt_mem_region_attr_get_type ( qurt_mem_region_attr_t ∗ *attr,* qurt_mem_region_type_t ∗ *type* )

Gets the memory type from the specified memory region attribute structure.

#### Associated data types

qurt_mem_region_attr_t
qurt_mem_region_type_t

#### Parameters

| | | |
|---|---|---|
| in | *attr* | Pointer to the memory region attribute structure. |
| out | *type* | Pointer to the destination variable for the memory type. |

#### Safe API

No

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 21.37 qurt_mem_region_attr_get_virtaddr()

## 21.37.1 Function Documentation

### 21.37.1.1 static void qurt_mem_region_attr_get_virtaddr ( qurt_mem_region_attr_t * *attr,* unsigned int * *addr* )

Gets the memory region virtual address from the specified memory region attribute structure.

#### Associated data types

qurt_mem_region_attr_t

#### Parameters

| | | |
|---|---|---|
| in | *attr* | Pointer to the memory region attribute structure. |
| out | *addr* | Pointer to the destination variable for the memory region virtual address. |

#### Safe API

No

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 21.38 qurt_mem_region_attr_get_physaddr_64()

## 21.38.1 Function Documentation

### 21.38.1.1 static void qurt_mem_region_attr_get_physaddr_64 ( qurt_mem_region_-attr_t ∗ *attr,* qurt_paddr_64_t ∗ *addr_64* )

Gets the memory region 64-bit physical address from the specified memory region attribute structure.

#### Associated data types

qurt_mem_region_attr_t
qurt_paddr_64_t

#### Parameters

| | | |
|---|---|---|
| in | *attr* | Pointer to the memory region attribute structure. |
| out | *addr_64* | Pointer to the destination variable for the memory region 64-bit physical address. |

#### Safe API

No

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 21.39 qurt_mem_region_attr_init()

## 21.39.1 Function Documentation

### 21.39.1.1 void qurt_mem_region_attr_init ( qurt_mem_region_attr_t ∗ *attr* )

Initializes the specified memory region attribute structure with default attribute values:

- Mapping: QURT_MEM_MAPPING_VIRTUAL
- Cache mode: QURT_MEM_CACHE_WRITEBACK
- Physical address: -1
- Virtual address: -1
- Memory type: QURT_MEM_REGION_LOCAL
- Size: -1

**NOTE** The memory physical address attribute must be explicitly set by calling the qurt_mem_region_attr_set_physaddr() function. The size and pool attributes are set directly as parameters in the memory region create operation.

#### Associated data types

qurt_mem_region_attr_t

#### Parameters

| | | |
|---|---|---|
| in,out | *attr* | Pointer to the destination structure for the memory region attributes. |

#### Safe API

No

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 21.40 qurt_mem_region_attr_set_bus_attr()

## 21.40.1 Function Documentation

### 21.40.1.1 static void qurt_mem_region_attr_set_bus_attr ( qurt_mem_region_attr_t * *attr,* unsigned *abits* )

Sets the (A1, A0) bus attribute bits in the specified memory region attribute structure.

For more information on the bus attribute bits, see the 80-VB419-92.

#### Associated data types

qurt_mem_region_attr_t

#### Parameters

| | | |
|---|---|---|
| in,out | *attr* | Pointer to the memory region attribute structure. |
| in | *abits* | The (A1, A0) bits to use with the memory region, expressed as a 2-bit binary number. |

#### Safe API

No

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 21.41 qurt_mem_region_attr_set_cache_mode()

## 21.41.1 Function Documentation

### 21.41.1.1 static void qurt_mem_region_attr_set_cache_mode ( qurt_mem_region_attr_t * *attr,* qurt_mem_cache_mode_t *mode* )

Sets the cache operation mode in the specified memory region attribute structure.

For more information on the cache, see 80-VB419-92.

#### Associated data types

qurt_mem_region_attr_t
qurt_mem_cache_mode_t

#### Parameters

| | | |
|---|---|---|
| in,out | *attr* | Pointer to the memory region attribute structure. |
| in | *mode* | Cache mode. Values:<br>• QURT_MEM_CACHE_WRITEBACK<br>• QURT_MEM_CACHE_WRITETHROUGH<br>• QURT_MEM_CACHE_WRITEBACK_NONL2CACHEABLE<br>• QURT_MEM_CACHE_WRITETHROUGH_NONL2CACHEABLE<br>• QURT_MEM_CACHE_WRITEBACK_L2CACHEABLE<br>• QURT_MEM_CACHE_WRITETHROUGH_L2CACHEABLE<br>• QURT_MEM_CACHE_NONE |

#### Safe API

No

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 21.42 qurt_mem_region_attr_set_mapping()

## 21.42.1 Function Documentation

### 21.42.1.1 static void qurt_mem_region_attr_set_mapping ( qurt_mem_region_attr_t * *attr,* qurt_mem_mapping_t *mapping* )

Sets the memory mapping in the specified memory region attribute structure.

The mapping value indicates how the memory region is mapped in virtual memory.

#### Associated data types

qurt_mem_region_attr_t
qurt_mem_mapping_t

#### Parameters

| | | |
|---|---|---|
| in,out | *attr* | Pointer to the memory region attribute structure. |
| in | *mapping* | Mapping. Values:<br>• QURT_MEM_MAPPING_VIRTUAL<br>• QURT_MEM_MAPPING_PHYS_CONTIGUOUS<br>• QURT_MEM_MAPPING_IDEMPOTENT<br>• QURT_MEM_MAPPING_VIRTUAL_FIXED<br>• QURT_MEM_MAPPING_NONE<br>• QURT_MEM_MAPPING_VIRTUAL_RANDOM<br>• QURT_MEM_MAPPING_INVALID |

#### Safe API

No

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 21.43 qurt_mem_region_attr_set_physaddr()

## 21.43.1 Function Documentation

### 21.43.1.1 static void qurt_mem_region_attr_set_physaddr ( qurt_mem_region_attr_t * *attr,* qurt_paddr_t *addr* )

Sets the memory region 32-bit physical address in the specified memory attribute structure.

**NOTE** The physical address attribute is explicitly set only for memory regions with physical contiguous mapping. Otherwise QuRT automatically sets it when the memory region is created.

#### Associated data types

qurt_mem_region_attr_t
qurt_paddr_t

#### Parameters

| | | |
|---|---|---|
| in,out | *attr* | Pointer to the memory region attribute structure. |
| in | *addr* | Memory region physical address. |

#### Safe API

No

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 21.44 qurt_mem_region_attr_set_physaddr_64()

## 21.44.1 Function Documentation

### 21.44.1.1 static void qurt_mem_region_attr_set_physaddr_64 ( qurt_mem_region_-attr_t ∗ *attr,* qurt_paddr_64_t *addr_64* )

Sets the memory region 64-bit physical address in the specified memory attribute structure.

**NOTE** The physical address attribute is explicitly set only for memory regions with physical contiguous mapping. Otherwise it is automatically set by QuRT when the memory region is created.

#### Associated data types

qurt_mem_region_attr_t
qurt_paddr_64_t

#### Parameters

| | | |
|---|---|---|
| in,out | *attr* | Pointer to the memory region attribute structure. |
| in | *addr_64* | Memory region 64-bit physical address. |

#### Safe API

No

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 21.45 qurt_mem_region_attr_set_type()

## 21.45.1 Function Documentation

### 21.45.1.1 static void qurt_mem_region_attr_set_type ( qurt_mem_region_attr_t ∗ *attr,* qurt_mem_region_type_t *type* )

Sets the memory type in the specified memory region attribute structure.

The type indicates whether the memory region is local to an application or shared between applications. For more information, see 80-VB419-92.

#### Associated data types

qurt_mem_region_attr_t
qurt_mem_region_type_t

#### Parameters

| | | |
|---|---|---|
| in,out | *attr* | Pointer to memory region attribute structure. |
| in | *type* | Memory type. Values:<br>• QURT_MEM_REGION_LOCAL<br>• QURT_MEM_REGION_SHARED |

#### Safe API

No

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 21.46 qurt_mem_region_attr_set_virtaddr()

## 21.46.1 Function Documentation

### 21.46.1.1 static void qurt_mem_region_attr_set_virtaddr ( qurt_mem_region_attr_t * *attr,* qurt_addr_t *addr* )

Sets the memory region virtual address in the specified memory attribute structure.

#### Associated data types

qurt_mem_region_attr_t
qurt_addr_t

#### Parameters

| | | |
|---|---|---|
| in,out | *attr* | Pointer to the memory region attribute structure. |
| in | *addr* | Memory region virtual address. |

#### Safe API

No

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 21.47 qurt_mem_region_create()

## 21.47.1 Function Documentation

### 21.47.1.1 int qurt_mem_region_create ( qurt_mem_region_t ∗ *region,* qurt_size_t *size,* qurt_mem_pool_t *pool,* qurt_mem_region_attr_t ∗ *attr* )

Creates a memory region with the specified attributes.

The application initializes the memory region attribute structure with qurt_mem_region_attr_init() and qurt_mem_region_attr_set_bus_attr().

If the virtual address attribute is set to its default value (Section 21.39.1.1), the virtual address of the memory region is automatically assigned any available virtual address value.

If the memory mapping attribute is set to virtual mapping, the physical address of the memory region is also automatically assigned.

**NOTE** The physical address attribute is explicitly set in the attribute structure only for memory regions with physical-contiguous-mapped mapping.

Memory regions are always assigned to memory pools. The pool value specifies the memory pool that the memory region is assigned to.

**NOTE** If attr is specified as NULL, the memory region is created with default attribute values (Section 21.39.1.1). QuRT predefines the memory pool object qurt_mem_default_pool (Section 21), which allocates memory regions in SMI memory.

#### Associated data types

qurt_mem_region_t
qurt_size_t
qurt_mem_pool_t
qurt_mem_region_attr_t

#### Parameters

| | | |
|---|---|---|
| out | *region* | Pointer to the memory region object. |
| in | *size* | Memory region size (in bytes). If size is not an integral multiple of 4K, it is rounded up to a 4K boundary. |
| in | *pool* | Memory pool of the region. |
| in | *attr* | Pointer to the memory region attribute structure. |

#### Returns

QURT_EOK – Memory region successfully created.
QURT_EMEM – Not enough memory to create the region.
QURT_EINVALID – Invalid cache attributes or permissions provided in the attribute.

#### Safe API

No

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

### Dependencies

None.

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:40:42 GMT
hyungchul.won@lge.com

# 21.48 qurt_mem_region_delete()

## 21.48.1 Function Documentation

### 21.48.1.1 int qurt_mem_region_delete ( qurt_mem_region_t *region* )

Deletes the specified memory region.

If the caller application creates the memory region, it is removed and the system reclaims its assigned memory.

If a different application creates the memory region (and is shared with the caller application), only the local memory mapping to the region is removed; the system does not reclaim the memory.

#### Associated data types

qurt_mem_region_t

#### Parameters

| | | |
|---|---|---|
| in | *region* | Memory region object. |

#### Returns

QURT_EOK – Region successfully deleted.
QURT_ELOCKED – Buffer is locked, mapping delete failed.

#### Safe API

No

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 21.49 qurt_mem_region_query()

## 21.49.1 Function Documentation

### 21.49.1.1 int qurt_mem_region_query ( qurt_mem_region_t * *region_handle,* qurt_addr_t *vaddr,* qurt_paddr_t *paddr* )

Queries a memory region.

This function determines whether a dynamically-created memory region (Section 21.47.1.1) exists for the specified virtual or physical address. When a memory region has been determined to exist, its attributes are accessible (Section 21.28.1.1).

**NOTE** This function returns QURT_EFATAL if QURT_EINVALID is passed to both vaddr and paddr (or to neither).

#### Associated data types

qurt_mem_region_t
qurt_paddr_t

#### Parameters

| | | |
|---|---|---|
| `out` | *region_handle* | Pointer to the memory region object (if it exists). |
| `in` | *vaddr* | Virtual address to query; if vaddr is specified, paddr must be set to the value QURT_EINVALID. |
| `in` | *paddr* | Physical address to query; if paddr is specified, vaddr must be set to the value QURT_EINVALID. |

#### Returns

QURT_EOK – Query successfully performed.
QURT_EMEM – Region not found for the specified address.
QURT_EFATAL – Invalid input parameters.

#### Safe API

No

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:46:42 GMT hyungchul.won@me.com

# 21.50 qurt_mem_region_query_64()

## 21.50.1 Function Documentation

### 21.50.1.1 int qurt_mem_region_query_64 ( qurt_mem_region_t * *region_handle,* qurt_addr_t *vaddr,* qurt_paddr_64_t *paddr_64* )

Determines whether a dynamically created memory region (Section 21.47.1.1) exists for the specified virtual or physical address. When a memory region has been determined to exist, its attributes are accessible (Section 21.28.1.1).

**NOTE** This function returns QURT_EFATAL if QURT_EINVALID is passed to both vaddr and paddr (or to neither).

#### Associated data types

qurt_mem_region_t
qurt_addr_t
qurt_paddr_64_t

#### Parameters

| | | |
|---|---|---|
| out | *region_handle* | Pointer to the memory region object (if it exists). |
| in | *vaddr* | Virtual address to query; if vaddr is specified, paddr must be set to the value QURT_EINVALID. |
| in | *paddr_64* | 64-bit physical address to query; if paddr is specified, vaddr must be set to the value QURT_EINVALID. |

#### Returns

QURT_EOK – Success.
QURT_EMEM – Region not found for the specified address.
QURT_EFATAL – Invalid input parameters.

#### Safe API

No

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 21.51 qurt_mem_syncht()

## 21.51.1 Function Documentation

### 21.51.1.1 static void qurt_mem_syncht ( void )

Performs heavy-weight synchronization of memory transactions.

This operation does not return until all previous memory transactions (cached and uncached load/store, mem_locked, and so on) that originated from the current thread are complete and globally observable.

**NOTE** This operation is implemented as a wrapper for the Hexagon syncht instruction.

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 21.52 qurt_mapping_attr_get()

## 21.52.1 Function Documentation

### 21.52.1.1 int qurt_mapping_attr_get ( qurt_addr_t *vaddr,* unsigned int *pid,* qurt_-mapping_attr_t ∗ *attr* )

Gets the mapping attributes for a specified virtual address and PID.

#### Associated data types

qurt_addr_t
qurt_mapping_attr_t

#### Parameters

| | | |
|---|---|---|
| in | *vaddr* | Virtual address for which the attributes are required. |
| in | *pid* | process id for the target process |
| out | *attr* | Pointer to the mapping attribute structure. |

#### Returns

0 – Success.
QURT_EINVALID – Incorrect virtual address or pid

#### Safe API

No

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 21.53 qurt_mapping_attr_get_cache_mode()

## 21.53.1 Function Documentation

### 21.53.1.1 static void qurt_mapping_attr_get_cache_mode ( qurt_mapping_attr_t * *attr,* qurt_mem_cache_mode_t * *cache_mode* )

Gets the cache operation mode in the specified memory mapping attribute structure.

#### Associated data types

qurt_mapping_attr_t
qurt_mem_cache_mode_t

#### Parameters

| | | |
|---|---|---|
| in | *attr* | Pointer to the memory mapping attribute structure. |
| out | *cache_mode* | Pointer to the destination variable for cache mode. |

#### Safe API

No

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 21.54 qurt_mapping_attr_get_physaddr()

### 21.54.1 Function Documentation

#### 21.54.1.1 static void qurt_mapping_attr_get_physaddr ( qurt_mapping_attr_t ∗ *attr,* qurt_paddr_64_t ∗ *physaddr* )

Gets the physical memory address in the specified memory mapping attribute structure.

**Associated data types**

qurt_mapping_attr_t
qurt_paddr_64_t

**Parameters**

| | | |
|---|---|---|
| in | *attr* | Pointer to the memory mapping attribute structure. |
| out | *physaddr* | Pointer to the destination variable for physical address. |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 21.55 qurt_mapping_attr_get_perms()

## 21.55.1 Function Documentation

### 21.55.1.1 static void qurt_mapping_attr_get_perms ( qurt_mapping_attr_t ∗ *attr,* qurt_perm_t ∗ *perms* )

Gets the permissions in the specified memory mapping attribute structure.

#### Associated data types

qurt_mapping_attr_t
qurt_perm_t

#### Parameters

| | | |
|---|---|---|
| in | *attr* | Pointer to the memory mapping attribute structure. |
| out | *perms* | Pointer to the destination variable for permissions. |

#### Safe API

No

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 21.56 qurt_mapping_attr_get_size()

## 21.56.1 Function Documentation

### 21.56.1.1 static void qurt_mapping_attr_get_size ( qurt_mapping_attr_t ∗ *attr,* unsigned int ∗ *size* )

Gets the size in the specified memory mapping attribute structure; represents the size of the TLB entry that covers the virtual address.

#### Associated data types

qurt_mapping_attr_t

#### Parameters

| | | |
|---|---|---|
| in | *attr* | Pointer to the memory mapping attribute structure. |
| out | *size* | Pointer to the destination variable for size. |

#### Safe API

No

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 21.57 qurt_mapping_attr_get_64()

## 21.57.1 Function Documentation

### 21.57.1.1 int qurt_mapping_attr_get_64 ( qurt_addr_64_t *vaddr,* unsigned int *pid,* qurt_mapping_attr64_t ∗ *attr* )

Gets the mapping attributes for a specified virtual address and PID. It supports both HSV32 and HSV39 mapping.

#### Associated data types

qurt_addr_64_t
qurt_mapping_attr64_t

#### Parameters

| | | |
|---|---|---|
| in | *vaddr* | Virtual address for which the attributes are required. |
| in | *pid* | process id for the target process |
| out | *attr* | Pointer to the mapping attribute structure. |

#### Returns

0 – Success.
QURT_EINVALID – Incorrect virtual address or pid

#### Safe API

No

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 21.58 qurt_physmem_get_addr_width()

## 21.58.1 Function Documentation

### 21.58.1.1 unsigned qurt_physmem_get_addr_width ( unsigned *is_hsv39,* unsigned ∗ *addr_width,* unsigned ∗ *sid_width* )

Get the width of physical address

**Parameters**

| | | |
|---|---|---|
| in | *is_hsv39* | Indicate whether it is HSV32 (0) or HSV39 (1) |
| out | *addr_width* | Pointer to the width of address only |
| out | *sid_width* | Pointer to the width of SID |

**Returns**

returns QURT_EOK

**Safe API**

No

**Dependencies**

None

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 21.59 qurt_physmem_encode_page()

## 21.59.1 Function Documentation

### 21.59.1.1 int qurt_physmem_encode_page ( uint32_t *ppage,* uint32_t *sid,* unsigned *is_hsv39,* uint32_t ∗ *enc_ppage* )

Encode the physical page and sid into where enc_ppage points to.

**Parameters**

| | | |
|---|---|---|
| in | *ppage* | The physical page of the memory |
| in | *sid* | The SID of the memory |
| in | *is_hsv39* | Indicate whether it is HSV32 (0) or HSV39 (1) |
| out | *enc_ppage* | The pointer of encoded phyical page |

**Returns**

returns QURT_EOK/error.

**Note**

In case of HSV39, the page unit is 1MB. Otherwise it is 4K. This API is expected to be used rarely.

**Safe API**

No

**Dependencies**

None

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 21.60 qurt_physmem_decode_page()

## 21.60.1 Function Documentation

### 21.60.1.1 int qurt_physmem_decode_page ( uint32_t *enc_ppage,* unsigned *is_hsv39,* uint32_t ∗ *ppage,* uint32_t ∗ *sid* )

Decode the encoded physical page into physical page and SID separately.

**Parameters**

| | | |
|---|---|---|
| in | *enc_ppage* | The encoded physical page |
| in | *is_hsv39* | Indicate whether it is HSV32 (0) or HSV39 (1) |
| out | *ppage* | The pointer of physical page |
| out | *sid* | the pointer of SID |

**Returns**

returns QURT_EOK/error

**Note**

In case of HSV39, the page unit is 1MB. Otherwise it is 4K. This API is expected to be used rarely.

**Safe API**

No

**Dependencies**

None

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 21.61 qurt_mem_preempt_virtual_memory()

## 21.61.1 Function Documentation

### 21.61.1.1 int qurt_mem_preempt_virtual_memory ( unsigned int *pid,* qurt_addr_64_t *vaddr,* qurt_size_64_t *size,* unsigned int *flag* )

Preempt/disable access to a virtual memory range within a process domain.

The API does following operation - Invalidate the virtual memory range range fom MMU and pagetable. Based on the additional flag, the interface perform teardown of other compute resources due to memory preemption, for example if flag QURT_USER_DMA_TLBSYNC is set, issue a DMTLBSynch() for user DMA engines.

**Note**

Access to preempted virtual address shall suspend the faulted thread. A thread suspeded on a preempted virtual address shall become runnable once the virutal memory range is resumed by qurt_mem_resume_virtual_memory().

The API is avaiable only to secure resource manager(SRM)

**Associated data types**

#unsigned int
qurt_addr_64_t qurt_size_64_t

**Parameters**

| | | |
|---|---|---|
| in | *pid* | Process identifier. |
| in | *vaddr* | Base virtual address for preempted memory, shall be 4K aligned. |
| in | *size* | Total size in bytes |
| in | *flag* | Attribute to indiacate a teardown for additonal resources for example - QURT_USER_DMA_TLBSYNC |

**Returns**

On success, returns QURT_EOK. For failure, returns. QURT_FAILED: Error QURT_EINVALID: vpage and size out of range. QURT_ENORESOURCE: resource busy for operation, retry.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 09:10:42 GMT hyungchul.w@lg.com

# 21.62 qurt_mem_resume_virtual_memory()

## 21.62.1 Function Documentation

### 21.62.1.1 int qurt_mem_resume_virtual_memory ( unsigned int *pid,* qurt_addr_64_t *vaddr,* qurt_size_64_t *size,* unsigned int *flag* )

Resume/enable access to a virtual memory range within a process domain. Transition the suspended thread(s) on a preempted range to runnable.

#### Note

A user of this API shall ensure that a full pre-empted range is resumed. Resume to partial range may not awaken all suspended threads. Resume to out of bound or a non TLB aligned size shall result in error. The API is avaiable only to secure resource manager(SRM)

#### Associated data types

#unsigned int
qurt_addr_64_t qurt_size_64_t

#### Parameters

| | | |
|---|---|---|
| in | *pid* | Process identifier. |
| in | *vaddr* | Base virtual address for preempted memory, shall be 4K aligned. |
| in | *size* | Total size in bytes |
| in | *flag* | Reserved |

#### Returns

On success, returns QURT_EOK. For failure, returns. QURT_FAILED: Error QURT_EINVALID: vpage and size out of range. QURT_ENORESOURCE: resource busy for operation, retry.

#### Safe API

No

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 03:20:42 GMT hyungchul.won@qc.com

## 21.63 Memory Management Data Types

- Memory pools are represented in QuRT as objects of type qurt_mem_pool_t.
- Memory regions are represented as objects of type qurt_mem_region_t.
- Memory region attributes are stored in structures of type qurt_mem_region_attr_t.
- Memory region types are stored as values of type qurt_mem_region_type_t.
- Memory region mappings are specified as values of type qurt_mem_mapping_t.
- Cache types are specified as values of type qurt_mem_cache_type_t.
- Cache modes are specified as values of type qurt_mem_cache_mode_t.
- Cache operation codes are specified as values of type qurt_mem_cache_op_t.
- Cache control operations are stored in opaque structures of type qurt_mem_cache_ctrl_t.
- QuRT pre-initializes the memory pool object qurt_mem_default_pool.

### 21.63.1 Define Documentation

#### 21.63.1.1 #define QURT_POOL_REMOVE_ALL_OR_NONE 1

#### 21.63.1.2 #define QURT_MEM_CACHE_WRITEBACK_NONL2CACHEABLE 0U

QuRT cache mode type. Write back non-L2-cacheable.

#### 21.63.1.3 #define QURT_MEM_CACHE_WRITETHROUGH_NONL2CACHEABLE 1U

Write through non-L2-cacheable.

#### 21.63.1.4 #define QURT_MEM_CACHE_DEVICE_SFC 2U

Enables placing limitations on the number of outstanding transactions.

#### 21.63.1.5 #define QURT_MEM_CACHE_DEVICE 4U

Volatile memory-mapped device. Access to device memory cannot be cancelled by interrupts, re-ordered, or replayed.

#### 21.63.1.6 #define QURT_MEM_CACHE_WRITETHROUGH 5U

Write through.

#### 21.63.1.7 #define QURT_MEM_CACHE_NONE_SHARED 6U

Normal uncached memory that can be shared with other subsystems.

#### 21.63.1.8 #define QURT_MEM_CACHE_WRITEBACK 7U

Write back.

#### 21.63.1.9 #define QURT_MEM_CACHE_INVALID 10U

Reserved as an invalid cache type.

Qualcomm Confidential - May Contain Trade Secrets 2026-03-27 09:40:32 GMT hyungchulwon@hyne.com

#### 21.63.1.10 #define QURT_MEM_CACHE_WRITEBACK_L2CACHEABLE QURT_MEM_-CACHE_WRITEBACK

Write back L2 cacheable.

#### 21.63.1.11 #define QURT_MEM_CACHE_WRITETHROUGH_L2CACHEABLE QURT_M-EM_CACHE_WRITETHROUGH

Write through L2 cacheable.

#### 21.63.1.12 #define QURT_MEM_CACHE_NONE QURT_MEM_CACHE_DEVICE

Deprecated – use QURT_MEM_CACHE_DEVICE instead.

### 21.63.2 Data Structure Documentation

#### 21.63.2.1 struct qurt_mem_region_attr_t

QuRT memory region attributes type.

#### 21.63.2.2 struct qurt_mem_pool_attr_t

QuRT user physical memory pool type.

#### 21.63.2.3 struct qurt_mem_pool_status_t

QuRT memory pool status type.

**Data fields**

| Type | Parameter | Description |
|---|---|---|
| qurt_size_t | contig_size | Largest contiguous free memory in bytes. |
| qurt_size_t | free_size | Total free memory in bytes. |
| qurt_size_t | total_size | Total declared memory in bytes. |

#### 21.63.2.4 struct qurt_mem_physpool_attr64_t

QuRT user 64 bit physical memory pool type.

### 21.63.3 Typedef Documentation

#### 21.63.3.1 typedef unsigned int qurt_addr_t

QuRT address type.

#### 21.63.3.2 typedef unsigned int qurt_paddr_t

QuRT physical memory address type.

#### 21.63.3.3 typedef unsigned long long qurt_addr_64_t

QuRT 64-bit memory address type.

Qualcomm Confidential - May Contain Trade Secrets 2026-03-27 05:40:42 GMT hyunschulwon@lge.com

#### 21.63.3.4 typedef unsigned long long qurt_paddr_64_t

QuRT 64-bit physical memory address type.

#### 21.63.3.5 typedef unsigned long long qurt_size_64_t

QuRT 64-bit size.

#### 21.63.3.6 typedef unsigned int qurt_mem_region_t

QuRT memory regions type.

#### 21.63.3.7 typedef unsigned int qurt_mem_fs_region_t

QuRT memory FS region type.

#### 21.63.3.8 typedef unsigned int qurt_mem_pool_t

QuRT memory pool type.

#### 21.63.3.9 typedef unsigned int qurt_size_t

QuRT size type.

#### 21.63.3.10 typedef unsigned char qurt_mem_cache_mode_t

#### 21.63.3.11 typedef unsigned char qurt_perm_t

### 21.63.4 Enumeration Type Documentation

#### 21.63.4.1 enum qurt_mem_mapping_t

QuRT memory region mapping type.

**Enumerator:**

***QURT_MEM_MAPPING_VIRTUAL*** Default mode. The region virtual address range maps to an available contiguous area of physical memory. For the most efficient use of virtual memory, the QuRT system chooses the base address in physical memory. This works for most memory use cases.

***QURT_MEM_MAPPING_PHYS_CONTIGUOUS*** The region virtual address space must be mapped to a contiguous area of physical memory. This is necessary when the memory region is accessed by external devices that bypass Hexagon virtual memory addressing. The base address in physical memory must be explicitly specified.

***QURT_MEM_MAPPING_IDEMPOTENT*** Region virtual address space maps to the identical area of physical memory.

***QURT_MEM_MAPPING_VIRTUAL_FIXED*** Virtual address space of the region maps either to the specified area of physical memory or (if no area is specified) to available physical memory. Use this mapping to create regions from virtual space that was reserved by calling qurt_mem_region_create() with mapping.

***QURT_MEM_MAPPING_NONE*** Reserves a virtual memory area (VMA). Remapping a virtual range is not permitted without first deleting the memory region. When such a region is deleted, its corresponding virtual memory addressing remains intact.

***QURT_MEM_MAPPING_VIRTUAL_RANDOM*** System chooses a random virtual address and maps it to available contiguous physical addresses.

***QURT_MEM_MAPPING_PHYS_DISCONTIGUOUS*** While virtual memory is contiguous, allocates in discontiguous physical memory blocks. This helps when there are smaller contiguous blocks than the requested size. Physical address is not provided as part of the get_attr call

***QURT_MEM_MAPPING_INVALID*** Reserved as an invalid mapping type.

### 21.63.4.2 enum qurt_mem_cache_type_t

QuRT cache type; specifies data cache or instruction cache.

**Enumerator:**

***QURT_MEM_ICACHE*** Instruction cache.
***QURT_MEM_DCACHE*** Data cache.

### 21.63.4.3 enum qurt_mem_cache_op_t

QuRT cache operation code type.

**Enumerator:**

***QURT_MEM_CACHE_FLUSH*** Flush.
***QURT_MEM_CACHE_INVALIDATE*** Invalidate
***QURT_MEM_CACHE_FLUSH_INVALIDATE*** Flush invalidate.
***QURT_MEM_CACHE_FLUSH_ALL*** Flush all.
***QURT_MEM_CACHE_FLUSH_INVALIDATE_ALL*** Flush invalidate all.
***QURT_MEM_CACHE_TABLE_FLUSH_INVALIDATE*** Table flush invalidate.
***QURT_MEM_CACHE_FLUSH_INVALIDATE_L2*** L2 flush invalidate.

### 21.63.4.4 enum qurt_mem_region_type_t

QuRT memory region type.

**Enumerator:**

***QURT_MEM_REGION_LOCAL*** Local.
***QURT_MEM_REGION_SHARED*** Shared.
***QURT_MEM_REGION_USER_ACCESS*** User access.
***QURT_MEM_REGION_FS*** FS.
***QURT_MEM_REGION_INVALID*** Reserved as an invalid region type.

### 21.63.4.5 enum qurt_cache_type_t

**Enumerator:**

***HEXAGON_L1_I_CACHE*** Hexagon L1 instruction cache.
***HEXAGON_L1_D_CACHE*** Hexagon L1 data cache.
***HEXAGON_L2_CACHE*** Hexagon L2 cache.

### 21.63.4.6 enum qurt_cache_partition_size_t

**Enumerator:**

***FULL_SIZE*** Fully shared cache, without partitioning.
***HALF_SIZE*** 1/2 for main, 1/2 for auxiliary.
***THREE_QUARTER_SIZE*** 3/4 for main, 1/4 for auxiliary.
***SEVEN_EIGHTHS_SIZE*** 7/8 for main, 1/8 for auxiliary; for L2 cache only.

Qualcomm Confidential - May Contain Trade Secrets 2026-03-27 05:40:13 GMT hyungchul.wonelge.com

## 21.63.5 Variable Documentation

### 21.63.5.1 qurt_mem_pool_t qurt_mem_default_pool

Memory pool object.

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:40:42 GMT
hyungchul.won@lge.com

## 21.64 Macros

### 21.64.1 Define Documentation

#### 21.64.1.1 #define QURT_SYSTEM_ALLOC_VIRTUAL 1U

Allocates available virtual memory in the address space of all processes.

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:40:42 GMT
hyungchul.won@lge.com

# 22 Memory Mapping

Memory mapping services are accessed with the following QuRT functions.

- qurt_mem_mmap()
- qurt_mem_mmap2()
- qurt_mem_mmap_by_name()
- qurt_mem_mprotect2()
- qurt_mem_mprotect()
- qurt_mem_munmap()
- qurt_mem_munmap2()
- qurt_mem_munmap3()
- qurt_mem_pool_status_get()
- Memory Mapping Macros
- Memory Mapping Types

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:40:42 GMT
hyungchul.won@lge.com

## 22.1 qurt_mem_mmap()

### 22.1.1 Function Documentation

#### 22.1.1.1 void∗ qurt_mem_mmap ( int *client_handle,* qurt_mem_pool_t *pool,* qurt_mem_region_t ∗ *pRegion,* void ∗ *addr,* size_t *length,* int *prot,* int *flags,* int *fd,* unsigned long long *offset* )

Creates a memory mapping with the specified attributes. This API allows the root process caller to create mapping on behalf of a user process. If the client_handle belongs to a valid user process, the resulting mapping is created for the process. If -1 is passed in place of client_handle, the API creates mapping for the underlying process of the caller.

**NOTE** If the specified attributes are not valid, an error result is returned.

**NOTE** Safety Applications shall only use -1 for the fd parameter.

**Parameters**

| | | |
|---|---|---|
| out | *client_handle* | Client handle to use for this mapping (optional). |
| in | *pool* | Optional argument that specifies a pool handle if the user wants to allocate memory from a specific pool. The default value for this argument is NULL. |
| in | *pRegion* | Map region. This argument is unused, and the default value is NULL. |
| in | *addr* | Virtual memory address. |
| in | *length* | Size of mapping in bytes. |
| in | *prot* | Mapping access permissions (R/W/X). |
| in | *flags* | Mapping modes.<br>• QURT_MAP_NAMED_MEMSECTION<br>• QURT_MAP_FIXED<br>• QURT_MAP_NONPROCESS_VPOOL<br>• QURT_MAP_TRYFIXED<br>• QURT_MAP_ANON<br>• QURT_MAP_PHYSADDR<br>• QURT_MAP_VA_ONLY |
| in | *fd* | File designator. |
| in | *offset* | Offset in file. |

**Returns**

Valid virtual address – Success.
QURT_MAP_FAILED – Mapping creation failed.

**Safe API**

Yes

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 09:14:42 GMT hyungchul.kim@qualcomm.com

## 22.2 qurt_mem_mmap2()

### 22.2.1 Function Documentation

#### 22.2.1.1 void∗ qurt_mem_mmap2 ( int *client_handle,* qurt_mem_pool_t *pool,* qurt_mem_region_t ∗ *pRegion,* void ∗ *addr,* size_t *length,* int *prot,* int *flags,* int *fd,* unsigned long long *offset* )

Creates a memory mapping with the specified attributes. Returns a more descriptive error code in case of failure. This API allows the root process caller to create mapping on behalf of a user process. If the client_handle belongs to a valid user process, the resulting mapping is created for the process. If -1 is passed in place of client_handle, the API creates mapping for the underlying process of the caller.

**NOTE** If the specified attributes are not valid, an error result is returned.

**NOTE** Safety Applications shall only use -1 for the fd parameter.

**Parameters**

| | | |
|---|---|---|
| out | *client_handle* | Client handle to use for this mapping (optional). |
| in | *pool* | Optional argument that allows the user to specify a pool handle when the user wants to allocate memory from a specific pool. Default value for this argument is NULL. |
| in | *pRegion* | Map region (unused argument); default value is NULL. |
| in | *addr* | Virtual memory address. |
| in | *length* | Size of mapping in bytes. |
| in | *prot* | Mapping access permissions (R/W/X). Cache attributes, bus attributes, User mode. |
| in | *flags* | Mapping modes; Shared, Private, or Anonymous. |
| in | *fd* | File designator. |
| in | *offset* | Offset in file. |

**Returns**

Valid virtual address – Success.
QURT_EMEM – Physical address is not available.
QURT_EFAILED – VA is not available or mapping failed.
QURT_EINVALID – Invalid argument was passed (for example, an unaligned VA/PA).

**Safe API**

Yes

Qualcomm Confidential - May Contain Trade Secrets 2025-02-27 05:40:27 GMT hyungchul.won@lge.com

## 22.3 qurt_mem_mmap_by_name()

### 22.3.1 Function Documentation

#### 22.3.1.1 void∗ qurt_mem_mmap_by_name ( const char ∗ *name,* void ∗ *addr,* size_t *length,* int *prot,* int *flags,* unsigned long long *offset* )

Creates a memory mapping for a named-memsection using the specified attributes. Specify the named memsection in cust_config.xml.

**NOTE** If the specified attributes are not valid or the named memsection is not found, an error result is returned.

**Parameters**

| | | |
|---|---|---|
| in | *name* | Name of the memsection in cust_config.xml that specifies this mapping. The name must be less than 25 characters. |
| in | *addr* | Virtual memory address. |
| in | *length* | Size of mapping in bytes. |
| in | *prot* | Mapping access permissions (R/W/X). Cache attributes, bus attributes, User mode |
| in | *flags* | Mapping modes, such as Shared, Private, or Anonymous. |
| in | *offset* | Offset relative to the physical address range specified in memsection. If offset + length exceeds size of memsection, failure is returned. |

**Returns**

Valid virtual address – Success.
QURT_MAP_FAILED – Mapping creation failed.

**Safe API**

Yes

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 22.4 qurt_mem_mprotect2()

### 22.4.1 Function Documentation

#### 22.4.1.1 int qurt_mem_mprotect2 ( int *client_handle,* const void ∗ *addr,* size_t *length,* int *prot* )

Changes access permissions and attributes on an existing mapping based on the client_handle argument.

**NOTE** If the specified virtual address is not found or invalid attributes are passed, an error code is returned.

**NOTE** When error is returned, it is possible that attributes or permissions are changed for some part of the mapping, and unchanged for the remaining mapping. Clients are not to use these mappings further.

**Parameters**

| | | |
|---|---|---|
| in | *client_handle* | Obtained from the current invocation function. See Section 3.4.1 in the Hexagon QuRT QDI User Guide (80-VB419-84). |
| in | *addr* | Virtual memory address. |
| in | *length* | Size of mapping in bytes. |
| in | *prot* | Mapping access permissions (R/W/X). Cache attributes, bus attributes, User mode. |

**Returns**

QURT_EOK – Successful permission change on the mapping.
QURT_EFATAL – Failed to change permissions on the mapping.
QURT_EINVALID – Invalid requested attributes or permissions.

**Safe API**

No

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 22.5 qurt_mem_mprotect()

### 22.5.1 Function Documentation

#### 22.5.1.1 int qurt_mem_mprotect ( const void ∗ *addr,* size_t *length,* int *prot* )

Changes access permissions and attributes on an existing mapping.

**NOTE** If the specified virtual address is not found or invalid attributes are passed, an error code is returned.

**NOTE** When error is returned, it is possible that attributes or permissions are changed for some part of the mapping, and unchanged for the remaining mapping. Clients are not to use these mappings further.

**Parameters**

| | | |
|---|---|---|
| in | *addr* | Virtual memory address. |
| in | *length* | Size of mapping in bytes. |
| in | *prot* | Mapping access permissions (R/W/X). Cache attributes, bus attributes, User mode. |

**Returns**

QURT_EOK – Successfully changes permissions on the mapping.
QURT_EFATAL – Failed to change permissions on the mapping.
QURT_EINVALID – Attributes / permissions requested are invalid.

**Safe API**

Yes

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 22.6 qurt_mem_munmap()

### 22.6.1 Function Documentation

#### 22.6.1.1 int qurt_mem_munmap ( void * *addr,* size_t *length* )

Removes an existing mapping.

**NOTE** If the specified mapping is not found in the context of the caller process or invalid attributes are passed, an error code is returned.

**Parameters**

| | | |
|---|---|---|
| in | *addr* | Virtual memory address. |
| in | *length* | Size of mapping in bytes. |

**Returns**

QURT_EOK – Successful change to permissions on the mapping.
QURT_EFATAL – Failed to change permissions on the mapping.
QURT_ELOCKED - Buffer is locked. Mapping delete failed.

**Safe API**

Yes

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 22.7 qurt_mem_munmap2()

### 22.7.1 Function Documentation

#### 22.7.1.1 int qurt_mem_munmap2 ( int *client_handle,* void ∗ *addr,* size_t *length* )

Removes an existing mapping for a specified process.

**NOTE** This API allows a root process entity, such as a driver, to remove mapping that was created for a user process. If the specified mapping is not found in the context of client handle or invalid attributes are passed, an error code is returned.

**Parameters**

| | | |
|---|---|---|
| out | *client_handle* | Client handle of the user process that owns this mapping. |
| in | *addr* | Virtual memory address. |
| in | *length* | Size of mapping in bytes. |

**Returns**

QURT_EOK – Successfully changes permissions on the mapping.
QURT_EFATAL – Failed to change permissions on the mapping.
QURT_ELOCKED - Buffer is locked. Mapping delete failed.

**Safe API**

No

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 22.8 qurt_mem_munmap3()

### 22.8.1 Function Documentation

#### 22.8.1.1 int qurt_mem_munmap3 ( int *client_handle,* void ∗ *addr,* size_t *length,* int *flags* )

Removes an existing mapping or reservation for a specified process.

**Parameters**

| | | |
|---|---|---|
| in | *client_handle* | Client handle of the user process that owns this mapping. |
| in | *addr* | Pointer to a virtual memory address. |
| in | *length* | Size of mapping in bytes. |
| in | *flags* | Specifies the flag. |

**Returns**

QURT_EOK – Successfully changes permissions on the mapping.
QURT_EFATAL – Failed to change permissions on the mapping.
QURT_ELOCKED - Buffer is locked. Mapping delete failed.

**Safe API**

No

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 22.9 qurt_mem_pool_status_get()

### 22.9.1 Function Documentation

#### 22.9.1.1 int qurt_mem_pool_status_get ( qurt_mem_pool_t *pool,* qurt_mem_pool_status_t ∗ *status* )

Gets the memory pool status.

Based on the pool handle, retrieves largest contiguous free memory, total free memory, and total memory declared for the pool in bytes. Fills in the memory status structure with the values.

**Associated data types**

qurt_mem_pool_t
qurt_mem_pool_status_t

**Parameters**

| | | |
|---|---|---|
| in | *pool* | Pool handle. |
| out | *status* | Pointer to the memory pool status structure. |

**Returns**

QURT_EOK – Success.
QURT_EINVALID – Corrupt handle; pool handle is invalid.

**Safe API**

No

**Note**

When secure PD is spawned through TRUSTED ROOT, API will NOT be supported for secure PD

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.w@lge.com

## 22.10 Memory Mapping Macros

### 22.10.1 Define Documentation

#### 22.10.1.1 #define QURT_MAP_NAMED_MEMSECTION 0x0004U

Named memsection.

#### 22.10.1.2 #define QURT_MAP_FIXED 0x0010U

Fixed virtual address.

#### 22.10.1.3 #define QURT_MAP_RENAME 0x0020U

Rename.

#### 22.10.1.4 #define QURT_MAP_NORESERVE 0x0040U

No reserve.

#### 22.10.1.5 #define QURT_MAP_INHERIT 0x0080U

Inherit.

#### 22.10.1.6 #define QURT_MAP_NONPROCESS_VPOOL 0x0100U

Use a virtual address outside of the default range of the processes. This option is only supported in the root process and only when virtual memory split is enabled in the XML. The root process can use this flag to create mapping for a user process, for example, if the virtual address is configured for a 3G/1G split, the root process can use this flag to create mapping in the top 1 GB area for the user process or the lower 3 GB area for the root process. This is useful for shared buffer use cases.

#### 22.10.1.7 #define QURT_MAP_HASSEMAPHORE 0x0200U

Has semaphore.

#### 22.10.1.8 #define QURT_MAP_TRYFIXED 0x0400U

Try to create a mapping for a virtual address that was passed. If the passed virtual address fails, use a random virtual address.

#### 22.10.1.9 #define QURT_MAP_WIRED 0x0800U

Wired.

#### 22.10.1.10 #define QURT_MAP_FILE 0x0000U

File.

#### 22.10.1.11 #define QURT_MAP_ANON 0x1000U

Allocate physical memory from the pool that was passed. By default, memory is allocated from the default physpool.

#### 22.10.1.12 #define QURT_MAP_VA_ONLY 0x2000U

Reserve a virtual address without mapping it.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:10:41 GMT hyungchul.kim@lge.com

#### 22.10.1.13 #define QURT_MAP_FAR_ADDR 0x4000U

Create a mapping for FAR address

#### 22.10.1.14 #define QURT_MAP_FAILED ((void ∗) -1)

Mapping creation failed.

#### 22.10.1.15 #define QURT_PROT_CACHE_MODE( *n* ) QURT_MMAP_BUILD(QURT_PROT_CACHE_BOUNDS,(n))

#### 22.10.1.16 #define QURT_PROT_BUS_ATTR( *n* ) QURT_MMAP_BUILD(QURT_PROT_BUS_BOUNDS,(n))

#### 22.10.1.17 #define QURT_PROT_USER_MODE( *n* ) QURT_MMAP_BUILD(QURT_PROT_USER_BOUNDS,(n))

#### 22.10.1.18 #define QURT_MAP_PHYSADDR QURT_MMAP_BUILD(QURT_MAP_PHYSADDR_BOUNDS,1U)

Use the physical address that was passed in offset field. This is allowed only for root process.

#### 22.10.1.19 #define QURT_MAP_TYPE( *n* ) QURT_MMAP_BUILD(QURT_MAP_TYPE_BOUNDS,(n))

#### 22.10.1.20 #define QURT_MAP_REGION( *n* ) QURT_MMAP_BUILD(QURT_MAP_REGION_BOUNDS,(n))

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 22.11 Memory Mapping Types

## 22.11.1 Data Structure Documentation

### 22.11.1.1 struct qurt_mapping_attr_t

Mapping attribute information

**Data fields**

| Type | Parameter | Description |
|---|---|---|
| qurt_paddr_64-_t | paddr | |
| qurt_size_t | size | |
| qurt_mem_-cache_mode_t | cache_mode | |
| qurt_perm_t | perms | |

### 22.11.1.2 struct qurt_mapping_attr64_t

**Data fields**

| Type | Parameter | Description |
|---|---|---|
| qurt_paddr_64-_t | paddr | |
| qurt_size_64_t | size | |
| unsigned int | sid | |
| qurt_mem_-cache_mode_t | cache_mode | |
| qurt_perm_t | perms | |

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 23 System Environment

## 23.1 Overview

Programs can access various properties of the QuRT system environment.

The system environment supports the following operations:

- Get the program heap - Return information on the program heap.
- Get the hardware timer - Return the memory address of the hardware timer.
- Get the architecture version - Return the Hexagon processor architecture version.
- Get the maximum hardware threads - Return the number of hardware threads supported in the Hexagon processor.
- Get the maximum pimutex priority - Return the maximum priority of the priority inheritance mutexes.
- Get the API version - Return the release version of the current QuRT RTOS.
- Get the process names - Return the names of processes in the system.

The maximum pimutex priority specifies the highest priority that a thread can be set to while it has the lock on a priority inheritance mutex. This value enables other threads which are not using pimutexes to run with a thread priority higher than the pimutex maximum priority.

**Functions**

System environment services are accessed with the following QuRT functions.

- qurt_sysenv_get_app_heap()
- qurt_sysenv_get_arch_version()
- qurt_sysenv_get_max_hw_threads()
- qurt_sysenv_get_max_pi_prio()
- qurt_sysenv_get_process_name2()
- qurt_sysenv_get_process_name()
- qurt_sysenv_get_stack_profile_count()
- qurt_sysenv_get_instance_id()
- qurt_sysenv_get_hw_threads()
- Data Types

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:40 GMT Hyungchul.won@lge.com

## 23.2 qurt_sysenv_get_app_heap()

### 23.2.1 Function Documentation

#### 23.2.1.1 int qurt_sysenv_get_app_heap ( qurt_sysenv_app_heap_t * *aheap* )

Gets information on the program heap from the kernel.

**Associated data types**

qurt_sysenv_app_heap_t

**Parameters**

| | | |
|---|---|---|
| out | *aheap* | Pointer to information on the program heap. |

**Returns**

QURT_EOK – Success.
QURT_EVAL – Invalid parameter.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 23.3 qurt_sysenv_get_arch_version()

### 23.3.1 Function Documentation

#### 23.3.1.1 int qurt_sysenv_get_arch_version ( qurt_arch_version_t * *vers* )

Gets the Hexagon processor architecture version from the kernel.

**Associated data types**

qurt_arch_version_t

**Parameters**

| | | |
|---|---|---|
| out | *vers* | Pointer to the Hexagon processor architecture version. |

**Returns**

QURT_EOK – Success.
QURT_EVAL – Invalid parameter

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 23.4 qurt_sysenv_get_max_hw_threads()

### 23.4.1 Function Documentation

#### 23.4.1.1 int qurt_sysenv_get_max_hw_threads ( qurt_sysenv_max_hthreads_t * *mhwt* )

Gets the maximum number of hardware threads supported in the Hexagon processor. The API includes the disabled hardware threads to reflect the maximum hardware thread count. For example, if the image is configured for four hardware threads and hthread_mask is set to 0x5 in cust_config.xml, only HW0 and HW2 are initialized by QuRT. HW1 and HW3 are not used at all. Under such a scenario, qurt_sysenv_get_max_hw_threads() still returns four.

**Associated data types**

qurt_sysenv_max_hthreads_t

**Parameters**

| | | |
|---|---|---|
| `out` | *mhwt* | Pointer to the maximum number of hardware threads supported in the Hexagon processor. |

**Returns**

QURT_EOK – Success.
QURT_EVAL – Invalid parameter.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 23.5 qurt_sysenv_get_max_pi_prio()

### 23.5.1 Function Documentation

#### 23.5.1.1 int qurt_sysenv_get_max_pi_prio ( qurt_sysenv_max_pi_prio_t * *mpip* )

Gets the maximum priority inheritance mutex priority from the kernel.

**Associated data types**

qurt_sysenv_max_pi_prio_t

**Parameters**

| | | |
|---|---|---|
| out | *mpip* | Pointer to the maximum priority inheritance mutex priority. |

**Returns**

QURT_EOK – Success.
QURT_EVAL – Invalid parameter.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 23.6 qurt_sysenv_get_process_name2()

### 23.6.1 Function Documentation

#### 23.6.1.1 int qurt_sysenv_get_process_name2 ( int *client_handle,* qurt_sysenv_procname_t ∗ *pname* )

Gets information on the system environment process names based on the client_handle argument.

**Associated data types**

qurt_sysenv_procname_t

**Parameters**

| | | |
|---|---|---|
| in | *client_handle* | Obtained from the current invocation function. See Section 3.4.1 in the Hexagon QuRT QDI User Guide (80-VB419-84). |
| out | *pname* | Pointer to information on the process names in the system. |

**Returns**

QURT_EOK – Success.
QURT_EVAL – Invalid parameter.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 23.7 qurt_sysenv_get_process_name()

### 23.7.1 Function Documentation

#### 23.7.1.1 int qurt_sysenv_get_process_name ( qurt_sysenv_procname_t * *pname* )

Gets information on the system environment process names from the kernel.

**Associated data types**

qurt_sysenv_procname_t

**Parameters**

| | | |
|---|---|---|
| out | *pname* | Pointer to information on the process names in the system. |

**Returns**

QURT_EOK – Success.
QURT_EVAL – Invalid parameter.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 23.8 qurt_sysenv_get_stack_profile_count()

### 23.8.1 Function Documentation

#### 23.8.1.1 int qurt_sysenv_get_stack_profile_count ( qurt_sysenv_stack_profile_count_t * *count* )

Gets information on the stack profile count from the kernel.

**Associated data types**

qurt_sysenv_stack_profile_count_t

**Parameters**

| | | |
|---|---|---|
| out | *count* | Pointer to information on the stack profile count. |

**Returns**

QURT_EOK – Success.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 23.9 qurt_sysenv_get_instance_id()

### 23.9.1 Function Documentation

#### 23.9.1.1 int qurt_sysenv_get_instance_id ( unsigned int * *inst_id* )

This function returns the instance ID of the current subsystem upon invocation.

**NOTE** Field names of Instance ID Register: INSTANCE_NUM: the instance number of the subsystem. ID: ID value of subsystem. Use QURT_GET_INSTANCE_ID_INST_NUM(x) to get INSTANCE_NUMBER from inst_id. Use QURT_GET_INSTANCE_ID_ID(x) to get ID from inst_id.

**Parameters**

| | | |
|---|---|---|
| `out` | *inst_id* | Pointer to data of instance id. |

**Returns**

QURT_EOK – Success.
QURT_EINVALID – Invalid parameter.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 23.10 qurt_sysenv_get_hw_threads()

### 23.10.1 Function Documentation

#### 23.10.1.1 int qurt_sysenv_get_hw_threads ( qurt_sysenv_hthreads_t * *mhwt* )

Gets the number of hardware threads initialized by QuRT in Hexagon processor. For example, if the image is configured for four hardware threads and hthread_mask is set to 0x5 in cust_config.xml, QuRT only initializes HW0 and HW2. HW1 and HW3 are not used. In this scenario, qurt_sysenv_get_hw_threads() returns 2.

**Associated data types**

qurt_sysenv_hthreads_t

**Parameters**

| | | |
|---|---|---|
| `out` | *mhwt* | Pointer to the number of hardware threads active in the Hexagon processor. |

**Returns**

QURT_EOK – Success.
QURT_EVAL – Invalid parameter.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 23.11 Data Types

This section describes data types for thread services.

### 23.11.1 Data Structure Documentation

#### 23.11.1.1 struct qurt_sysenv_swap_pools_t

QuRT swap pool information type.

#### 23.11.1.2 struct qurt_sysenv_app_heap_t

QuRT application heap information type.

#### 23.11.1.3 struct qurt_arch_version_t

QuRT architecture version information type.

#### 23.11.1.4 struct qurt_sysenv_max_hthreads_t

QuRT maximum hardware threads information type.

#### 23.11.1.5 struct qurt_sysenv_hthreads_t

QuRT active hardware threads information type.

#### 23.11.1.6 struct qurt_sysenv_max_pi_prio_t

QuRT maximum pi priority information type.

#### 23.11.1.7 struct qurt_sysenv_procname_t

QuRT process name information type.

#### 23.11.1.8 struct qurt_sysenv_stack_profile_count_t

QuRT stack profile count information type.

#### 23.11.1.9 struct qurt_sysevent_error_t

QuRT system error event type.

**Data fields**

| Type | Parameter | Description |
|---|---|---|
| unsigned int | thread_id | Thread ID. |
| unsigned int | fault_pc | Fault PC. |
| unsigned int | sp | Stack pointer. |
| unsigned int | badva | Virtual data address where the exception occurred. |
| unsigned int | cause | QuRT error result. |
| unsigned int | ssr | Supervisor status register. |
| unsigned int | fp | Frame pointer. |
| unsigned int | lr | Link register. |
| unsigned int | pid | PID of the process to which this thread belongs. |

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyunchul.won@lge.com

### 23.11.1.10 struct qurt_sysevent_error_1_t

**Data fields**

| Type | Parameter | Description |
|---|---|---|
| unsigned int | thread_id | Thread ID. |
| unsigned int | fault_pc | Fault PC. |
| unsigned int | sp | Stack pointer. |
| unsigned int | badva | Virtual data address where the exception occurred. |
| unsigned int | cause | QuRT error result. |
| unsigned int | ssr | Supervisor status register. |
| unsigned int | fp | Frame pointer. |
| unsigned int | lr | Link register. |
| unsigned int | pid | PID of the process to which this thread belongs. |
| unsigned int | fkey | Framekey. |
| unsigned int | reserved1 | Reserved. |
| unsigned int | reserved2 | Reserved. |
| unsigned int | reserved3 | Reserved. |

### 23.11.1.11 struct qurt_sysevent_pagefault_t

QuRT page fault error event information type.

**Data fields**

| Type | Parameter | Description |
|---|---|---|
| qurt_thread_t | thread_id | Thread ID of the page fault thread. |
| unsigned int | fault_addr | Accessed address that caused the page fault. |
| unsigned int | ssr_cause | SSR cause code for the page fault. |

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 24 Profiling

## 24.1 Overview

Threads use profiling to determine the cycle counts for selected parts of an application. The collected data can be used to determine the CPU utilization of a QuRT thread (or the entire QuRT application system).

Profiling is performed with the following operations:

- Enable profiling – Enable counting of running and idle processor cycles.
- Reset profile thread ID processor cycles – Set the per-hardware-thread running processor cycle counts to zero for the specified QuRT thread.
- Get profile thread ID processor cycles – Get the current per-hardware-thread running processor cycle counts for the specified QuRT thread.
- Get profile thread processor cycles – Get the current running processor cycle count for the current QuRT thread.
- Get profile thread thread cycles – Get the current running thread cycle count for the current QuRT thread.
- Reset profile idle processor cycles – Set the per-hardware-thread idle processor cycle counts to zero.
- Get profile idle processor cycles – Get the current per-hardware-thread idle processor cycle counts.
- Get core processor cycles – Get the current running processor cycle count since the processor was reset.

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:40:42 GMT
hyunghul.won@lge.com

Profiling supports thread-specific cycle counting for both the running (i.e., executing) and idle (i.e., not executing) cycles. The counts can be reset, which enables cycle counting to be performed on specific parts of an application.

All but one of the profile cycle counts are expressed in terms of processor cycles (i.e, the number of actual processor cycles executed by all hardware threads) as opposed to thread cycles (i.e., the number of cycles executed by a specific hardware thread). Assuming six hardware threads, the relation between these two cycle types is expressed in the following equation:

```
thread_cycles = processor_cycles / 6
```

The enable profiling operation is used to selectively enable or disable profiling (which is disabled by default).

**Note:** Enabling profiling does not automatically reset the cycle counts – this must be done explicitly by calling the reset operations before starting cycle counting.

The get profile thread ID processor cycles operation returns the current per-hardware thread running cycle counts for the specified QuRT thread (Section 3.1). This operation returns an array containing the current running cycle count for each hardware thread. Each count value represents the number of processor cycles

that have elapsed on the corresponding hardware thread while that thread has been scheduled for the specified QuRT thread.

The get profile thread processor cycles operation returns the current running cycle count for the specified QuRT thread (Section 3.1). The count value represents the number of processor cycles that have elapsed on all hardware threads while that thread has been scheduled for the specified QuRT thread.

**Note:** This count value is equivalent to summing the per-hardware-thread cycle count values returned by the get profile thread ID processor cycles operation.

The get profile thread cycles operation returns the current running cycle counts for the current QuRT thread, expressed in terms of thread cycles (as defined above).

The get profile idle processor cycles operation returns the current idle cycle count (i.e., the number of cycles a hardware thread is in IDLE state and not executing any instructions). This operation returns an array containing the current idle cycle count for each hardware thread. Each count value represents the number of processor cycles that have elapsed on the corresponding hardware thread while that thread has been in Wait mode.

**Note:** Cycles executed in the kernel get classified as idle or running according to the state that the current thread was in (i.e., idle or running) when the transition to the kernel occurred.

The get core processor cycles operation returns the number of processor cycles executed since the Hexagon processor was last reset. This value is based on the hardware core clock, which varies in speed according to the processor clock frequency (and which differs from the system clock described in Section 16.1).

In a given time duration, the relationship between the number of cycles elapsed by this operation, and the values returned by the get profile thread/idle processor cycles operations (both described above), is expressed by the following equation:

```
total_PCYCLES = run_pcycles + idle_pcycles
```

In this equation total_PCYCLES is the value returned by the get core processor cycles operation.

run_pcycles and idle_pcycles are defined in terms of the cycle count values returned by the get profile thread/idle processor cycles operations:

```
for (<all QuRT threads>)     for (i = 0; i < MAX_HW_THREADS; i++)
run_pcycles += profile_thread_pcycles[i] / 6;
for (i = 0; i < MAX_HW_THREADS; i++)
idle_pcycles += profile_idle_pcycles[i] / 6;
```

Because the cycle counts are summed on a per-thread basis, it is necessary in the above code to convert each processor cycle count to a thread cycle count (by dividing by 6).

**Note:** Because the hardware core clock stops running when the Hexagon processor shuts down (due to all its hardware threads being idle), the cycle values returned by this operation should be treated as relative rather than absolute.

### Computing CPU utilization

The CPU utilization for a QuRT thread (or an entire QuRT application system) indicates how many of the cycles that were executed in a given period of time by the Hexagon processor were used by a specific thread (or by the application system):

```
CPU_utilization = run_pcycles / total_PCYCLES
```

In this equation run_pcycles is the cycle count value returned by the get profile thread processor cycles operation (described above).

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:43 GMT hyunchul.won@lge.com

total_PCYCLES is the value returned by the get core processor cycles operation (also described above).

Note, however, that the Hexagon processor may have spent part of the specified time period in Power-saving mode, where the hardware core clock is completely shut down (because all the hardware threads are idle). In this case the value in total_PCYCLES does not represent the absolute time.

To accurately compute the CPU utilization in this case, total_PCYCLES must be adjusted by the core clock shutdown time. The shutdown time (also called the ALL_WAIT period) can be computed from the QuRT system clock using the following equation:

```
ALL_WAIT_pcycles = ((total_sclk_samples / QTIMER_clock_freq)*
core_clock_freq) - total_PCYCLES
```

In this equation total_sclk_samples is the number of cycles elapsed in the QuRT system clock (Section 16.1).

total_PCYCLES is the value returned by the get core processor cycles operation. QTIMER_clock_freq is 19.2 MHz on all target systems.

core_clock_freq is the Hexagon processor core clock frequency (which is specific to each target system).

Taking the ALL_WAIT period into consideration, the adjusted CPU utilization is:

```
CPU_utilization = run_pcycles / (total_PCYCLES + ALL_WAIT_pcycles)
```

**Note:** The ALL_WAIT_pcycles equation assumes that the Hexagon processor core clock frequency does not change during the time interval profiled. If the clock frequency does change in this interval, the input values need to be corrected because the weight of each sample is different.

For more information on profiling QuRT threads see Appendix A.

**Functions**

Profiling services are accessed with the following QuRT functions.

- qurt_get_core_pcycles()
- qurt_profile_enable()
- qurt_profile_enable2()
- qurt_profile_get()
- qurt_profile_get_idle_pcycles()
- qurt_profile_get_thread_pcycles()
- qurt_get_hthread_pcycles()
- qurt_get_hthread_commits()
- qurt_profile_get_threadid_pcycles()
- qurt_profile_reset_idle_pcycles()
- qurt_profile_reset_threadid_pcycles()
- Data Types
- Constants and Macros

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 24.2 qurt_get_core_pcycles()

### 24.2.1 Function Documentation

#### 24.2.1.1 unsigned long long int qurt_get_core_pcycles ( void )

Gets the count of executed core processor cycles.

Returns the number of running processor cycles that have executed since the Hexagon processor was last reset.

This value is based on the hardware core clock, which varies in speed according to the processor clock frequency.

**NOTE** Because the hardware core clock stops running when the processor shuts down (due to all of the hardware threads in idle), treat the cycle values returned by this operation as relative rather than absolute.

Thread cycle counts are valid only in the v4 Hexagon processor.

**Returns**

Integer – Count of core processor cycles.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 24.3 qurt_profile_enable()

### 24.3.1 Function Documentation

#### 24.3.1.1 void qurt_profile_enable ( int *enable* )

Enables profiling.

Enables or disables cycle counting of the running and idle processor cycles. Profiling is disabled by default.

**NOTE** Enabling profiling does not automatically reset the cycle counts – reset must explicitly call the reset operations before starting cycle counting. Cycle counting starts from the instant it is enabled using this API, and halts on profiling disable.

**Parameters**

| | | |
|---|---|---|
| `in` | *enable* | Profiling. Values:<br>• 0 – Disable profiling<br>• 1 – Enable profiling |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 24.4 qurt_profile_enable2()

### 24.4.1 Function Documentation

#### 24.4.1.1 int qurt_profile_enable2 ( qurt_profile_param_t *param*, qurt_thread_t *thread_id*, int *enable* )

Starts profiling of a specific parameter on a specific thread (as applicable).

**Parameters**

| | | |
|---|---|---|
| in | *param* | Profiling parameter. |
| in | *thread_id* | ID of the thread (if applicable) for which the specified paramter must be profiled. |
| in | *enable* | QURT_PROFILE_DISABLE – disable<br>QURT_PROFILE_ENABLE – enable |

**Returns**

QURT_EOK – Success
QURT_EALREADY – Measurement already in progress or already stopped
QURT_ENOTHREAD – Thread does not exist
QURT_EINVALID – Invalid profiling parameter

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 24.5 qurt_profile_get()

### 24.5.1 Function Documentation

#### 24.5.1.1 int qurt_profile_get ( qurt_profile_param_t *param*, qurt_thread_t *thread_id*, qurt_profile_result_t ∗ *result* )

Gets the value of the profiling parameter that was previously enabled.

**Parameters**

| | | |
|---|---|---|
| in | *param* | Profiling parameter. |
| in | *thread_id* | ID of thread (if applicable) for which the specified profiling paramter must be retrieved. |
| out | *result* | Profiling result associated with the parameter for the specified thread (if applicable). |

**Returns**

QURT_EOK – Success
QURT_EFAILED – Operation failed; profiling was not enabled
QURT_ENOTHREAD – Thread does not exist
QURT_EINVALID – Invalid profiling parameter

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 24.6 qurt_profile_get_idle_pcycles()

### 24.6.1 Function Documentation

#### 24.6.1.1 void qurt_profile_get_idle_pcycles ( unsigned long long * *pcycles* )

Gets the current idle processor cycle counts for a maximum of six hardware threads. Use qurt_profile_get_idle_pcycles2() to read pcycles without limits on maximum hardware threads.

This operation accepts a pointer to a user-defined array, and writes to the array the current idle cycle count for each hardware thread.

Each count value represents the number of processor cycles that have elapsed on the corresponding hardware thread while that thread has been in Wait mode.

**NOTE** This operation does not return the idle cycles that occur when the Hexagon processor shuts down (due to all of the hardware threads in idle). Idle cycle counts accumulate irrespective of whether profiling is enabled, and reset on qurt_profile_reset_idle_pcycles().

**Parameters**

| | | |
|---|---|---|
| `out` | *pcycles* | User array where the function stores the current idle cycle count values. Array size should be a minimum of the number of hardware threads intended. |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 24.7 qurt_profile_get_thread_pcycles()

### 24.7.1 Function Documentation

#### 24.7.1.1 unsigned long long int qurt_profile_get_thread_pcycles ( void )

Gets the count of the running processor cycles for the current thread.

Returns the current running processor cycle count for the current QuRT thread.

**NOTE** Profiling shall be enabled first to start the cycle count. The cycles accumulate after the profiling is enabled and resets on qurt_profile_reset_threadid_pcycles().

**Returns**

Integer – Running processor cycle count for the current thread.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 24.8 qurt_get_hthread_pcycles()

## 24.8.1 Function Documentation

### 24.8.1.1 unsigned int qurt_get_hthread_pcycles ( int *n* )

Reads the GCYCLE_nT register to allow performance measurement when N threads are in Run mode.

**NOTE** Returns 0 when the architecture is earlier than v67 or for an invalid hardware thread ID.

**Parameters**

| | | |
|---|---|---|
| in | *n* | Threads in run mode. Valid values are 1 through maximum hardware threads. |

**Returns**

Value read from the GCYCLE_nT register, which indicates the total number of pcycles that executed from reset to the current point of execution when n threads are in Run mode

**Safe API**

No

**Dependencies**

PMU must be enabled.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 24.9 qurt_get_hthread_commits()

### 24.9.1 Function Documentation

#### 24.9.1.1 unsigned int qurt_get_hthread_commits ( int *n* )

Reads the GCOMMIT_nT register to allow performance measurement when N threads are in Run mode.

**NOTE** Returns 0 when the architecture is earlier than v67 or for an invalid hardware thread ID.

**Parameters**

| | | |
|---|---|---|
| in | *n* | Threads in run mode. Valid values: 1 through maximum hardware threads. |

**Returns**

Value read from the GCOMMIT_nT register. This value indicates the total number of packets committed from reset to the current point of execution when n threads are in Run mode.

**Safe API**

No

**Dependencies**

PMU must be enabled.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 24.10 qurt_profile_get_threadid_pcycles()

### 24.10.1 Function Documentation

#### 24.10.1.1 void qurt_profile_get_threadid_pcycles ( int *thread_id,* unsigned long long ∗ *pcycles* )

Gets the running per-hardware-thread cycle counts for the specified QuRT thread for a maximum of six hardware threads.

Each count value represents the number of processor cycles that have elapsed on the corresponding hardware thread while that thread is scheduled for the specified QuRT thread.

**NOTE** Profiling shall be enabled first to start the cycle count. The cycles accumulate after profiling is enabled and reset on qurt_profile_reset_threadid_pcycles().

**Parameters**

| | | |
|---|---|---|
| in | *thread_id* | Valid thread identifier. |
| out | *pcycles* | Pointer to a user array where the function stores the running cycle count values. Array size should be a minimum of the intended number of hardware threads. |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 24.11 qurt_profile_reset_idle_pcycles()

### 24.11.1 Function Documentation

#### 24.11.1.1 void qurt_profile_reset_idle_pcycles ( void )

Sets the per-hardware-thread idle cycle counts to zero.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 24.12 qurt_profile_reset_threadid_pcycles()

### 24.12.1 Function Documentation

#### 24.12.1.1 void qurt_profile_reset_threadid_pcycles ( int *thread_id* )

Sets the per-hardware-thread running cycle counts to zero for the specified QuRT thread.

**Parameters**

| | | |
|---|---|---|
| in | *thread_id* | Thread identifier. |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 24.13 Data Types

Profiling related data types

### 24.13.1 Data Structure Documentation

#### 24.13.1.1 union qurt_profile_result_t

Profiling results.

**Data fields**

| Type | Parameter | Description |
|---|---|---|
| struct qurt_-profile_result_t | thread_ready_-time | Result associated with QURT_PROFILE_PARAM_THREAD_READY_TIME. |

#### 24.13.1.2 struct qurt_profile_result_t.thread_ready_time

Result associated with QURT_PROFILE_PARAM_THREAD_READY_TIME.

**Data fields**

| Type | Parameter | Description |
|---|---|---|
| unsigned int | ticks | Cumulative ticks the thread was ready. |

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 24.14 Constants and Macros

Profiling related constants and macros.

### 24.14.1 Define Documentation

#### 24.14.1.1 #define QURT_PROFILE_DISABLE 0

Disable profiling.

#### 24.14.1.2 #define QURT_PROFILE_ENABLE 1

Enable profiling.

#### 24.14.1.3 #define QURT_PROFILE_PARAM_THREAD_READY_TIME 0U

Values for qurt_profile_param_t. Uses ID's from 00h to 7Fh Profile thread ready time.

#### 24.14.1.4 #define QURT_PROFILE_REG_GROUP_CYCLE_NT 0x81U

Values for qurt_profile_reg_group_t. Uses ID's from 80h to FFh CYCLE_0T to CYCLE_nT.

#### 24.14.1.5 #define QURT_PROFILE_REG_GROUP_COMMIT_NT 0x82U

COMMIT_0T to COMMIT_nT.

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:40:42 GMT
hyungchul.won@lge.com

# 25 Device tree

## 25.1 Overview

An embedded platform has hardware components that can be discoverable (i.e., can be enumerated at runtime) like USB and non-discoverable (i.e., need to know what is connected to the rest of the system) like SPI. The challenge of describing this non-discoverable hardware can be resolved by either directly describing them in OS/boot loader code using C data structures like in Devcfg (now deprecated) or by using a Device Tree.

DeviceTree provides a mechanism to describe configuration data in a separate file that can be loaded and accessed (via APIs) at runtime.Configuration changes do not require to be re-build.

Advantages of DT:

- Industry standard specification for Linux, Zephyr and FreeBSD.
- Familiar syntax for developers and OEM's migrating from Linux
- Community support
- Binary compatibility

**NOTE:** For Device Tree feature to be enabled from QuRT the following config `CONFIG\_QURT\_DT` flag needs to be set.

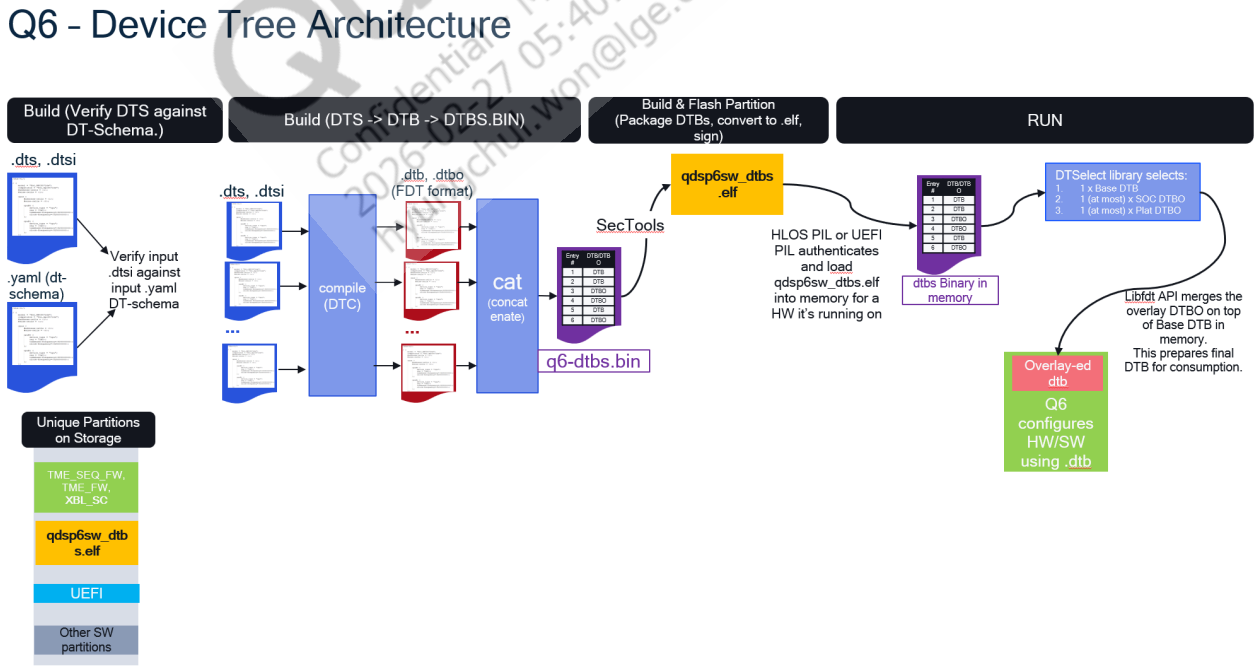


**Figure 25.1 Q6 DT Architecture**

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

### 25.1.1 QuRT Kernel Locating DT image

New 32-bit registers for passing the sw-defined boot parameters to the QDSP6 subsystem are proposed. They will be populated by TZ(ADSP/CDSP) or TME(MPSS). To accomodate 32 bits for OEM Variant ID, version 1 of boot params are introduced.

| Bits | | Version 0 | Bits | Version 1 |
|---|---|---|---|---|
| QDSP6SS_BOOT_PARAMS0 | | | | |
| 31:0 | DTB_PA_LOW | Low-order 32 bits of Physical Address where Device Tree image is loaded. | | Same as version 0 |
| QDSP6SS_BOOT_PARAMS1 | | | | |
| 31:0 | DTB_PA_HIGH | High-order 32 bits of Physical Address where Device Tree image is loaded. | 31:24 | Version Identifier (Bits 31:24, set to 1 for Version 1), |
| | | | 23:0 | High 24 bits of Physical Address where DT is loaded. |
| QDSP6SS_BOOT_PARAMS2 | | | | |
| 31:16 | CHIP_FAMILY | Chip Family field, as defined by ChipInfo driver | | Same as version 0 |
| 15:0 | CHIP_ID | Chip ID field, as defined by ChipInfo driver | | Same as version 0 |
| QDSP6SS_BOOT_PARAMS3 | | | | |
| 31:24 | RESERVED | Reserved for future use; Loader shall set to 0 | 31:24 | Low 16 bits of OEM Variant ID |
| 31:16 | OEM Variant ID | | 31:16 | |
| 15:8 | CHIP_VER_MAJ | Chip Major version, as defined by ChipInfo driver | | Same as version 0 |
| 7:0 | CHIP_VER_MIN | Chip Minor version, as defined by ChipInfo driver | | Same as version 0 |
| QDSP6SS_BOOT_PARAMS4 | | | | |
| 31:24 | PLAT_TYPE | Platform Type, as defined by PlatformInfo driver | | Same as version 0 |
| 23:16 | PLAT_SUBTYPE | Platform SubType, as defined by PlatformInfo driver | | Same as version 0 |
| 15:8 | PLAT_VER_MAJ | Platform Major version number, as defined by PlatformInfo driver | | Same as version 0 |
| 7:0 | PLAT_VER_MIN | Platform Minor version number, as defined by PlatformInfo driver | | Same as version 0 |
| QDSP6SS_BOOT_PARAMS5 | | | | |
| 31:0 | | Size of Device Tree Image in Bytes | 31:16 | High 16 bits of OEM Variant ID |
| | | | 15:0 | Size of Device Tree Image in 4kb |

**Figure 25.2 Q6_BOOT_PARAMS**

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungsul.won@lge.com

### 25.1.2 Read the Registers

**NOTE:** Bits are numbered from 31 (most significant) to 0 (least significant). Fields are 32 bits wide, unless otherwise specified.

The CFGBASE register points to the Hexagon Subsystem Config Table. All addresses are right shifted by 16 when stored.

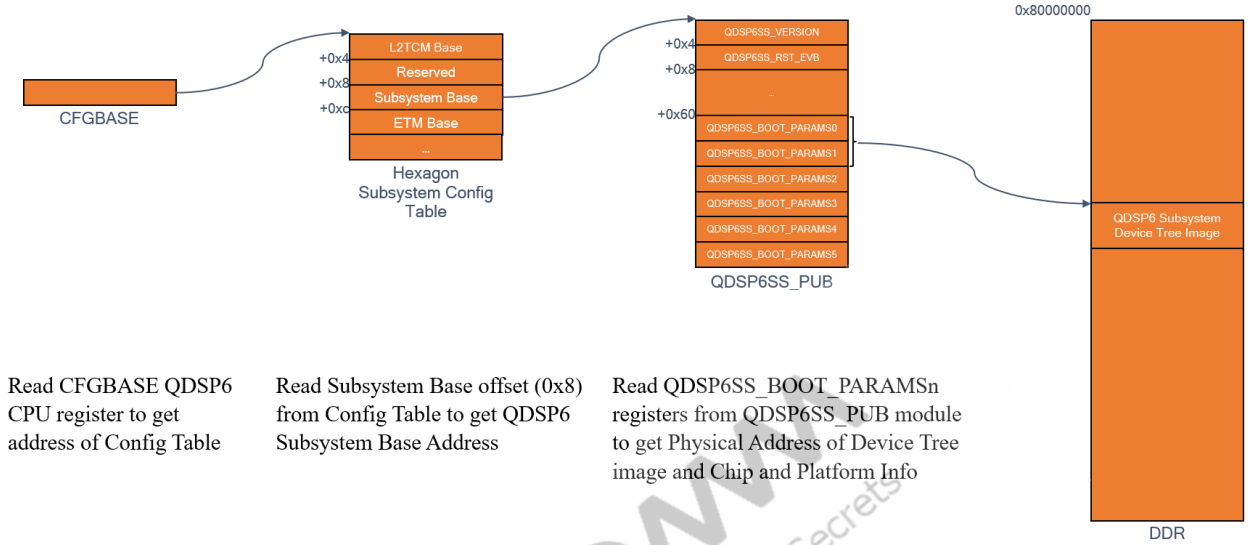


**Figure 25.3 QuRT Kernel Locating DT image**

### 25.1.3 QDI and Device Tree

Leverage Device Tree Principles into QDI to create a more complete driver model by dynamically tying QDI drivers and DT devices to simplify initialization.

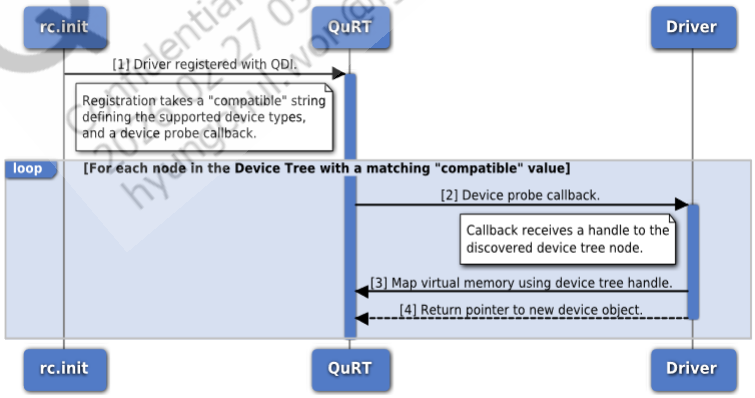


**Figure 25-4 DT_QDI_Driver_Model**

The QDI extension scons method is augmented with the below keyword parameters to add device tree aware QDI registration via scons. Omitting the parameters will result in the original functionality

```
env.QurtQdiRegister(          "duck",                                  ->Name
    of the driver
```

```
                                        "duck_create",                      ->
                                           Drivers create function
                                        "duck_invocation",                  ->
                                           Drivers invocation function
                                        "duck_destroy",                     ->
                                           Drivers destroy function
                                        target="duck_qdi_init.c",           ->
                                           Auto-generated file name
                                        autogen_func_name = "duck_init",    ->
                                           Name of init function
                                        rcinit_dep_list= ['timer'],         ->
                                           RCINIT dependency list
                                        singleton=2,                        ->
                                           Singleton per process
                                        island_capable=1,                   ->
                                           Island mode driver
                                        probe="duck_probe",                 ->
                                           function to probe the device
                                        compatible="qcom,duck",             ->
                                           Compatibe parameter to bind device &
                                           driver
                                        do_rcinit = True                    -> Do
                                           RCINIT for duck driver automatically
                                    )
```

**NOTE:** We have started supporting multiple-compatible strings where a driver can give a list of devices it can bind with instead of a single string. This helps clients with an overhead of registering driver again for a new device

Example:

```
compatible={[}"device1","device2","device3"{]}
```

**Figure 25.5 Multi compatible strings support**

**NOTE:** It is also possible to register a driver directly with QDI using qurt_qdi_dt_register instead of via SCons, in which case the compatible and probe attributes must be included in the qurt_qdi_obj_t opener object and AddRCInitFunc should be used to register the driver initialization function.

- compatible: Device tree compatible string which will be used to identify compatible devices for initialization. function specified with the above parameter will get called for each

- create_device: Specialized opener for devices, will additionally pass back device tree node associated with device, see more information in QDI Enhancement doc
- driverSafeName: String used in symbols and filenames (if not specified by 'target'). As of writing symbols and files include, file generated by this build (if target not specified), the device structure, the set singleton function generated as part of the builder and the init function also generated as part of the builder. If this value is not specified it will default to the device name, with leading slashes removed and converted to underscores.

### 25.1.4 Device Tree Overlay

QuRT provides device tree overlay support from kernel version 4.4.3 to modify kernel's live tree, and have the modification affecting the state of the kernel in a way that is reflecting the changes. Since the kernel mainly deals with devices, any new device node that result in an active device should have it created while if the device node is either disabled or removed all together, the affected device should be deregistered.

In the below example you can verify if an overlay is applied using QDTE.

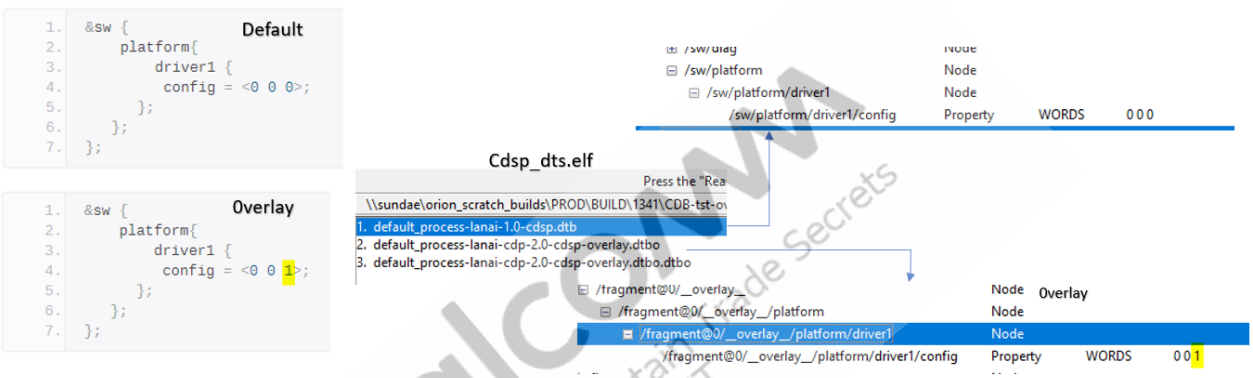


**Figure 25.6 DT_Overlay_Example**

### 25.1.5 Supported API

- `qurt_dt_blob_id_get`: Returns the Blob ID for the Blob type passed
- `qurt_dt_isr_register`: Device tree aware registration of an interrupt service routine (ISR) to an ISR thread.
- `qurt_dt_mapping_create` and `qurt_dt_mapping_create2`: Creates a memory mapping from the specified property of the specified device tree node.

### 25.1.6 FAQ's

**Few general things to check while enabling DT**

1. Firstly check if chipinfo and platforminfo are updated properly else it would result in loading on incorrect blob ID (i.e. DEFAULT_BLOB_ID in most cases)
2. Check q6dtb.elf is being generated or not
3. Check q6dtb.elf file is included in contents.xml file so the it gets included in NIN-HLOS.bin file
4. Check TZ/TME has DTB support and is able to authenticate the dtb firmware. Else it would result in XPU violation.
5. Check if QURT is able to access the DTB blob.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.wan@lge.com

### Functions

DT services are accessed with the following QuRT functions.

- qurt_devtree_init()
- qurt_dt_mapping_create()
- qurt_dt_isr_register()
- qurt_dt_blob_id_get()
- Constants and Macros

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 25.2 qurt_devtree_init()

### 25.2.1 Function Documentation

#### 25.2.1.1 int qurt_devtree_init ( void )

Device tree initialization function which retreives the blob and then set the handle. It infers the boot params register's and correspondinly provide handle to dtb for setting the configuration.

**Returns**

QURT_EOK – Successfully initialized DT
QURT_EFAILED – Failure in DT initialization

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 25.3 qurt_dt_mapping_create()

### 25.3.1 Function Documentation

#### 25.3.1.1 int qurt_dt_mapping_create ( fdt_node_handle ∗ *devtreeNode,* int *flags,* char ∗ *regionName,* int *regionIdx,* unsigned long long ∗ *vaddr,* unsigned long long ∗ *size* )

Creates a memory mapping from the specified property of the specified device tree node. Returns virtual addresses and sizes.

**Parameters**

| | | |
|---|---|---|
| in | *devtreeNode* | Device tree node offset. |
| in | *flags* | Flags to configure memory. Overloaded as property index when reg_name is NULL. |
| in | *regionName* | Identifies the property to use for mapping, which should resemble a region. |
| in | *regionIdx* | Region index for mapping. |
| out | *vaddr* | Return pointer for the virtual region address. |
| out | *size* | Return pointer for the virtual region size. |

**Returns**

Result code to indicate success or failure.

**Safe API**

No

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 25.4 qurt_dt_isr_register()

### 25.4.1 Function Documentation

#### 25.4.1.1 int qurt_dt_isr_register ( fdt_node_handle ∗ *dt_node,* int *dt_int_index,* char ∗ *dt_int_name,* qurt_thread_t *isr_thread_id,* unsigned short *prio,* unsigned short *flags,* void(∗)(void ∗, int) *isr,* void ∗ *arg* )

Device tree aware registration of an interrupt service routine (ISR) to an ISR thread. The interrupt defined in the specified device tree node is enabled when this function returns success.

**Associated data types**

qurt_thread_t
#fdt_node_handle

**Parameters**

| | | |
|---|---|---|
| in | *dt_node* | Device tree node that specifies the interrupt property. |
| in | *dt_int_index* | Index of the specific interrupt to use within the device tree node structure. Specify either dt_int_index or int_name and use -1 if string is used. |
| in | *dt_int_name* | Name of the specific interrupt to use within the device tree node structure. Specify either dt_int_name or int_index and use NULL when index is used. |
| in | *isr_thread_id* | ISR thread ID returned from qurt_isr_create(), defined by qurt_isr_register2(). |
| in | *prio* | Priority of the ISR defined by qurt_isr_register2(). |
| in | *flags* | Defines the ACK type. Values :<br>QURT_INT_NON_DELAYED_ACK – ISR is acknowledged by the interrupt handle routine in the kernel.<br>QURT_INT_DELAYED_ACK – Client chooses to acknowledge. Defined by qurt_isr_register2(). |
| in | *isr* | ISR with the prototype void isr (void ∗arg, int int_num), defined by qurt_isr_register2(). |
| in | *arg* | First argument of the ISR when it is called to service the interrupt, defined by qurt_isr_register2(). |

**Returns**

QURT_EOK – Successfully registered the ISR for the interrupt
QURT_EINT – Interrupt not configured
QURT_EINVALID – Invalid thread ID
QURT_EDISABLED – Feature is disabled
QURT_EDUPLICATE – Interrupt is already registered

**Safe API**

No

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 15:42:41 hyungchul@qualcomm.com

### Dependencies

Create the thread ID qurt_isr_create(). ISR registration completed with qurt_isr_register2().

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:40:42 GMT
hyungchul.won@lge.com

## 25.5 qurt_dt_blob_id_get()

### 25.5.1 Function Documentation

#### 25.5.1.1 int qurt_dt_blob_id_get ( unsigned int *blob_type* )

Returns the blob ID for the blob type passed. The value returned from this API can be passed as a blob ID parameter to DTBExtnLib APIs.

**Parameters**

| | | |
|---|---|---|
| in | *blob_type* | Blob type to look up. |

**Returns**

Blob ID for the passed blob type.

**Safe API**

No

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 25.6 Constants and Macros

This section describes constants and macros for DT services.

### 25.6.1 Define Documentation

#### 25.6.1.1 #define DEFAULT_BLOB_ID 0

Device Tree Blob Types

#### 25.6.1.2 #define QURT_DT_MAPPING_FAILED (-1)

QURT device tree mapping macros.

#### 25.6.1.3 #define QURT_DT_BLOB_TYPE_ROOT 0

Device tree type for the root PD device tree. The root PD device tree typically describes the hardware in the subsystem. This is the /soc portion of the device tree.

#### 25.6.1.4 #define QURT_DT_BLOB_TYPE_LOCAL 1

Device tree type for the local device tree. The local device tree typically contains the software settings. This is the /sw portion of the device tree.

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:40:42 GMT
hyungchul.won@lge.com

# 26 Performance Monitor

## 26.1 Overview

Threads use the performance monitor to measure code performance in real time during application execution.

The performance monitor unit (PMU) is a hardware feature in the Hexagon processor. It is controlled by accessing a set of dedicated processor registers. For more information refer to [80-VB419-92].

The performance monitor is controlled in QuRT with the following operations:

- Enable performance monitor – Enable the performance monitor unit.
- Get PMU register – Get the current value of the specified PMU register.
- Set PMU register – Set the value of the specified PMU register.

**Functions**
Performance monitor services are accessed with the following QuRT functions.

- qurt_pmu_enable()
- qurt_pmu_get()
- qurt_pmu_set()
- qurt_pmu_get_pmucnt()
- Constants and Macros

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 26.2 qurt_pmu_enable()

### 26.2.1 Function Documentation

#### 26.2.1.1 void qurt_pmu_enable ( int *enable* )

Enables or disables the Hexagon processor PMU. By default, profiling is disabled.

**NOTE** On PMU disable, all the configuration registers(PMUCFG, PMUEVTCFGx, PMUSTIDx) get reset. PMUCNTx registers don't get reset. So make sure to reconfigure all the PMU configuration registers before enabling PMU again. On explicit writing to PMUEVTCFGx registers, QuRT resets the respective PMUCNTx registers.

**Parameters**

| | | |
|---|---|---|
| in | *enable* | Performance monitor. Values:<br>• 0 – Disable performance monitor<br>• 1 – Enable performance monitor |

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 26.3 qurt_pmu_get()

### 26.3.1 Function Documentation

#### 26.3.1.1 unsigned int qurt_pmu_get ( int *reg_id* )

Gets the PMU register.

Returns the current value of the specified PMU register.

**Parameters**

| | | |
|---|---|---|
| in | *reg_id* | PMU register. Values:<br>• QURT_PMUCNT0 to QURT_PMUCNT7<br>• QURT_PMUCFG<br>• QURT_PMUEVTCFG<br>• QURT_PMUEVTCFG1<br>• QURT_PMUSTID0 to QURT_PMUSTID1 – Reads the entire PMUSTIDx register<br>• QURT_PMUCNTSTID0 to QURT_PMUCNTSTID7 – Reads STID match for the respective counter |

**Returns**

Integer – Current value of the specified PMU register.

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 26.4 qurt_pmu_set()

### 26.4.1 Function Documentation

#### 26.4.1.1 void qurt_pmu_set ( int *reg_id,* unsigned int *reg_value* )

Sets the value of the specified PMU register.

**NOTE** Set PMUEVTCFG to automatically clear the PMU registers PMUCNT0 through PMUCNT3. Similarly, writes to PMUEVTCFG1 clears PMUCNT4 through PMUCNT7

**Parameters**

| | | |
|---|---|---|
| in | *reg_id* | PMU register. Values:<br>• QURT_PMUCNT0 to QURT_PMUCNT7<br>• QURT_PMUCFG<br>• QURT_PMUEVTCFG<br>• QURT_PMUEVTCFG1<br>• QURT_PMUSTID0 & QURT_PMUSTID1 – Writes to the entire PMUSTIDx register<br>• QURT_PMUCNTSTID0 to QURT_PMUCNTSTID7 – Writes to the STID match for the respective counter |
| in | *reg_value* | Register value. |

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 26.5 qurt_pmu_get_pmucnt()

### 26.5.1 Function Documentation

#### 26.5.1.1 int qurt_pmu_get_pmucnt ( void * *buf* )

Reads PMU counters in a single trap.

**Parameters**

| | | |
|---|---|---|
| `out` | *buf* | Pointer to a buffer to save values read from PMU counters. Buffer size should be at least 32 bytes to read all eight PMU counters. |

**Returns**

QURT_EOK – Successful read.
QURT_EFATAL – Failure.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 26.6 Constants and Macros

This section describes constants for performance monitor services.

### 26.6.1 Define Documentation

#### 26.6.1.1 #define QURT_PMUCNT0 0

#### 26.6.1.2 #define QURT_PMUCNT1 1

#### 26.6.1.3 #define QURT_PMUCNT2 2

#### 26.6.1.4 #define QURT_PMUCNT3 3

#### 26.6.1.5 #define QURT_PMUCFG 4

#### 26.6.1.6 #define QURT_PMUEVTCFG 5

#### 26.6.1.7 #define QURT_PMUCNT4 6

#### 26.6.1.8 #define QURT_PMUCNT5 7

#### 26.6.1.9 #define QURT_PMUCNT6 8

#### 26.6.1.10 #define QURT_PMUCNT7 9

#### 26.6.1.11 #define QURT_PMUEVTCFG1 10

#### 26.6.1.12 #define QURT_PMUSTID0 11

#### 26.6.1.13 #define QURT_PMUSTID1 12

#### 26.6.1.14 #define QURT_PMUCNTSTID0 13

#### 26.6.1.15 #define QURT_PMUCNTSTID1 14

#### 26.6.1.16 #define QURT_PMUCNTSTID2 15

#### 26.6.1.17 #define QURT_PMUCNTSTID3 16

#### 26.6.1.18 #define QURT_PMUCNTSTID4 17

#### 26.6.1.19 #define QURT_PMUCNTSTID5 18

#### 26.6.1.20 #define QURT_PMUCNTSTID6 19

#### 26.6.1.21 #define QURT_PMUCNTSTID7 20

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:10:42 GMT hyunchul.wo@lge.com

# 27 Error Results

QuRT functions return error results in one of two ways:

- As function result values
- As values passed to the user-defined exception handler
  QuRT defines a set of standard symbols for the error result values. This section lists the symbols and their corresponding values.

## 27.0.1 Define Documentation

### 27.0.1.1 #define QURT_EOK 0

Operation successfully performed.

### 27.0.1.2 #define QURT_EVAL 1

Wrong values for the parameters. The specified page does not exist.

### 27.0.1.3 #define QURT_EMEM 2

Not enough memory to perform the operation.

### 27.0.1.4 #define QURT_EINVALID 4

Invalid argument value; invalid key.

### 27.0.1.5 #define QURT_EFAILED 12

Operation failed.

### 27.0.1.6 #define QURT_ENOTALLOWED 13

Operation not allowed.

### 27.0.1.7 #define QURT_ENOREGISTERED 20

No registered interrupts.

### 27.0.1.8 #define QURT_ETLSAVAIL 23

No free TLS key is available.

### 27.0.1.9 #define QURT_ETLSENTRY 24

TLS key is not already free.

Qualcomm Confidential - May Contain Trade Secrets 2026-03-27 05:40:42 GMT hyungchowon@lge.com

#### 27.0.1.10 #define QURT_EINT 26

Invalid interrupt number (not registered).

#### 27.0.1.11 #define QURT_ESIG 27

Invalid signal bitmask (cannot set more than one signal at a time).

#### 27.0.1.12 #define QURT_ENOTHREAD 30

Thread no longer exists.

#### 27.0.1.13 #define QURT_EALIGN 32

Not aligned.

#### 27.0.1.14 #define QURT_EDEREGISTERED 33

Interrupt is already deregistered.

#### 27.0.1.15 #define QURT_EEXISTS 35

File or message queue already exists.

#### 27.0.1.16 #define QURT_ENAMETOOLONG 36

Name too long for message queue creation.

#### 27.0.1.17 #define QURT_EPRIVILEGE 36

Caller does not have privilege for this operation.

#### 27.0.1.18 #define QURT_ECANCEL 37

A cancellable request was canceled because the associated process was asked to exit.

#### 27.0.1.19 #define QURT_EISLANDUSEREXIT 44

User call has resulted in island exit.

#### 27.0.1.20 #define QURT_ENOISLANDENTRY 45

Island mode had not yet been entered.

#### 27.0.1.21 #define QURT_EISLANDINVALIDINT 46

Exited Island mode due to an invalid island interrupt.

#### 27.0.1.22 #define QURT_ETIMEDOUT 47

Operation timed-out.

#### 27.0.1.23 #define QURT_EALREADY 48

Operation already in progress.

#### 27.0.1.24 #define QURT_ERETRY 49

Retry the operation.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:10:42 GMT hsungchunwon@lg.com

#### 27.0.1.25 #define QURT_EDISABLED 50

Resource disabled.

#### 27.0.1.26 #define QURT_EDUPLICATE 51

Duplicate resource.

#### 27.0.1.27 #define QURT_EBADR 53

Invalid request descriptor.

#### 27.0.1.28 #define QURT_ETLB 54

Exceeded maximum allowed TLBs.

#### 27.0.1.29 #define QURT_ENOTSUPPORTED 55

Operation not supported.

#### 27.0.1.30 #define QURT_ENORESOURCE 56

No resource.

#### 27.0.1.31 #define QURT_EDTINIT 57

Problem with device tree intialization.

#### 27.0.1.32 #define QURT_EBUFLOCK 58

Buffer lock failed because it was already locked many times.

#### 27.0.1.33 #define QURT_ELOCKED 59

Current operation failed as the buffer is locked.

#### 27.0.1.34 #define QURT_ECLOSED 60

object on which client is waiting, have closed

#### 27.0.1.35 #define QURT_EMSGSIZE 90

Message queue msg_len is greater than mq_msgsize attribute of the message queue.

#### 27.0.1.36 #define QURT_ENOTCONFIGURED 91

Interrupt is not configured.

#### 27.0.1.37 #define QURT_EBANDWIDTHLIMIT 92

Message queue send exceeds the bandwidth limit.

#### 27.0.1.38 #define QURT_ECFIVIOLATION 93

CFI violation detected.

#### 27.0.1.39 #define QURT_EDESTROY 94

Destroy request was made to waiting threads.

Qualcomm Confidential - May Contain Trade Secrets 2026-03-27 05:10:42 GMT hungchunwon@lge.com

#### 27.0.1.40 #define QURT_EHMXNOTAVAIL 95

HMX is not available to the target thread.

#### 27.0.1.41 #define QURT_ETIMERADJUST 97

Timer list adjustment has to be done

#### 27.0.1.42 #define QURT_EFATAL -1

Fatal error.

#### 27.0.1.43 #define QURT_EXCEPT_PRECISE 0x01U

Precise exception occurred. For this cause code, Cause2 is SSR[7:0].

#### 27.0.1.44 #define QURT_EXCEPT_NMI 0x02U

NMI occurred; Cause2 is not defined.

#### 27.0.1.45 #define QURT_EXCEPT_TLBMISS 0x03U

TLBMISS RW occurred; for this cause code, Cause2 is SSR[7:0].

#### 27.0.1.46 #define QURT_EXCEPT_RSVD_VECTOR 0x04U

Interrupt raised on a reserved vector, which must never occur. Cause2 is not defined.

#### 27.0.1.47 #define QURT_EXCEPT_ASSERT 0x05U

Kernel assert. Cause2 QURT_ABORT_∗ are listed below.

#### 27.0.1.48 #define QURT_EXCEPT_BADTRAP 0x06U

trap0(num) called with unsupported num. Cause2 is 0.

#### 27.0.1.49 #define QURT_EXCEPT_UNDEF_TRAP1 0x07U

Trap1 is not supported. Use of Trap1 causes this error. Cause2 is not defined.

#### 27.0.1.50 #define QURT_EXCEPT_EXIT 0x08U

Application called qurt_exit() or qurt_exception_raise_nonfatal(). Can be called from C library. Cause2 is "[Argument passed to qurt_exception_raise_nonfatal() & 0xFF]".

#### 27.0.1.51 #define QURT_EXCEPT_TLBMISS_X 0x0AU

TLBMISS X (execution) occurred. Cause2 is not defined.

#### 27.0.1.52 #define QURT_EXCEPT_STOPPED 0x0BU

Running thread stopped due to fatal error on other hardware thread. Cause2 is not defined.

#### 27.0.1.53 #define QURT_EXCEPT_FATAL_EXIT 0x0CU

Application called qurt_fatal_exit(). Cause2 is not defined.

Qualcomm Confidential - May Contain Trade Secrets 2021-02-27 05:10:42 GMT hyungsoul.kim@lg.com

#### 27.0.1.54 #define QURT_EXCEPT_INVALID_INT 0x0DU

Kernel received an invalid L1 interrupt. Cause2 is not defined.

#### 27.0.1.55 #define QURT_EXCEPT_FLOATING_POINT 0x0EU

Kernel received a floating point error. Cause2 is not defined.

#### 27.0.1.56 #define QURT_EXCEPT_DBG_SINGLE_STEP 0x0FU

Cause2 is not defined.

#### 27.0.1.57 #define QURT_EXCEPT_TLBMISS_RW_ISLAND 0x10U

Read write miss in Island mode. Cause2 QURT_TLB_MISS_RW_MEM∗ are listed below.

#### 27.0.1.58 #define QURT_EXCEPT_TLBMISS_X_ISLAND 0x11U

Execute miss in Island mode. For this cause code, Cause2 is SSR[7:0].

#### 27.0.1.59 #define QURT_EXCEPT_SYNTHETIC_FAULT 0x12U

Synthetic fault with user request that kernel detected. Cause2 QURT_SYNTH_∗ are listed below.

#### 27.0.1.60 #define QURT_EXCEPT_INVALID_ISLAND_TRAP 0x13U

Invalid trap in Island mode. Cause2 is trap number.

#### 27.0.1.61 #define QURT_EXCEPT_UNDEF_TRAP0 0x14U

trap0(num) was called with an unsupported number. Cause2 is trap number.

#### 27.0.1.62 #define QURT_EXCEPT_PRECISE_DMA_ERROR 0x28U

Precise DMA error. Cause2 is DM4[15:8]. Badva is DM5 register.

#### 27.0.1.63 #define QURT_EXCEPT_KERNEL_STACK_CORRUPT 0x29U

Kernel stack corrupted during a trap0 handling

#### 27.0.1.64 #define QURT_ECODE_UPPER_LIBC (0U << 16)

Upper 16 bits is zero for libc.

#### 27.0.1.65 #define QURT_ECODE_UPPER_QURT (0U << 16)

Upper 16 bits is zero for QuRT.

#### 27.0.1.66 #define QURT_ECODE_UPPER_ERR_SERVICES (2U << 16)

Upper 16 bits is two for error service.

#### 27.0.1.67 #define QURT_SYNTH_ERR 0x01U

#### 27.0.1.68 #define QURT_SYNTH_INVALID_OP 0x02U

#### 27.0.1.69 #define QURT_SYNTH_DATA_ALIGNMENT_FAULT 0x03U

Qualcomm Confidential - May Contain Trade Secrets 2026-03-27 05:19:42 GMT hungchunwon@lg.com

#### 27.0.1.70 #define QURT_SYNTH_FUTEX_INUSE 0x04U

#### 27.0.1.71 #define QURT_SYNTH_FUTEX_BOGUS 0x05U

#### 27.0.1.72 #define QURT_SYNTH_FUTEX_ISLAND 0x06U

#### 27.0.1.73 #define QURT_SYNTH_FUTEX_DESTROYED 0x07U

#### 27.0.1.74 #define QURT_SYNTH_PRIVILEGE_ERR 0x08U

#### 27.0.1.75 #define QURT_ABORT_FUTEX_WAKE_MULTIPLE 0x01U

Abort cause – futex wake multiple.

#### 27.0.1.76 #define QURT_ABORT_WAIT_WAKEUP_SINGLE_MODE 0x02U

Abort cause – thread waits to wake up in Single Threaded mode.

#### 27.0.1.77 #define QURT_ABORT_TCXO_SHUTDOWN_NOEXIT 0x03U

Abort cause – call TCXO shutdown without exit.

#### 27.0.1.78 #define QURT_ABORT_FUTEX_ALLOC_QUEUE_FAIL 0x04U

Abort cause – futex allocation queue failure - QURTK_futexhash_lifo empty.

#### 27.0.1.79 #define QURT_ABORT_INVALID_CALL_QURTK_WARM_INIT 0x05U

Abort cause – invalid QURTK_warm_init() call in NONE CONFIG_POWER_MGMT mode.

#### 27.0.1.80 #define QURT_ABORT_THREAD_SCHEDULE_SANITY 0x06U

Abort cause – sanity schedule thread is not supposed to run on the current hardware thread.

#### 27.0.1.81 #define QURT_ABORT_REMAP 0x07U

Remap in the page table; the correct behavior must remove mapping if necessary.

#### 27.0.1.82 #define QURT_ABORT_NOMAP 0x08U

No mapping in page table when removing a user mapping.

#### 27.0.1.83 #define QURT_ABORT_INVALID_MEM_MAPPING_TYPE 0x0AU

Invalid memory mapping type when creating qmemory.

#### 27.0.1.84 #define QURT_ABORT_NOPOOL 0x0BU

No pool available to attach.

#### 27.0.1.85 #define QURT_ABORT_LIFO_REMOVE_NON_EXIST_ITEM 0x0CU

Cannot allocate more futex waiting queue.

#### 27.0.1.86 #define QURT_ABORT_ASSERT 0x0EU

Assert abort.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 09:40:12 GMT hyunchul.wu@qce.com

#### 27.0.1.87 #define QURT_ABORT_FATAL 0x0FU

Fatal error; must never occur.

#### 27.0.1.88 #define QURT_ABORT_FUTEX_RESUME_INVALID_QUEUE 0x10U

Abort cause – invalid queue ID in futex resume.

#### 27.0.1.89 #define QURT_ABORT_FUTEX_WAIT_INVALID_QUEUE 0x11U

Abort cause – invalid queue ID in futex wait.

#### 27.0.1.90 #define QURT_ABORT_FUTEX_RESUME_INVALID_FUTEX 0x12U

Abort cause – invalid futex object in hashtable.

#### 27.0.1.91 #define QURT_ABORT_NO_ERHNDLR 0x13U

No registered error handler.

#### 27.0.1.92 #define QURT_ABORT_ERR_REAPER 0x14U

Exception in the reaper thread.

#### 27.0.1.93 #define QURT_ABORT_FREEZE_UNKNOWN_CAUSE 0x15U

Abort in thread freeze operation.

#### 27.0.1.94 #define QURT_ABORT_FUTEX_WAIT_WRITE_FAILURE 0x16U

During futex wait processing, cannot perform a necessary write operation to userland data; most likely due to a DLPager eviction.

#### 27.0.1.95 #define QURT_ABORT_ERR_ISLAND_EXP_HANDLER 0x17U

Exception in Island exception handler task.

#### 27.0.1.96 #define QURT_ABORT_L2_TAG_DATA_CHECK_FAIL 0x18U

Detected error in L2 tag/data during warm boot. The L2 tag/data check is done when CONFIG_DEBUG_L2_POWER_COLLAPSE is enabled.

#### 27.0.1.97 #define QURT_ABORT_ERR_SECURE_PROCESS 0x19U

Abort error in secure process.

#### 27.0.1.98 #define QURT_ABORT_ERR_EXP_HANDLER 0x20U

No exception handler, or the handler caused an exception.

#### 27.0.1.99 #define QURT_ABORT_ERR_NO_PCB 0x21U

PCB of the thread context failed initialization, PCB was NULL.

#### 27.0.1.100 #define QURT_ABORT_NO_PHYS_ADDR 0x22U

Unable to find the physical address for the virtual address.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:46:42 GMT hyunchul.wan@qti.com

#### 27.0.1.101 #define QURT_ABORT_OUT_OF_FASTINT_CONTEXTS 0x23U

Fast interrupt contexts exhausted.

#### 27.0.1.102 #define QURT_ABORT_CLADE_ERR 0x24U

Fatal error seen with CLADE interrupt.

#### 27.0.1.103 #define QURT_ABORT_ETM_ERR 0x25U

Fatal error seen with ETM interrupt.

#### 27.0.1.104 #define QURT_ABORT_ECC_DED_ASSERT 0x26U

ECC two-bit DED error.

#### 27.0.1.105 #define QURT_ABORT_VTLB_ERR 0x27U

Fatal error in the VTLB layer.

#### 27.0.1.106 #define QURT_ABORT_TLB_ENCODE_DECODE_FAILURE 0x28U

Failure during the TLB encode or decode operation.

#### 27.0.1.107 #define QURT_ABORT_VTLB_WALKOBJS_BOUND_FAILURE 0x29U

Failure to lookup entry in the page table.

#### 27.0.1.108 #define QURT_ABORT_PHY_MEMORY_OWNERSHIP_FAILURE 0x30U

Failure to claim PHY memory ownership.

#### 27.0.1.109 #define QURT_ABORT_JTLB_SIZE_CHECK_FAIL 0x31U

Configured JTLB size is more than the actual size in hardware

#### 27.0.1.110 #define QURT_ABORT_AUTOSTACK_ASSERT 0x32U

Error while handling stack flimit exception.

#### 27.0.1.111 #define QURT_TLB_MISS_X_FETCH_PC_PAGE 0x60U

#### 27.0.1.112 #define QURT_TLB_MISS_X_2ND_PAGE 0x61U

#### 27.0.1.113 #define QURT_TLB_MISS_X_ICINVA 0x62U

#### 27.0.1.114 #define QURT_TLB_MISS_RW_MEM_READ 0x70U

#### 27.0.1.115 #define QURT_TLB_MISS_RW_MEM_WRITE 0x71U

#### 27.0.1.116 #define QURT_FLOATING_POINT_EXEC_ERR 0xBFU

Execute floating-point.

#### 27.0.1.117 #define QURT_AUTOSTACKV2_CANARY_NOT_MATCH 0xC1U

Cause2 - autostackv2 - 8 bits

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:20:42 GMT kwunchowong@lg.com

#### 27.0.1.118 #define QURT_CFI_VIOLATION 0xC3U

Cause2 - CFI violation - 8 bits

#### 27.0.1.119 #define QURT_FP_EXCEPTION_ALL 0x1FU << 25

#### 27.0.1.120 #define QURT_FP_EXCEPTION_INEXACT 0x1U << 29

#### 27.0.1.121 #define QURT_FP_EXCEPTION_UNDERFLOW 0x1U << 28

#### 27.0.1.122 #define QURT_FP_EXCEPTION_OVERFLOW 0x1U << 27

#### 27.0.1.123 #define QURT_FP_EXCEPTION_DIVIDE0 0x1U << 26

#### 27.0.1.124 #define QURT_FP_EXCEPTION_INVALID 0x1U << 25

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:40:42 GMT
hyungchul.won@lge.com

# 28 Function Tracing

QuRT supports function tracing to assist in debugging applications.

- qurt_trace_changed()
- qurt_trace_get_marker()
- qurt_etm_set_pc_range()
- qurt_etm_set_range()
- qurt_etm_set_atb()
- qurt_etm_set_sync_period()
- qurt_stm_trace_set_config()
- Data Types
- Macros

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:40:42 GMT
hyungchul.won@lge.com

# 28.1 qurt_trace_changed()

## 28.1.1 Function Documentation

### 28.1.1.1 int qurt_trace_changed ( unsigned int *prev_trace_marker,* unsigned int *trace_mask* )

Determines whether specific kernel events have occurred.

Returns a value that indicates whether the specified kernel events are recorded in the kernel trace buffer since the specified kernel trace marker was obtained.

The prev_trace_marker parameter specifies a kernel trace marker that was obtained via a call to qurt_trace_get_marker(). For more information on the mask value, see the description of the trace_mask element in 80-VB419-92.

**NOTE** Used with qurt_trace_get_marker(), this function determines whether certain kernel events occurred in a block of code.

This function cannot determine whether a specific kernel event type has occurred unless that event type has been enabled in the trace_mask element of the system configuration file.

QuRT supports the recording of interrupt and context switch events only (such as a trace_mask value of 0x3).

#### Parameters

| | | |
|---|---|---|
| in | *prev_trace_marker* | Previous kernel trace marker. |
| in | *trace_mask* | Mask value that indicates which kernel events to check for. |

#### Returns

1 – Kernel events of the specified type have occurred since the specified trace marker was obtained.
0 – No kernel events of the specified type have occurred since the specified trace marker was obtained.

#### Safe API

No

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lg.com

## 28.2 qurt_trace_get_marker()

### 28.2.1 Function Documentation

#### 28.2.1.1 unsigned int qurt_trace_get_marker ( void )

Gets the kernel trace marker.

Returns the current value of the kernel trace marker. The marker consists of a hardware thread identifier and an index into the kernel trace buffer. The trace buffer records kernel events.

**NOTE** Use this function with qurt_trace_changed() to determine whether certain kernel events occurred in a block of code.

**Returns**

Integer – Kernel trace marker.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 28.3 qurt_etm_set_pc_range()

### 28.3.1 Function Documentation

#### 28.3.1.1 unsigned int qurt_etm_set_pc_range ( unsigned int *range_num,* unsigned int *low_addr,* unsigned int *high_addr* )

Sets the PC address range for ETM filtering. Depending on the Hexagon core design, a maximum of four PC ranges are supported.

**Parameters**

| | | |
|---|---|---|
| in | *range_num* | 0 to 3. |
| in | *low_addr* | Lower boundary of PC address range. |
| in | *high_addr* | Higher boundary of PC address range. |

**Returns**

QURT_ETM_SETUP_OK – Success.
QURT_ETM_SETUP_ERR – Failure.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 28.4 qurt_etm_set_range()

### 28.4.1 Function Documentation

#### 28.4.1.1 unsigned int qurt_etm_set_range ( unsigned int *addr_source_type,* unsigned int *trig_block_num,* unsigned int *pid,* unsigned int *low_addr,* unsigned int *high_addr* )

Sets the address range for ETM filtering. Allows the user to select the source type of addresses; QURT_ETM_SOURCE_PC and QURT_ETM_SOURCE_DATA.

**Parameters**

| | | |
|---|---|---|
| in | *addr_source_type* | Type of the address source:<br>• QURT_ETM_SOURCE_PC<br>• QURT_ETM_SOURCE_DATA |
| in | *trig_block_num* | 0 to 3. |
| in | *pid* | PID of the process.<br>• Any valid PID number enables the ASID-based trace filtering.<br>• QURT_ETM_NO_PID disables the ASID-based trace filtering. |
| in | *low_addr* | Lower boundary of PC address range. |
| in | *high_addr* | Higher boundary of PC address range. |

**Returns**

QURT_ETM_SETUP_OK – Success.
QURT_ETM_SETUP_ERR – Failure.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 28.5 qurt_etm_set_atb()

### 28.5.1 Function Documentation

#### 28.5.1.1 unsigned int qurt_etm_set_atb ( unsigned int *flag* )

Sets the advanced trace bus (ATB) state to notify QuRT that the ATB is actively enabled or disabled. QuRT performs the corresponding actions at low power management.

**Parameters**

| | | |
|---|---|---|
| in | *flag* | Values:<br>QURT_ATB_ON<br>QURT_ATB_OFF |

**Returns**

QURT_ETM_SETUP_OK – Success.
QURT_ETM_SETUP_ERR – Failure

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 28.6 qurt_etm_set_sync_period()

### 28.6.1 Function Documentation

#### 28.6.1.1 unsigned int qurt_etm_set_sync_period ( unsigned int *sync_type,* unsigned int *period* )

Sets the period for types of synchronization trace packets.

ASYNC defines the period between alignment synchronization packets. Period is in terms of bytes in the packet stream.

ISYNC defines the period between instruction synchronization packets. Period is per thread and is defined as the bytes sent out for that thread.

GSYNC is the defined period in thread cycles between GSYNC packets.

**Parameters**

| | | |
|---|---|---|
| in | *sync_type* | Type of synchronization packets:<br>QURT_ETM_ASYNC_PERIOD<br>QURT_ETM_ISYNC_PERIOD<br>QURT_ETM_GSYNC_PERIOD |
| in | *period* | Period value. |

**Returns**

QURT_ETM_SETUP_OK – Success.
QURT_ETM_SETUP_ERR – Failure.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 28.7 qurt_stm_trace_set_config()

### 28.7.1 Function Documentation

#### 28.7.1.1 unsigned int qurt_stm_trace_set_config ( qurt_stm_trace_info_t * *stm_-config_info* )

Sets up a STM port for tracing events.

**Associated data types**

qurt_stm_trace_info_t

**Parameters**

| | | |
|---|---|---|
| in | *stm_config_info* | Pointer to the STM trace information used to set up the trace in the kernel. The strucure must have the following:<br>• One port address per hardware thread<br>• Event ID for context switches<br>• Event ID for interrupt tracing<br>• Header or marker to identify the beginning of the trace. |

**Returns**

QURT_EOK – Success.
QURT_EINVALID – Failure; possibly because the passed port address is not in the page table.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 28.8 Data Types

### 28.8.1 Data Structure Documentation

#### 28.8.1.1 struct qurt_stm_trace_info_t

STM trace information.

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:40:42 GMT
hyungchul.won@lge.com

## 28.9 Macros

### 28.9.1 Define Documentation

#### 28.9.1.1 #define QURT_ETM_SOURCE_PC 0U

ETM memory source of SAC∗ is PC.

#### 28.9.1.2 #define QURT_ETM_SOURCE_DATA 1U

ETM memory source of SAC∗ is data.

#### 28.9.1.3 #define QURT_ETM_ASYNC_PERIOD 0

Async.

#### 28.9.1.4 #define QURT_ETM_ISYNC_PERIOD 1

Isync.

#### 28.9.1.5 #define QURT_ETM_GSYNC_PERIOD 2

Gsync.

#### 28.9.1.6 #define QURT_ETM_SETUP_OK 0

ETM setup OK.

#### 28.9.1.7 #define QURT_ETM_SETUP_ERR 1

ETM setup error.

#### 28.9.1.8 #define QURT_ATB_OFF 0

ATB off.

#### 28.9.1.9 #define QURT_ATB_ON 1

ATB on.

#### 28.9.1.10 #define QURT_TRACE( *str,* ... ) __VA_ARGS__

Function tracing is implemented with the QURT_TRACE debug macro, which optionally generates printf statements both before and after every function call that is passed as a macro argument.

For example, in the following macro calls in the source code:

```
QURT_TRACE(myfunc, my_func(33))
```

generates the following debug output:

```
myfile:nnn: my_func >>> calling my_func(33)
myfile:nnn: my_func >>> returned my_func(33)
```

The debug output includes the source file and line number of the function call, along with the text of the call. Compile the client source file with -D __FILENAME__ defined for its file name.

The library function qurt_printf() generates the debug output. The QURT_DEBUG symbol controls generation of the debug output. If this symbol is not defined, function tracing is not generated.

**NOTE** The debug macro is accessed through the QuRT API header file.

Qualcomm Confidential - May Contain Trade Secrets 2025-02-21 09:40:42 GMT hyungbum.wg@lge.com

# 29 Atomic Operations

QuRT kernel atomic operations are accessed with the following QuRT functions.

- qurt_atomic_set()
- qurt_atomic_and()
- qurt_atomic_and_return()
- qurt_atomic_or()
- qurt_atomic_or_return()
- qurt_atomic_xor()
- qurt_atomic_xor_return()
- qurt_atomic_set_bit()
- qurt_atomic_clear_bit()
- qurt_atomic_change_bit()
- qurt_atomic_add()
- qurt_atomic_add_return()
- qurt_atomic_add_unless()
- qurt_atomic_sub()
- qurt_atomic_sub_return()
- qurt_atomic_inc()
- qurt_atomic_inc_return()
- qurt_atomic_dec()
- qurt_atomic_dec_return()
- qurt_atomic_compare_and_set()
- qurt_atomic_barrier()
- qurt_atomic64_set()
- qurt_atomic64_and_return()
- qurt_atomic64_or()
- qurt_atomic64_or_return()
- qurt_atomic64_xor_return()

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:40:42 GMT
hyungchul.won@lge.com

- qurt_atomic64_set_bit()
- qurt_atomic64_clear_bit()
- qurt_atomic64_change_bit()
- qurt_atomic64_add()
- qurt_atomic64_add_return()
- qurt_atomic64_sub_return()
- qurt_atomic64_inc()
- qurt_atomic64_inc_return()
- qurt_atomic64_dec_return()
- qurt_atomic64_compare_and_set()
- qurt_atomic64_barrier()

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 29.1 qurt_atomic_set()

### 29.1.1 Function Documentation

#### 29.1.1.1 static QURT_INLINE unsigned int qurt_atomic_set ( unsigned int * *target,* unsigned int *value* )

Sets the atomic variable with the specified value.

**NOTE** The function retries until load lock and store conditional is successful.

**Parameters**

| | | |
|---|---|---|
| `in,out` | *target* | Pointer to the atomic variable. |
| `in` | *value* | Value to set. |

**Returns**

Value successfuly set.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 29.2 qurt_atomic_and()

### 29.2.1 Function Documentation

#### 29.2.1.1 static QURT_INLINE void qurt_atomic_and ( unsigned int ∗ *target,* unsigned int *mask* )

Bitwise AND operation of the atomic variable with mask.

**NOTE** The function retries until load lock and store conditional is successful.

**Parameters**

| | | |
|---|---|---|
| `in,out` | *target* | Pointer to the atomic variable. |
| `in` | *mask* | Mask for bitwise AND. |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 29.3 qurt_atomic_and_return()

### 29.3.1 Function Documentation

#### 29.3.1.1 static QURT_INLINE unsigned int qurt_atomic_and_return ( unsigned int * *target,* unsigned int *mask* )

Bitwise AND operation of the atomic variable with mask.

**NOTE** The function retries until load lock and store conditional is successful.

**Parameters**

| | | |
|---|---|---|
| `in,out` | *target* | Pointer to the atomic variable. |
| `in` | *mask* | Mask for bitwise AND. |

**Returns**

AND result of atomic variable with mask.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 29.4 qurt_atomic_or()

### 29.4.1 Function Documentation

#### 29.4.1.1 static QURT_INLINE void qurt_atomic_or ( unsigned int * *target,* unsigned int *mask* )

Bitwise OR operation of the atomic variable with mask.

**NOTE** The function retries until load lock and store conditional is successful.

**Parameters**

| | | |
|---|---|---|
| `in,out` | *target* | Pointer to the atomic variable. |
| `in` | *mask* | Mask for bitwise OR. |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 29.5 qurt_atomic_or_return()

### 29.5.1 Function Documentation

#### 29.5.1.1 static QURT_INLINE unsigned int qurt_atomic_or_return ( unsigned int * *target,* unsigned int *mask* )

Bitwise OR operation of the atomic variable with mask.

**NOTE** The function retries until load lock and store conditional is successful.

**Parameters**

| | | |
|---|---|---|
| `in,out` | *target* | Pointer to the atomic variable. |
| `in` | *mask* | Mask for bitwise OR. |

**Returns**

Returns the OR result of the atomic variable with mask.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 29.6 qurt_atomic_xor()

### 29.6.1 Function Documentation

#### 29.6.1.1 static QURT_INLINE void qurt_atomic_xor ( unsigned int * *target,* unsigned int *mask* )

Bitwise XOR operation of the atomic variable with mask.

**NOTE** The function retries until load lock and store conditional is successful.

**Parameters**

| | | |
|---|---|---|
| `in,out` | *target* | Pointer to the atomic variable. |
| `in` | *mask* | Mask for bitwise XOR. |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 29.7 qurt_atomic_xor_return()

### 29.7.1 Function Documentation

#### 29.7.1.1 static QURT_INLINE unsigned int qurt_atomic_xor_return ( unsigned int * *target,* unsigned int *mask* )

Bitwise XOR operation of the atomic variable with mask.

**NOTE** The function retries until load lock and store conditional is successful.

**Parameters**

| | | |
|---|---|---|
| `in,out` | *target* | Pointer to the atomic variable. |
| `in` | *mask* | Mask for bitwise XOR. |

**Returns**

XOR result of atomic variable with mask.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 29.8 qurt_atomic_set_bit()

### 29.8.1 Function Documentation

#### 29.8.1.1 static QURT_INLINE void qurt_atomic_set_bit ( unsigned int * *target,* unsigned int *bit* )

Sets a bit in the atomic variable at a specified position.

**NOTE** The function retries until load lock and store conditional is successful.

**Parameters**

| | | |
|---|---|---|
| `in,out` | *target* | Pointer to the atomic variable. |
| `in` | *bit* | Bit position to set. |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 29.9 qurt_atomic_clear_bit()

### 29.9.1 Function Documentation

#### 29.9.1.1 static QURT_INLINE void qurt_atomic_clear_bit ( unsigned int * *target,* unsigned int *bit* )

Clears a bit in the atomic variable at a specified position.

**NOTE** The function retries until load lock and store conditional is successful.

**Parameters**

| | | |
|---|---|---|
| in,out | *target* | Pointer to the atomic variable. |
| in | *bit* | Bit position to clear. |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 29.10 qurt_atomic_change_bit()

### 29.10.1 Function Documentation

#### 29.10.1.1 static QURT_INLINE void qurt_atomic_change_bit ( unsigned int * *target,* unsigned int *bit* )

Toggles a bit in a atomic variable at a bit position.

**NOTE** The function retries until load lock and store conditional is successful.

**Parameters**

| | | |
|---|---|---|
| `in,out` | *target* | Pointer to the atomic variable. |
| `in` | *bit* | Bit position to toggle. |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 29.11 qurt_atomic_add()

## 29.11.1 Function Documentation

### 29.11.1.1 static QURT_INLINE void qurt_atomic_add ( unsigned int ∗ *target,* unsigned int *v* )

Adds an integer to atomic variable.

**NOTE** The function retries until load lock and store conditional is successful.

**Parameters**

| | | |
|---|---|---|
| in,out | *target* | Pointer to the atomic variable. |
| in | *v* | Integer value to add. |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 29.12 qurt_atomic_add_return()

### 29.12.1 Function Documentation

#### 29.12.1.1 static QURT_INLINE unsigned int qurt_atomic_add_return ( unsigned int * *target,* unsigned int *v* )

Adds an integer to atomic variable.

**NOTE** The function retries until load lock and store conditional is successful.

**Parameters**

| | | |
|---|---|---|
| `in,out` | *target* | Pointer to the atomic variable. |
| `in` | *v* | Integer value to add. |

**Returns**

Result of arithmetic sum.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 29.13 qurt_atomic_add_unless()

### 29.13.1 Function Documentation

#### 29.13.1.1 static QURT_INLINE unsigned int qurt_atomic_add_unless ( unsigned int * *target,* unsigned int *delta,* unsigned int *unless* )

Adds the delta value to an atomic variable unless the current value in the target matches the unless variable.

**NOTE** The function retries until load lock and store conditional are successful.

**Parameters**

| | | |
|---|---|---|
| in,out | *target* | Pointer to the atomic variable. |
| in | *delta* | Value to add to the current value. |
| in | *unless* | Perform the addition only when the current value is not equal to this unless value. |

**Returns**

TRUE – 1 - Addition was performed.
FALSE – 0 - Addition was not done.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 29.14 qurt_atomic_sub()

### 29.14.1 Function Documentation

#### 29.14.1.1 static QURT_INLINE void qurt_atomic_sub ( unsigned int ∗ *target,* unsigned int *v* )

Subtracts an integer from an atomic variable.

**NOTE** The function retries until load lock and store conditional is successful.

**Parameters**

| | | |
|---|---|---|
| in,out | *target* | Pointer to the atomic variable. |
| in | *v* | Integer value to subtract. |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 29.15 qurt_atomic_sub_return()

## 29.15.1 Function Documentation

### 29.15.1.1 static QURT_INLINE unsigned int qurt_atomic_sub_return ( unsigned int * *target,* unsigned int *v* )

Subtracts an integer from an atomic variable.

**NOTE** The function retries until load lock and store conditional is successful.

**Parameters**

| | | |
|---|---|---|
| in,out | *target* | Pointer to the atomic variable. |
| in | *v* | Integer value to subtract. |

**Returns**

Result of arithmetic subtraction.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 29.16 qurt_atomic_inc()

## 29.16.1 Function Documentation

### 29.16.1.1 static QURT_INLINE void qurt_atomic_inc ( unsigned int * *target* )

Increments an atomic variable by one.

**NOTE** The function retries until load lock and store conditional is successful.

#### Parameters

| | | |
|---|---|---|
| `in,out` | *target* | Pointer to the atomic variable. |

#### Safe API

No

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 29.17 qurt_atomic_inc_return()

## 29.17.1 Function Documentation

### 29.17.1.1 static QURT_INLINE unsigned int qurt_atomic_inc_return ( unsigned int * *target* )

Increments an atomic variable by one.

**NOTE** The function retries until load lock and store conditional is successful.

**Parameters**

| | | |
|---|---|---|
| in,out | *target* | Pointer to the atomic variable. |

**Returns**

Incremented value.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 29.18 qurt_atomic_dec()

## 29.18.1 Function Documentation

### 29.18.1.1 static QURT_INLINE void qurt_atomic_dec ( unsigned int * *target* )

Decrements an atomic variable by one.

**NOTE** The function retries until load lock and store conditional is successful.

#### Parameters

| | | |
|---|---|---|
| `in,out` | *target* | Pointer to the atomic variable. |

#### Safe API

No

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 29.19 qurt_atomic_dec_return()

## 29.19.1 Function Documentation

### 29.19.1.1 static QURT_INLINE unsigned int qurt_atomic_dec_return ( unsigned int * *target* )

Decrements an atomic variable by one.

**NOTE** The function retries until load lock and store conditional is successful.

**Parameters**

| | | |
|---|---|---|
| in,out | *target* | Pointer to the atomic variable. |

**Returns**

Decremented value.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 29.20 qurt_atomic_compare_and_set()

## 29.20.1 Function Documentation

### 29.20.1.1 static QURT_INLINE unsigned int qurt_atomic_compare_and_set ( unsigned int * *target,* unsigned int *old_val,* unsigned int *new_val* )

Compares the current value of the atomic variable with the specified value and set to a new value when compare is successful.

**NOTE** The function retries until load lock and store conditional is successful.

**Parameters**

| | | |
|---|---|---|
| `in,out` | *target* | Pointer to the atomic variable. |
| `in` | *old_val* | Old value to compare. |
| `in` | *new_val* | New value to set. |

**Returns**

FALSE – Specified value is not equal to the current value.
TRUE --Specified value is equal to the current value.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 29.21 qurt_atomic_barrier()

## 29.21.1 Function Documentation

### 29.21.1.1 static QURT_INLINE void qurt_atomic_barrier ( void )

Allows the compiler to enforce order constraints on memory operations issued before and after the function.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 29.22 qurt_atomic64_set()

### 29.22.1 Function Documentation

#### 29.22.1.1 static QURT_INLINE unsigned long long qurt_atomic64_set ( unsigned long long ∗ *target,* unsigned long long *value* )

Sets the 64-bit atomic variable with the specified value.

**NOTE** The function retries until load lock and store conditional is successful.

**Parameters**

| | | |
|---|---|---|
| `in,out` | *target* | Pointer to the atomic variable. |
| `in` | *value* | 64-bit value to set. |

**Returns**

Successfuly set value.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 29.23 qurt_atomic64_and_return()

## 29.23.1 Function Documentation

### 29.23.1.1 static QURT_INLINE unsigned long long qurt_atomic64_and_return ( unsigned long long * *target,* unsigned long long *mask* )

Bitwise AND operation of a 64-bit atomic variable with mask.

**NOTE** The function retries until load lock and store conditional is successful.

**Parameters**

| | | |
|---|---|---|
| `in,out` | *target* | Pointer to the atomic variable. |
| `in` | *mask* | 64-bit mask for bitwise AND. |

**Returns**

AND result of 64-bit atomic variable with mask.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 29.24 qurt_atomic64_or()

## 29.24.1 Function Documentation

### 29.24.1.1 static QURT_INLINE void qurt_atomic64_or ( unsigned long long ∗ *target,* unsigned long long *mask* )

Bitwise OR operation of a 64-bit atomic variable with mask.

**NOTE** The function retries until load lock and store conditional is successful.

**Parameters**

| | | |
|---|---|---|
| `in,out` | *target* | Pointer to the atomic variable. |
| `in` | *mask* | 64-bit mask for bitwise OR. |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 29.25 qurt_atomic64_or_return()

## 29.25.1 Function Documentation

### 29.25.1.1 static QURT_INLINE unsigned long long qurt_atomic64_or_return ( unsigned long long ∗ *target,* unsigned long long *mask* )

Bitwise OR operation of a 64-bit atomic variable with mask.

**NOTE** The function retries until load lock and store conditional is successful.

**Parameters**

| | | |
|---|---|---|
| `in,out` | *target* | Pointer to the atomic variable. |
| `in` | *mask* | 64-bit mask for bitwise OR. |

**Returns**

OR result of the atomic variable with mask.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 29.26 qurt_atomic64_xor_return()

## 29.26.1 Function Documentation

### 29.26.1.1 static QURT_INLINE unsigned long long qurt_atomic64_xor_return ( unsigned long long ∗ *target,* unsigned long long *mask* )

Bitwise XOR operation of 64-bit atomic variable with mask.

**NOTE** The function retries until load lock and store conditional is successful.

**Parameters**

| | | |
|---|---|---|
| `in,out` | *target* | Pointer to the atomic variable. |
| `in` | *mask* | 64-bit mask for bitwise XOR. |

**Returns**

XOR result of atomic variable with mask.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 29.27 qurt_atomic64_set_bit()

## 29.27.1 Function Documentation

### 29.27.1.1 static QURT_INLINE void qurt_atomic64_set_bit ( unsigned long long * *target,* unsigned int *bit* )

Sets a bit in a 64-bit atomic variable at a specified position.

**NOTE** The function retries until load lock and store conditional is successful.

**Parameters**

| | | |
|---|---|---|
| in,out | *target* | Pointer to the atomic variable. |
| in | *bit* | Bit position to set. |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 29.28 qurt_atomic64_clear_bit()

## 29.28.1 Function Documentation

### 29.28.1.1 static QURT_INLINE void qurt_atomic64_clear_bit ( unsigned long long * *target,* unsigned int *bit* )

Clears a bit in a 64-bit atomic variable at a specified position.

**NOTE** The function retries until load lock and store conditional is successful.

**Parameters**

| | | |
|---|---|---|
| `in,out` | *target* | Pointer to the atomic variable. |
| `in` | *bit* | Bit position to clear. |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 29.29 qurt_atomic64_change_bit()

## 29.29.1 Function Documentation

### 29.29.1.1 static QURT_INLINE void qurt_atomic64_change_bit ( unsigned long long * *target,* unsigned int *bit* )

Toggles a bit in a 64-bit atomic variable at a bit position.

**NOTE** The function retries until load lock and store conditional is successful.

**Parameters**

| | | |
|---|---|---|
| in,out | *target* | Pointer to the atomic variable. |
| in | *bit* | Bit position to toggle. |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 29.30 qurt_atomic64_add()

## 29.30.1 Function Documentation

### 29.30.1.1 static QURT_INLINE void qurt_atomic64_add ( unsigned long long * *target,* unsigned long long *v* )

Adds a 64-bit integer to 64-bit atomic variable.

**NOTE** The function retries until load lock and store conditional is successful.

**Parameters**

| | | |
|---|---|---|
| `in,out` | *target* | Pointer to the atomic variable. |
| `in` | *v* | 64-bit integer value to add. |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 29.31 qurt_atomic64_add_return()

## 29.31.1 Function Documentation

### 29.31.1.1 static QURT_INLINE unsigned long long qurt_atomic64_add_return ( unsigned long long * *target,* unsigned long long *v* )

Adds a 64-bit integer to 64-bit atomic variable.

**NOTE** The function retries until load lock and store conditional is successful.

**Parameters**

| | | |
|---|---|---|
| `in,out` | *target* | Pointer to the atomic variable. |
| `in` | *v* | 64-bit integer value to add. |

**Returns**

Result of arithmetic sum.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 29.32 qurt_atomic64_sub_return()

## 29.32.1 Function Documentation

### 29.32.1.1 static QURT_INLINE unsigned long long qurt_atomic64_sub_return ( unsigned long long ∗ *target,* unsigned long long *v* )

Subtracts a 64-bit integer from an atomic variable.

**NOTE** The function retries until load lock and store conditional is successful.

**Parameters**

| | | |
|---|---|---|
| in,out | *target* | Pointer to the atomic variable. |
| in | *v* | 64-bit integer value to subtract. |

**Returns**

Result of arithmetic subtraction.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 29.33 qurt_atomic64_inc()

## 29.33.1 Function Documentation

### 29.33.1.1 static QURT_INLINE void qurt_atomic64_inc ( unsigned long long * *target* )

Increments a 64-bit atomic variable by one.

**NOTE** The function retries until load lock and store conditional is successful.

#### Parameters

| | | |
|---|---|---|
| `in,out` | *target* | Pointer to the atomic variable. |

#### Safe API

No

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 29.34 qurt_atomic64_inc_return()

### 29.34.1 Function Documentation

#### 29.34.1.1 static QURT_INLINE unsigned long long qurt_atomic64_inc_return ( unsigned long long ∗ *target* )

Increments a 64-bit atomic variable by one

**NOTE** The function retries until load lock and store conditional is successful.

**Parameters**

| | | |
|---|---|---|
| in,out | *target* | Pointer to the atomic variable. |

**Returns**

Incremented value.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 29.35 qurt_atomic64_dec_return()

## 29.35.1 Function Documentation

### 29.35.1.1 static QURT_INLINE unsigned long long qurt_atomic64_dec_return ( unsigned long long ∗ *target* )

Decrements a 64-bit atomic variable by one.

**NOTE** The function retries until load lock and store conditional is successful.

**Parameters**

| | | |
|---|---|---|
| in,out | *target* | Pointer to the atomic variable. |

**Returns**

Decremented value.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 29.36 qurt_atomic64_compare_and_set()

## 29.36.1 Function Documentation

### 29.36.1.1 static QURT_INLINE int qurt_atomic64_compare_and_set ( unsigned long long ∗ *target,* unsigned long long *old_val,* unsigned long long *new_val* )

Compares the current value of an 64-bit atomic variable with the specified value and sets to a new value when compare is successful.

**NOTE** The function continues to retry until load lock and store conditional is successful.

#### Parameters

| | | |
|---|---|---|
| in,out | *target* | Pointer to the atomic variable. |
| in | *old_val* | 64-bit old value to compare. |
| in | *new_val* | 64-bit new value to set. |

#### Returns

FALSE – Specified value is not equal to the current value.
TRUE – Specified value is equal to the current value.

#### Safe API

No

#### Dependencies

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 29.37 qurt_atomic64_barrier()

## 29.37.1 Function Documentation

### 29.37.1.1 static QURT_INLINE void qurt_atomic64_barrier ( void )

Allows the compiler to enforce order constraints on memory operations issued before and after the function.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 30 QuRT Callbacks

The QuRT RTOS defines a callback function that enables users to perform program-specific operations during certain QuRT system events.

The QuRT callback framework provides a safe mechanism for root process drivers to execute callback functions in a user process. The framework hosts dedicated worker threads in corresponding processes that handle the execution of the callback fuction. This ensures that the callbacks occur in context of the appropriate process thread, in result maintaining privilege boundaries.

Prerequisites for use of this framework are:

1. Driver is a QDI driver and client communicates with drivers using QDI invocations.

2. Callback configuration is specified in cust_config.xml for the user process that intends to use this framework.

qurt_cb_data_t is the public data structure that allows a client to store all the required information about the callback, including the callback function and the arguments to pass to this function when it executes. The client uses the QDI interface to register this structure with the root driver.

The callback framework provides two APIs that a root driver can use to invoke callbacks. These functions are described in the qurt_qdi.h header file.

qurt_qdi_cb_invoke_async() invokes an asynchronous callback, wherein the invoking driver does not wait for the callback to finish executing.

qurt_qdi_cb_invoke_sync() invokes a synchronous callback. Upon invocation, the invoking thread is suspended until the callback function finishes execution.

- __hexagon_bsp_init()
- qurt_cb_data_set_cbarg()
- qurt_cb_data_set_cbfunc()
- qurt_cb_data_init()
- Data Types

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.kim@lge.com

## 30.1 __hexagon_bsp_init()

**void __hexagon_bsp_init(void)**

Initializes the Hexagon processor.

__hexagon_bsp_init is called by the QuRT boot code during system startup. It enables the application system to perform application-specific system initialization. The callback function has the following properties:

- It must be implemented in assembly language.
- It can access device memory (I/O registers) only through physical addresses.
- No stack is available.
- Registers R24-27 and R31 are callee-saved registers.
- Register R31 contains the function return address.
- It executes in Supervisor mode. The function is called before the Hexagon memory management unit (MMU) is enabled, and before the kernel is started.

**NOTE** __hexagon_bsp_init is invoked only if its symbol name is defined as a function in the application code.

The function code has the following form:

```
.falign
.global __hexagon_bsp_init
.type __hexagon_bsp_init, @function
// function assembly code goes here
jumpr r31
.size __hexagon_bsp_init, .-__hexagon_bsp_init
```

**Parameters**

None.

**Returns**

None.

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 30.2 qurt_cb_data_set_cbarg()

### 30.2.1 Function Documentation

#### 30.2.1.1 static void qurt_cb_data_set_cbarg ( qurt_cb_data_t * *cb_data,* unsigned *cb_arg* )

Sets up the callback argument. This function sets up the argument passed to the callback function when it executes.

**Associated data types**

qurt_cb_data_t

**Parameters**

| | | |
|---|---|---|
| in | *cb_data* | Pointer to the callback data structure. |
| in | *cb_arg* | Argument for the callback function. |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 30.3 qurt_cb_data_set_cbfunc()

### 30.3.1 Function Documentation

#### 30.3.1.1 static void qurt_cb_data_set_cbfunc ( qurt_cb_data_t * *cb_data,* void * *cb_func* )

Sets up the callback function in the callback registration data structure.

**Associated data types**

qurt_cb_data_t

**Parameters**

| | | |
|---|---|---|
| in | *cb_data* | Pointer to the callback data structure. |
| in | *cb_func* | Pointer to the callback function. |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 30.4 qurt_cb_data_init()

### 30.4.1 Function Documentation

#### 30.4.1.1 static void qurt_cb_data_init ( qurt_cb_data_t * *cb_data* )

Initializes the callback data structure. The entity that registers a callback with the root process driver must call this function to initialize the callback registration data structure to the default value.

**Associated data types**

qurt_cb_data_t

**Parameters**

| | | |
|---|---|---|
| in | *cb_data* | Pointer to the callback data structure. |

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 30.5 Data Types

This section describes data types for callbacks.

### 30.5.1 Data Structure Documentation

#### 30.5.1.1 struct qurt_cb_data_t

Callback registration data structure. This data structure is used by a client that attempts to register a callback with a QDI driver. This data structure holds the address of callback function and the argument supplied to the callback function when it executes.

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:40:42 GMT
hyungchul.won@lge.com

# 31 HVX

The Hexagon Vector Extension (HVX) is an optional component of the Hexagon DSP for vector operations.

To get the HVX hardware configuration that the chipset supports, use this API.

- qurt_hvx_get_units()
- qurt_hvx_lock()
- qurt_hvx_unlock

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:40:42 GMT
hyungchul.won@lge.com

## 31.1 qurt_hvx_get_units()

### 31.1.1 Function Documentation

#### 31.1.1.1 int qurt_hvx_get_units ( void )

Gets the HVX hardware configuration that the chipset supports.

**NOTE** The function returns the HVX hardware configuration supported by the chipset.

**Returns**

Bitmask of the units: 1X64, 2X64, 4X64, 1X128, 2X128, and so on.

- QURT_HVX_HW_UNITS_2X126B_4X64B – V60, V62, or V65 HVX
- QURT_HVX_HW_UNITS_4X128B_0X64B – V66 CDSP or newer
- 0 – not available

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 31.2 qurt_hvx_lock()

### 31.2.1 Function Documentation

#### 31.2.1.1 int qurt_hvx_lock ( qurt_hvx_mode_t *lock_mode* )

Locks one HVX unit specified by the HVX mode.

**NOTE** Input variable can be 128B_MODE or 64B_MODE. If an HVX unit in this mode is available, this function locks the unit and returns right away. If the current HVX mode is different from the requested mode, the current thread is blocked. When all HVX units become idle, QuRT changes the mode, locks the HVX unit, and returns.

Starting from Q6v65 with HVX context switch support, qurt_hvx_lock() is mapped as qurt_hvx_set(64_BYTE or 128_BYTE).

**Associated data types**

#qurt_mode_t

**Parameters**

| | | |
|---|---|---|
| in | *lock_mode* | #QURT_HVX_MODE_64B or #QURT_HVX_MODE_128B. |

**Returns**

QURT_EOK – Success
Other value – Failure

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 31.3 qurt_hvx_unlock

### 31.3.1 Function Documentation

#### 31.3.1.1 int qurt_hvx_unlock ( void )

Unlocks the HVX unit held by this software thread.

**NOTE** Starting from Q6v65 with HVX context switch support, qurt_hvx_unlock() maps as qurt_hvx_set(QURT_HVX_RELEASE_CONTEXT).

**Returns**

QURT_EOK – Successful return
Other values – Failure

**Safe API**

Yes

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 32 Predefined Symbols

QuRT predefines the symbol QURT_API_VERSION to support backwards compatibility of the QuRT API. This symbol returns a numeric value which represents a specific compatible version of the QuRT API.

QURT_API_VERSION is redefined with a new value only when a new version of the QuRT API is released that adds new API functions, or introduces changes to the existing API functions that make them incompatible with the previous API version.

Use the symbol in conditional compilation directives to write QuRT program code that works with more than one version of the QuRT API.

For example, consider the case of a QuRT API function which is redefined in a new version of the QuRT API (for example, version N) to have a second argument. The program code can then be written to conditionally use either version of this function:

```
#if QURT_API_VERSION < N
result = qurt_func (arg1);
#else /* QURT_API_VERSION < N */
result = qurt_func (arg1, arg2);
#endif /* QURT_API_VERSION < N */
```

**NOTE** The value of QURT_API_VERSION remains unchanged across multiple QuRT releases as long as the API compatibility is not affected by the new releases.

- qurt_api_version()
- Constants and Macros

### 32.0.1 Define Documentation

#### 32.0.1.1 #define QURT_UPDATE_PROCESS_STATE 0

Qualcomm Confidential - May Contain Trade Secrets 2025-02-27 05:40:42 GMT hyunghul.won@lge.com

## 32.1 qurt_api_version()

### 32.1.1 Function Documentation

#### 32.1.1.1 unsigned int qurt_api_version ( void )

Gets the QuRT API version. Usage:

```
if QURT_API_VERSION >= QURT_API_VERSION_ENCODE(1,4,0,0)
 qurt_func_2(a,b,c);
else
 qurt_func(a);
endif
```

**Returns**

QuRT API version.

**Safe API**

No

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 32.2 Constants and Macros

### 32.2.1 Define Documentation

#### 32.2.1.1 #define QURT_API_VER_MAJOR 1

Each field of the QURT_API_VERSION definitions is an 8-bit unsigned integer. The main release updates the first three fields: Major, Minor, and Release.

- QURT_API_VERSION = Major, Minor, Release. Patch releases are supported by adding the patch field.
- QURT_API_VERSION = Major, Minor, Release, Patch.

Increment the major version for incompatible API changes.

#### 32.2.1.2 #define QURT_API_VER_MINOR 5

Increment the minor version for backward-compatible enhancements in the API set.

#### 32.2.1.3 #define QURT_API_VER_RELEASE 11

Increment the release version for each release within a MAJOR.MINOR release.

#### 32.2.1.4 #define QURT_API_VER_PATCH 0

Increment the patch version when new API content is introduced on an older LTS release.

#### 32.2.1.5 #define QURT_API_VERSION_ENCODE( *major, minor, release, patch* )

**Value:**

```
((((major) & 0xFF) << 24) | (((minor) & 0xFF) << 16) | \
        (((release) & 0xFF) << 8) | ((patch) & 0xFF))
```

Update the QURT_API_VERSION function macro.

#### 32.2.1.6 #define QURT_API_VERSION

**Value:**

```
QURT_API_VERSION_ENCODE(QURT_API_VER_MAJOR, QURT_API_VER_MINOR, \
        QURT_API_VER_RELEASE, QURT_API_VER_PATCH)
```

Update the QURT_API_VERSION macro.

Qualcomm Confidential - May Contain Trade Secrets 2020-02-27 05:40:42 GMT hyungchul.yoon@lge.com

# 33 Note

Note macros and types.

- Macros
- Types

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:40:42 GMT
hyungchul.won@lge.com

## 33.1 Macros

### 33.1.1 Define Documentation

#### 33.1.1.1 #define QURT_NOTE_TYPE_POOL_INFO 14

Identifies note holding pool information

#### 33.1.1.2 #define QURT_NOTE_TYPE_SEGMENT_INFO 15

Identifies note holding segment to pool mapping information

#### 33.1.1.3 #define QURT_NOTE_TYPE_ARB_INFO 16

Identifies note holding ARB info

#### 33.1.1.4 #define QURT_NOTE_NAME "QUALCOMM_QDSP6"

Name of the note the loader expects for loader-specific functions

#### 33.1.1.5 #define QURT_NOTE_NAME_SIZE 16

Size of name element in the note

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:40:42 GMT
hyungchul.won@lge.com

## 33.2 Types

### 33.2.1 Data Structure Documentation

#### 33.2.1.1 struct qurt_note_t

General purpose note.

#### 33.2.1.2 struct qurt_note_type_pool_info_t

Note holding pool information consumed by the dynamic loader.

#### 33.2.1.3 struct qurt_note_type_pool_seg_info_t

Note holding segment to pool mapping information consumed by the dynamic loader.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 34 OS services

OS service APIs.

- Macros

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:40:42 GMT
hyungchul.won@lge.com

## 34.1 Macros

### 34.1.1 Define Documentation

#### 34.1.1.1 #define QURT_OS_SERVICE_THREAD "/os/thread"

Thread service.

#### 34.1.1.2 #define QURT_OS_SERVICE_FS_HUB "/os/fs_hub"

File system hub.

#### 34.1.1.3 #define QURT_OS_SERVICE_CALLBACK "/os/callback"

QDI callback service.

#### 34.1.1.4 #define QURT_OS_SERVICE_INTERRUPTS "/os/interrupt"

Interrupt service.

#### 34.1.1.5 #define QURT_OS_SERVICE_PROXY "/os/proxy"

QDI proxy serice.

#### 34.1.1.6 #define QURT_OS_SERVICE_MEMORY "/os/memory"

Memory management service.

#### 34.1.1.7 #define QURT_OS_SERVICE_MEMPOOL "/os/mempool"

Pool management service.

#### 34.1.1.8 #define QURT_OS_SERVICE_PROCESS "/os/process"

Process management service.

#### 34.1.1.9 #define QURT_OS_SERVICE_MMAP "/os/mem_mapper"

Memory mapper service.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 35 Physical memory manager

Physical memory manager APIs.

- func_qurt_pmem_acquire()
- func_qurt_pmem_release()
- Physical memory management macros

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:40:42 GMT
hyungchul.won@lge.com

## 35.1 func_qurt_pmem_acquire()

### 35.1.1 Function Documentation

#### 35.1.1.1 int qurt_pmem_acquire ( unsigned int *ppage,* unsigned int *pnum* )

Acquire the ownership of a specific physical memory region.

**NOTE** The ownership will be the caller

**Parameters**

| | | |
|---|---|---|
| in | *ppage* | Starting physical page number |
| in | *pnum* | Number of physical pages |

**Returns**

QURT_PMEM_NO_PRIV – Have no privilege to claim the ownership.
QURT_PMEM_OVERLAP – The whole or part of the range has been owned
QURT_PMEM_SUCCESS – Succeed to claim ownership.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 35.2 func_qurt_pmem_release()

### 35.2.1 Function Documentation

#### 35.2.1.1 int qurt_pmem_release ( unsigned int *ppage,* unsigned int *pnum* )

Release the ownership of a specific physical memory region.

**Parameters**

| | | |
|---|---|---|
| in | *ppage* | The start of physical page number |
| in | *pnum* | The numbers of physical pages |

**Returns**

QURT_PMEM_NO_PRIV – Have no privilege to claim the ownership.
QURT_PMEM_NOT_EXIST – The physical memory range is not usable.
QURT_PMEM_OUTSTANDING_MAPPING – There is outstanding mapping in this range
QURT_PMEM_SUCCESS – Succeed to claim ownership.

**Safe API**

No

**Dependencies**

None.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 35.3 Physical memory management macros

### 35.3.1 Define Documentation

#### 35.3.1.1 #define QURT_PMEM_SUCCESS 0

#### 35.3.1.2 #define QURT_PMEM_NO_PRIV 1

#### 35.3.1.3 #define QURT_PMEM_RETRY 2

#### 35.3.1.4 #define QURT_PMEM_OVERLAP 3

#### 35.3.1.5 #define QURT_PMEM_NOT_EXIST 4

#### 35.3.1.6 #define QURT_PMEM_INIT_FAILURE 5

#### 35.3.1.7 #define QURT_PMEM_OUTSTANDING_MAPPING 6

#### 35.3.1.8 #define QURT_PMEM_GENERIC_FAILURE 7

#### 35.3.1.9 #define QURT_PMEM_ENTRY_FOUND 8

#### 35.3.1.10 #define QURT_PMEM_REACH_END 9

#### 35.3.1.11 #define QURT_PMEM_UNCLAIMED 10

#### 35.3.1.12 #define QURT_PMEM_ALREADY_CLAIMED 11

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# 36 Printf

Prototypes of printf APIs.

- printf()
- vprintf()

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:40:42 GMT
hyungchul.won@lge.com

# 36.1 printf()

## 36.1.1 Function Documentation

### 36.1.1.1 int qurt_printf ( const char * *format,* ... )

Prints formatted output to the console.

**Parameters**

| | | |
|---|---|---|
| in | *format* | specifies how the output should be formatted. |

**Returns**

positive value – the number of characters printed negative value – In case of error

**Safe API**

Yes

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

## 36.2 vprintf()

### 36.2.1 Function Documentation

#### 36.2.1.1 int qurt_vprintf ( const char ∗ *format,* va_list *args* )

Prints formatted output with a variable argument list.

**Parameters**

| | | |
|---|---|---|
| in | *vaddr* | Virtual address for which the attributes are required. |
| in | *pid* | process id for the target process |
| out | *attr* | Pointer to the mapping attribute structure. |

**Returns**

positive value – the number of characters printed negative value – In case of error

**Safe API**

Yes

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@lge.com

# A Thread-level Profiling

## A.1 Overview

The profiling support in QuRT (Section 24.1) can be used to profile the execution of one or more QuRT threads individually, or the entire QuRT application system as a whole.

The following sections describe the procedure for profiling QuRT threads. The description is presented in terms of a client/server model:

- The client resides outside the system, and is connected by some means to the server that is using the QuRT system.
- The client sends the profiling information in units of packets.
- The server processes the packets and plots a graph displaying the CPU utilization.

## A.2 Server Behavior

The server receives and processes the following events:

- Start command from client
- Timer expiry
- Stop command from client

**Start command**
The start command specifies the sampling period for profiling. The use of a sampling period limits the overhead imposed on the overall system by the profiling task.

Upon receiving the start command, the server initializes its state by performing the following steps:

1. Record the system clock using qurt_sysclock_attr_get_hw_ticks(). This value is referred to as tick_base.
2. Record the PCYCLE count from the core using qurt_get_core_pcycles(). This value is referred to as pcycle_base.
3. Clear the pcycles of all threads of the system (or alternatively a specific subset of threads) using qurt_profile_reset_threadid_pcycles().
4. Clear the idle thread pcycles using qurt_profile_reset_idle_pcycles().
5. Start a periodic timer (Section 15.1) with the period specified by the sampling period received from the start command.
6. Enable QuRT profiling using qurt_profile_enable(1).

**Timer expiry**
The timer expiry triggers the start of the collection of the profiling information. The server performs the

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT yungchuwen@lge.com

following steps when the timer expires.

1. Record the system clock using qurt_sysclock_attr_get_hw_ticks(). This value is referred to as tick_base. Compute the value ticks using the following equation: ticks = new_tick_base – tick_base
2. Record the PCYCLE count from the core using qurt_get_core_pcycles().This value is referred to as pcycle_base. Compute the value total_pcycles using the following equation: total_pcycles = new_pcycle_base – pcycle_base
3. Obtain the run time information of a thread using qurt_profile_get_thread_pcycles(). This value is referred to as pcycles.
4. For each thread being profiled, construct a packet with the following information:
   - ticks
   - total_pcycles
   - pcycles
   - core_clock_freq
   - thread_ID
5. Send the constructed packets to the client.

**Stop command**
Upon receiving the stop command, the server performs the following steps:

1. Stop the periodic timer started by the start command.
2. Disable QuRT profiling using qurt_profile_enable(0).

## A.3 Client Behavior

The client accepts user input to start and stop profiling. It receives the packets sent by the server, and converts the information to absolute time.

When the client issues a start command, it resets to zero both the run time of each thread and the total run time. Assume that the client maintains the following values:

- prev_thread_pcycles
- prev_ticks
- thread_run_time
- system_run_time

All of these values are set to 0 when the client issues the start command.

Given the above values, the following logic can be used to determine the run time and CPU utilization of a QuRT thread.

```
net_run_pcycles = pcycles - prev_pcycles;
net_ticks = ticks - prev_ticks;
thread_run_time = thread_run_time +
(net_run_pcycles / (6 * core_clock_freq));
system_run_time = system_run_time +
(net_ticks / QTIMER_clk_freq);
prev_pcycles = pcycles;
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:41:42 GMT hyungchul.yun@lge.com

```
prev_ticks = ticks;
```

Note that this works even if the core clock frequency changes during the course of the profiling. Any change in the core clock frequency is limited to only a single iteration; therefore, the error accumulated is insignificant.

**Note:** The QTIMER clock frequency used above is fixed at 19.2 MHz on all target systems.

## A.4 Profiling the System

System profiling can be performed efficiently without having to profile all the QuRT threads in the system.

The client can request the server to send IDLE information. For example, the server sends the IDLE thread information using the same parameters used for thread profiling; the only difference is that pcycles represents the IDLE thread run time.

The IDLE thread run time is equivalent to the IDLE time of the hardware thread (i.e., the duration the hardware thread spent in the WAIT state).

With minor modifications, the same logic used for thread profiling can be used to determine the run time and CPU utilization of the system:

```
net_run_pcycles = pcycles - prev_pcycles;
net_total_pcycles = total_pcycles - prev_total_pcycles;
net_ticks = ticks - prev_ticks;
run_time = net_run_time +
((net_total_pcycles - net_run_pcycles) /
(6 * core_clock_freq));
system_run_time = system_run_time +
(net_ticks / sleep_clk_freq);
prev_pcycles = pcycles;
prev_total_pcycles = total_pcycles;
prev_ticks = ticks;
```

This makes it possible to plot the CPU utilization of each hardware thread of the system without going through all the threads in the system.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.woo@lge.com

# B Debugging Errors and Cause Codes

## B.1 Debugging Errors and Exceptions

The QuRT error handling routine populates the QURT_error_info data structure with error information.

This can be used to identify:

- Nonfatal (recoverable) errors, seen when:
  - User program raises an ASSERT using a QuRT API
  - User program results in an exception
- Fatal (non-recoverable) errors, seen:
  - When the program exception handler promotes a nonfatal error as a fatal error
  - When a QuRT program code (kernel) raises a fatal error (ASSERT)
  - When an exception is seen in Supervisor mode.
  - On imprecise exceptions

To triage crash dumps:

- Identify the reason behind the fatal errors by looking for values in QURT_error_info.status.cause and QURT_error_info.status.cause2 from the cause-cause2 table.
- For fatal errors, local and global registers are saved into QURT_error_info.locregs and QURT_error_info.globregs.
- At times, the application triggers the fatal error and in most cases is the program exception handler handling nonfatal errors. Use QURT_error_info.user_errors to triage nonfatal errors.
- For nonfatal exceptions, the local registers for the faulting thread are saved into TCB. Can be accessed by QURT_error_info.user_errors.entry[counter].error_tcb.
- For nonfatal errors raised through qurt_exception_raise_nonfatal(), only callee-saved registers are saved into TCB.

Qualcomm Confidential - May Contain Trade Secrets 2025-02-27 05:40:12 GMT hyunseoul.won@lge.com

## B.2 Cause Codes

Cause and cause2 are error codes to distinguish multiple errors. All cause and cause2 error codes can range from 1 to 255, and every cause can have 1 to 255 error codes. Hence the system can have up to 255 * 255 unique error codes.

The combination of cause and cause2 is represented as ((cause2 « 8) 'Logical OR' (cause) ).

Some Cause2 codes are statically defined, whereas some are obtained from SSR[7:0] cause codes. SSR cause codes are defined in 80-V9418-27 and 80-V9418-25. All possible combinations are listed in Tables B.5 and B.6.

## B.3 Debugging a Fatal Error

| Error | Description |
|---|---|
| QURT_error_info.status.status | Indicates whether an error occurred. |
| QURT_error_info.status.cause | Cause code for fatal error, see Table B.5. |
| QURT_error_info.status.cause2 | Cause2 code for fatal error, see Table B.6. |
| QURT_error_info.status.fatal | Indicates whether a fatal error occurred. A user error can result in a fatal error if the exception handler is not registered |
| QURT_error_info.status.hw_tnum | Indicates the index of QURT_error_info.locregs[], where the context is saved if the error is a fatal error. |
| QURT_error_info.global_regs | Contains the values of the global registers of Q6. |
| QURT_error_info.local_regs [QURT_error_info.status.hw_tnum] | Provides the CPU context if the error is a supervisor error. |

## B.4 Debugging a Nonfatal Error

| Error | Description |
|---|---|
| QURT_error_info.user_errors | All user errors are logged here |
| QURT_error_info.user_errors.counter | Index to last logged error |
| QURT_error_info.user_errors.entry[0...counter] | Structure for logged error |
| QURT_error_info.user_ errors.entry[0...counter].error_tcb | TCB for the user error |
| QURT_error_info.user_ errors.entry[0...counter].error_tcb.error | Information about error; Cause, Cause2, Badva, and HWTID |
| QURT_error_info.user_ errors.entry[0...counter].error_code | ((cause2 « 8) 'Logical OR' (cause) ). See Table B.6. |
| QURT_error_info.user_ errors.entry[0...counter].hw_thread | Hardware thread ID for error. |
| QURT_error_info.user_ errors.entry[0...counter].pcycle | Pcycle for error. |

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungdeul.won@lge.com

## B.5 Cause

| Cause | Value | Description |
|---|---|---|
| QURT_EXCEPT_PRECISE | 0x01 | Precise exception occurred. |
| QURT_EXCEPT_NMI | 0x02 | NMI occurred. |
| QURT_EXCEPT_TLBMISS | 0x03 | TLBMISS RW occurred. |
| QURT_EXCEPT_RSD_VECTOR | 0x04 | Interrupt was raised on reserved vector, should never happen. |
| QURT_EXCEPT_ASSERT | 0x05 | Kernel assert. |
| QURT_EXCEPT_BADTRAP | 0x06 | Trap0 was called with unsupported num. |
| QURT_EXCEPT_UNDEF_TRAP1 | 0x07 | Trap1 is not supported; using Trap1 causes this error. |
| QURT_EXCEPT_EXIT | 0x08 | Application called qurt_exit(), or called qurt_exception_raise_nonfatal(). Can call from C library. |
| QURT_EXCEPT_TLBMISS_X | 0x0A | TLBMISS X (execution) occurred. |
| QURT_EXCEPT_STOPPED | 0x0B | Running thread stopped due to fatal error on another hardware thread. |
| QURT_EXCEPT_FATAL_EXIT | 0x0C | Call ended because of an internal error |
| QURT_EXCEPT_INVALID_INT | 0x0D | Kernel received an invalid L1 interrupt. |
| QURT_EXCEPT_FLOATING_POINT | 0x0E | Kernel received a floating point error. |
| QURT_EXCEPT_DBG_SINGLE_STEP | 0x0F | Cause2 is not defined. |
| QURT_EXCEPT_TLBMISS_RW_ ISLAND | 0x10 | Read write miss in Island mode. |
| QURT_EXCEPT_TLBMISS_X_ISLAND | 0x11 | Execute miss in Island mode. |
| QURT_EXCEPT_SYNTHETIC_FAULT | 0x12 | Synthetic fault with user request that the kernel detected. See Table B.6. |
| QURT_EXCEPT_INVALID_ ISLAND_TRAP | 0x13 | Invalid trap in Island mode. |
| QURT_EXCEPT_UNDEF_TRAP0 | 0x14 | Trap0 called with an unsupported number. |
| QURT_EXCEPT_PRECISE_DMA_ ERROR | 0x28 | Precise DMA error. Cause2 is DM4[15:8]. Badva is DM5 register. |

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 00:20:42 GMT hyungchul.won@lge.com

## B.6 Cause 2

| Cause | Possible cause 2 | Value | Cause2 description |
|---|---|---|---|
| QURT_EXCEPT_PRECISE | SSR[7:0] | – | – |
| QURT_EXCEPT_NMI | Not defined. | – | – |
| QURT_EXCEPT_TLBMISS | SSR[7:0] | – | – |
| QURT_EXCEPT_RSD_ VECTOR | Not defined. | – | – |
| QURT_EXCEPT_ASSERT | QURT_ABORT_FUTEX_ WAKE_MULTIPLE | 0x01 | Abort cause; futex wake multiple |
| | QURT_ABORT_WAIT_ WAKEUP_SINGLE_MODE | 0X02 | Abort cause; thread waits to wake up in Single-threaded mode |
| | QURT_ABORT_TCXO_ SHUTDOWN_NOEXIT | 0x03 | Abort cause; TCXO shutdown is called without exit |
| | QURT_ABORT_FUTEX_ ALLOC_QUEUE_FAIL | 0x04 | Abort cause; futex allocation queue fail |
| | QURT_ABORT_INVALID_ CALL_QURTK_WARM_INIT | 0x05 | Abort cause; invalid QURTK_warm_init() call in NONE CON-FIG_POWER_MGMT mode |
| | QURT_ABORT_THREAD_ SCHEDULE_SANITY | 0x06 | Abort cause; sanity schedule thread is not supposed to run on the current hardware thread |
| | QURT_ABORT_REMAP | 0x07 | Remap in the page table; the correct behavior is to always remove mapping |
| | QURT_ABORT_NOMAP | 0x08 | No mapping in the page table when removing a user mapping |
| | QURT_ABORT_OUT_ OF_SPACES | 0x09 | – |
| | QURT_ABORT_INVALID_ MEM_MAPPING_TYPE | 0x0A | Invalid memory mapping type when creating qmemory |
| | QURT_ABORT_NOPOOL | 0x0B | No pool available to attach |
| | QURT_ABORT_LIFO_ REMOVE_NON_EXIST_ ITEM | 0x0C | Cannot allocate more futex waiting queue |
| | QURT_ABORT_ARG_ERROR | 0x0D | – |
| | QURT_ABORT_ASSERT | 0x0E | Assert abort |
| | QURT_ABORT_FATAL | 0x0F | Fatal error that shall never happen |
| | QURT_ABORT_FUTEX_ RESUME_INVALID_QUEUE | 0x10 | Abort cause; invalid queue ID in futex resume |

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:40:42 GMT hyungchul.won@qc.com

| Cause | Possible cause 2 | Value | Cause2 description |
|---|---|---|---|
| | QURT_ABORT_FUTEX_ WAIT_INVALID_QUEUE | 0x11 | Abort cause; invalid queue ID in futex wait |
| | QURT_ABORT_FUTEX_ RESUME_INVALID_FUTEX | 0x12 | Abort cause; invalid futex object in hashtable |
| | QURT_ABORT_NO_ ERHNDLR | 0x13 | No registered error handler |
| | QURT_ABORT_ERR_ REAPER | 0x14 | Exception in reaper thread |
| | QURT_ABORT_FREEZE_ UNKNOWN_CAUSE | 0x15 | Abort in thread freeze operation |
| | QURT_ABORT_FUTEX_ WAIT_WRITE_FAILURE | 0x16 | Unable to perform a necessary write operation to userland data during futex wait processing; most likely due to a DL Pager eviction. |
| | QURT_ABORT_ERR_ ISLAND_EXP_HANDLER | 0x17 | Exception in Island exception handler task. |
| | QURT_ABORT_L2_ TAG_DATA_CHECK_FAIL | 0x18 | Detected error in L2 Tag/Data during warm boot, The L2 Tag/Data check is done when CONFIG_DEBUG_L2 is enabled. |
| | QURT_ABORT_ERR_ SECURE_PROCESS | 0x19 | Abort error secure process. |
| | QURT_ABORT_ERR_EXP_ HANDLER | 0x20 | Either no exception handler or handler itself caused an exception. |
| | QURT_ABORT_ERR_NO_ PCB | 0x21 | Thread context PCB failed initialization, PCB was NULL. |
| | QURT_ABORT_NO_PHYS_ ADDR | 0x22 | Unable to find the physical address for the virtual address. |
| | QURT_ABORT_OUT_OF_ FASTINT_CONTEXTS | 0x23 | Fast interrupt contexts exhausted. |
| | QURT_ABORT_CLADE_ERR | 0x24 | Fatal error seen with CLADE interrupt. |
| | QURT_ABORT_ETM_ERR | 0x25 | Fatal error seen with ETM interrupt. |
| | QURT_ABORT_ECC_DED_ ASSERT | 0x26 | ECC two-bit DED error. |
| | QURT_ABORT_VTLB_ERR | 0x27 | Fatal error in the VTLB layer. |
| | QURT_ABORT_TLB_ENCODE_ DECODE_FAILURE | 0x28 | Failure during the TLB encode or decode operation. |

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:10:27 GMT hyunchul.wee@lge.com

| Cause | Possible cause 2 | Value | Cause2 description |
|---|---|---|---|
| | QURT_ABORT_VTLB_ WALKOBJS_BOUND_ FAILURE | 0x29 | Failure to look up entry in the page table. |
| QURT_EXCEPT_BADTRAP | 0 | – | – |
| QURT_EXCEPT_UNDEF_ TRAP1 | Not defined | – | – |
| QURT_EXCEPT_EXIT | Argument passed to qurt_exception_raise_nonfatal() and 0XFF | – | – |
| QURT_EXCEPT_TLBMISS_X | SSR[7:0] | – | – |
| QURT_EXCEPT_STOPPED | Not defined | – | – |
| QURT_EXCEPT_FATAL_EXIT | Not defined | – | – |
| QURT_EXCEPT_INVALID_INT | Not defined | – | – |
| QURT_EXCEPT_FLOATING_ POINT | Not defined | – | – |
| QURT_EXCEPT_DBG_ SINGLE_STEP | Not defined | – | – |
| QURT_EXCEPT_TLBMISS_ RW_ISLAND | QURT_TLB_MISS_RW_MEM_ READ | 0x70 | |
| | QURT_TLB_MISS_RW_MEM_ WRITE | 0x71 | – |
| QURT_EXCEPT_TLBMISS_X_ ISLAND | SSR[7:0] | – | – |
| QURT_EXCEPT_SYNTHETIC_ FAULT | QURT_SYNTH_ERR | 0X01 | – |
| | QURT_SYNTH_INVALID_OP | 0X02 | – |
| | QURT_SYNTH_DATA_ ALIGNMENT_FAULT * | 0X03 | – |
| | QURT_SYNTH_FUTEX_INUSE * | 0X04 | – |
| | QURT_SYNTH_FUTEX_ BOGUS * | 0X05 | – |
| | QURT_SYNTH_FUTEX_ ISLAND * | 0X06 | – |
| | QURT_SYNTH_FUTEX_ DESTROYED * | 0X07 | – |
| | QURT_SYNTH_PRIVILEGE_ ERR | 0X08 | – |
| QURT_EXCEPT_INVALID_ ISLAND_TRAP | Trap number | – | – |
| QURT_EXCEPT_UNDEF_ TRAP0 | Trap number | – | |

*** Badva is the object address for these cause2 codes.**

Qualcomm Confidential - May Contain Trade Secrets 2026-01-07 03:14:42 PST hyunchulryu@lsi.com

# C References

## C.1 Related Documents

| Title | Number |
|---|---|
| **Qualcomm Technologies** | |
| *Hexagon QuRT RTOS System Build Guide* | 80-VB419-79 |
| *Hexagon QuRT Performance Profiling User Guide* | 80-VB419-92 |
| *Hexagon QuRT QDI User Guide* | 80-VB419-84 |
| *Hexagon V5X Programmer's Reference Manual* | 80-N2040-8 |
| *Hexagon V66 Programmer's Reference Manual* | 80-N2040-42 |
| *Hexagon V67 Programmer's Reference Manual* | 80-N2040-45 |
| *Hexagon V68 Programmer's Reference Manual* | 80-N2040-46 |
| *Hexagon V69 Programmer's Reference Manual* | 80-N2040-46 |
| *Hexagon V71 Programmer's Reference Manual* | 80-N2040-46 |
| *Hexagon V73 Programmer's Reference Manual* | 80-N2040-46 |
| *Hexagon V66 Architecture System-Level Specification* | 80-V9418-25 |
| *Hexagon V67 Architecture System-Level Specification* | 80-V9418-27 |
| ***Resources*** | |
| *Abraham Silberschatz, Peter Baer Galvin, and Greg Gagne. Operating System Concepts. John Wiley and Sons, 2008.* | ISBN No. 0470128720 |

Qualcomm Confidential - May Contain Trade Secrets 2025-02-17 03:40:42 GMT hyungsul.yun@lge.com

## C.2 Acronyms and Terms

| Acronym or term | Definition |
|---|---|
| API | Application programming interface |
| Application | Category of user program (multimedia, modem firmware, modem software) |
| ATB | Advanced trace bus |
| BSP | Board support package |
| CDSP | Compute DSP |
| DED | Double error detection |
| DSP | Digital signal processor |

| Acronym or term | Definition |
|---|---|
| DTB | Device tree blob |
| EBI | External bus interface (memory type) |
| ELF | Executable and linking format |
| ETM | Embedded trace macro |
| Edge-triggered | Interrupt triggered by a rising or falling transition on the interrupt request line |
| HVX | Hexagon Vector Extensions |
| Interrupt | Externally generated processor event that interrupts the normal flow of program control |
| IST | Interrupt service thread |
| L2VIC | Second-level vector interrupt controller |
| Level-triggered | Interrupt triggered by a high or low level on the interrupt request line |
| LPM | Low-power memory |
| LR | Link register |
| MMU | Memory management unit |
| NMI | Nonmaskable interrupt |
| PCB | Process control block |
| PDC | Peripheral DMA controller |
| PID | Process ID. |
| PMU | Performance monitor unit – Hexagon processor feature that measures code performance |
| Priority | User-defined thread attribute that prioritizes thread execution |
| QDI | QuRT driver invocation – Set of facilities that support the implementation of device drivers in the QuRT system |
| QuRT | Real time operating system for the Hexagon processor |
| RTOS | Real-time operating system |
| SAC | Secure access control |
| SMI | Stack memory interface |
| SRM | System resource manager |
| SSR | Supervisor status register |
| STID | Software thread ID |
| TCB | Task control block |
| TCM | Tightly coupled memory |
| TCXO | Temperature compensated crystal oscillator |
| TLB | Translation lookaside buffer |
| TLS | Thread local storage |
| UGP | User general pointer |
| User program | Complete program that makes calls to the QuRT API to perform RTOS operations |
| USR | User status register |
| VA | Virtual address |
| VMA | Virtual memory area |

QUALCOMM
Confidential - May Contain Trade Secrets
2025-05-27 05:40:42 GMT
kwangsoul.won@lge.com

# LEGAL INFORMATION

**Your access to and use of this material, along with any documents, software, specifications, reference board files, drawings, diagnostics and other information contained herein (collectively this "Material"), is subject to your (including the corporation or other legal entity you represent, collectively "You" or "Your") acceptance of the terms and conditions ("Terms of Use") set forth below. If You do not agree to these Terms of Use, you may not use this Material and shall immediately destroy any copy thereof.**

**1) Legal Notice.**

This Material is being made available to You solely for Your internal use with those products and service offerings of Qualcomm Technologies, Inc. ("**Qualcomm Technologies**"), its affiliates and/or licensors described in this Material, and shall not be used for any other purposes. If this Material is marked as "**Qualcomm Internal Use Only**", no license is granted to You herein, and You must immediately (a) destroy or return this Material to Qualcomm Technologies, and (b) report Your receipt of this Material to qualcomm.support@qti.qualcomm.com. This Material may not be altered, edited, or modified in any way without Qualcomm Technologies' prior written approval, nor may it be used for any machine learning or artificial intelligence development purpose which results, whether directly or indirectly, in the creation or development of an automated device, program, tool, algorithm, process, methodology, product and/or other output. Unauthorized use or disclosure of this Material or the information contained herein is strictly prohibited, and You agree to indemnify Qualcomm Technologies, its affiliates and licensors for any damages or losses suffered by Qualcomm Technologies, its affiliates and/or licensors for any such unauthorized uses or disclosures of this Material, in whole or part.

Qualcomm Technologies, its affiliates and/or licensors retain all rights and ownership in and to this Material. No license to any trademark, patent, copyright, mask work protection right or any other intellectual property right is either granted or implied by this Material or any information disclosed herein, including, but not limited to, any license to make, use, import or sell any product, service or technology offering embodying any of the information in this Material.

THIS MATERIAL IS BEING PROVIDED "AS IS" WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESSED, IMPLIED, STATUTORY OR OTHERWISE. TO THE MAXIMUM EXTENT PERMITTED BY LAW, QUALCOMM TECHNOLOGIES, ITS AFFILIATES AND/OR LICENSORS SPECIFICALLY DISCLAIM ALL WARRANTIES OF TITLE, MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, SATISFACTORY QUALITY, COMPLETENESS OR ACCURACY, AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MOREOVER, NEITHER QUALCOMM TECHNOLOGIES, NOR ANY OF ITS AFFILIATES AND/OR LICENSORS, SHALL BE LIABLE TO YOU OR ANY THIRD PARTY FOR ANY EXPENSES, LOSSES, USE, OR ACTIONS HOWSOEVER INCURRED OR UNDERTAKEN BY YOU IN RELIANCE ON THIS MATERIAL.

Certain product kits, tools and other items referenced in this Material may require You to accept additional terms and conditions before accessing or using those items.

Technical data specified in this Material may be subject to U.S. and other applicable export control laws. Transmission contrary to U.S. and any other applicable law is strictly prohibited.

Nothing in this Material is an offer to sell any of the components or devices referenced herein.

This Material is subject to change without further notification.

In the event of a conflict between these Terms of Use and the Website Terms of Use on www.qualcomm.com, the *Qualcomm Privacy Policy* referenced on www.qualcomm.com, or other legal statements or notices found on prior pages of the Material, these Terms of Use will control. In the event of a conflict between these Terms of Use and any other agreement (written or click-through, including, without limitation any non-disclosure agreement) executed by You and Qualcomm Technologies or a Qualcomm Technologies affiliate and/or licensor with respect to Your access to and use of this Material, the other agreement will control.

These Terms of Use shall be governed by and construed and enforced in accordance with the laws of the State of California, excluding the U.N. Convention on International Sale of Goods, without regard to conflict of laws principles. Any dispute, claim or controversy arising out of or relating to these Terms of Use, or the breach or validity hereof, shall be adjudicated only by a court of competent jurisdiction in the county of San Diego, State of California, and You hereby consent to the personal jurisdiction of such courts for that purpose.

**2) Trademark and Product Attribution Statements.**

Qualcomm is a trademark or registered trademark of Qualcomm Incorporated. Arm is a registered trademark of Arm Limited (or its subsidiaries) in the U.S. and/or elsewhere. The Bluetooth® word mark is a registered trademark owned by Bluetooth SIG, Inc. Other product and brand names referenced in this Material may be trademarks or registered trademarks of their respective owners.

Snapdragon and Qualcomm branded products referenced in this Material are products of Qualcomm Technologies, Inc. and/or its subsidiaries. Qualcomm patented technologies are licensed by Qualcomm Incorporated.

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:40:42 GMT
sungchul.woo@lge.com